

# Teoria Completa

**Dataprev 2026**

Analista de Tecnologia da Informação — Perfil 3  
Desenvolvimento de Software

---

Banca: **FGV** · Prova: **11/10/2026, 13h–17h** · Lotação: **Natal/RN**

Módulo I (Gerais) + Módulo II (Específicos, peso 2,5)

---

Complemento da **apostila/** (revisão rápida, com dicas de banca):  
aqui cada conceito do edital é explicado do zero, com exemplos resolvidos,  
sem pressa e sem atalho. Use a apostila para revisar; use este livro  
quando um conceito não estiver fixado.

Compilado em 30 de julho de 2026

Uso pessoal de estudo — material derivado, não substitui o edital oficial.

# Sumário

|          |                                                                                  |           |
|----------|----------------------------------------------------------------------------------|-----------|
| <b>I</b> | <b>Módulo II — Conhecimentos Específicos (peso 2,5, 30 questões)</b>             | <b>1</b>  |
| <b>3</b> | <b>Engenharia de Software</b>                                                    | <b>2</b>  |
| 3.1      | Modelos de ciclo de vida . . . . .                                               | 2         |
| 3.1.1    | Modelo V: cada fase de construção tem seu par de teste . . . . .                 | 3         |
| 3.1.2    | Verificação x validação . . . . .                                                | 4         |
| 3.1.3    | RUP: fase x disciplina . . . . .                                                 | 4         |
| 3.2      | Maturidade de processo: CMMI e MPS.BR . . . . .                                  | 5         |
| 3.3      | Engenharia de requisitos . . . . .                                               | 6         |
| 3.3.1    | Requisito funcional x não funcional . . . . .                                    | 7         |
| 3.3.2    | Processo de engenharia de requisitos . . . . .                                   | 7         |
| 3.4      | Metodologias ágeis . . . . .                                                     | 8         |
| 3.4.1    | Scrum: framework, não metodologia . . . . .                                      | 8         |
| 3.4.2    | Kanban, XP, Lean . . . . .                                                       | 9         |
| 3.5      | Testes de software . . . . .                                                     | 9         |
| 3.5.1    | Níveis e tipos . . . . .                                                         | 9         |
| 3.5.2    | Testes de desempenho: carga, estresse, volume . . . . .                          | 10        |
| 3.5.3    | Técnicas de teste caixa-preta: partição de equivalência e valor limite . . . . . | 10        |
| 3.5.4    | Cobertura de caixa-branca . . . . .                                              | 11        |
| 3.5.5    | TDD, BDD, caixa-preta x caixa-branca, RPA . . . . .                              | 11        |
| 3.6      | Manutenção de software . . . . .                                                 | 12        |
| 3.7      | Mensuração: Ponto de Função e Story Points . . . . .                             | 12        |
| 3.7.1    | Ponto de Função (APF): medir de fora, independente da linguagem . . . . .        | 12        |
| 3.7.2    | Ponto de Função x Story Points . . . . .                                         | 13        |
| 3.8      | DevOps e estratégias de entrega . . . . .                                        | 14        |
| 3.9      | Design de software . . . . .                                                     | 14        |
| <b>4</b> | <b>Padrões de Projeto e UML</b>                                                  | <b>16</b> |
| 4.1      | Padrões de projeto (GoF) . . . . .                                               | 16        |
| 4.1.1    | Criacionais . . . . .                                                            | 17        |
| 4.1.2    | Estruturais . . . . .                                                            | 18        |
| 4.1.3    | Comportamentais . . . . .                                                        | 19        |
| 4.1.4    | GRASP . . . . .                                                                  | 20        |
| 4.1.5    | SOLID . . . . .                                                                  | 21        |
| 4.1.6    | Padrões arquiteturais (fora do catálogo GoF) . . . . .                           | 21        |

|          |                                                                                |           |
|----------|--------------------------------------------------------------------------------|-----------|
| 4.1.7    | Reuso e anti-padrões . . . . .                                                 | 22        |
| 4.2      | UML . . . . .                                                                  | 22        |
| 4.2.1    | Os 14 diagramas em duas famílias . . . . .                                     | 22        |
| 4.2.2    | Diagrama de classes . . . . .                                                  | 23        |
| 4.2.3    | Casos de uso . . . . .                                                         | 24        |
| 4.2.4    | Os quatro diagramas de interação . . . . .                                     | 24        |
| 4.2.5    | Atividade e máquina de estados . . . . .                                       | 25        |
| <b>5</b> | <b>Arquitetura de Software</b>                                                 | <b>26</b> |
| 5.1      | Servidor web x servidor de aplicação . . . . .                                 | 26        |
| 5.2      | Estilos de arquitetura: uma sequência histórica . . . . .                      | 26        |
| 5.2.1    | Camadas lógicas ( <i>layers</i> ) x camadas físicas ( <i>tiers</i> ) . . . . . | 28        |
| 5.3      | Integração: REST, SOAP e web services . . . . .                                | 28        |
| 5.3.1    | Mensageria: comunicação assíncrona . . . . .                                   | 30        |
| 5.4      | Ambientes de rede corporativa . . . . .                                        | 30        |
| 5.5      | Nuvem e escalabilidade . . . . .                                               | 31        |
| 5.5.1    | Containers e virtualização . . . . .                                           | 32        |
| 5.6      | Transações distribuídas . . . . .                                              | 32        |
| 5.7      | Tópicos complementares . . . . .                                               | 33        |
| <b>6</b> | <b>Banco de Dados</b>                                                          | <b>35</b> |
| 6.1      | Modelagem em três níveis . . . . .                                             | 35        |
| 6.1.1    | Metadados . . . . .                                                            | 36        |
| 6.1.2    | Avaliação de modelos de dados . . . . .                                        | 36        |
| 6.2      | Modelo relacional e integridade . . . . .                                      | 37        |
| 6.2.1    | Do MER ao relacional: como cada cardinalidade vira tabela . . . . .            | 38        |
| 6.2.2    | Entidade fraca x entidade associativa . . . . .                                | 39        |
| 6.3      | Normalização . . . . .                                                         | 39        |
| 6.3.1    | Decomposição passo a passo: um exemplo completo . . . . .                      | 40        |
| 6.4      | SQL: DDL x DML x DCL x TCL . . . . .                                           | 41        |
| 6.4.1    | VIEW, GRANT e controle de transação . . . . .                                  | 42        |
| 6.4.2    | Notação Crow's Foot e chave surrogada . . . . .                                | 42        |
| 6.4.3    | Gatilho x procedimento armazenado . . . . .                                    | 43        |
| 6.5      | Propriedades ACID . . . . .                                                    | 44        |
| 6.6      | Concorrência: níveis de isolamento . . . . .                                   | 44        |
| 6.7      | Relacional x multidimensional: OLTP x OLAP . . . . .                           | 45        |
| 6.8      | NoSQL e novos armazenamentos . . . . .                                         | 45        |
| 6.8.1    | Teorema CAP . . . . .                                                          | 46        |
| 6.8.2    | Data Lake x Data Warehouse x Lakehouse . . . . .                               | 46        |
| 6.9      | ETL x ELT . . . . .                                                            | 47        |
| 6.10     | Índices . . . . .                                                              | 47        |

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| 6.11     | Big Data: os "V"s . . . . .                                              | 48        |
| <b>7</b> | <b>Business Intelligence (BI)</b>                                        | <b>49</b> |
| 7.1      | O que é BI, e por que não basta consultar o banco transacional . . . . . | 49        |
| 7.2      | Data Warehouse . . . . .                                                 | 50        |
| 7.2.1    | Inmon x Kimball: por onde se começa a construir . . . . .                | 50        |
| 7.3      | ETL: a ponte para o DW . . . . .                                         | 51        |
| 7.4      | OLAP e modelagem dimensional . . . . .                                   | 51        |
| 7.4.1    | Star Schema x Snowflake Schema . . . . .                                 | 52        |
| 7.4.2    | Montando um esquema estrela: as quatro decisões, em ordem . . . . .      | 53        |
| 7.4.3    | Granularidade: o grão da fato . . . . .                                  | 53        |
| 7.4.4    | Tipos de fato . . . . .                                                  | 54        |
| 7.4.5    | Dimensões lentamente mutantes (SCD) . . . . .                            | 54        |
| 7.5      | Sistemas de suporte à decisão (SSD/DSS) . . . . .                        | 55        |
| 7.6      | Mapeamento de fontes de dados . . . . .                                  | 55        |
| 7.7      | Data Mining . . . . .                                                    | 55        |
| 7.7.1    | CRISP-DM: o ciclo de um projeto de mineração . . . . .                   | 56        |
| <b>8</b> | <b>Segurança da Informação</b>                                           | <b>57</b> |
| 8.1      | A tríade CIA: o critério que organiza tudo . . . . .                     | 57        |
| 8.2      | Criptografia . . . . .                                                   | 58        |
| 8.3      | HTTPS, SSL e TLS . . . . .                                               | 59        |
| 8.4      | Controle de acesso . . . . .                                             | 60        |
| 8.5      | OWASP Top 10 . . . . .                                                   | 60        |
| 8.6      | Desenvolvimento seguro . . . . .                                         | 61        |
| 8.7      | X.800: a arquitetura de segurança OSI . . . . .                          | 62        |
| 8.8      | Gestão de riscos . . . . .                                               | 62        |
| 8.9      | Continuidade de negócio: RTO x RPO . . . . .                             | 63        |
| 8.10     | Deteção e resposta a incidentes . . . . .                                | 64        |
| 8.11     | ISO/IEC 27001 e 27002 . . . . .                                          | 64        |
| <b>9</b> | <b>Programação</b>                                                       | <b>66</b> |
| 9.1      | Ecossistema Spring . . . . .                                             | 66        |
| 9.2      | Formatos de dados: XML, JSON, XSLT . . . . .                             | 67        |
| 9.3      | Mobile e low-code . . . . .                                              | 67        |
| 9.4      | Java EE / web <i>server-side</i> : JSF e Primefaces . . . . .            | 68        |
| 9.5      | Versionamento com Git . . . . .                                          | 69        |
| 9.6      | CI, entrega contínua e implantação contínua . . . . .                    | 70        |
| 9.7      | Contêineres . . . . .                                                    | 70        |
| 9.8      | Qualidade de código . . . . .                                            | 71        |
| 9.9      | Blockchain . . . . .                                                     | 71        |

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| 9.10 Estruturas de dados . . . . .                                         | 71        |
| 9.11 Leitura ativa de código . . . . .                                     | 73        |
| <b>10 Java</b>                                                             | <b>75</b> |
| 10.1 Exceções: <i>checked</i> x <i>unchecked</i> . . . . .                 | 75        |
| 10.2 Polimorfismo: <i>overload</i> x <i>override</i> . . . . .             | 75        |
| 10.3 Orientação a objetos e SOLID . . . . .                                | 76        |
| 10.3.1 Liskov em detalhe: a violação mais cobrada . . . . .                | 77        |
| 10.4 Interface x classe abstrata . . . . .                                 | 78        |
| 10.5 Coleções ( <i>Collections</i> ) . . . . .                             | 78        |
| 10.5.1 String: imutabilidade e <i>StringBuilder</i> . . . . .              | 79        |
| 10.6 JVM, JPA e Java EE . . . . .                                          | 80        |
| 10.6.1 Anotações que aparecem em leitura de código . . . . .               | 81        |
| 10.7 Recursos modernos (Java 17/21 LTS) . . . . .                          | 82        |
| 10.7.1 <i>var</i> , <i>record</i> e <i>switch</i> como expressão . . . . . | 82        |
| 10.8 Leitura ativa de código em Java . . . . .                             | 83        |
| 10.8.1 <i>Checked/unchecked</i> disfarçadas em <i>try-catch</i> . . . . .  | 83        |
| 10.8.2 Escopo de variável . . . . .                                        | 84        |
| 10.8.3 Armadilhas de threads virtuais: <i>pinning</i> . . . . .            | 84        |
| <b>11 Frontend</b>                                                         | <b>85</b> |
| 11.1 HTML e CSS . . . . .                                                  | 85        |
| 11.2 SPA x PWA . . . . .                                                   | 86        |
| 11.2.1 Onde o HTML é montado: CSR, SSR, SSG e <i>hydration</i> . . . . .   | 87        |
| 11.3 Frameworks . . . . .                                                  | 87        |
| 11.4 Ajax . . . . .                                                        | 88        |
| 11.5 UX, usabilidade e acessibilidade . . . . .                            | 88        |
| 11.6 Mobile: multiplataforma x nativo . . . . .                            | 89        |
| 11.7 Leitura ativa de código: CSS e React . . . . .                        | 89        |
| 11.7.1 Pegadinhas visuais em CSS . . . . .                                 | 90        |
| 11.7.2 Escopo e <i>closures</i> em JavaScript . . . . .                    | 90        |
| 11.7.3 Renderização no React . . . . .                                     | 91        |
| <b>12 Gestão e Governança de TI</b>                                        | <b>93</b> |
| 12.1 Gerenciamento de projetos (PMBOK) . . . . .                           | 93        |
| 12.1.1 Projeto, programa e portfólio . . . . .                             | 93        |
| 12.1.2 PMBOK 6ª edição: processos, grupos e áreas . . . . .                | 94        |
| 12.1.3 PMBOK 7ª edição: princípios e domínios de desempenho . . . . .      | 94        |
| 12.1.4 Tripla restrição e abordagens de execução . . . . .                 | 95        |
| 12.2 Scrum, Kanban e Lean . . . . .                                        | 96        |
| 12.2.1 Scrum: os time-boxes . . . . .                                      | 96        |

|           |                                                                    |            |
|-----------|--------------------------------------------------------------------|------------|
| 12.2.2    | Kanban: fluxo contínuo e limite de WIP . . . . .                   | 97         |
| 12.2.3    | Lean . . . . .                                                     | 99         |
| 12.3      | ITIL 4 . . . . .                                                   | 99         |
| 12.3.1    | Cocriação de valor: a mudança de eixo da v4 . . . . .              | 99         |
| 12.3.2    | Incidente, problema, requisição e mudança . . . . .                | 99         |
| 12.3.3    | Sistema de Valor de Serviço . . . . .                              | 100        |
| 12.4      | COBIT 2019 . . . . .                                               | 102        |
| 12.4.1    | Governança × gestão . . . . .                                      | 102        |
| 12.4.2    | 5 domínios, 40 objetivos . . . . .                                 | 103        |
| 12.4.3    | Os dois conjuntos de princípios . . . . .                          | 103        |
| 12.5      | BPMN (modelagem de processos) . . . . .                            | 104        |
| 12.6      | Gestão de riscos (ISO 31000) . . . . .                             | 106        |
| 12.7      | Roteiro de revisão do capítulo . . . . .                           | 106        |
| <b>13</b> | <b>Redes de Computadores</b>                                       | <b>107</b> |
| 13.1      | Modelos de referência: OSI e TCP/IP . . . . .                      | 107        |
| 13.1.1    | Encapsulamento: a ideia que sustenta o modelo em camadas . . . . . | 107        |
| 13.2      | Equipamentos de rede por camada . . . . .                          | 109        |
| 13.3      | TCP × UDP . . . . .                                                | 109        |
| 13.4      | Endereçamento e roteamento IP . . . . .                            | 110        |
| 13.4.1    | IPv4, IPv6 e faixas privadas . . . . .                             | 110        |
| 13.4.2    | Sub-redes: calculando hosts utilizáveis . . . . .                  | 111        |
| 13.4.3    | Roteamento e distância administrativa . . . . .                    | 112        |
| 13.4.4    | ACL Cisco e <i>wildcard mask</i> . . . . .                         | 112        |
| 13.4.5    | MPLS, VLAN e NAT . . . . .                                         | 113        |
| 13.5      | Tipos de rede corporativa . . . . .                                | 113        |
| 13.6      | Segurança de rede . . . . .                                        | 113        |
| 13.6.1    | Firewall: <i>stateless</i> × <i>stateful</i> . . . . .             | 114        |
| 13.6.2    | SSH substituindo o Telnet . . . . .                                | 114        |
| 13.6.3    | TLS: por que a criptografia é híbrida . . . . .                    | 115        |
| 13.7      | Roteiro de revisão do capítulo . . . . .                           | 115        |
| <b>14</b> | <b>Órfãos — Administração de BD e Temas Coringa</b>                | <b>116</b> |
| 14.1      | Administração de Dados × Administração de Banco de Dados . . . . . | 116        |
| 14.2      | Recuperação e transações: o protocolo ARIES . . . . .              | 117        |
| 14.3      | Armazenamento físico e desempenho . . . . .                        | 117        |
| 14.4      | Backup e recuperação . . . . .                                     | 118        |
| 14.5      | Segurança e acesso no SGBD . . . . .                               | 119        |
| 14.6      | Arquitetura de computadores e sistema operacional . . . . .        | 120        |
| 14.6.1    | Modelo de Von Neumann . . . . .                                    | 120        |
| 14.6.2    | CPU: Unidade de Controle e ULA . . . . .                           | 120        |

|           |                                                                      |            |
|-----------|----------------------------------------------------------------------|------------|
| 14.6.3    | Ciclo de instrução e <i>pipeline</i> . . . . .                       | 120        |
| 14.6.4    | Sistemas de numeração e complemento de dois . . . . .                | 121        |
| 14.6.5    | ROM, firmware e periféricos . . . . .                                | 121        |
| 14.6.6    | Memória virtual: paginação, MMU, TLB e <i>thrashing</i> . . . . .    | 122        |
| 14.6.7    | E/S bloqueante, troca de contexto e DMA . . . . .                    | 122        |
| 14.7      | Sistemas de informação corporativos . . . . .                        | 123        |
| 14.8      | Aprendizado de máquina . . . . .                                     | 123        |
| 14.9      | Mapa: onde estão os demais temas coringa . . . . .                   | 125        |
| 14.10     | Roteiro de revisão do capítulo . . . . .                             | 126        |
| <b>II</b> | <b>Módulo I — Conhecimentos Gerais (peso 1, 40 questões)</b>         | <b>127</b> |
| <b>15</b> | <b>Língua Portuguesa</b>                                             | <b>128</b> |
| 15.1      | Compreensão e interpretação de textos . . . . .                      | 128        |
| 15.1.1    | Três níveis de leitura . . . . .                                     | 128        |
| 15.1.2    | Tipos textuais x gêneros textuais . . . . .                          | 129        |
| 15.1.3    | Coerência x coesão . . . . .                                         | 130        |
| 15.2      | Ortografia . . . . .                                                 | 130        |
| 15.3      | Coesão textual: referenciação, conectores e tempos verbais . . . . . | 130        |
| 15.3.1    | Coesão referencial . . . . .                                         | 131        |
| 15.3.2    | Conectores e o valor semântico que carregam . . . . .                | 131        |
| 15.3.3    | Correlação/sequência de tempos verbais . . . . .                     | 132        |
| 15.4      | Classes de palavras . . . . .                                        | 132        |
| 15.5      | Sintaxe do período: coordenação e subordinação . . . . .             | 132        |
| 15.5.1    | Oração e período . . . . .                                           | 133        |
| 15.5.2    | Coordenação . . . . .                                                | 133        |
| 15.5.3    | Subordinação . . . . .                                               | 134        |
| 15.5.4    | Orações reduzidas . . . . .                                          | 136        |
| 15.5.5    | Roteiro de classificação . . . . .                                   | 136        |
| 15.6      | Pontuação . . . . .                                                  | 137        |
| 15.7      | Concordância verbal . . . . .                                        | 137        |
| 15.7.1    | Regra geral e sujeito composto . . . . .                             | 138        |
| 15.7.2    | Verbos impessoais . . . . .                                          | 138        |
| 15.7.3    | Sujeito coletivo e sujeito oracional . . . . .                       | 138        |
| 15.8      | Concordância nominal . . . . .                                       | 138        |
| 15.9      | Regência verbal e nominal . . . . .                                  | 139        |
| 15.10     | Crase . . . . .                                                      | 140        |
| 15.11     | Colocação pronominal . . . . .                                       | 141        |
| 15.12     | Reescrita e significação . . . . .                                   | 142        |

|                                                                          |            |
|--------------------------------------------------------------------------|------------|
| <b>16 Língua Inglesa</b>                                                 | <b>143</b> |
| 16.1 Como ler para prova: <i>skimming</i> e <i>scanning</i>              | 143        |
| 16.2 Referência pronominal                                               | 143        |
| 16.3 Conectivos e o valor lógico que carregam                            | 144        |
| 16.4 Verbos modais                                                       | 145        |
| 16.5 Vocabulário em contexto                                             | 146        |
| 16.6 Compreensão global: ideia principal, propósito e gênero             | 146        |
| 16.7 Questões de julgamento (afirmativas verdadeiro/falso)               | 147        |
| 16.8 Roteiro de revisão do capítulo                                      | 148        |
| <b>17 Raciocínio Lógico-Matemático</b>                                   | <b>149</b> |
| 17.1 Porcentagem e taxas sucessivas                                      | 149        |
| 17.2 Razão, proporção e regra de três                                    | 149        |
| 17.3 Análise combinatória                                                | 150        |
| 17.4 Probabilidade                                                       | 151        |
| 17.5 Médias, mediana, moda e desvio padrão                               | 151        |
| 17.6 Juros simples e compostos                                           | 152        |
| 17.7 Progressões aritmética e geométrica                                 | 152        |
| 17.8 Matrizes                                                            | 153        |
| 17.9 Conjuntos: inclusão-exclusão                                        | 153        |
| 17.10 Geometria plana essencial                                          | 153        |
| 17.11 Lógica sentencial: proposições e tabelas-verdade                   | 153        |
| 17.12 Equivalências lógicas                                              | 154        |
| 17.13 Argumentação: validade e falácias                                  | 155        |
| 17.14 Lógica de primeira ordem: quantificadores                          | 156        |
| 17.15 Roteiro de revisão do capítulo                                     | 156        |
| <b>18 Atualidades e Inteligência Artificial</b>                          | <b>157</b> |
| 18.1 Atualidades: o viés socioambiental                                  | 157        |
| 18.2 Fundamentos de Inteligência Artificial                              | 158        |
| 18.2.1 IA, Machine Learning e Deep Learning: uma relação de contêiner    | 158        |
| 18.2.2 Tipos de aprendizado                                              | 158        |
| 18.3 Viés e variância: o trade-off central do aprendizado supervisionado | 159        |
| 18.4 Métricas de avaliação                                               | 160        |
| 18.5 Modelos generativos e LLMs                                          | 160        |
| 18.6 Ética, governança e privacidade em IA                               | 161        |
| 18.7 Regulação da IA: em vigor x ainda em tramitação                     | 162        |
| 18.8 Reconhecendo o padrão em cenários de IA aplicada                    | 163        |
| 18.9 Interseção IA × ESG: energia e sustentabilidade                     | 163        |
| 18.10 Roteiro de revisão do capítulo                                     | 164        |

|                                                                                        |            |
|----------------------------------------------------------------------------------------|------------|
| <b>19 Legislação — Segurança da Informação e Proteção de Dados</b>                     | <b>165</b> |
| 19.1 LGPD (Lei 13.709/2018) . . . . .                                                  | 165        |
| 19.1.1 Dado pessoal e dado pessoal sensível . . . . .                                  | 165        |
| 19.1.2 Agentes de tratamento: o papel vem do poder de decisão . . . . .                | 165        |
| 19.1.3 Bases legais: duas listas diferentes, arts. 7º e 11 . . . . .                   | 166        |
| 19.1.4 Princípios (art. 6º): finalidade, adequação e necessidade . . . . .             | 167        |
| 19.1.5 Decisão automatizada e explicabilidade (art. 20) . . . . .                      | 167        |
| 19.1.6 Segurança e boas práticas (Capítulo VII): a lei que vira requisito de sistema . | 168        |
| 19.1.7 Sanções administrativas (art. 52): um regime, não uma tabela de multa . . .     | 168        |
| 19.1.8 ANPD e CNPD: dois órgãos com funções opostas . . . . .                          | 169        |
| 19.2 Marco Civil da Internet (Lei 12.965/2014) . . . . .                               | 169        |
| 19.2.1 Guarda de registros . . . . .                                                   | 169        |
| 19.2.2 Neutralidade de rede . . . . .                                                  | 170        |
| 19.2.3 Sanções do art. 12 . . . . .                                                    | 170        |
| 19.2.4 Art. 19: responsabilidade dos provedores por conteúdo de terceiros . . . . .    | 170        |
| 19.3 LAI — Lei de Acesso à Informação (Lei 12.527/2011) . . . . .                      | 171        |
| 19.4 Delitos informáticos: invasão de dispositivo (art. 154-A do CP) . . . . .         | 172        |
| 19.5 Roteiro de revisão do capítulo . . . . .                                          | 173        |

## Parte I

### Módulo II — Conhecimentos Específicos (peso 2,5, 30 questões)

## Capítulo 3

# Engenharia de Software

### Ficha do edital

Engenharia de requisitos (classificação, processo, técnicas de elicitação); testes (unitários, integração, ágeis, usabilidade, automatizados, TDD, ciclo de vida, RPA); metodologias ágeis (Scrum, Kanban, XP); padrões de desenvolvimento e reuso; codificação; Ponto de Função e Story Points; DevOps; design de software.

Engenharia de software é a disciplina de organizar **como** um sistema é construído, não o que ele faz por dentro (isso é o Capítulo 9) nem como seus componentes se distribuem (isso é o Capítulo 5). Este capítulo segue uma ordem lógica: primeiro os **modelos de processo** (o "formato" do projeto do início ao fim), depois **maturidade organizacional**, depois os **blocos que preenchem qualquer processo** — requisitos, testes, metodologias ágeis, medição, entrega e design.

### 3.1 Modelos de ciclo de vida

#### O que um modelo de ciclo de vida decide

Todo projeto de software passa pelas mesmas atividades essenciais: levantar requisitos, projetar, construir, testar, implantar. O que muda de um **modelo de ciclo de vida** para outro não é o **que se faz**, é a **ordem e a repetição**: as atividades acontecem uma vez, em sequência rígida? Repetem em ciclos? Entregam aos pedaços?

- **Cascata (*waterfall*)**: fases **sequenciais e estanques** — uma termina, a próxima começa, sem voltar atrás. Requisitos **congelados** logo no início. Funciona quando o problema é **bem compreendido e estável** (ex.: sistema regulatório com especificação fechada por lei). Falha quando o requisito muda no meio do caminho, porque o modelo não tem mecanismo para incorporar mudança sem recomeçar.
- **Incremental**: o sistema é entregue em **pedaços funcionais que se somam** ao longo do tempo — cada incremento já é utilizável. Reduz o risco de "só ver o produto pronto no fim".
- **Iterativo**: o mesmo produto é **refinado** em ciclos sucessivos (a primeira versão é rascunho grosseiro, a segunda melhora, e assim por diante) — o foco é refinar, não necessariamente entregar pedaço novo a cada ciclo (a diferença com incremental é sutil e por isso a maioria dos processos modernos combina os dois: "iterativo-incremental").
- **Espiral (Boehm)**: iterativo, mas com um passo extra em cada volta da espiral — **análise de risco explícita** antes de seguir adiante. Nasceu para projetos grandes e arriscados, onde o custo de errar é alto o bastante para justificar parar e avaliar risco a cada ciclo.
- **Prototipação**: constrói uma versão **parcial e rápida** (às vezes descartável) só para **validar requisito com o cliente** antes de investir no sistema real — serve quando o requisito é **incerto ou volátil**, o oposto do cenário ideal para cascata.
- **Ágil**: iterativo-incremental com ciclos **curtos e adaptação contínua** — assume que requisito muda e projeta o processo em torno disso, em vez de tentar blindar o processo

contra mudança.

### Reconhecendo o modelo pelo enunciado

"O projeto define todo o escopo antes de iniciar a codificação; cada fase só começa quando a anterior está formalmente aprovada" → **cascata** (sequencial, sem sobreposição, aprovação formal entre fases). "A equipe constrói uma versão simplificada da tela principal só para o cliente confirmar o fluxo antes de programar o sistema de verdade" → **prototipação** (o objetivo é validar requisito, não entregar produto).

### Incremental e iterativo não são sinônimos, embora andem juntos

**Incremental** responde "o que é entregue" (pedaços que se somam, cada um com valor de uso). **Iterativo** responde "como se chega lá" (ciclos que refinam o que já existe). É possível ser incremental sem ser muito iterativo (entregar módulo A, depois B, depois C, sem voltar a refinar A) e vice-versa. Scrum, na prática, é os dois ao mesmo tempo: cada Sprint refina o produto (iterativo) e normalmente adiciona um incremento utilizável (incremental) — por isso os processos ágeis modernos costumam ser descritos com o termo composto.

#### 3.1.1 Modelo V: cada fase de construção tem seu par de teste

##### A lógica "quem definiu, confere"

O Modelo V é uma variação do cascata desenhada em V: o **ramo descendente** (esquerda) vai do requisito ao código, detalhando cada vez mais; o **ramo ascendente** (direita) sobe testando, e cada nível de teste **verifica exatamente o que foi decidido no nível espelhado do outro lado**.

| Fase (ramo descendente)                     | Nível de teste que a verifica |
|---------------------------------------------|-------------------------------|
| Requisitos do usuário                       | <b>Teste de aceitação</b>     |
| Especificação do sistema                    | Teste de sistema              |
| Projeto arquitetural (módulos e interfaces) | <b>Teste de integração</b>    |
| Projeto detalhado (algoritmos)              | Teste de unidade              |
| Codificação                                 | (vértice do V)                |

Por que essa lógica faz sentido: quem **acordou** o requisito com o usuário é quem consegue dizer, no fim, se ele foi atendido — daí requisito ↔ aceitação. Quem **decidiu** a divisão em módulos e suas interfaces (projeto arquitetural) é quem consegue verificar se os módulos, integrados, funcionam como planejado — daí arquitetura ↔ integração. O par nunca cruza: requisito não casa com teste de unidade (unidade verifica o **projeto detalhado**, não a visão do usuário), e código não casa com aceitação.

### 3.1.2 Verificação x validação

#### Duas perguntas diferentes sobre o mesmo software

O Modelo V chama de "verificação" todo o ramo ascendente, mas os dois termos não são sinônimos — e a confusão entre eles é o distrator mais recorrente de todo o bloco, porque as duas palavras soam parecidas em português e a resposta muda inteira conforme qual delas o enunciado descreve.

|                              | Verificação                                                                             | Validação                                                                         |
|------------------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| A pergunta                   | "estamos construindo o produto <b>corretamente</b> ?"                                   | "estamos construindo o <b>produto certo</b> ?"                                    |
| Compara com                  | a <b>especificação</b> (documento da fase anterior)                                     | a <b>necessidade real</b> do usuário                                              |
| Quando                       | ao longo de todo o desenvolvimento, fase a fase                                         | principalmente perto da entrega, com o usuário                                    |
| Como se faz                  | revisão, inspeção, <i>walkthrough</i> , análise estática, teste de unidade e integração | teste de aceitação, teste alfa/beta, homologação, protótipo avaliado pelo cliente |
| Precisa executar o software? | <b>não necessariamente</b> — revisar documento já é verificar                           | na prática, <b>sim</b>                                                            |

**A âncora de uma palavra:** verificação olha para o *papel* (a especificação); validação olha para a *pessoa* (o usuário).

A consequência prática que resolve qualquer cenário: **um sistema pode passar na verificação e falhar na validação**. Ele implementa exatamente o que o documento pedia — só que o documento pedia a coisa errada. É o sistema que atende a cada linha do requisito e não serve para o trabalho de ninguém: verificação impecável, validação reprovada. O caminho inverso também existe, mais raro: o produto resolve a necessidade real apesar de divergir do que a especificação escrita dizia.

#### Os dois erros de troca mais comuns

Definir **validação** como "conformidade com a especificação" (isso é verificação); ou definir **verificação** como "atendimento às necessidades do usuário" (isso é validação). Um terceiro erro: tratar as duas como sinônimos de "teste" — teste é uma **técnica**, usada pelas duas (existe teste de unidade, que verifica, e teste de aceitação, que valida). No Modelo V, a ponta de baixo (unidade/integração/sistema) é verificação; a ponta de cima (aceitação) é validação.

### 3.1.3 RUP: fase x disciplina

#### Os dois eixos independentes do RUP

O RUP (*Rational Unified Process*) é iterativo-incremental e dirigido por casos de uso. A estrutura que sustenta qualquer questão sobre ele tem dois eixos que **não se misturam**:

**Fase** é o eixo do **tempo** — o projeto passa por Concepção, Elaboração, Construção e Transição, **nessa ordem, uma única vez**, cada fase terminando num marco de decisão. **Disciplina**

(*workflow*) é o eixo do **conteúdo** — Requisitos, Análise e Projeto, Implementação, Teste, Implantação, mais as de apoio (Gerência de Projeto, Configuração e Mudança, Ambiente). O ponto central: **as disciplinas atravessam todas as fases**. Não existe "a fase de testes—testa-se em todas as quatro fases, só que em intensidade diferente a cada uma (o gráfico clássico do RUP desenha as fases como colunas e as disciplinas como linhas horizontais; cada linha tem uma "corcova" de maior intensidade em certa coluna, mas nenhuma linha começa e termina dentro de uma coluna só).

| Fase       | Pergunta que ela responde                                                                             | Marco                          |
|------------|-------------------------------------------------------------------------------------------------------|--------------------------------|
| Concepção  | vale a pena fazer? escopo, viabilidade, caso de negócio                                               | Objetivos do ciclo de vida     |
| Elaboração | <b>como fazer sem que dê errado?</b> — ataca os <b>riscos</b> e firma a <b>arquitetura executável</b> | Arquitetura do ciclo de vida   |
| Construção | constrói o grosso do produto, iteração a iteração                                                     | Capacidade operacional inicial |
| Transição  | entrega ao usuário: implantação, treinamento, ajuste                                                  | Liberação do produto           |

**Por que risco e arquitetura vivem na Elaboração:** a lógica é econômica — resolver o risco arquitetural **antes** de gastar o orçamento da Construção. Descobrir um problema de arquitetura já na Construção é caro demais para corrigir.

#### Erros de leitura comuns no RUP

Tratar disciplina como fase ("a fase de testes do RUP" não existe: teste é disciplina, presente nas quatro fases). Deslocar o risco para a Concepção (lá se avalia viabilidade, não se ataca risco técnico) ou para a Construção (lá já é tarde e caro). Chamar o RUP de "cascata": ele é iterativo **dentro** de cada fase; o que é sequencial é só a **ordem** das quatro fases, não o trabalho interno de cada uma.

## 3.2 Maturidade de processo: CMMI e MPS.BR

#### Que problema a maturidade de processo resolve

Modelos de maturidade avaliam **quão maduro é o processo** de uma organização — não a qualidade de um produto específico. O CMMI nasceu de uma pergunta prática do Departamento de Defesa americano nos anos 1980: *como saber, antes de contratar, se um fornecedor vai entregar?* Olhar o produto anterior não bastava — um projeto que deu certo pode ter dado certo por acaso, ou porque alguém virou noites. O que se queria medir era a **capacidade de repetir** o resultado.

Daí a definição central: **maturidade é o quanto o resultado depende do processo, e não das pessoas**. Uma organização madura entrega parecido com equipes diferentes, porque o jeito de trabalhar está descrito, é seguido e é medido. Uma organização imatura entrega bem quando calha de ter a pessoa certa no projeto certo.

## CMMI — os cinco níveis, por estágios

| Nº | Nível                        | O que caracteriza                                                            |
|----|------------------------------|------------------------------------------------------------------------------|
| 1  | Inicial                      | processo imprevisível, reativo, dependente de heróis                         |
| 2  | Gerenciado                   | gerenciado <b>por projeto</b> (cada projeto planeja e monitora do seu jeito) |
| 3  | <b>Definido</b>              | processo <b>padronizado no âmbito da organização</b> , não projeto a projeto |
| 4  | Gerenciado Quantitativamente | processo <b>medido e controlado por estatística</b>                          |
| 5  | Em Otimização                | melhoria contínua a partir da medição                                        |

O corte mais importante para entender (e o mais cobrado) é do nível 2 para o 3: no **nível 2**, o projeto A tem cronograma e controle de mudanças, o projeto B ao lado faz diferente — e os dois podem estar bem gerenciados, porque a disciplina existe **por projeto**. No **nível 3**, existe um **processo-padrão da organização**, e cada projeto o adapta (*tailoring*) dentro de regras — um desenvolvedor que muda de projeto reconhece o jeito de trabalhar, e o aprendizado de um projeto vira ativo de todos.

Do 3 para cima, a evolução é sobre **medir**: no **nível 4**, a organização não diz mais "costuma dar certo", diz "a densidade de defeito fica entre tais limites— previsibilidade quantitativa. No **nível 5**, essa medição é usada para **mudar o processo de propósito** e verificar se a mudança realmente melhorou o resultado — melhoria contínua baseada em dado, não em opinião.

**Dois representações, duas perguntas diferentes:** a representação **por estágios** (a tabela acima) responde "que nota tem esta **organização**?— um número só, comparável entre fornecedores, e foi para isso que o modelo nasceu. A representação **contínua** responde "quão **capaz** somos **nesta área específica** (por exemplo, só Engenharia de Requisitos)?— permite investir numa área sem ter de elevar tudo junto. Por isso o vocabulário muda: estágios dá **maturidade** à organização inteira; contínua dá **capacidade** a cada área de processo isolada.

## MPS.BR — a versão brasileira, de G a A

Mesma ideia do CMMI, escala própria (MR-MPS-SW) com **sete níveis identificados por letras**, evoluindo **de G para A**: G (Parcialmente Gerenciado), F (Gerenciado), E (Parcialmente Definido), D (Largamente Definido), C (Definido), B (Gerenciado Quantitativamente), A (Em Otimização). O G é o ponto de partida (organização menos madura); o A é o topo.

## Trocadas clássicas nos dois modelos

Chamar o nível 3 do CMMI de "Gerenciado"(ele é o **Definido** — o Gerenciado é o 2). Inverter a ordem do topo, pondo Gerenciado Quantitativamente depois de Em Otimização (a ordem certa é Definido → Gerenciado Quantitativamente → Em Otimização). Achar que o MPS.BR começa no A — é o contrário, começa no G. E lembrar que a escala tem **cinco** níveis no CMMI e **sete** no MPS.BR — não são a mesma contagem.

## 3.3 Engenharia de requisitos

### 3.3.1 Requisito funcional x não funcional

#### O que o sistema faz x quão bem ele faz

| Tipo                       | O que descreve                                                                  | Exemplo                                                                    |
|----------------------------|---------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| <b>Funcional (RF)</b>      | O QUE o sistema faz — uma função, um comportamento observável                   | "emitir saldo da conta"                                                    |
| <b>Não funcional (RNF)</b> | COMO / QUÃO BEM — uma qualidade ou restrição sobre como as funções se comportam | "saldo em tempo real", desempenho, segurança, usabilidade, disponibilidade |

O mecanismo por trás da distinção: um RF sobrevive sozinho como frase com verbo de ação ("o sistema deve emitir", "o usuário deve poder cadastrar"). Um RNF **qualifica** um RF ou o sistema como um todo — ele não tem sentido sem alguma função para qualificar ("rápido— rápido fazendo o quê?"). Por isso RNF costuma vir como adjetivo/advérbio de qualidade ("seguro", "disponível 99,9% do tempo", "em tempo real"), enquanto RF vem como verbo de ação.

#### Separando RF de RNF num enunciado

"O sistema deve permitir que o usuário emita boletos (1). O sistema deve processar até 1000 requisições por segundo (2). O boleto deve ser emitido em até 2 segundos após a solicitação (3). O sistema deve estar em conformidade com a LGPD (4).— (1) é RF (ação: emitir boletos); (2), (3) e (4) são RNF (desempenho, desempenho/tempo de resposta, e conformidade legal, respectivamente — este último é um RNF do tipo "externo/legal").

#### Regra prática

Ao ler um enunciado longo, marque cada **verbo de ação** do sistema (RF) e cada palavra de **qualidade/restrrição** ("deve ser rápido/seguro/disponível/acessível/auditável") (RNF). RNF se subdivide em tipos (desempenho, usabilidade, segurança, produto, organizacional, externo/legal) — quando a questão pede o **tipo** de RNF, não só "é RNF", associe a palavra-chave ao tipo correspondente.

### 3.3.2 Processo de engenharia de requisitos

#### As cinco etapas, em ordem

**Elicitação** (levantamento) → **análise** (entender, priorizar, resolver conflitos) → **especificação** (documentar formalmente) → **validação** (confirmar com o interessado que é isso mesmo) → **gerenciamento** (rastrear mudanças ao longo do projeto — os requisitos evoluem, e cada mudança precisa de rastreabilidade até o que ela impacta).

Técnicas de **elicitação** (todas legítimas, escolhidas conforme o contexto): entrevista, questionário, **brainstorming**, workshop, observação (o analista acompanha o usuário no trabalho real, útil quando o usuário não sabe verbalizar o que faz), prototipação, análise de documentos existentes (manuais, sistemas legados).

**Cuidado — confusão comum**

Brainstorming não é técnica "informal demais para prova de requisito" — é técnica de elicitação plenamente válida e cobrada como tal. E a ordem do processo é sempre elicitar **antes** de codificar, nunca o inverso — requisito que "surge depois da implementação" é sinal de processo mal seguido, não de uma etapa válida do processo.

## 3.4 Metodologias ágeis

### 3.4.1 Scrum: framework, não metodologia

#### Por que "framework" e não "metodologia"

Scrum define **papéis, eventos e artefatos** — uma estrutura mínima — mas não dita **como** o time deve executar cada atividade dentro dela (isso é "metodologia": um passo a passo prescritivo). O Scrum Guide é propositalmente enxuto: ele diz "existe um Sprint Planning", não "faça o Sprint Planning exatamente assim". Por isso é chamado de framework: a moldura é fixa, o conteúdo dentro dela é decidido pelo time.

O trabalho é organizado em **Sprints**: ciclos de tamanho fixo (*time-boxed*), normalmente de 1 a 4 semanas, sempre com a mesma duração dentro de um mesmo projeto.

**Papéis (*accountabilities*, termo do Scrum Guide 2020):**

- **Product Owner:** dono do **valor** do produto — decide o que entra no Product Backlog e em que ordem, representando o interesse do negócio/cliente.
- **Scrum Master:** **remove impedimentos** e serve ao time — garante que o Scrum seja entendido e praticado, mas **não manda** no time nem distribui tarefas.
- **Developers:** quem constrói o incremento a cada Sprint.

**Eventos:** a Sprint em si, mais quatro eventos dentro dela — **Sprint Planning** (o que será feito e como, no início), **Daily Scrum** (sincronização diária de 15 minutos), **Sprint Review** (inspeciona o **incremento** com os interessados, no fim), **Sprint Retrospective** (inspeciona o **processo** do time, também no fim, após a Review).

**Artefatos e seus compromissos (Scrum Guide 2020):** cada artefato tem um compromisso associado, que existe para dar foco e permitir medir progresso — três pares fixos que não se misturam:

| Artefato        | Compromisso                           | O que ele fixa                    |
|-----------------|---------------------------------------|-----------------------------------|
| Product Backlog | <b>Meta do Produto</b> (Product Goal) | o objetivo de longo prazo         |
| Sprint Backlog  | <b>Meta da Sprint</b> (Sprint Goal)   | o objetivo único da Sprint        |
| Increment       | <b>Definition of Done</b>             | o padrão de qualidade do "pronto" |

#### Review x Retrospective num mesmo dia

No último dia de uma Sprint de duas semanas, o time faz duas reuniões seguidas. Na primeira, mostra a nova funcionalidade de exportar relatórios para o cliente e outros interessados, que dão feedback sobre o que priorizar a seguir — isso é **Sprint Review** (foco no **produto**). Na segunda, só o time se reúne e discute que o retrabalho de testes atrasou a Sprint, e decide

mudar a forma de escrever casos de teste na próxima — isso é **Sprint Retrospective** (foco no processo).

#### Ligações que a confusão costuma trocar

**Sprint Review = produto; Sprint Retrospective = processo** — inverter as duas é o erro mais comum. Nos compromissos, a confusão mais comum é ligar Definition of Done ao Product Backlog ou ao Sprint Backlog (ela é sempre do **Incremento**) e a Meta do Produto ao Incremento (ela é sempre do **Product Backlog**). Guie-se pelo horizonte de tempo: produto (longo prazo) → Product Backlog; Sprint (curto prazo, uma iteração) → Sprint Backlog; "está pronto ou não" → Incremento. E no Daily, quando surge risco à Sprint, o Scrum Master **facilita a solução do time** — não redistribui tarefas sozinho, nem assume a tarefa do desenvolvedor: quem decide como resolver é o próprio time.

### 3.4.2 Kanban, XP, Lean

| Framework                         | Ideia central                                                                                                                                                                                                     |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Kanban                            | Fluxo <b>contínuo</b> , sem ciclos fechados (sem Sprint); limita o <b>WIP</b> ( <i>work in progress</i> , trabalho simultâneo em curso) para evitar sobrecarga; entrega assim que cada item termina, não em lote. |
| XP ( <i>Extreme Programming</i> ) | Conjunto de <b>práticas de engenharia</b> : programação em par ( <i>pair programming</i> ), TDD, integração contínua, refatoração constante, cliente presente no time.                                            |
| Lean                              | Elimina desperdício e otimiza o fluxo de valor de ponta a ponta — originado na manufatura (Toyota), adaptado ao software.                                                                                         |
| Ágil híbrido                      | Combina práticas ágeis com elementos de métodos tradicionais (por exemplo, um cronograma macro fixo por contrato, com execução interna em Sprints).                                                               |

#### Cuidado — confusão comum

Kanban **não** tem Sprint — "Kanban organiza o trabalho em sprints" é sempre falso. Scrum **tem** *time-box* fixo; Kanban é fluxo contínuo, sem prazo fechado por ciclo (o controle é pelo limite de WIP, não pelo calendário).

## 3.5 Testes de software

### 3.5.1 Níveis e tipos

| Nível/tipo              | O que verifica                                                                                                           |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Unitário                | a menor unidade isolada (função/método), sem depender de outras partes do sistema                                        |
| Integração              | a interação entre módulos/componentes já unidos                                                                          |
| Sistema                 | o sistema completo, de ponta a ponta                                                                                     |
| Aceitação               | se o sistema atende ao que o cliente/usuário precisava (é o teste que <b>valida</b> , na terminologia da Seção anterior) |
| Usabilidade             | se a interface é intuitiva e a experiência de uso é boa                                                                  |
| Regressão               | se uma mudança recente não quebrou algo que já funcionava                                                                |
| Fumaça ( <i>smoke</i> ) | verificação rápida e superficial das funções principais, para decidir se vale a pena testar o resto                      |

### 3.5.2 Testes de desempenho: carga, estresse, volume

#### Os três se confundem porque todos "sobrecarregam" o sistema

O que muda entre eles é **até onde** se vai e **o que** está sendo sobrecarregado:

- **Carga (*load*)**: roda o volume de uso **esperado em produção** e observa se o sistema aguenta o que foi previsto/contratado.
- **Estresse (*stress*)**: vai **além** do previsto, aumentando progressivamente até achar o **ponto de ruptura** — e observa como o sistema se comporta ao quebrar (degrada com elegância? cai tudo de uma vez? se recupera sozinho depois?).
- **Volume**: sobrecarrega com muita **massa de dados armazenada** (um banco com 500 milhões de registros), não necessariamente muitos usuários simultâneos.

#### Cuidado — confusão comum

"Ultrapassar o previsto até o sistema deixar de responder" é **estresse**; "comportar-se bem no volume contratado/esperado" é **carga**. O gatilho textual é a palavra **além/ultrapassa** (estresse) versus **esperado/previsto** (carga). Volume é massa de **dados**, não de usuários simultâneos — não confunda com carga só porque os dois envolvem "muito".

### 3.5.3 Técnicas de teste caixa-preta: partição de equivalência e valor limite

#### Duas técnicas para escolher casos de teste sem testar tudo

Testar **toda** entrada possível é inviável (um campo numérico de 0 a 1000 tem 1001 valores possíveis). As duas técnicas clássicas de caixa-preta resolvem isso escolhendo **poucos** casos que representam bem o espaço de entrada:

- **Partição de equivalência**: divide as entradas possíveis em **classes** que, supõe-se, o sistema trata do mesmo jeito — testando **um** valor de cada classe, testa-se a classe inteira. Para um campo que aceita idade de 18 a 65, há três classes: "menor que 18" (inválida), "18 a 65" (válida), "maior que 65" (inválida) — basta um valor de cada.
- **Análise de valor limite**: em vez de um valor qualquer da classe, testa **exatamente as**

**fronteiras** entre classes (17, 18, 65, 66) — porque é ali que a maioria dos defeitos reais de código acontece (erro de ">" no lugar de "≥", por exemplo).

As duas se complementam: partição de equivalência decide **onde** procurar; valor limite decide o **ponto exato** mais eficiente dentro de cada fronteira.

### 3.5.4 Cobertura de caixa-branca

#### Critérios que medem quanto do código os testes exercitam

- **Cobertura de comandos** (*statement*): cada linha executada ao menos uma vez.
- **Cobertura de decisões** (*branch*): cada decisão avaliada **como verdadeira e como falsa** pelo menos uma vez.
- **Cobertura de caminhos**: cada combinação possível de caminhos pelo código — o critério mais forte, e em geral inviável de atingir por completo (o número de caminhos cresce exponencialmente com o número de decisões).

Força crescente: comandos < decisões < caminhos. **100% de cobertura de decisões implica 100% de cobertura de comandos; a recíproca é falsa** — cobrir toda linha não garante ter testado toda combinação de verdadeiro/falso de cada condição.

#### Por que 100% de comandos não é 100% de decisões

Código: `if (a > 10) { x = 1; }` — um `if` sem `else`. Um único caso de teste com `a = 20` faz **todas as linhas** executarem: cobertura de comandos em 100%. Mas a condição `a > 10` nunca foi avaliada como **falsa** nesse teste — cobertura de decisões fica em 50%, mesmo sem existir nenhum bloco `else` escrito no código. Um segundo caso com `a = 5` completaria a cobertura de decisões.

### 3.5.5 TDD, BDD, caixa-preta x caixa-branca, RPA

**TDD (*Test-Driven Development*)**: escreve o **teste antes** do código de produção, no ciclo **red** (teste falha, porque o código ainda não existe) → **green** (escreve o código mínimo que faz o teste passar) → **refactor** (melhora o código mantendo os testes passando). O foco é tanto garantir cobertura quanto **guiar o design** — escrever o teste primeiro obriga a pensar na interface antes da implementação.

**BDD (*Behavior-Driven Development*)**: evolução do TDD que especifica o comportamento esperado em linguagem **acessível ao negócio**, não só a quem programa — tipicamente em Gherkin (*Dado/Quando/Então*), permitindo que stakeholders não técnicos leiam e validem os cenários de teste.

**Caixa-preta** (testa entrada/saída, sem olhar o código por dentro) x **caixa-branca** (baseado na estrutura interna/no código-fonte) — partição de equivalência e valor limite são caixa-preta; cobertura de comandos/decisões/caminhos é caixa-branca.

**RPA (*Robotic Process Automation*)**: automatiza tarefas repetitivas de **interface** (robôs de software que clicam, digitam, copiam dados entre sistemas) — **não é técnica de teste**, é automação de processo de negócio; aparece no edital de eng. de software porque frequentemente confundido com automação de teste.

## 3.6 Manutenção de software

### Classificação pela causa da alteração, não pelo momento

A ISO/IEC 14764 classifica o que se faz com o software **depois de entregue**. O critério é sempre **a causa** da mudança:

| Tipo       | Causa da alteração                                                                                                                        |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Corretiva  | <b>corrigir defeito</b> já detectado (erro relatado, falha em produção)                                                                   |
| Adaptativa | acompanhar <b>mudança do ambiente</b> externo ao software: sistema operacional, SGBD, plataforma, hardware, legislação                    |
| Perfectiva | <b>melhorar</b> o que já funciona: desempenho, manutenibilidade, requisito novo pedido pelo usuário (sem que nada estivesse "quebrado")   |
| Preventiva | corrigir uma <b>falha latente</b> antes que ela se manifeste — reduzir risco futuro identificado numa análise, não um defeito já relatado |

### Classificando um caso ambíguo

"O time atualizou o driver de conexão com o banco de dados porque o fabricante descontinuou a versão anterior, sem que houvesse defeito relatado nem funcionalidade nova.— é **adaptativa**: a causa é o **ambiente** (o fabricante mudou algo fora do controle do sistema). Não é corretiva (não havia defeito a corrigir), não é perfectiva (não se melhorou nada do ponto de vista do usuário) e não é preventiva (não havia uma falha latente identificada e sim uma obsolescência externa).

### Cuidado — confusão comum

"Evolutiva" não é um dos quatro tipos da classificação — é um rótulo informal que às vezes aparece em prova como se fosse categoria da norma, mas a ISO/IEC 14764 reconhece só as quatro da tabela acima.

## 3.7 Mensuração: Ponto de Função e Story Points

### 3.7.1 Ponto de Função (APF): medir de fora, independente da linguagem

#### O problema de contrato que a APF resolve

A Análise de Pontos de Função resolve uma pergunta prática: *como pagar por software sem pagar por linha de código?* Medir por linhas de código premia quem escreve mal e pune quem usa uma linguagem expressiva — a mesma funcionalidade pode sair em 500 linhas de uma linguagem e 80 de outra, e não faz sentido a segunda valer menos.

A saída foi medir do **lado de fora**: conta-se o que o sistema **faz para o usuário** (o que ele entrega em termos de negócio), não como foi construído por dentro. Por isso a contagem parte de **telas, relatórios e arquivos lógicos** — artefatos visíveis ao usuário — e o resultado é o

**mesmo** qualquer que seja a linguagem, a equipe ou a ferramenta usada para construir. É essa independência de tecnologia que torna a APF apta a comparar fornecedores diferentes num mesmo contrato.

Contam-se cinco tipos de função, em dois grupos:

|                             |                                                                                                                                                                                        |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Funções de transação</b> | de o que <b>atravessa</b> a fronteira do sistema: <b>EE</b> (Entrada Externa), <b>SE</b> (Saída Externa), <b>CE</b> (Consulta Externa)                                                 |
| <b>Funções de dados</b>     | de o que o sistema <b>guarda ou lê</b> : <b>ALI</b> (Arquivo Lógico Interno, mantido dentro do sistema), <b>AIE</b> (Arquivo de Interface Externa, só lido, mantido por outro sistema) |

Os dois critérios que resolvem quase toda classificação:

- **ALI x AIE se decide pela manutenção**, não pela leitura. Se o sistema em análise **mantém** o dado (cria, atualiza, apaga), é ALI; se ele só **consulta** um dado que outro sistema mantém, é AIE — mesmo que a leitura seja constante.
- **SE x CE se decide pelo dado derivado**. Se a saída envolve **cálculo, totalização ou lógica** que gera informação nova a partir dos dados brutos, é Saída Externa; se é recuperação pura, sem processamento que agregue valor, é Consulta Externa.

### Classificando telas

Uma tela de cadastro de cliente, que grava o registro no banco → **EE** (Entrada Externa: dado entra e é mantido). Um relatório que soma o total de vendas do mês e mostra o percentual de crescimento em relação ao mês anterior → **SE** (Saída Externa: há cálculo/ totalização). Uma tela que só busca e exibe o cadastro de um cliente pelo CPF, sem processar nada → **CE** (Consulta Externa: recuperação pura). A tabela de clientes, mantida por este mesmo sistema → **ALI**. Uma tabela de CEPs, consultada mas mantida por um sistema externo dos Correios → **AIE**.

### 3.7.2 Ponto de Função x Story Points

|                        |    | Ponto de Função (APF)                                              | Story Points                                                    |
|------------------------|----|--------------------------------------------------------------------|-----------------------------------------------------------------|
| Natureza               |    | <b>objetiva</b> , padronizada por norma (IFPUG)                    | <b>relativa/subjetiva</b> , calibrada pelo time                 |
| Independência do time? | do | sim — dois analistas certificados chegam ao mesmo número           | não — o mesmo item vale pontos diferentes para times diferentes |
| Bom para               |    | contrato de escopo fechado, comparação entre projetos/fornecedores | estimativa ágil interna, planejamento de Sprint                 |
| Baseado em             |    | funções do sistema (EE, SE, CE, ALI, AIE)                          | esforço/complexidade/incerteza percebidos pelo time             |

A confusão mais comum inverte os dois: dizer que Story Point é objetivo e comparável entre times, ou que serve a contrato de escopo fechado — é exatamente o oposto. Ponto de Função é o **padronizado e comparável**; Story Points é o **relativo ao time que o usa**, e por isso não

deveria ser usado para comparar a "produtividade" de dois times diferentes.

### 3.8 DevOps e estratégias de entrega

#### CI, CD e DevOps

**CI (Integração Contínua)**: integra e testa o código com frequência (idealmente a cada *commit*), pegando conflitos e regressões cedo. **CD**: tem dois sentidos que convivem — **Entrega Contínua** (o software fica sempre em estado **pronto para ir** a produção, mesmo que o "ir" seja manual) e **Implantação Contínua** (o "ir" para produção também é automático). Na prática de prova, "fornecer rapidamente uma nova versão em produção" é sempre **CD**, nunca **CI** — **CI** para antes de chegar a produção.

**DevOps** une desenvolvimento e operações num fluxo só, reduzindo o atrito entre "quem constrói" e "quem opera". **DevSecOps** injeta verificações de segurança dentro desse mesmo pipeline, em vez de deixar a segurança para uma revisão isolada no fim.

#### Estratégias de implantação

Formas de colocar uma nova versão em produção reduzindo o risco de queda:

- **Blue-green**: mantém **dois ambientes idênticos** (azul = versão atual em produção, verde = nova versão) e troca o tráfego de um para o outro de uma vez — rollback é instantâneo, basta apontar de volta para o azul.
- **Canary**: libera a nova versão para uma **fatia pequena** dos usuários primeiro, monitora, e só então expande gradualmente para todos — reduz o raio de impacto de um defeito não detectado em teste.
- **Rolling update**: substitui as instâncias **aos poucos**, uma ou algumas de cada vez, sem nunca derrubar a capacidade total do serviço.
- **Feature toggle** (*feature flag*): a nova funcionalidade já está no código em produção, mas **desligada** por uma chave de configuração — liga-se para todos, para um grupo, ou nem se liga, sem precisar de novo deploy.

### 3.9 Design de software

#### Alto nível x baixo nível; arquitetura x design

**Design de alto nível**: estrutura geral do sistema — módulos e como eles interagem, próximo do que se chama de **arquitetura**. **Design de baixo nível**: detalhe interno de cada módulo — funções, métodos, algoritmos, estruturas de dados específicas.

**Arquitetura** trata de decisões **amplas e estruturais** (quais componentes existem, como se comunicam); **design** (no sentido estrito) trata de decisões **detalhadas** dentro de um componente já definido. Não é verdade que "arquitetura só importa em projeto grande" — toda aplicação tem uma arquitetura, ainda que implícita e não documentada; o risco de não decidir conscientemente é que ela emerge de forma desorganizada.

Padrões de projeto (GoF) e notação UML têm tratamento completo no Capítulo 4 — este capítulo cobre o **processo** em volta do design; a técnica de design orientado a objetos propriamente dita está lá.

**Regra prática**

Ao ler uma questão que descreve uma decisão de projeto, pergunte: essa decisão **muda a forma como os componentes se relacionam** (quantos serviços existem, como se comunicam, onde os dados moram)? Se sim, é arquitetura/alto nível. Ela só **muda o algoritmo/estrutura dentro de um componente** sem afetar quem fala com quem? Se sim, é design/baixo nível.

## Capítulo 4

# Padrões de Projeto e UML

### Ficha do edital

"Padrões de projeto", "design de software", "padrões de desenvolvimento e reuso", "análise e projeto orientados a objetos". UML não está listada com todas as letras no edital do Perfil 3 (aparece explícita no Perfil 2), mas é vizinha direta desses itens e a FGV cobra com frequência em provas de TI.

## 4.1 Padrões de projeto (GoF)

### O que é — e o que não é — um padrão de projeto

Um padrão de projeto **não é código pronto, nem biblioteca, nem framework**. É a descrição de uma solução que já foi reinventada tantas vezes, por gente diferente, para o mesmo problema estrutural, que virou vocabulário compartilhado. Como todo verbete de dicionário técnico, tem quatro partes: um **nome** (para poder falar sobre ele sem reexplicar), o **problema** em que se aplica, a **solução** em termos de classes e objetos que colaboram, e as **consequências** — porque toda escolha de design tem um preço (mais classes, mais indireção, mais complexidade para quem lê o código pela primeira vez).

Essa definição já entrega o principal erro de quem estuda padrão decorando a estrutura: aplicar um padrão **sem ter o problema** correspondente é *over-engineering*. Uma *Abstract Factory* para criar um único tipo de objeto, sem variação nenhuma, acrescenta duas classes e zero flexibilidade real — o padrão existe para resolver um problema, não para demonstrar conhecimento de padrões.

O catálogo GoF (*Gang of Four*, os quatro autores do livro que o consolidou em 1994) reúne **23 padrões**, divididos em três famílias: **criacionais** (5), **estruturais** (7), **comportamentais** (11).

### O fio que liga quase todos os 23

Por trás da variedade, a maioria dos padrões do catálogo faz a **mesma** manobra estrutural: **isola o que varia atrás de uma interface estável**, e prefere **composição a herança** para poder trocar essa parte variável em tempo de execução, sem recompilar nem alterar quem usa a interface. É o fio que liga Strategy, State, Bridge, Decorator, Abstract Factory e a maior parte dos outros — e é por isso que "programe para uma interface, não para uma implementação" e "prefira composição a herança" são os dois lemas mais citados do livro original.

O que muda de família para família é **o que** exatamente está sendo isolado:

| Família              | Isola...                                             | Pergunta que ela responde                                                          |
|----------------------|------------------------------------------------------|------------------------------------------------------------------------------------|
| Criacionais (5)      | <i>qual</i> objeto se instancia, e quando            | "como criar isto sem amarrar o código à classe concreta?"                          |
| Estruturais (7)      | <i>como</i> as peças se encaixam                     | "como compor/embrulhar objetos para obter uma interface ou um comportamento novo?" |
| Comportamentais (11) | <i>quem</i> conversa com quem, e qual algoritmo roda | "como distribuir a responsabilidade da interação em tempo de execução?"            |

#### 4.1.1 Criacionais

| Padrão           | Intenção (gatilho no enunciado)                                                                                                                                 |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Singleton        | garantir <b>uma única instância</b> e ponto global de acesso ("apenas uma conexão/configuração no sistema todo")                                                |
| Factory Method   | delegar a criação a <b>subclasses</b> ; criar objeto <b>sem acoplar à classe concreta</b> ("decidir em runtime qual objeto criar", via herança)                 |
| Abstract Factory | criar <b>famílias</b> inteiras de objetos relacionados sem especificar classes concretas ("kit de componentes de UI para Windows e para Mac")                   |
| Builder          | construir objeto complexo <b>passo a passo</b> , separando o processo de construção da representação final ("montar um objeto com muitos parâmetros opcionais") |
| Prototype        | criar novos objetos <b>clonando</b> um protótipo existente ("copiar um objeto já configurado em vez de criar do zero")                                          |

#### Factory Method x Abstract Factory

Um sistema de notificação precisa enviar mensagens por e-mail ou SMS, decidido em tempo de execução por configuração. Uma classe abstrata `NotificadorFactory` tem o método `criar()`, e cada subclasse (`EmailFactory`, `SmsFactory`) decide qual notificador concreto instanciar — isso é **Factory Method**: uma família de objetos, delegada a subclasses por herança. Agora imagine que o sistema de UI precisa gerar um **conjunto inteiro** de componentes visualmente consistentes: botão, campo de texto e janela, todos no estilo "Windows" ou todos no estilo "Mac", nunca misturados. Uma `UIFactory` abstrata declara `criarBotao()`, `criarCampo()`, `criarJanela()`; `WindowsFactory` e `MacFactory` implementam a família inteira de uma vez — isso é **Abstract Factory**: não um objeto, uma **família** de objetos relacionados que precisam ser consistentes entre si.

**Cuidado — confusão comum**

Factory Method resolve **um** produto por vez, via herança (uma subclasse por variante). Abstract Factory resolve uma **família** inteira de produtos relacionados, via composição (a fábrica concreta é injetada, não subclassificada por quem a usa). Singleton é sempre **uma** instância — nunca "mais de uma quando necessário", isso descaracteriza o padrão.

**4.1.2 Estruturais**

| Padrão    | Intenção (gatilho)                                                                                                               |
|-----------|----------------------------------------------------------------------------------------------------------------------------------|
| Adapter   | fazer interfaces <b>incompatíveis</b> trabalharem juntas ("adaptar uma API legada à interface nova que o sistema espera")        |
| Facade    | interface <b>simplificada</b> para um subsistema complexo ("uma fachada única escondendo vários serviços internos")              |
| Proxy     | um <b>substituto</b> que controla o acesso ao objeto real (carregamento tardio, cache, controle de permissão)                    |
| Decorator | adicionar responsabilidades a um objeto <b>dinamicamente</b> , sem usar herança ("empilhar comportamentos em tempo de execução") |
| Composite | tratar objeto individual e composição de objetos <b>de forma uniforme</b> (estrutura em árvore, relação todo-parte recursiva)    |
| Bridge    | separar <b>abstração</b> de <b>implementação</b> para as duas variarem de forma independente                                     |
| Flyweight | compartilhar objetos semelhantes para <b>economizar memória</b> (muitas instâncias quase idênticas)                              |

**Adapter x Facade x Decorator x Proxy — os quatro que se confundem**

Um sistema novo espera receber pagamentos por uma interface `ProcessadorPagamento.pagar(valor)`, mas a biblioteca do banco só oferece `BancoLegado.executarTransacao(valorEmCentavos, moeda)`. Uma classe `AdapterBanco` implementa `ProcessadorPagamento` e, por dentro, converte a chamada para o formato do banco legado — isso é **Adapter**: faz duas interfaces que já existem e não combinam conversarem.

O mesmo sistema, para fechar um pedido, precisa chamar `Estoque`, `Pagamento`, `NotaFiscal` e `Envio`, em sequência determinada. Uma classe `FachadaPedido.finalizar(pedido)` esconde essas quatro chamadas atrás de um método só — isso é **Facade**: simplifica o acesso a um subsistema que já funciona, sem mudar nenhuma das peças internas.

Agora um sistema de café precisa somar o preço base a extras escolhidos (leite, chantilly, calda) sem criar uma subclasse para cada combinação possível. `CafeComLeite` envolve um `Cafe` e soma seu próprio preço ao preço de quem envolve; `CafeComChantilly` faz o mesmo em volta do resultado anterior — isso é **Decorator**: cada camada **acrescenta comportamento/dado**, empilhável em qualquer combinação.

Por fim, uma imagem de 50MB só deve ser carregada do disco quando realmente for exibida na

tela. Uma classe **ProxyImagem** implementa a mesma interface de **Imagem**, mas só instancia a **ImagemReal** (que faz o carregamento pesado) na primeira chamada a **exibir()** — isso é **Proxy**: **controla o acesso** ao objeto real, sem acrescentar comportamento novo à funcionalidade em si.

#### A diferença que sobra depois da estrutura parecida

Adapter, Facade e Decorator todos "envolvem" outro objeto, e por isso a estrutura de código fica parecida. O que os separa é a **intenção**: Adapter **traduz** uma interface existente para outra que o cliente espera (o objetivo é compatibilidade); Facade **simplifica** o acesso a várias peças que já são compatíveis entre si (o objetivo é conveniência); Decorator **acrescenta** funcionalidade nova, mantendo a mesma interface (o objetivo é extensão). E Proxy **controla acesso** sem mudar a função — é a diferença que separa Proxy de Decorator: o primeiro decide *se e quando* delega para o objeto real; o segundo sempre delega, e soma algo à resposta.

### 4.1.3 Comportamentais

| Padrão                  | Intenção (gatilho)                                                                                                                                |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Strategy                | família de algoritmos <b>intercambiáveis</b> , escolhidos por <b>quem usa</b> , em runtime ("trocar a regra de cálculo sem um <b>if</b> gigante") |
| Observer                | notificar automaticamente <b>dependentes</b> quando o estado de um objeto muda ( <i>publish-subscribe</i> , sistema de eventos)                   |
| Command                 | encapsular uma <b>requisição como objeto</b> (permite desfazer/refazer, enfileirar, logar operações)                                              |
| State                   | mudar o comportamento conforme o <b>estado interno</b> , e é o <b>próprio objeto/estado</b> quem decide a próxima transição                       |
| Template Method         | define o <b>esqueleto</b> fixo de um algoritmo, deixando passos específicos para subclasses sobrescreverem                                        |
| Iterator                | percorre uma coleção <b>sem expor sua estrutura interna</b>                                                                                       |
| Chain of Responsibility | passa a requisição por uma <b>cadeia</b> de manipuladores até que algum a trate                                                                   |
| Mediator                | centraliza a comunicação entre vários objetos, reduzindo acoplamento <b>N-para-N</b> para N-para-1                                                |
| Memento                 | captura e restaura o estado interno de um objeto sem violar seu encapsulamento ( <i>snapshot</i> , base do "desfazer")                            |
| Visitor                 | adiciona novas operações a uma estrutura de objetos <b>sem alterar as classes</b> dessa estrutura                                                 |
| Interpreter             | define uma <b>gramática</b> para uma linguagem simples e um interpretador que avalia sentenças escritas nela                                      |

### Do if gigante ao Strategy

O cálculo de frete começa assim, dentro do serviço de pedidos: `if (tipo == "sedex") {...} else if (tipo == "pac") {...} else if (tipo == "retirada") {...}`. Refatorando com Strategy: cria-se uma interface `CalculoFrete` com o método `calcular(pedido)`, e uma classe por regra (`FreteSedex`, `FretePac`, `FreteRetirada`). O serviço passa a **receber** a estratégia já escolhida e só chamar `calcular` — ele deixa de conhecer as regras concretas, dependendo só da interface.

O ganho tem nome: acrescentar uma transportadora nova virou **criar uma classe**, não **editar** um método já existente — é o *Open/Closed* do SOLID (Seção 4.1.5 adiante) em ação. E o serviço passou a depender de uma abstração (`CalculoFrete`), não de implementações concretas — é o *Dependency Inversion* do mesmo princípio. Repare que o padrão não eliminou a decisão de **qual** regra usar; ele só a **moveu** para fora, para quem monta o objeto do serviço.

### Strategy x State — mesma estrutura, intenção oposta

Os dois são, por dentro, a mesma estrutura de código: um objeto delegando comportamento a outro através de uma interface comum. A diferença está em **quem decide a troca**. No **Strategy**, quem escolhe a estratégia é o **cliente**, de fora, e as estratégias não sabem umas das outras (trocar o cálculo de frete não afeta o cálculo de imposto). No **State**, quem troca é o **próprio objeto**, conforme seu estado interno, e os estados costumam conhecer para qual estado ir a seguir ("pedido pago" sabe que o próximo é "pedido enviado"). Regra de leitura: se o enunciado descreve um **ciclo de vida com transições** (pedido pendente → pago → enviado → entregue), é State; se descreve **alternativas equivalentes** para fazer a mesma tarefa (calcular frete de três jeitos diferentes), é Strategy.

### Observer x Mediator

**Observer** é o padrão de notificação: um sujeito muda de estado e **avisa** uma lista de interessados que se inscreveram para saber disso (a comunicação é sempre "sujeito → observadores", um-para-muitos). **Mediator** **centraliza a comunicação** entre vários objetos que, de outra forma, teriam que se conhecer uns aos outros (N-para-N vira N-para-1 através do mediador) — não é sobre notificar mudança de estado, é sobre reduzir quantas classes conhecem quantas outras classes diretamente.

## 4.1.4 GRASP

### Princípios de responsabilidade, não padrões prontos

GRASP (*General Responsibility Assignment Software Patterns*) não é uma lista de estruturas de código como o GoF — é um conjunto de **princípios** para decidir **qual classe deve receber qual responsabilidade**: Information Expert, Creator, Controller, Low Coupling, High Cohesion, Polymorphism, Pure Fabrication, Indirection, Protected Variations.

Dois deles resolvem sozinhos a maior parte das perguntas de "a qual classe cabe esta responsabilidade":

- **Information Expert**: a responsabilidade vai para a classe que **já tem os dados** necessários para cumpri-la. Quem calcula o total de um pedido é a classe `Pedido`, que já conhece seus itens — não uma `CalculadoraDeTotais` externa, que precisaria pedir tudo emprestado (o que criaria acoplamento desnecessário).
- **Creator**: quem cria um objeto é quem o **agrega, contém ou usa de perto**. `Pedido`

cria seus `ItemDePedido`, porque é quem os contém e usa diretamente.

Os dois existem pelo mesmo motivo: manter juntos o dado e o comportamento que o usa é o que produz **alta coesão** (a classe faz uma coisa coerente) e **baixo acoplamento** (menos classes precisam se conhecer para o trabalho acontecer) — que são, eles próprios, dois outros princípios da mesma lista.

#### 4.1.5 SOLID

##### Os cinco princípios — tratamento completo com armadilhas de Java no Capítulo 10

- **S — *Single Responsibility***: uma classe deve ter **uma responsabilidade**, ou seja, um único motivo para mudar. Uma classe que salva no banco e formata relatório e valida regra de negócio tem três motivos para mudar — e muda toda vez que qualquer um dos três muda, mesmo sem relação com os outros dois.
- **O — *Open/Closed***: aberta à **extensão**, fechada à **modificação**. Acrescentar comportamento novo deve ser possível **sem editar** código já testado e em produção — é exatamente o que o exemplo de Strategy acima demonstrou.
- **L — *Liskov***: um subtipo deve poder substituir seu tipo base em qualquer lugar do programa **sem quebrar o comportamento esperado**. O exemplo clássico de violação: `Quadrado` herda de `Retangulo` e sobrescreve `setLargura` para também mudar a altura (mantendo os lados iguais) — um código que usa `Retangulo` esperando poder mudar largura e altura independentemente quebra quando recebe um `Quadrado`.
- **I — *Interface Segregation***: várias interfaces **pequenas e específicas** são melhores que uma interface "gorda" que obriga quem implementa a fornecer métodos que não usa.
- **D — *Dependency Inversion***: módulos de alto nível não devem depender de módulos de baixo nível — ambos devem depender de **abstrações**. Na prática: o serviço depende da interface `CalculoFrete`, nunca da classe concreta `FreteSedex`.

##### ISP x SRP

Os dois falam em "pequeno e específico", e por isso se confundem, mas tratam de coisas diferentes: **SRP** é sobre quantas **responsabilidades** uma **classe** tem; **ISP** é sobre quantos **métodos desnecessários** uma **interface** obriga a implementar. Uma classe pode violar SRP mesmo implementando uma interface enxuta, e uma interface pode violar ISP mesmo sendo implementada por uma classe de responsabilidade única.

#### 4.1.6 Padrões arquiteturais (fora do catálogo GoF)

- **MVC** (*Model-View-Controller*): separa dados (*Model*), apresentação (*View*) e controle do fluxo (*Controller*) em três responsabilidades distintas.
- **MVVM / MVP**: variações do MVC com *binding* de dados automático (MVVM) ou um *presenter* que intermedeia view e model (MVP) — comuns em front-end e mobile.
- **DAO** (*Data Access Object*) / **Repository**: isolam o código de acesso a dados (SQL, ORM) do resto da aplicação.
- **DTO** (*Data Transfer Object*): objeto simples, sem lógica de negócio, usado só para transportar dados entre camadas.

### 4.1.7 Reuso e anti-padrões

**Reuso por herança** ("é-um": `Cachorro extends Animal`) x **reuso por composição** ("tem-um": `Carro tem um Motor`). A recomendação consolidada é **preferir composição a herança** (*favor composition over inheritance*) sempre que a relação não for genuinamente um tipo-subtipo estável, porque herança acopla fortemente a subclasse aos detalhes internos da superclasse (qualquer mudança na base pode quebrar a subclasse sem aviso).

**Anti-padrões** (más práticas recorrentes, batizadas do mesmo jeito que os padrões, para poder nomear o erro): **God Object** (uma classe que faz tudo, violando SRP em escala máxima), **Spaghetti Code** (fluxo de controle emaranhado, sem estrutura reconhecível), **Golden Hammer** (usar sempre a mesma ferramenta/solução para problemas diferentes, mesmo quando não é a mais adequada), **Copy-Paste Programming** (duplicar código em vez de extrair uma abstração comum).

## 4.2 UML

### Notação, não método

A UML é uma **linguagem de notação**, não um processo: ela padroniza o **desenho**, não diz **quando** desenhar cada diagrama — isso é papel do processo de desenvolvimento (RUP, Scrum; Capítulo 3). O problema que ela resolve é concreto: antes da padronização, cada empresa tinha sua própria simbologia, e o mesmo losango podia significar coisas diferentes em dois documentos da mesma equipe. Padronizar a notação é o que permite discutir projeto sem primeiro precisar combinar o que cada símbolo quer dizer. A especificação vigente da OMG é a **UML 2.5.1**.

### 4.2.1 Os 14 diagramas em duas famílias

| Família         | Foca em                                                              | Diagramas principais                                                                                                               |
|-----------------|----------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Estruturais     | estrutura <b>estática</b> — como o sistema é organizado num instante | <b>Classe</b> , Objeto, Componente, Implantação ( <i>deployment</i> ), Pacote, Estrutura composta, Perfil                          |
| Comportamentais | comportamento <b>dinâmico</b> — como o sistema se move no tempo      | <b>Caso de uso</b> , <b>Sequência</b> , <b>Atividade</b> , <b>Estado</b> (máquina de estados), Comunicação, Interação geral, Tempo |

A pergunta-teste: o diagrama descreveria o sistema **parado, num instante** (estrutural) ou descreve **o que acontece ao longo do tempo** (comportamental)? Classe é sempre estrutural; sequência, atividade, estado e caso de uso são sempre comportamentais.

### 4.2.2 Diagrama de classes

#### Relacionamentos e o que cada notação significa

| Relacionamento          | Significado                                              | Notação                           |
|-------------------------|----------------------------------------------------------|-----------------------------------|
| Associação              | ligação estrutural entre classes                         | linha                             |
| Agregação               | todo-parte <b>fraca</b> (a parte sobrevive sem o todo)   | losango <b>vazio</b>              |
| Composição              | todo-parte <b>forte</b> (a parte morre junto com o todo) | losango <b>cheio</b>              |
| Herança / Generalização | é-um (subtipo do tipo base)                              | seta de triângulo vazio           |
| Dependência             | usa temporariamente (parâmetro, variável local)          | seta tracejada                    |
| Realização              | implementa uma interface                                 | linha tracejada + triângulo vazio |

**Multiplicidade** (quantas instâncias do outro lado participam): 1 (exatamente uma), 0..1 (zero ou uma), 1..\* (uma ou mais), \* (zero ou mais). **Visibilidade**: + público, - privado, # protegido, ~ pacote.

A classe se desenha em **três compartimentos**: nome, atributos, operações. O atributo, na forma completa, é onde mora o detalhe mais cobrado em leitura de diagrama:

```
- status : String [1] = "aberto"
```

Nessa ordem: visibilidade, nome, tipo, multiplicidade, e **valor padrão** — o valor que a instância assume quando ninguém informa outro na criação.

#### Agregação x composição

Uma **Universidade** tem **Departamentos** — se a universidade fechar, os departamentos, como entidades daquela universidade, deixam de existir junto (não fazem sentido soltos): **composição**, losango cheio do lado de **Universidade**. Um **Departamento** tem **Professores** — se o departamento fechar, os professores continuam existindo (podem ser realocados para outro departamento): **agregação**, losango vazio do lado de **Departamento**.

#### Extremidade de associação como atributo, e associação qualificada

Cada ponta de uma associação tem papel, multiplicidade e navegabilidade — e uma ponta navegável **pode ser representada como atributo** da classe do outro lado. "Pedido associa-se a 1 Cliente" desenhado como linha e "Pedido tem o atributo cliente: Cliente" escrito no compartimento de atributos dizem a **mesma** coisa em UML — são duas notações de um único modelo, não duas modelagens diferentes. Uma **associação qualificada** (um qualificador desenhado como um pequeno retângulo na ponta da linha, geralmente uma chave) **particiona** o conjunto do outro lado e costuma reduzir a multiplicidade de \* para 0..1: dado um **Banco** e um **numeroConta**, chega-se a no máximo uma **Conta** — é um índice de busca, não uma relação todo-parte, o que a torna um distrator plausível de agregação em leitura de diagrama.

### 4.2.3 Casos de uso

Modela **requisitos funcionais** do ponto de vista de quem usa o sistema: **ator** (bonequinho, alguém/algo externo que interage), **caso de uso** (elipse, uma funcionalidade completa do ponto de vista do ator), **fronteira do sistema** (retângulo que delimita o que está dentro).

- «**include**»: o comportamento incluído é **sempre** executado como parte do caso de uso base (obrigatório) — ex.: "Fazer Pedido" *include* "Autenticar Usuário".
- «**extend**»: o comportamento é **opcional/condicional**, só ocorre em certas circunstâncias — ex.: "Fazer Pedido" pode ser *extended* por "Aplicar Cupom de Desconto", que só roda se o cliente tiver um cupom.

#### Cuidado — confusão comum

Incluir é o comportamento que **sempre** acontece; *extend* é o que **às vezes** acontece. É comum a leitura inverter os dois — o teste mental é: "esse passo é parte obrigatória do fluxo principal, ou é um desvio condicional que só ocorre em certas situações?"

### 4.2.4 Os quatro diagramas de interação

Todos mostram objetos trocando mensagens; a diferença é a ênfase

| Diagrama                        | Enfatiza                                                                                         | Como se lê a ordem                                                                 |
|---------------------------------|--------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <b>Sequência</b>                | a <b>ordem cronológica</b> das mensagens                                                         | pela posição vertical: de cima para baixo                                          |
| <b>Comunicação</b>              | a <b>estrutura de ligações</b> entre os objetos (quem está conectado a quem)                     | pela <b>numeração</b> das mensagens (1, 1.1, 1.2) — sem eixo de tempo explícito    |
| <b>Tempo</b> ( <i>timing</i> )  | a <b>mudança de estado</b> de cada participante ao longo de uma <b>escala de tempo explícita</b> | pelo eixo horizontal de tempo; usado quando o requisito envolve duração/tempo real |
| <b>Visão geral de interação</b> | como <b>vários cenários</b> se encadeiam entre si                                                | é um diagrama de atividades cujos nós são, eles mesmos, interações                 |

**Sequência** e **comunicação** são **semanticamente equivalentes** — dá para converter um no outro sem perder informação, já que ambos mostram as mesmas mensagens entre os mesmos objetos. A diferença é puramente de ênfase visual: sequência é melhor para ver **quando** cada coisa acontece; comunicação é melhor para ver **quem fala com quem**. Diagrama de **atividade** é diferente dos quatro: ali não há troca de mensagens entre participantes nomeados, há um **fluxo de trabalho** genérico — ações, decisões, paralelismo.

#### Regra prática

Se o enunciado pede a **ordem exata em que as mensagens acontecem**, é **sequência**. Se pede **quem está ligado a quem** estruturalmente, sem se importar com quando, é **comunicação**. Se

envolve **duração/tempo real**, é **timing**. Se é sobre um **fluxo de trabalho** sem participantes trocando mensagem nomeada, é **atividade**, não um diagrama de interação.

#### 4.2.5 Atividade e máquina de estados

**Atividade:** fluxo de trabalho/algoritmo — nós de ação, decisão (losango), bifurcação/junção (barra, representa paralelismo), início/fim. Parecido com um fluxograma; usado para modelar processos de negócio ou a lógica de um algoritmo.

**Máquina de estados:** modela os **estados possíveis** de um objeto e as **transições** entre eles, disparadas por eventos — o objeto se comporta diferente conforme o estado em que está. É o diagrama que casa diretamente com o padrão de projeto **State** (Seção anterior): cada estado do diagrama pode virar uma classe concreta que implementa a interface do estado.

##### Roteiro para ligar diagrama a conceito

Caso de uso ↔ requisitos funcionais (Capítulo 3). Classes ↔ orientação a objetos/SOLID. Máquina de estados ↔ padrão de projeto State. Componente e Implantação (*deployment*) ↔ decisões de arquitetura (Capítulo 5).

## Capítulo 5

# Arquitetura de Software

### Ficha do edital

Arquitetura de software; interoperabilidade; arquitetura e linguagem orientada a serviços; web services; mensageria; API, Swagger; arquitetura orientada a objetos; aplicações para ambiente web; servidor de aplicações x servidor web; internet/extranet/intranet/portal; arquitetura hexagonal; microsserviços (orquestração e API gateway); containers; transações distribuídas.

## 5.1 Servidor web x servidor de aplicação

### Onde a requisição HTTP para, e onde a lógica de negócio roda

|          | Servidor web                                  | Servidor de aplicação                                         |
|----------|-----------------------------------------------|---------------------------------------------------------------|
| Função   | atende HTTP, serve conteúdo estático/dinâmico | executa <b>lógica de negócio</b> , integra com banco de dados |
| Exemplos | Apache, NGINX                                 | JBoss/WildFly, WebLogic                                       |

No mundo Java a distinção é literal, não só uma divisão de responsabilidade conceitual: o servidor de aplicação implementa a especificação **Java EE/Jakarta EE completa** — ele traz o **container web** (que processa Servlet/JSP, a mesma coisa que um servidor web faz) e, **além disso**, o **container EJB** para os componentes de negócio, mais transações distribuídas, JMS (mensageria) e injeção de dependência. O Tomcat, por causa disso, é um caso interessante: ele é servidor web **mais** container web, mas **não** é um servidor de aplicação completo, porque não tem o container EJB.

### Cuidado — confusão comum

Servidor web **não** processa lógica de negócio — ele recebe a requisição HTTP e serve conteúdo; quem executa a regra de negócio é o servidor de aplicação. Servidor de aplicação não é "sempre mais leve" (é o oposto: carrega containers a mais), não "dispensa a JVM" (roda sobre ela) e não "deixa de suportar HTTP" (suporta, porque contém o próprio container web dentro de si).

## 5.2 Estilos de arquitetura: uma sequência histórica

### Cada estilo nasceu resolvendo a dor do anterior

Os estilos de arquitetura não são uma lista de opções paralelas e equivalentes — eles formam uma **sequência histórica**, e entender por que cada um surgiu é o que permite responder a uma questão de cenário ("qual estilo resolve esta dor"), em vez de decorar definições soltas. O **monólito** incomodava porque uma equipe travava a outra: subir a correção de uma única tela

obrigava a reimplantar o sistema inteiro. A **SOA** respondeu quebrando o sistema em **serviços** com contrato explícito e baixo acoplamento — mas concentrou a inteligência de integração num **barramento** central (o ESB), e esse barramento virou o novo gargalo e ponto único de falha. Os **microserviços** responderam à SOA invertendo a filosofia: em vez de um barramento inteligente com canais burros, a ideia virou **canais burros e serviços espertos** — cada serviço decide sozinho, sem depender de um intermediário central pesado.

- **Monolítica**: tudo num artefato único, implantado junto. Simples de começar; acopla e dificulta escalar partes isoladamente.
- **Cliente-servidor, N camadas, P2P, barramento de mensagens**: formas intermediárias de distribuir responsabilidade entre nós.
- **SOA (orientada a serviços)**: serviços reutilizáveis, contrato explícito, baixo acoplamento, interoperabilidade entre sistemas heterogêneos. Web services são a tecnologia comum de implementação. O barramento típico é o **ESB (Enterprise Service Bus)**: o intermediário que **roteia**, **transforma** (converte formato/protocolo entre sistemas que não se falam nativamente) e **orquestra** as mensagens entre serviços.
- **Microserviços**: serviços pequenos, autônomos, **implantáveis independentemente**, cada um com **seu próprio banco de dados** (não compartilham base — isso é o que reduz o acoplamento). Comunicação leve, tipicamente REST ou mensageria.
- **Arquitetura hexagonal (*Ports & Adapters*)**: separa a **lógica de negócio** do mundo externo através de **portas** (interfaces que o núcleo define) e **adaptadores** (implementações concretas dessas portas) — permite trocar UI, banco de dados ou serviço externo sem tocar no núcleo de negócio.

### O preço dos microserviços — por que nem sempre valem a pena

Todo ganho de microserviço vem com uma conta correspondente:

| Ganho                                     | Preço correspondente                                                                                                                                                     |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| implantar um serviço sem tocar nos outros | precisa de automação de <i>deploy</i> e observabilidade distribuída — sem isso, achar a causa de um erro fica pior do que num monólito                                   |
| escalar só a parte que aperta             | a chamada entre serviços virou <b>chamada de rede</b> : pode falhar, tem latência, exige repetição e <i>timeout</i>                                                      |
| cada serviço com seu próprio banco        | acabou o JOIN entre eles, e acabou a transação ACID atravessando os dois — daí a necessidade de padrões como Saga e a aceitação de consistência eventual (ver Seção 5.6) |
| equipes autônomas por serviço             | só compensa se as equipes <b>forem</b> realmente várias; para um time pequeno, é custo sem benefício correspondente                                                      |

A regra prática: microserviço resolve um problema **organizacional** (várias equipes que precisam trabalhar em paralelo sem se travar) e de **escala independente** (um componente específico recebe muito mais carga que os outros). Se o cenário descreve sistema pequeno, equipe única ou forte necessidade de consistência transacional entre módulos, um monólito (eventualmente modular internamente) costuma ser a resposta mais apropriada.

**Cuidado — confusão comum**

Microserviços **não** compartilham o mesmo banco de dados — compartilhar aumentaria o acoplamento, contrariando o princípio central do estilo. Baixo acoplamento significa que uma mudança em um serviço **não** obriga mudança em outro. Um item que descreve "o barramento que intermedeia, roteia e transforma mensagens entre serviços" está descrevendo um **ESB** — não confundir com ETL (move dados para um *data warehouse*), CDN (entrega conteúdo estático) ou DNS (resolve nomes).

**5.2.1 Camadas lógicas (*layers*) x camadas físicas (*tiers*)****Duas ideias que o português funde numa palavra só**

|                       |                                                                                                     |
|-----------------------|-----------------------------------------------------------------------------------------------------|
| <b>Layer (lógica)</b> | como as <b>responsabilidades</b> do software estão organizadas: apresentação, negócio, persistência |
| <b>Tier (física)</b>  | em <b>quantos nós</b> o software está implantado: máquinas, processos, servidores diferentes        |

Uma coisa não obriga a outra. Uma aplicação pode ter **três layers e um único tier**: as três responsabilidades bem separadas no código-fonte (classes de apresentação, de negócio, de persistência, cada uma no seu lugar), mas tudo rodando dentro do mesmo processo, na mesma máquina — e a aplicação continua sendo, do ponto de vista de implantação, **monolítica**. O inverso também existe: um cliente-servidor com dois tiers físicos pode manter apresentação e negócio bem separados logicamente dentro de cada tier.

**Cuidado — confusão comum**

Não inverta as definições (layer é organização lógica de responsabilidade; tier é distribuição física em nós). E desconfie de "três camadas lógicas são **necessariamente** implantadas em três nós físicos— não são; o MVC, por exemplo, é um padrão de organização **lógica** (apresentação/modelo/controle), não um padrão de implantação em tiers.

**5.3 Integração: REST, SOAP e web services****O que REST é de verdade — recurso, representação, verbo**

REST não é "API que devolve JSON"— é um **estilo arquitetural** com uma ideia central: tudo que interessa é um **recurso**, identificado por uma URI, e o cliente nunca manipula o recurso diretamente — ele troca **representações** dele (JSON, XML, HTML). Dessa ideia central decorrem as restrições do estilo:

- **Interface uniforme**: o conjunto de verbos é fixo e igual para todo recurso — GET lê, POST cria, PUT substitui por inteiro, PATCH altera em parte, DELETE remove. Por isso a URI deve nomear uma **coisa**, não uma **ação**: `/pedidos/42` está de acordo com o estilo; `/buscarPedido?id=42` contraria, porque põe o verbo na URI quando ele já deveria estar só no método HTTP.
- **Sem estado (*stateless*)**: cada requisição carrega tudo o que é necessário para ser entendida sozinha, sem depender de uma sessão guardada no servidor. O ganho não é ideológico: se o servidor não guarda sessão, **qualquer** instância do servidor consegue

atender **qualquer** requisição — e é exatamente isso que permite colocar um balanceador de carga na frente e escalar horizontalmente sem precisar de "afinidade de sessão" (a mesma requisição sempre indo para a mesma instância).

- **Cliente-servidor, cache, sistema em camadas:** o cliente não precisa saber se está falando com o servidor final ou com um intermediário — por isso um proxy, um gateway ou uma CDN cabem no caminho da requisição sem quebrar nada do lado do cliente.

A corrente de raciocínio que sustenta "por que REST escala bem": ausência de estado no servidor → qualquer instância pode atender qualquer requisição → escala horizontal sem restrição de afinidade.

## REST x SOAP

|          | REST                                  | SOAP                                                  |
|----------|---------------------------------------|-------------------------------------------------------|
| Natureza | estilo arquitetural, sobre HTTP       | protocolo baseado em XML                              |
| Formato  | JSON (comum), leve                    | XML (envelope), verboso                               |
| Contrato | OpenAPI/Swagger                       | WSDL                                                  |
| Estado   | <b>stateless</b> por definição        | pode incorporar padrões WS-*                          |
| Vantagem | leve, escala, independe de plataforma | contratos formais, padrões de segurança (WS-Security) |

## Cuidado — confusão comum

**Cacheável** é uma restrição **distinta** de *stateless* — a resposta se declara cacheável ou não, e quem a guarda pode ser o **cliente** ou um **intermediário** (proxy, gateway, CDN). Cache no caminho é **permitido** pelo REST, é o que deixa uma CDN servir conteúdo estático de um nó próximo do usuário. O que **não** pode ficar guardado no servidor é o **estado da sessão**, não o cache da resposta — não confunda os dois. **PUT** substitui o recurso **inteiro**; **PATCH** altera só uma **parte** dele. Dizer que "SOAP não tem contrato formal" contradiz o próprio SOAP, que usa WSDL como contrato.

## UDDI, WSDL, SOAP: três papéis, não um sinônimo do outro

O trio clássico dos web services WS-\*: **SOAP** é o formato de **mensagem** trocada; **WSDL** é o **contrato** (descreve o que o serviço oferece, quais operações, quais parâmetros); **UDDI** é o diretório (legado) para **descoberta/registro** de web services — onde procurar quem oferece o quê. **Swagger/OpenAPI** cumpre, no mundo REST, o papel que o WSDL cumpre no mundo SOAP: **OpenAPI** é a especificação padrão para documentar/contratar APIs REST (endpoints, parâmetros, respostas); **Swagger** é o conjunto de ferramentas (Swagger UI, editor, geração de código) construído em cima dela.

### 5.3.1 Mensageria: comunicação assíncrona

#### Desacopla quem envia de quem processa

Numa chamada REST síncrona, quem envia **espera** a resposta. Na mensageria, quem produz a mensagem **não espera** quem a consome — os dois lados só compartilham um intermediário, o **broker** (RabbitMQ, Kafka).

| Modelo                            | Como funciona                                                         | Exemplo       |
|-----------------------------------|-----------------------------------------------------------------------|---------------|
| Fila ( <i>queue</i> )             | mensagem consumida por <b>um</b> consumidor ( <i>point-to-point</i> ) | RabbitMQ, SQS |
| <i>Publish/Subscribe</i> (tópico) | mensagem entregue a <b>vários</b> assinantes ao mesmo tempo           | Kafka, SNS    |

**Apache Kafka** é uma plataforma de **streaming de eventos**: produtores publicam em **tópicos** (particionados para paralelismo), consumidores leem organizados em **grupos**, e a plataforma guarda o **log de eventos** (o que permite reprocessar mensagens antigas, diferente de uma fila tradicional que descarta a mensagem ao ser consumida).

Benefícios da mensageria: **desacoplamento** (produtor e consumidor não precisam estar disponíveis ao mesmo tempo), **absorção de picos** (a fila funciona como *buffer* quando o consumidor não acompanha o ritmo do produtor), **resiliência** (se o consumidor cair, a mensagem espera na fila). Casa naturalmente com **microserviços** e com os padrões **Saga** (Seção 5.6) e **Event Sourcing** (guardar o histórico de eventos como fonte da verdade, em vez de só o estado final).

#### Cuidado — confusão comum

Fila entrega para **um** consumidor; tópico/*pub-sub* entrega para **vários** — não inverta os dois. Mensageria é sempre **assíncrona**, ao contrário de uma chamada REST tradicional, que é síncrona.

## 5.4 Ambientes de rede corporativa

| Termo    | Alcance                                                                                                        |
|----------|----------------------------------------------------------------------------------------------------------------|
| Internet | rede pública global                                                                                            |
| Intranet | rede interna da organização (acesso só de funcionários)                                                        |
| Extranet | acesso controlado estendido a parceiros/clientes externos autorizados — o meio-termo entre intranet e internet |
| Portal   | ponto único que centraliza acesso a informação/serviços diversos                                               |

## 5.5 Nuvem e escalabilidade

### IaaS, PaaS, SaaS — onde passa a linha de responsabilidade

Os três modelos respondem à mesma pergunta: **até onde vai o provedor, e onde começa a sua responsabilidade**. A pilha completa de uma aplicação é sempre a mesma — rede, armazenamento, servidores, virtualização, sistema operacional, *middleware*, tempo de execução, dados, aplicação — e cada modelo apenas move a linha de corte mais para cima na pilha.

|             | Você cuida de                                                                     | Exemplo típico                                     |
|-------------|-----------------------------------------------------------------------------------|----------------------------------------------------|
| <b>IaaS</b> | sistema operacional para cima: <i>patch</i> , tempo de execução, aplicação, dados | máquina virtual alugada                            |
| <b>PaaS</b> | aplicação e dados; o resto é responsabilidade do provedor                         | plataforma que recebe seu código-fonte e o executa |
| <b>SaaS</b> | só os <b>dados</b> e a configuração de uso                                        | software pronto, acessado pelo navegador           |

Duas consequências importantes. Primeira: **quanto mais "as a Service", menos controle** você tem — não é só menos trabalho operacional, é também menos liberdade para customizar a infraestrutura subjacente. Segunda, a **responsabilidade compartilhada em segurança**: mesmo em SaaS, **o dado continua sendo seu** — o provedor responde pela segurança *da* nuvem (a infraestrutura em si), e o cliente responde pela segurança *na* nuvem (quem tem acesso, como o dado é classificado, se a conta usa segundo fator de autenticação). A afirmação de que SaaS "transfere integralmente a responsabilidade pelos dados ao provedor" é falsa.

### Escalabilidade horizontal x vertical

**Escalabilidade vertical (*scale up*)**: adicionar recursos à **instância existente** (mais CPU, mais RAM na mesma máquina) — tem um teto físico e costuma exigir reinício. **Escalabilidade horizontal (*scale out*)**: **adicionar mais instâncias/nós** rodando em paralelo — teoricamente sem teto, mas exige que a aplicação seja capaz de distribuir carga entre instâncias (o que REST *stateless* viabiliza diretamente).

**Elasticidade**: ajustar a capacidade **automaticamente** conforme a demanda sobe e desce, sem intervenção manual. **Cloud bursting**: uma aplicação que roda normalmente em infraestrutura própria "estoura" para a nuvem pública só nos picos de demanda, voltando ao normal depois.

### Serverless / FaaS

"Sem servidor" é um nome enganoso: **existe** servidor, só que é o **provedor** quem o gerencia, não o cliente. O código é publicado como uma **função** e o provedor a executa em resposta a **eventos** (uma requisição HTTP, uma mensagem numa fila, um arquivo novo num *bucket*).

Características cobradas em prova: escala **de zero** até muitas execuções simultâneas conforme os eventos chegam; **cobrança pelo tempo e pelos recursos efetivamente consumidos** durante a execução (não por instância ficando parada); execução **sem estado** entre uma invocação e a próxima (o estado precisa ir para um armazenamento externo, como um banco); **cold start** — a latência maior na primeira invocação depois de um período ocioso, porque o ambiente de execução precisa ser inicializado; e um **limite de tempo por execução**, que desaconselha serverless para processamento longo e contínuo.

**Balanceador de carga x CDN — gargalos diferentes**

Um site de e-commerce está lento para **todos** os usuários porque as poucas instâncias da aplicação estão sobrecarregadas de processamento → a solução é um **balanceador de carga**, que reparte as requisições entre mais instâncias. O mesmo site está rápido para usuários no Brasil, mas lento para usuários na Europa, ao carregar imagens e arquivos estáticos → a solução é uma **CDN**, que replica esse conteúdo estático em pontos de presença geograficamente distribuídos e o entrega do nó mais próximo do usuário. Os dois atacam gargalos diferentes (processamento x latência geográfica de conteúdo estático) e por isso se somam na prática, em vez de serem alternativas mutuamente exclusivas.

**Cuidado — confusão comum**

"Adicionar recursos à instância" é sempre **vertical**, nunca horizontal. Em serverless: "não há servidor, o código roda no dispositivo que dispara o evento" é falso (há servidor, do lado do provedor); "o estado da execução anterior permanece disponível" é falso (é sem estado entre invocações); "a latência da primeira invocação é igual à das demais" ignora o *cold start*; "remove o limite de tempo por execução" é falso — o limite existe e é justamente o que desaconselha processamento longo em FaaS.

**5.5.1 Containers e virtualização**

**Container (Docker) x máquina virtual:** container compartilha o **kernel** do sistema operacional hospedeiro (por isso é leve e sobe rápido); máquina virtual tem um **sistema operacional convidado completo**, rodando sobre um **hipervisor** (isola mais, mas pesa mais e sobe mais devagar). **Kubernetes** orquestra containers em escala (agenda onde cada container roda, reinicia os que caem, distribui carga entre eles).

**Hipervisor Tipo 1** (*bare-metal*, roda diretamente sobre o hardware, sem sistema operacional hospedeiro no meio: ESXi, Hyper-V, KVM) x **Tipo 2** (*hosted*, roda como um programa em cima de um sistema operacional já instalado: VirtualBox, VMware Workstation).

**Virtualização total** (o sistema operacional convidado não sabe que está virtualizado, roda sem modificação) x **paravirtualização** (o sistema operacional convidado é modificado para cooperar diretamente com o hipervisor via *hypercalls*, o que é mais rápido por evitar simulação completa de hardware).

**Cuidado — confusão comum**

Container **não** é sinônimo de VM — a diferença central é compartilhar o kernel (container) versus ter sistema operacional próprio completo (VM).

**5.6 Transações distribuídas****O problema nasce quando cada serviço ganha o próprio banco**

Dentro de um único SGBD, o ACID (Capítulo 6) é "de graça": existe um só log de transações, um só gerenciador de bloqueios, e ele decide sozinho se a transação efetiva ou desfaz. Distribuído entre vários bancos (um por microserviço), não há árbitro único — "debitar no serviço de contas e reservar no serviço de estoque" são **duas** transações, em **dois** bancos diferentes, ligadas por uma rede que pode falhar no meio do caminho. Não existe uma operação que efetive as

duas ao mesmo tempo de forma atômica; o que existe são **protocolos que tentam coordenar** isso, cada um pagando um preço diferente.

- **2PC (*Two-Phase Commit*)**: um coordenador central pergunta a todos os participantes se estão prontos para efetivar; só faz o *commit* se **todos** confirmarem (unanimidade). É **bloqueante**: se o coordenador cair no meio do processo, os participantes ficam travados esperando, sem saber se devem efetivar ou desfazer.
- **Saga**: uma sequência de transações locais independentes, cada uma com uma ação de **compensação** correspondente caso algo dê errado mais adiante na sequência (se reservar estoque funcionou mas debitar o cartão falhou, uma ação compensatória libera o estoque reservado). É o padrão típico para transações que atravessam microsserviços.
- **Raft**: algoritmo de **consenso**, que resolve um problema **diferente** do 2PC — não pergunta "todos aceitam efetivar esta transação?", pergunta "em que **sequência de operações** este grupo de réplicas concorda?". Funciona por **eleição de líder** (o líder ordena as operações e replica o log para os demais) e decide por **maioria** (quórum) — o que o torna **tolerante a falhas**: o sistema sobrevive à queda de uma minoria das réplicas, inclusive do próprio líder, que é substituído por uma nova eleição. É o algoritmo que sustenta o *etcd* (e, por consequência, o próprio Kubernetes).

#### 2PC x Raft — o par central desta seção

O 2PC é **commit atômico**: exige **unanimidade** e **bloqueia** se o coordenador cair (ponto único de falha). O Raft é **consenso**: decide por **maioria** e **continua funcionando** mesmo com uma minoria de réplicas fora do ar. Trocar "unanimidade" por "maioria" de um algoritmo para o outro é o erro mais comum ao descrevê-los, e afirmar que o 2PC "tolera a falha do coordenador" é falso — é exatamente a fraqueza estrutural do protocolo.

## 5.7 Tópicos complementares

### DDD — Domain-Driven Design

Abordagem de modelagem que organiza o software em torno do **domínio de negócio**, não da tecnologia. Conceitos centrais: **Aggregate** (fronteira de consistência transacional — protege as invariantes de um conjunto de entidades relacionadas, e não expõe referência direta a cada entidade interna a quem está fora do agregado), **Repository** (abstração de coleção/persistência do agregado, sem que classes externas instanciem entidades internas diretamente), **Factory** (encapsula a criação de objetos complexos do domínio), **Ubiquitous Language** (vocabulário compartilhado entre especialistas de domínio e código-fonte — o nome da classe no código deve ser o mesmo termo que o especialista de negócio usa). Eventos de domínio são tipicamente **imutáveis**, porque registram um fato que já ocorreu no passado (não faz sentido "editar" um fato passado).

### Service mesh x API gateway

**API gateway**: ponto único de entrada dos microsserviços, cuidando do tráfego **norte-sul** (de fora para dentro do sistema) — roteamento, autenticação, limitação de taxa, agregação de respostas. **Service mesh** (ex.: Istio): cuida da comunicação **serviço-a-serviço**, tráfego **leste-oeste** (dentro do sistema, entre os próprios microsserviços) — descoberta de serviço, balanceamento, retentativa, criptografia entre serviços, tudo de forma transparente à aplicação.

**Kubernetes: taint e toleration**

Funcionam de um jeito contraintuitivo em relação ao nome. O **taint** é aplicado ao **nó** e **afasta** pods: "não escalone nada aqui, a menos que aceite esta marca". A **toleration** é aplicada ao **pod** e diz "eu tolero essa marca", tornando o pod **elegível** para aquele nó — útil para reservar nós especiais (com GPU, banco de dados, ou o nó mestre) só para cargas de trabalho específicas. Tolerar **não é atrair**: um pod que tolera o taint **pode** ser escalonado para lá, não é **obrigado** a ir — quem efetivamente atrai um pod para um nó específico é *node affinity/nodeSelector*, um mecanismo diferente.

**IaC — Infraestrutura como Código**

Provisiona ambiente de infraestrutura de forma **declarativa** (descreve o estado desejado, não o passo a passo) e **versionada** (o ambiente vira arquivo de texto sob controle de versão, revisável e reproduzível). Ferramentas típicas: Terraform, Ansible.

## Capítulo 6

# Banco de Dados

### Ficha do edital

Modelagem de dados (conceitual, lógica e física); abordagem relacional e multidimensional; normalização; integridade referencial; metadados; modelagem dimensional; SQL; DDL; DML; SGBD; propriedades de banco de dados; NoSQL; banco de dados em memória; *data lakes* e soluções para big data; dados estruturados e não estruturados; avaliação de modelos de dados; técnicas de integração e ingestão (ETL/ELT, transferência de arquivos, integração via base de dados).

## 6.1 Modelagem em três níveis

### A mesma pergunta, respondida três vezes, com liberdade decrescente

Modelar um banco é responder três vezes à mesma pergunta de negócio, cada vez com **menos** liberdade de escolha. *O que existe no mundo do negócio?* — existem servidores, e cada servidor tem dependentes. *Como isso vira estrutura de dados?* — vira duas tabelas, ligadas por uma chave. *Como isso roda neste produto específico?* — vira um `CREATE TABLE` com tipos concretos, um índice na coluna de busca e uma escolha de armazenamento.

A separação não é burocracia: os três níveis **mudam em ritmos diferentes**. A regra "todo dependente pertence a um servidor" sobrevive à troca do SGBD Oracle pelo PostgreSQL; o tipo `NUMBER(5)` específico do Oracle não sobrevive. Quem mistura os níveis (decide tipo de coluna enquanto ainda discute com o usuário o que é uma entidade) acaba redesenhando o negócio toda vez que troca de infraestrutura.

| Nível      | O que é                                                           | Artefato                                 |
|------------|-------------------------------------------------------------------|------------------------------------------|
| Conceitual | visão de negócio, independente de SGBD                            | Modelo Entidade-Relacionamento (MER/DER) |
| Lógico     | estrutura no modelo de dados escolhido (relacional, documento...) | tabelas, chaves, tipos genéricos         |
| Físico     | implementação no SGBD específico                                  | DDL, índices, particionamento            |

O critério para saber em que nível uma decisão pertence é **o que já foi decidido até ali**: o **conceitual** decide entidades, atributos e relacionamentos — não sabe o que é tabela nem chave estrangeira; é o nível discutido **com o usuário de negócio**, sem jargão técnico. O **lógico** já escolheu o *paradigma* (relacional, documento, grafo) e traduz o conceitual para ele — entidades viram tabelas, relacionamentos viram chaves estrangeiras ou tabelas associativas — mas ainda **não** escolheu o produto/SGBD. O **físico** já escolheu o produto e decide o que só

existe ali dentro: tipo exato de coluna, índice, partição, *tablespace* — é o único nível em que "desempenho" é uma resposta legítima à pergunta "por que essa decisão".

A informação **só se acrescenta** de um nível para o outro: cada nível carrega tudo que o anterior decidiu e soma o seu. É por isso que a ordem não é invertível — não se parte de um índice físico para descobrir qual é a entidade de negócio.

### 6.1.1 Metadados

#### Metadado é dado sobre o dado, não o dado

**Metadado** descreve a estrutura, o significado, a origem e as regras de um dado — nunca o valor dele. Se "1988-10-05" é o dado, o metadado é "a coluna chama-se **data\_admissao**, é do tipo DATE, não aceita nulo, veio do sistema de folha de pagamento e significa a data de entrada em exercício do servidor". Divide-se em três famílias:

|                    |                                                                                                                                                                                                     |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Técnico</b>     | o que o SGBD precisa para funcionar: tabelas, colunas, tipos, chaves, índices, restrições. Mora no <b>dicionário de dados</b> (catálogo do sistema), consultável por SQL                            |
| <b>De negócio</b>  | o que a <b>pessoa</b> precisa entender: definição do termo, regra de cálculo, dono do dado ( <i>data owner</i> ), grau de sigilo. Conteúdo do <b>glossário de negócio</b>                           |
| <b>Operacional</b> | o que aconteceu com o dado ao longo do tempo: data da última carga, volume processado, erros, e a <b>linhagem</b> ( <i>data lineage</i> ) — de que fonte ele veio e por quais transformações passou |

A **linhagem** é a que mais rende em questão de cenário: quando um relatório não fecha, é ela que responde "de onde saiu este número, e o que foi feito com ele no caminho". Sem metadado operacional, um pipeline de ETL vira uma caixa-preta — ninguém consegue dizer qual regra transformou o quê.

#### Cuidado — confusão comum

Metadado **não é** o conteúdo/valor da coluna ("o CPF do servidor" é dado, não metadado) — metadado é a **descrição** desse dado. Dicionário de dados (catálogo **técnico**) e glossário de negócio (definições em linguagem de **negócio**) não são a mesma coisa.

### 6.1.2 Avaliação de modelos de dados

#### Critérios objetivos, não gosto pessoal

"O modelo está bom?" tem critérios concretos, não é opinião:

- **Completude:** o modelo cobre todos os requisitos levantados — nenhuma entidade, atributo ou regra ficou de fora.
- **Corretude:** o que está desenhado corresponde à realidade do negócio — cardinalidades, opcionalidade e domínios refletem o que realmente acontece.
- **Não redundância / grau de normalização:** cada fato mora num único lugar (ou a redundância é **intencional** e justificada, como num Data Warehouse).

- **Integridade:** chaves primárias e estrangeiras declaradas, restrições explícitas no modelo em vez de confiadas ao código da aplicação.
- **Aderência a padrão:** convenção de nomes, notação e tipos uniformes — o que permite outra pessoa ler o modelo sem um tradutor.
- **Legibilidade e manutenibilidade:** o modelo se entende sem quem o desenhou por perto, e uma mudança de regra de negócio não obriga a redesenhar meia base.

O corte que decide entre os dois primeiros critérios: **completude** pergunta "falta algo?"; **corretude** pergunta "o que está lá está certo?". Um modelo pode ser **completo e errado** (tem tudo, mas com uma cardinalidade invertida) ou **correto e incompleto** (o que está desenhado está certo, mas faltou modelar o requisito de auditoria).

## 6.2 Modelo relacional e integridade

### A ideia central de Codd (1970) e sua consequência inevitável

O modelo relacional tem uma ideia central: **todo dado é uma tabela, e a única forma de ligar duas tabelas é o valor de um campo coincidir com o valor de outro** — não há ponteiro, não há endereço de memória, não há ordem garantida entre linhas. A consequência inevitável: se o vínculo é por **valor**, alguém precisa **garantir que o valor referenciado exista**, senão a ligação aponta para o nada. É disso que trata a integridade referencial.

Caso concreto: se `DEPENDENTE.servidor_id` vale 47 e não existe servidor com id 47, aquela linha é lixo — nenhuma consulta com `JOIN` vai trazê-la de volta corretamente, porque não há do outro lado quem ela referencia. A restrição de integridade referencial é o que **impede essa linha de nascer** no banco.

### Vocabulário básico

- **Chave primária (PK):** identifica unicamente cada tupla (linha); não pode ser nula, precisa ser única.
- **Chave estrangeira (FK):** referencia a PK de outra tabela (ou, em casos recursivos, da mesma tabela).
- **Integridade referencial:** toda FK aponta para uma PK que existe, ou é nula.
- **Integridade de entidade:** a PK nunca pode ser nula.
- **Álgebra relacional:** seleção ( $\sigma$ ) corresponde ao `WHERE` do SQL e filtra **linhas**; projeção ( $\pi$ ) filtra **colunas**.

### As ações referenciais: o que fazer quando o "pai" é apagado

A pergunta que `ON DELETE` responde: apagaram o servidor 47, e existem dependentes apontando para ele. O que acontece com os "dependentes órfãos"?

| Ação        | O que o SGBD faz                                                                                                                                                |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CASCADE     | apaga (ou atualiza) os filhos junto — o dependente some com o servidor                                                                                          |
| SET NULL    | mantém o filho e zera a FK — exige que a coluna aceite NULL                                                                                                     |
| SET DEFAULT | mantém o filho e aponta a FK para um valor padrão, que precisa existir                                                                                          |
| RESTRICT    | <b>barra</b> a operação de imediato, se existirem filhos                                                                                                        |
| NO ACTION   | barra também, mas a checagem só ocorre no <b>fim do comando</b> — permite que um passo intermediário viole a restrição, desde que o estado final esteja íntegro |

RESTRICT e NO ACTION são quase idênticos: os dois recusam a operação, e a diferença é só **quando** a verificação acontece. CASCADE é o oposto dos dois — não recusa nada, ele propaga a mudança. Uma consequência prática: CASCADE numa tabela central pode apagar em silêncio uma quantidade grande de dados a partir de um único DELETE.

### 6.2.1 Do MER ao relacional: como cada cardinalidade vira tabela

| Cardinalidade | Implementação no modelo relacional                                                                          |
|---------------|-------------------------------------------------------------------------------------------------------------|
| 1:1           | FK em uma das duas tabelas (de preferência no lado de participação obrigatória), com restrição de unicidade |
| 1:N           | <b>FK no lado N</b> — a tabela "muitos" guarda a chave da tabela "um"                                       |
| N:M           | <b>tabela associativa</b> (intermediária), com as FKs das duas pontas formando a chave primária composta    |

#### Por que N:M precisa de uma terceira tabela

Um ALUNO pode se matricular em vários CURSOS, e um CURSO tem vários ALUNOs. Se a FK ficasse dentro de ALUNO (apontando para um curso só), cada aluno só poderia estar em um curso — perderia a possibilidade de matricular-se em vários. A solução é criar MATRICULA(aluno\_id, curso\_id), com as duas FKs formando a chave primária composta — cada linha dessa tabela representa uma combinação aluno-curso específica.

#### Cuidado — confusão comum

N:M **nunca** se resolve com FK direta numa das tabelas (isso só atende 1:N) nem com coluna multivalorada dos dois lados — coluna multivalorada viola a 1FN (Seção 6.3).

### 6.2.2 Entidade fraca x entidade associativa

**Entidade fraca:** não se identifica sozinha. Sua chave é a chave da **entidade proprietária** (forte) somada a uma **chave parcial** (discriminador) — por exemplo, **DEPENDENTE** só é identificado dentro do cadastro de um **SERVIDOR** específico, pelo nome. O relacionamento com o proprietário é **identificador**, e a existência da entidade fraca depende da entidade forte.

**Entidade associativa:** conceito diferente — é o **relacionamento N:M promovido a entidade**, usado quando esse relacionamento precisa, ele próprio, se relacionar com uma terceira entidade ou ganhar atributos próprios (por exemplo, **MATRICULA** da seção anterior, se ganhar o atributo **nota**, vira uma entidade associativa).

#### Cuidado — confusão comum

Entidade fraca **não dispensa** a chave do proprietário — é exatamente o oposto: sem ela, a entidade fraca não se identifica sozinha. Ter atributos próprios **não** torna uma entidade "forte"; o que define força é conseguir se identificar sozinha, sem depender da chave de outra entidade.

## 6.3 Normalização

### O problema real que a normalização resolve: anomalias

Normalizar não é obediência a uma regra estética — resolve um problema concreto chamado **anomalia**, que aparece sempre que uma tabela guarda **dois assuntos ao mesmo tempo**.

### Dependência funcional — a ferramenta que sustenta tudo

Escreve-se  $A \rightarrow B$  e lê-se "**A determina B**": dado um valor de A, existe **um único** valor de B possível.  $CPF \rightarrow nome$  é dependência funcional — um CPF corresponde a um único nome.  $nome \rightarrow CPF$  **não** é — homônimos existem, o mesmo nome pode ter CPFs diferentes.

Toda forma normal é uma restrição sobre **quais** dependências funcionais uma tabela tem o direito de conter. A chave determina todo o resto — isso é literalmente o que significa "ser chave". O que a normalização proíbe, forma a forma, é que **algo além da chave inteira** determine outra coisa:

- determinar a partir de **parte** da chave (não da chave inteira)  $\rightarrow$  viola a **2FN** (dependência parcial) — só é possível quando a chave é composta, por isso a 2FN não tem o que fazer numa tabela de chave simples;
- determinar a partir de um atributo **não-chave** (um atributo comum determinando outro)  $\rightarrow$  viola a **3FN** (dependência transitiva);
- determinar sendo algo que **não é chave candidata**  $\rightarrow$  viola a **BCNF** (forma normal de Boyce-Codd).

As três são a mesma frase ficando progressivamente mais rigorosa. Por isso a ordem é cumulativa: quem está em 3FN já está automaticamente em 2FN e em 1FN.

| FN   | Elimina                                                                             |
|------|-------------------------------------------------------------------------------------|
| 1FN  | atributos multivalorados / não atômicos (cada célula, um valor só)                  |
| 2FN  | dependência parcial da chave (só é relevante com PK composta)                       |
| 3FN  | dependência transitiva (atributo não-chave que depende de outro atributo não-chave) |
| BCNF | anomalia residual quando existe mais de uma chave candidata sobreposta              |

Normalizar **reduz redundância**; **desnormalizar de propósito** (caso típico: Data Warehouse, Capítulo 7) é aceitável quando o objetivo é desempenho de **leitura** — nunca de escrita.

### 6.3.1 Decomposição passo a passo: um exemplo completo

#### De uma tabela com anomalias a três tabelas em 3FN

Uma tabela de matrículas mal projetada, guardando aluno, curso, nota, nome e departamento do aluno, e chefe do departamento, tudo junto:

| aluno | curso | nota | nome_aluno | depto      | chefe_depto |
|-------|-------|------|------------|------------|-------------|
| 101   | BD01  | 8,5  | Ana        | Computação | Prof. Silva |
| 101   | ES02  | 7,0  | Ana        | Computação | Prof. Silva |
| 102   | BD01  | 9,0  | Bruno      | Computação | Prof. Silva |

A chave é composta: **(aluno, curso)** — é o par que identifica uma matrícula específica. As dependências funcionais presentes:

| Dependência               | Status                                                                |
|---------------------------|-----------------------------------------------------------------------|
| (aluno, curso) → nota     | legítima: depende da chave <b>inteira</b>                             |
| aluno → nome_aluno, depto | <b>parcial</b> : depende de só <b>metade</b> da chave (viola 2FN)     |
| depto → chefe_depto       | <b>transitiva</b> : um atributo não-chave determina outro (viola 3FN) |

**As três anomalias que isso causa:** (1) **de atualização** — o chefe da Computação muda; ele está repetido em toda linha de todo aluno do departamento, e se um `UPDATE` esquecer uma linha, a tabela passa a afirmar duas coisas contraditórias ao mesmo tempo; (2) **de inserção** — criou-se um departamento novo, ainda sem aluno nenhum; não há onde guardá-lo, porque a chave exige (aluno, curso) e não existe aluno; (3) **de exclusão** — o aluno 102 trancou e sua única matrícula foi apagada; junto com ela sumiu o próprio registro de que Bruno existe.

**Decompondo, uma forma normal por vez.** Já está em **1FN** (toda célula tem um valor só). Para **2FN**, elimina-se a dependência parcial: nome e departamento do aluno não dependem de *curso*, dependem só do *aluno* — separa-se uma tabela:

```
MATRICULA(aluno, curso, nota)
ALUNO(aluno, nome_aluno, depto, chefe_depto)
```

Para **3FN**, elimina-se a transitiva: dentro de **ALUNO**, o chefe não depende diretamente do aluno, depende do *departamento*, que por sua vez depende do aluno — separa-se mais uma:

```
MATRICULA(aluno, curso, nota)
ALUNO(aluno, nome_aluno, depto)
DEPARTAMENTO(depto, chefe_depto)
```

Cada fato passou a morar num único lugar, e as três anomalias desaparecem junto: trocar o chefe vira um **UPDATE** numa única linha; cadastrar um departamento novo não exige inventar um aluno; apagar a última matrícula de alguém não apaga o registro do aluno.

### Achar a chave a partir das dependências funcionais

A FGV costuma cobrar assim: "considere  $R(A, B, C, D, E)$  e  $F = \{A \rightarrow B, BC \rightarrow DE, \dots\}$ ; qual a forma normal mais alta?". O procedimento é mecânico: parta dos atributos que **nunca aparecem do lado direito** de nenhuma dependência — eles obrigatoriamente fazem parte de toda chave candidata. Feche esse conjunto aplicando as dependências até parar de crescer; se ele alcançar todos os atributos da relação, é chave. Só então classifique cada dependência restante como parcial, transitiva, ou fora de chave candidata — a forma normal mais alta é a última que **nenhuma** dependência viola.

## 6.4 SQL: DDL x DML x DCL x TCL

### A divisão é sobre o que cada grupo faz com a transação

DDL muda a **estrutura** do banco e, na maioria dos SGBD, confirma sozinha (*auto-commit*) — não se desfaz um **DROP TABLE** com **ROLLBACK**. DML muda os **dados** dentro de uma transação, e por isso é desfazível. É essa diferença de comportamento transacional — não a semelhança do efeito visível — que separa **TRUNCATE** (DDL) de **DELETE** (DML), mesmo que os dois "esvaziem a tabela".

| Sublinguagem      | Comandos                             | Observação                                                                  |
|-------------------|--------------------------------------|-----------------------------------------------------------------------------|
| DDL (definição)   | CREATE, ALTER, <b>DROP, TRUNCATE</b> | <b>TRUNCATE</b> é DDL (não DML), <i>auto-commit</i>                         |
| DML (manipulação) | SELECT, INSERT, UPDATE, DELETE       | <b>DELETE</b> é DML, aceita <b>WHERE</b> e é desfazível com <b>ROLLBACK</b> |
| DCL (controle)    | GRANT, REVOKE                        | gerencia permissões                                                         |
| TCL (transação)   | COMMIT, ROLLBACK, SAVEPOINT          | controla o ciclo de vida da transação                                       |

### A ordem lógica de execução, e o que ela explica

Você escreve o **SELECT** primeiro na sintaxe, mas o banco o resolve quase por último:

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

Duas consequências práticas que decorrem diretamente dessa ordem:

- O **WHERE** não enxerga um apelido definido no **SELECT**. Em `SELECT preco*2 AS dobro ... WHERE dobro > 100`, o **WHERE** roda **antes** de `dobro` existir — o banco acusa erro de coluna inexistente. Já `ORDER BY dobro` funciona normalmente, porque **ORDER BY** roda depois do **SELECT**.
- **WHERE** filtra linha; **HAVING** filtra grupo. O **WHERE** acontece **antes** do **GROUP BY**, quando nenhum grupo existe ainda; o **HAVING** acontece depois, e por isso é o único que pode comparar com o resultado de `COUNT(*)` ou `SUM()`.

#### Cuidado — confusão comum

**UNION** remove duplicatas; **UNION ALL** mantém todas as linhas, inclusive repetidas. **JOIN** sem cláusula **ON** produz um **produto cartesiano** (cada linha de uma tabela combinada com todas as linhas da outra).

### 6.4.1 VIEW, GRANT e controle de transação

**VIEW (visão)**: uma **tabela virtual** — é a consulta guardada com um nome, não os dados copiados fisicamente. Consultar a visão executa o **SELECT** que está por trás dela, então o resultado é **sempre atual**. Serve para simplificar uma consulta complexa e para **restringir acesso** (a visão expõe só as colunas/linhas permitidas, e o **GRANT** é dado sobre a visão, não sobre a tabela inteira). Uma visão **simples** costuma aceitar atualização; uma visão com **JOIN**, agregação ou **DISTINCT**, geralmente não. A **materialized view** é a exceção: ela **sim** armazena o resultado fisicamente em disco, e precisa ser **atualizada** (*refresh*) periodicamente para refletir mudanças na origem.

**GRANT/REVOKE (DCL)**: `GRANT SELECT, INSERT ON tabela TO usuario` concede privilégios; `REVOKE` os retira. Com `WITH GRANT OPTION`, quem recebeu o privilégio pode **repassá-lo** a terceiros.

**SAVEPOINT e ROLLBACK TO (TCL)**: `SAVEPOINT nome` marca um ponto **dentro** de uma transação em andamento; `ROLLBACK TO nome` desfaz só o que veio **depois** daquela marca, e a transação **continua aberta** — não equivale a um `COMMIT` nem a um `ROLLBACK` total. É um desfazer parcial.

#### Cuidado — confusão comum

Uma visão comum **não armazena** dados fisicamente (isso é exclusivo da *materialized view*). **GRANT** é **DCL**, não DDL nem DML. `ROLLBACK TO` **não encerra** a transação nem desfaz tudo — ele volta ao *savepoint* indicado, e a transação segue viva.

### 6.4.2 Notação Crow's Foot e chave surrogada

#### Crow's Foot: dois símbolos por ponta, dois significados

Cada ponta de uma linha de relacionamento traz **dois** símbolos, e a posição decide o papel: o que **encosta na entidade** é o **máximo** (a cardinalidade propriamente dita — um ou muitos); o símbolo **mais afastado** é o **mínimo** (a modalidade — zero ou um).

| Símbolo                     | Leitura                                    |
|-----------------------------|--------------------------------------------|
| Traço (barra)               | exatamente <b>um</b> (mínimo 1 / máximo 1) |
| Círculo                     | <b>zero</b> — opcional                     |
| Pé de galinha (três riscos) | <b>muitos</b>                              |
| Círculo + pé de galinha     | <b>zero ou muitos</b>                      |
| Traço + pé de galinha       | <b>um ou muitos</b> (obrigatório)          |

Tradução direta para DDL: o lado com **pé de galinha** é o lado **N**, e é nele que entra a **FK**; o círculo indica participação **opcional** (a FK aceita NULL); o traço indica participação **obrigatória** (NOT NULL).

### Chave surrogada x chave natural

**Chave surrogada** (substituta/artificial): chave primária **sem significado de negócio**, gerada pelo sistema (sequência, *identity*, UUID) — em oposição à **chave natural**, que vem do próprio domínio (CPF, matrícula). Vantagens da surrogada: é **estável** (um dado de negócio pode mudar — CPF corrigido, matrícula reemitida — sem exigir alterar a chave e todas as FKs que a referenciam), curta e uniforme para *join*. É padrão em Data Warehouse, onde também permite guardar **versões históricas** da mesma entidade de negócio (dimensão de mudança lenta, Capítulo 7).

### Cuidado — confusão comum

Chave surrogada **não substitui** a necessidade da chave natural na tabela — o campo de negócio continua presente, em geral com restrição **UNIQUE**. Em Crow's Foot, a inversão comum é ler a cardinalidade do lado errado da linha — o símbolo vale para a entidade que ele **toca**, não para a entidade do outro lado.

### 6.4.3 Gatilho x procedimento armazenado

|                | Gatilho ( <i>trigger</i> )                                                            | Procedimento armazenado                                     |
|----------------|---------------------------------------------------------------------------------------|-------------------------------------------------------------|
| Quem dispara   | o <b>próprio SGBD</b> , em resposta a um <b>evento</b> de dados                       | a <b>aplicação</b> (ou outro código), por chamada explícita |
| Como se invoca | não se invoca — é automático                                                          | <b>CALL/ EXECUTE</b>                                        |
| Amarrado a     | uma tabela + evento (INSERT, UPDATE, DELETE)                                          | nada específico; é código nomeado e reutilizável            |
| Parâmetros     | não recebe                                                                            | recebe (e pode retornar)                                    |
| Bom para       | auditoria, log, regra que <b>não pode depender</b> de a aplicação lembrar de chamá-la | rotina de negócio chamada sob demanda                       |

Palavra-chave para reconhecer o gatilho num enunciado: "**sem depender de a aplicação lembrar**".

## 6.5 Propriedades ACID

### As quatro letras, na transferência de R\$ 100 de A para B

Uma transação é um conjunto de operações que o banco trata como **uma só**. Debitar R\$ 100 da conta A e creditar R\$ 100 na conta B são dois **UPDATEs**, e nenhum estado intermediário entre eles é aceitável.

- **Atomicidade** — *tudo ou nada*. Se o servidor cair entre o débito e o crédito, o débito é desfeito. Nunca existe "saiu de A e não chegou em B". Quem entrega: **ROLLBACK** e o log de transações.
- **Consistência** — *estado válido antes, estado válido depois*. A soma dos saldos não mudou, e nenhuma restrição declarada (**CHECK**, chave estrangeira, **UNIQUE**) foi violada. Consistência é sobre **as regras do banco**, não sobre concorrência entre transações.
- **Isolamento** — *uma transação não enxerga o meio da outra*. Se um extrato rodar no exato instante da transferência, ele não pode ver o dinheiro debitado e ainda não creditado. É a letra mais cobrada porque é a única que tem **graus** (Seção seguinte).
- **Durabilidade** — *confirmado é para sempre*. Depois do **COMMIT**, o efeito sobrevive a uma queda de energia. Quem entrega: a gravação do log em disco **antes** da confirmação (**WAL**, *write-ahead log*).

### Cuidado — confusão comum

Consistência x isolamento se confundem: a primeira olha para as **regras declaradas** do banco; a segunda olha para **outras transações** concorrentes. Não confundir com o "C" do teorema CAP (Seção 6.8.1) — lá, consistência significa que todas as réplicas mostram o mesmo dado, um conceito diferente.

## 6.6 Concorrência: níveis de isolamento

### Quatro níveis, cada um admitindo certos fenômenos

O "I" do ACID não é tudo-ou-nada: o padrão SQL define quatro níveis, e cada um **admite** certos fenômenos de concorrência. Quanto maior o isolamento, **menor** a concorrência possível — é uma troca (*trade-off*), não um ganho de graça.

**Os três fenômenos:** **leitura suja** (*dirty read*): lê um dado alterado por outra transação que **ainda não confirmou** (e pode desfazer depois). **Leitura não repetível** (*non-repeatable read*): relê a **mesma linha** e o valor mudou, porque outra transação a atualizou e confirmou no meio do caminho. **Leitura fantasma** (*phantom read*): relê a **mesma faixa** de resultados e aparecem **linhas novas**, inseridas e confirmadas por outra transação.

| Nível            | Leitura suja  | Não repetível | Fantasma      |
|------------------|---------------|---------------|---------------|
| READ UNCOMMITTED | admite        | admite        | admite        |
| READ COMMITTED   | <b>impede</b> | admite        | admite        |
| REPEATABLE READ  | impede        | <b>impede</b> | admite        |
| SERIALIZABLE     | impede        | impede        | <b>impede</b> |

#### Cuidado — confusão comum

READ UNCOMMITTED é o **mais permissivo** (não o mais restritivo) — é justamente o que deixa ler dado ainda não confirmado. REPEATABLE READ elimina a leitura **não repetível**, mas **não** elimina a fantasma (que só é eliminada no SERIALIZABLE). Quanto mais alto o isolamento, **menor** a concorrência — nunca o contrário. Para diferenciar os dois últimos fenômenos: linha que **muda de valor** → não repetível; linha **nova que aparece** → fantasma.

## 6.7 Relacional x multidimensional: OLTP x OLAP

#### Duas cargas de trabalho opostas exigem dois modelos opostos

O sistema que registra uma venda precisa gravar **uma linha por vez, muitas vezes por segundo**, sem perder nada — a normalização serve bem a ele, porque cada fato mora num único lugar e a escrita fica barata. O sistema que responde "quanto vendemos por região no último trimestre" precisa **varrer milhões de linhas e somar** — e aí a normalização vira inimiga, porque cada JOIN a mais é trabalho a mais numa consulta que já é pesada.

Rodar as duas cargas na mesma base é o erro clássico: o relatório trava a produção. Separar em OLTP e OLAP é a resposta — e é isso que justifica o Data Warehouse existir como uma cópia desnormalizada dos dados, em vez de ser um capricho de arquitetura.

|                 | OLTP (transacional)    | OLAP (analítico)           |
|-----------------|------------------------|----------------------------|
| Uso             | operações do dia a dia | apoio à decisão / análise  |
| Escrita/leitura | muitas escritas curtas | leitura pesada, agregação  |
| Modelo          | relacional normalizado | multidimensional (cubos)   |
| Estrutura       | tabelas                | dimensões e métricas/fatos |

## 6.8 NoSQL e novos armazenamentos

#### Quatro famílias de NoSQL, e o que cada uma otimiza

NoSQL prioriza alta disponibilidade e **escalabilidade horizontal**, e costuma relaxar as garantias ACID em favor do modelo **BASE** (*Basically Available, Soft state, Eventual consistency*).

- **Chave-valor** (Redis): busca extremamente rápida por uma chave, sem estrutura interna imposta ao valor.

- **Documento** (MongoDB): documentos semiestruturados (tipo JSON), bom para dados que variam de forma entre registros.
- **Colunar** (Cassandra): otimizado para escrita massiva e leitura por coluna em grandes volumes.
- **Grafo** (Neo4j): nós e arestas, otimizado para percorrer **relacionamentos** complexos (rede social, detecção de fraude).

Banco NoSQL **não** é "sempre grafo" e não segue ACID estritamente por padrão — e não substitui um ERP/CRM fortemente transacional sem uma análise cuidadosa do que se perde.

### 6.8.1 Teorema CAP

#### Por que o CAP obriga a escolher: o cenário do cabo cortado

Duas réplicas do mesmo banco, uma no Rio e outra em Natal. O cabo de rede entre elas cai — isso é a **partição** (P). Chega uma escrita no Rio. O sistema tem exatamente duas saídas:

- **Aceitar a escrita**: o Rio responde ao usuário, mas Natal segue com o dado antigo — as réplicas divergiram. Abriu-se mão de **Consistência** para manter **Availability**: perfil **AP**.
- **Recusar a escrita** (ou travar até o cabo voltar): ninguém lê dado divergente, mas o serviço ficou indisponível para aquela operação. Abriu-se mão de **A** para manter **C**: perfil **CP**.

Não existe terceira saída — é por isso que é um teorema, não uma recomendação. "Escolha 2 de 3" é uma simplificação didática: numa rede real, a partição **não é opcional** (cabo cai eventualmente); P está sempre presente, e a escolha verdadeira é sempre entre **C ou A**. Um banco monolítico rodando num servidor só **não é "CA"** — ele simplesmente não é um sistema distribuído, e o teorema não se aplica a ele.

#### Regra prática

Pergunte **o que dói mais** no cenário descrito. Saldo bancário, sentença judicial, registro oficial, estoque de item único → dado errado é inaceitável → **CP**. Carrinho de compras, curta vida, *feed* social, telemetria → ficar fora do ar é o inaceitável, e a divergência se resolve depois → **AP**.

### 6.8.2 Data Lake x Data Warehouse x Lakehouse

**Data Lake**: guarda dado **bruto** (*schema-on-read* — a estrutura só é imposta na hora de ler, não na hora de gravar), aceita dado estruturado e não estruturado. **Data Warehouse**: guarda dado **tratado e modelado** (*schema-on-write* — a estrutura é definida antes de gravar). **Lakehouse**: combina os dois — a flexibilidade de ingestão do lake com as garantias de estrutura e qualidade do warehouse.

## 6.9 ETL x ELT

### A posição do "T" muda tudo

|               | ETL                                  | ELT                                                        |
|---------------|--------------------------------------|------------------------------------------------------------|
| Ordem         | Extrai → <b>Transforma</b> → Carrega | Extrai → Carrega → <b>Transforma</b>                       |
| Transformação | antes de carregar, fora do destino   | depois de carregar, dentro do próprio destino              |
| Bom para      | Data Warehouse tradicional           | destino com muito poder de processamento (nuvem, big data) |

ELT aproveita o processamento do próprio destino, o que o torna melhor para **grande** volume (não o contrário); ETL faz mais sentido quando a transformação precisa ocorrer antes/fora do destino, por limitação de processamento lá.

## 6.10 Índices

### O que um índice é, e por que existem tipos diferentes

Um índice é uma **estrutura separada da tabela**, que guarda os valores de uma coluna já **organizados**, junto com o endereço da linha correspondente. Não muda a tabela em si: acrescenta um caminho de acesso a mais. A troca é sempre a mesma — **leitura mais rápida, escrita mais lenta** (todo INSERT precisa atualizar o índice também) e mais espaço em disco. Índice não é grátis, e "criar índice em tudo" nunca é a resposta certa.

- **B+ Tree**: árvore balanceada com os valores **ordenados** nas folhas, encadeadas entre si. Como está ordenado, atende faixa (BETWEEN, >, <), ORDER BY e igualdade. É o padrão em todo SGBD, ideal para colunas de **alta cardinalidade** (muitos valores distintos: CPF, data, valor monetário).
- **Hash**: aplica uma função ao valor e vai direto ao "balde" correspondente. Ótimo para = e **inútil** para faixa — o hash embaralha a ordem de propósito, então "entre 10 e 20" não significa nada dentro dele.
- **Bitmap**: um vetor de bits por valor distinto. Só compensa em **baixa cardinalidade** (ex.: coluna ativo/inativo, ou UF com 27 valores), porque o número de vetores é o número de valores distintos. Combina vários filtros com operações lógicas de bits rapidamente — excelente em consulta analítica, ruim em tabela com muita escrita concorrente.

As duas perguntas que decidem o tipo certo: *a consulta pede faixa ou igualdade?* e *quantos valores distintos a coluna tem?*

## 6.11 Big Data: os "V"s

### O trio clássico, e o que veio depois

O trio **original** (Gartner, 2001) é **Volume, Velocidade e Variedade**. **Veracidade** e **Valor** são **extensões** posteriores (por isso os modelos "4V" e "5V"), assim como **Variabilidade**. Saber qual é o trio original importa porque a pergunta clássica de prova troca um membro do trio por um V de extensão ("volume, velocidade e valor" ou "volume, variedade e veracidade" soam plausíveis mas não são o trio original).

## Capítulo 7

# Business Intelligence (BI)

### Ficha do edital

Conceitos, fundamentos, técnicas e métodos de BI; sistemas de suporte à decisão (SSD) e gestão de conteúdo; arquitetura e aplicações de data warehouse com ETL e OLAP; data warehouse e data mining; visualização (BD individuais e cubos); mapeamento das fontes de dados; arquitetura de BI.

## 7.1 O que é BI, e por que não basta consultar o banco transacional

### A pergunta que fundou o assunto

BI é o conjunto de conceitos e ferramentas para transformar **dado** em **informação** e apoiar a **tomada de decisão** — a pirâmide clássica é dado → informação → conhecimento → inteligência/sabedoria, cada nível agregando contexto e interpretação ao anterior.

A pergunta concreta que justifica o assunto existir: *por que não rodar o relatório direto no banco do sistema transacional?* Três motivos:

- **Concorrência:** uma consulta que varre cinco anos de vendas segura recursos que o sistema transacional precisa para atender o caixa **agora** — rodar os dois no mesmo banco compete pelo mesmo recurso.
- **História:** o banco transacional guarda o estado **atual** — ele sabe onde o cliente mora **hoje**, não onde morava quando fez a compra que se quer analisar.
- **Integração** (o motivo mais caro de resolver): os dados relevantes estão espalhados em vários sistemas que discordam entre si — a mesma unidade organizacional com dois códigos diferentes em dois sistemas, a mesma data em dois formatos.

O BI existe para resolver os três de uma vez, e a arquitetura descrita a seguir é a solução estruturada para isso.

### Arquitetura de BI: o caminho do dado, ponta a ponta

fontes (OLTP, planilhas, externas) → **extração** → *staging* → **transformação** → **DW** (e seus *data marts*) → **OLAP/cubo** → visualização

Cada etapa existe por um motivo específico: a **extração** lê das fontes sem incomodá-las mais do que o necessário (janela de carga, leitura incremental do que mudou desde a última vez). A **staging area** é o pouso intermediário — dado bruto já fora da fonte, ainda não tratado — e existe para que a fonte seja liberada rapidamente e para que o tratamento possa ser refeito sem precisar consultar a fonte de novo. A **transformação** é onde a integração de fato acontece — é a etapa cara do processo, não a simples cópia. O **DW** guarda o resultado: integrado, histórico e **não volátil** (não se atualiza um fato passado; acrescenta-se um novo). **OLAP e visualização** são **consumo** — nenhum dos dois transforma dado; quem transforma é o ETL, e essa separação de responsabilidade é a resposta de vários itens de prova sobre "em que etapa isso acontece".

O **data mart** é o recorte do DW por assunto ou departamento (vendas, RH) — não é uma cópia menor por economia, é a entrega organizada no vocabulário de quem vai efetivamente usar aquele recorte.

## 7.2 Data Warehouse

### As quatro características de Inmon

Repositório central de dados **integrados, históricos e orientados a assunto**, destinado a análise, não a transação. As quatro características canônicas (Inmon): **orientado a assunto** (organizado em torno de temas de negócio, como "vendas" ou "cliente", não em torno de processos de sistema), **integrado** (dados de fontes diferentes reconciliados num formato e vocabulário únicos), **não volátil** (dado carregado não é alterado nem apagado, só acrescentado), **variante no tempo** (cada registro carrega uma dimensão temporal, permitindo reconstruir o estado passado).

### Data Warehouse x Data Lake x Lakehouse

**DW**: guarda dado **tratado e modelado** (*schema-on-write* — a estrutura é imposta antes de gravar). **Data Lake**: guarda dado **bruto**, estruturado e não estruturado (*schema-on-read* — a estrutura só é imposta na hora de ler). **Lakehouse**: une a governança do DW com a flexibilidade do Lake, tipicamente em camadas: **Bronze** (bruto, como chegou da fonte) → **Silver** (limpo, deduplicado, com tipos corretos) → **Gold** (modelado dimensionalmente, pronto para consumo de BI). Um projeto de Machine Learning costuma consumir o *Bronze* ou *Silver* (quer o dado com o máximo de granularidade e liberdade); um dashboard de BI consome o *Gold*.

### Cuidado — confusão comum

Data Lake **não substitui** o DW, e não guarda só dado estruturado (guarda os dois tipos). Lakehouse **combina** os dois — não é sinônimo de nenhum isoladamente.

### 7.2.1 Inmon x Kimball: por onde se começa a construir

#### Duas escolas, duas ordens de construção, dois riscos opostos

As duas escolas discordam sobre **qual peça se constrói primeiro** — e essa discordância tem consequência prática direta no cronograma e no risco do projeto.

**Inmon (*top-down*)**: constrói-se primeiro o **DW corporativo**, normalizado (até a 3FN), como fonte única de verdade da organização inteira; os **data marts** dimensionais são **derivados** dele depois. Integra melhor e resiste melhor a mudanças, mas o primeiro relatório demora — a área usuária espera o modelo corporativo inteiro ficar pronto antes de ver qualquer valor.

**Kimball (*bottom-up*)**: começa-se por um **data mart dimensional** de um único processo de negócio (vendas, por exemplo), já em esquema estrela, e vão se acrescentando outros processos aos poucos. Entrega valor cedo. A integração entre os marts não vem de um modelo central — vem das **dimensões conformadas**: a mesma dimensão Produto, com a mesma chave e o mesmo significado, compartilhada por todos os marts (a *bus architecture*).

O risco de cada abordagem é o espelho do outro: em Inmon, o risco é o projeto longo que nunca chega ao usuário; em Kimball, o risco é o **silo** — marts que não conversam entre si porque as dimensões não foram conformadas de propósito. É por isso que "dimensão conformada" não é

um detalhe de vocabulário: é a peça que faz o *bottom-up* efetivamente funcionar como um todo coerente, e não como ilhas isoladas.

### 7.3 ETL: a ponte para o DW

#### O que pertence a cada fase

*Extract* → *Transform* → *Load*. O objetivo é extrair de várias fontes, transformar num formato padronizado e consistente, e carregar no DW. ETL **não** é "criar dashboards" nem "treinar modelo de aprendizado de máquina" — isso vem depois, como consumo do dado já carregado. Como quase tudo interessante acontece dentro do *Transform*, o critério para separar as fases precisa ser mais fino que "transformar é mudar o dado":

| Fase             | O que é dela                                                                                                                                                                                                                                                                                                      |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Extract</b>   | ler das fontes (banco, arquivo, API), tipicamente só o que mudou desde a última carga. <b>Sem regra de negócio</b> — ela lê, não julga o conteúdo                                                                                                                                                                 |
| <b>Transform</b> | <b>limpeza</b> (nulos, formatos, valores inválidos), <b>padronização/conformidade</b> (o "de-para" que faz dois sistemas falarem a mesma língua), <b>deduplicação</b> , <b>junção</b> de fontes diferentes, <b>derivação</b> de medidas calculadas, geração da <b>chave surrogate</b> e aplicação da regra de SCD |
| <b>Load</b>      | gravar no destino — <b>dimensões antes da fato</b> (porque a fato referencia as chaves das dimensões, então elas precisam já existir), carga completa x incremental, índices e agregados construídos no fim                                                                                                       |

A regra prática que resolve a maior parte das questões de "a que fase pertence esta tarefa": se a tarefa **decide** alguma coisa sobre o conteúdo — padronizar, deduplicar, casar duas fontes, calcular —, é **Transform**. Ler é *Extract*; gravar é *Load*.

### 7.4 OLAP e modelagem dimensional

#### Cubo: dimensões + medidas

Um **cubo** organiza os dados em **dimensões** (tempo, produto, local — os eixos pelos quais se quer olhar o dado) e **métricas/fatos** (vendas, quantidade — os números que se quer medir). Três formas de **armazenar** fisicamente o cubo: **ROLAP** (o dado fica em tabelas relacionais comuns, e o cubo é montado sob demanda por consulta — escala melhor para grandes volumes), **MOLAP** (o cubo é pré-calculado e guardado numa estrutura própria — consulta mais rápida, mas carga mais pesada e demorada) e **HOLAP** (híbrido: detalhe fica no relacional, agregados mais usados ficam pré-calculados).

#### Cuidado — confusão comum

O modelo multidimensional serve para **análise**, não para transação — quem serve à transação é o modelo relacional/OLTP. Ao lembrar de ROLAP x MOLAP: o **M** de MOLAP é de

*multidimensional*, o cubo já pré-calculado. "Ferramentas de BI são incompatíveis com tabelas normalizadas" é falso — desnormalizar é uma escolha de desempenho, não uma exigência técnica das ferramentas.

### As operações OLAP

| Operação                  | O que faz                                                            |
|---------------------------|----------------------------------------------------------------------|
| <i>Drill-down</i>         | desce ao detalhe (ano → mês → dia)                                   |
| <i>Roll-up (drill-up)</i> | sobe à agregação (dia → mês → ano)                                   |
| <i>Slice</i>              | fatia <b>uma</b> dimensão (fixa um único valor nela)                 |
| <i>Dice</i>               | subcubo — filtra em <b>várias</b> dimensões e valores ao mesmo tempo |
| <i>Pivot (rotate)</i>     | gira a visão, trocando quais dimensões aparecem em linha e em coluna |

#### 7.4.1 Star Schema x Snowflake Schema

##### A mesma decisão de desnormalizar ou não, aplicada às dimensões

**Star Schema:** uma tabela fato central + dimensões **desnormalizadas** (cada dimensão numa única tabela larga, mesmo que isso repita dado). Menos *joins*, consulta mais rápida — é o padrão recomendado por Kimball.

**Snowflake Schema:** as dimensões são **normalizadas** em várias tabelas relacionadas entre si (por exemplo, **Produto** aponta para **Categoria**, que aponta para **Departamento**). Mais *joins*, mais lento para consulta analítica, mas economiza espaço e facilita manter a integridade de hierarquias que mudam.

##### Contando joins: por que o Star ganha em velocidade

Para responder "vendas do **departamento X** no trimestre":

|                      |                                                                                                                                     |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>Estrela</b>       | <b>Produto</b> já guarda categoria e departamento como colunas suas (desnormalizada): <b>2 joins</b> — fato → Produto, fato → Tempo |
| <b>Floco de neve</b> | <b>de</b> Produto → Categoria → Departamento, cada uma em sua própria tabela: <b>4 joins</b> para chegar à mesma resposta           |

Menos *joins* → consulta mais rápida → é por isso que o esquema estrela é o padrão recomendado para BI. O floco de neve troca essa velocidade por economia de espaço e integridade de hierarquia — uma escolha legítima em certos contextos, só não é o padrão default.

##### Cuidado — confusão comum

Não inverta os nomes: "estrela" com dimensão normalizada em várias tabelas é, na verdade, floco de neve. Snowflake **não** é o padrão recomendado por Kimball — é o Star.

### 7.4.2 Montando um esquema estrela: as quatro decisões, em ordem

#### O projeto dimensional tem uma ordem fixa

1. **Escolher o processo de negócio:** por exemplo, a venda no varejo.
2. **Declarar o grão:** "uma linha por item de um pedido" — é a decisão que governa todo o resto do projeto (ver Seção 7.4.3).
3. **Escolher as dimensões:** tudo que responde a "por qual recorte eu quero olhar isto?" — Tempo, Produto, Loja, Cliente.
4. **Escolher as medidas:** o que é **numérico e somável** no grão declarado — **quantidade**, **valor\_unitário**, **valor\_total**, **desconto**.

O resultado é uma **tabela fato** que contém **só duas coisas**: as **chaves estrangeiras** para as dimensões e as **medidas numéricas**. Nenhum texto descritivo mora na fato — o nome do produto fica na dimensão **Produto**. A única exceção com "nome próprio" é a **dimensão degenerada**: o **número\_do\_pedido** fica na fato porque é um identificador que não tem atributo nenhum ao redor que justifique uma dimensão própria.

### 7.4.3 Granularidade: o grão da fato

#### O que uma linha da tabela fato representa

Definir o grão é a **primeira** decisão do projeto dimensional, e é **irreversível** na prática: tudo o mais (dimensões, medidas, agregações possíveis) é escolhido depois dela, em função dela.

| Grão                                                       | Consequência                                                                                                       |
|------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>Fino</b> (detalhado) — "uma linha por atendimento"      | <b>amplia</b> as análises possíveis (dá para agregar depois em qualquer nível desejado); volume maior de dados     |
| <b>Grosso</b> (agregado) — "uma linha por unidade por dia" | tabela menor e consulta mais rápida, mas o detalhe está <b>perdido para sempre</b> — não dá para desagregar depois |

Regra de Kimball: **declare o grão no nível mais atômico que a fonte de dados permitir**. Agregados são construídos **a partir** do detalhe, como tabelas adicionais complementares — nunca no lugar do detalhe.

#### Cuidado — confusão comum

Escolher o grão mais detalhado **amplia** as análises possíveis (não restringe) — quem restringe é o agregado. Granularidade é definida pelo que **uma linha da fato representa**, não pela quantidade de dimensões. Não confundir **grão** (decisão do modelo, fixa desde a construção) com **nível de agregação de uma consulta** (decisão de quem consulta, via *drill-up*, variável a cada consulta).

#### 7.4.4 Tipos de fato

**Fato aditivo:** soma corretamente em **todas** as dimensões (ex.: valor de vendas — somar vendas de vários dias, produtos ou lojas sempre faz sentido). **Fato semiaditivo:** **não** soma em todas as dimensões, tipicamente não soma no eixo do **tempo** (ex.: saldo bancário, nível de estoque — somar o saldo de hoje com o de ontem não produz nada útil, mas somar o saldo de várias contas num mesmo dia faz sentido). **Fato não aditivo:** razões, percentuais, médias — nunca somam de forma útil em nenhuma dimensão. **Dimensão degenerada:** atributo de identificação (número do pedido) que fica na própria tabela fato, sem ganhar uma dimensão própria.

#### 7.4.5 Dimensões lentamente mutantes (SCD)

##### O que fazer quando um atributo de dimensão muda com o tempo

Um atributo de dimensão muda (o cliente troca de cidade, o produto troca de categoria). **SCD** (*Slowly Changing Dimension*) é a técnica que decide o que fazer com o histórico correspondente:

| Tipo          | O que faz                                                                                                                            | Efeito no histórico                                                                                       |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <b>Tipo 1</b> | <b>sobrescreve</b> o valor antigo                                                                                                    | histórico <b>perdido</b> ; os fatos passados passam a ser lidos com o valor novo                          |
| <b>Tipo 2</b> | <b>insere um novo registro</b> versionado (nova chave <i>surrogate</i> , com <i>data_início/data_fim</i> ou <i>flag</i> de corrente) | <b>preserva o histórico completo</b> — cada fato continua ligado à versão vigente na época em que ocorreu |
| <b>Tipo 3</b> | guarda o <b>valor anterior em outra coluna</b> da mesma linha                                                                        | preserva só <b>uma</b> mudança (o "valor anterior"), não a série toda de mudanças                         |

É o **Tipo 2** que responde ao cenário clássico: "as vendas antigas devem continuar associadas à cidade em que o cliente morava na época da compra". E é ele que justifica a **chave surrogate** da dimensão (Capítulo 6): como a mesma chave natural passa a ter várias versões ao longo do tempo, a fato precisa apontar para uma chave artificial que identifique a **versão** específica, não apenas a entidade genérica.

##### Cuidado — confusão comum

Aplicar Tipo 1 (sobrescrever) a um cenário que pede preservação de histórico é erro; aplicar Tipo 2 a um cenário de simples **correção de erro de digitação** também é erro no sentido oposto — correção de erro não é "mudança de negócio a preservar", é sobrescrita legítima (Tipo 1). O Tipo 3 preserva histórico só do **valor imediatamente anterior**, não da série completa — é o distrator "quase certo" mais comum entre os três.

## 7.5 Sistemas de suporte à decisão (SSD/DSS)

Apoiam gestores em problemas **estruturados, semiestruturados e não estruturados** — a característica central é oferecer flexibilidade suficiente para vários contextos de decisão diferentes, não se limitar a um único tipo de problema.

## 7.6 Mapeamento de fontes de dados

### Boas práticas, referenciadas pelo DAMA-DMBOK

**Fazer:** entrevistar *stakeholders*, analisar sistemas existentes, avaliar a **qualidade** e adequação do dado antes de usá-lo, documentar as fontes como parte da governança de dados. **Não fazer:** coletar **tudo sem discriminação**, incluindo dados inconsistentes ou irrelevantes, só "para maximizar a quantidade"— essa é a prática explicitamente **não** recomendada.

## 7.7 Data Mining

### Quatro tarefas, divididas por ter ou não rótulo prévio

**Data Mining** é o processo de descobrir padrões, correlações e conhecimento não trivial em grandes volumes de dados.

| Tarefa               | Tem rótulo?                     | Cenário típico                                                                                  |
|----------------------|---------------------------------|-------------------------------------------------------------------------------------------------|
| Classificação        | <b>sim</b> (supervisionada)     | prever se o cliente vai inadimplir (classes já conhecidas de antemão)                           |
| Regressão            | <b>sim</b> (supervisionada)     | prever um <b>valor numérico</b> (faturamento do próximo mês)                                    |
| Clusterização        | <b>não</b> (não supervisionada) | <b>descobrir grupos</b> de clientes com comportamento parecido, sem rótulo definido previamente |
| Regras de associação | <b>não</b>                      | descobrir <b>o que é comprado junto</b> (análise de cesta de compras)                           |

Em regras de associação: **suporte** é a frequência do conjunto de itens no total de transações (quantas vezes A e B aparecem juntos, sobre o total); **confiança** é a força da implicação  $A \rightarrow B$  (das vezes que A aparece, em quantas B também aparece).

### Cuidado — confusão comum

Classificação x clusterização é o par mais confundido: se o enunciado diz "sem rótulos prévios" ou "descobrir grupos desconhecidos", é clusterização; se as classes já existem e o modelo aprende a atribuí-las a novos casos, é classificação. "Produtos comprados juntos" é sempre **regra de**

**associação**, nunca clusterização, mesmo que ambas sejam técnicas não supervisionadas.

### 7.7.1 CRISP-DM: o ciclo de um projeto de mineração

Seis fases, iterativas, começando pelo negócio

|   | Fase                           | O que acontece                                                                               |
|---|--------------------------------|----------------------------------------------------------------------------------------------|
| 1 | Entendimento do <b>negócio</b> | objetivos e critérios de sucesso do negócio — é por aqui que o ciclo começa, não pelos dados |
| 2 | Entendimento dos <b>dados</b>  | coleta inicial, exploração, avaliação de qualidade                                           |
| 3 | <b>Preparação</b> dos dados    | limpeza, seleção, integração, formatação                                                     |
| 4 | <b>Modelagem</b>               | escolha da técnica, treino, ajuste de parâmetros                                             |
| 5 | <b>Avaliação</b>               | os resultados atendem aos <b>objetivos de negócio</b> definidos na fase 1?                   |
| 6 | <b>Implantação</b>             | colocar o modelo em operação e monitorá-lo                                                   |

O ciclo é **iterativo**, não uma cascata rígida: volta-se de qualquer fase para uma anterior conforme necessário, e a seta entre implantação e entendimento do negócio fecha o círculo, reiniciando o ciclo com o aprendizado acumulado.

#### Cuidado — confusão comum

Não confundir a **avaliação** (fase 5, mede contra os objetivos de **negócio**) com a validação técnica do modelo (métricas estatísticas), que é parte da **modelagem** (fase 4). O ciclo não começa pelo entendimento **dos dados** — começa pelo **negócio**.

#### KPI x métrica

Toda métrica é um número medido; **KPI** (*Key Performance Indicator*) é uma métrica **especificamente atrelada a um objetivo estratégico** da organização — nem toda métrica é KPI, mas todo KPI é uma métrica.

## Capítulo 8

# Segurança da Informação

### Ficha do edital

Políticas e procedimentos de segurança; ISO/IEC 27001:2022 e 27002:2022; confidencialidade, integridade e disponibilidade; mecanismos de segurança (controle de acesso, OAuth2, SSO); gestão de riscos (ameaça, vulnerabilidade, impacto); SDL e OWASP Top 10; análise estática e dinâmica de código (SAST e DAST). Também: HTTPS, SSL/TLS.

## 8.1 A tríade CIA: o critério que organiza tudo

### Cada controle existe para sustentar um dos três pilares

A tríade não é uma lista decorativa de três palavras — é o **critério de classificação** de todo o resto do capítulo. Cada controle (cifra, hash, backup, RBAC, WAF) existe para sustentar um dos três pilares, e a pergunta que resolve metade das questões do bloco é sempre a mesma: *qual pilar este controle protege?*

- **Confidencialidade:** só quem tem direito acessa/**vê** o dado (cifra, controle de acesso, mascaramento).
- **Integridade:** o dado permanece **íntegro**, não alterado indevidamente (hash, assinatura digital, trilha de auditoria).
- **Disponibilidade:** o dado/serviço fica **acessível** quando necessário (redundância, backup, plano de continuidade).
- Complementos frequentes: autenticidade, não repúdio, legalidade.

O detalhe que separa quem entendeu de quem decorou: os três pilares **competem entre si**. Aumentar confidencialidade quase sempre custa disponibilidade — um segundo fator de autenticação barra o invasor e também atrasa o usuário legítimo que perdeu o celular; cifrar o backup protege o dado roubado, mas o torna inútil se a chave de cifra se perder. Por isso uma questão de cenário raramente pede "o mais seguro—" pede o **equilíbrio adequado ao contexto**, e a alternativa que maximiza um pilar ignorando os outros costuma ser a errada.

### Cuidado — confusão comum

Mapeie o verbo do cenário para o pilar certo: "restringir quem vê" é confidencialidade, não integridade; "ficar no ar" é disponibilidade; "alteração indevida do conteúdo" é integridade.

## 8.2 Criptografia

### Assimétrica: a mesma matemática, duas direções, dois objetivos

O par de chaves tem uma propriedade central, e ela é a origem de toda a confusão sobre o tema: **o que uma chave fecha, só a outra abre**. Como as duas direções funcionam, a mesma matemática serve a dois objetivos diferentes — e é a **direção escolhida** que decide qual.

|                   | Para SIGILO                              | Para ASSINATURA                                 |
|-------------------|------------------------------------------|-------------------------------------------------|
| Cifra com         | a <b>pública do destinatário</b>         | a <b>privada do emissor</b>                     |
| Abre com          | a <b>privada do destinatário</b>         | a <b>pública do emissor</b>                     |
| Quem consegue ler | só o destinatário — só ele tem a privada | <b>qualquer um</b> : a pública é pública        |
| Logo, garante     | <b>confidencialidade</b>                 | <b>autenticidade, integridade e não repúdio</b> |

A lógica de cada uma, resumida numa frase: para **sigilo**, eu quero que *só você* leia, então uso o cadeado que *só você* abre — sua chave pública. Para **assinatura**, eu quero provar que *fui eu*, então uso a chave que *só eu* tenho — minha privada; que qualquer um consiga abrir e verificar não é defeito, é o ponto central: é assim que qualquer pessoa consegue confirmar a assinatura. Duas notas importantes. Primeira: assina-se o **hash** do documento, não o documento inteiro — criptografia assimétrica é lenta, e o hash tem tamanho fixo e pequeno. Segunda: na prática, nem o **sigilo** usa assimétrica para o volume de dados. O TLS faz o que se chama de **envelope**: usa a assimétrica **só para combinar** uma chave simétrica de sessão entre as partes, e daí em diante cifra tudo com a simétrica, que é rápida. "A assimétrica substituiu a simétrica" é falso — elas se combinam, cada uma no papel em que é boa.

### Simétrica x assimétrica, lado a lado

|            | Simétrica                       | Assimétrica (chave pública) |
|------------|---------------------------------|-----------------------------|
| Chaves     | uma chave secreta compartilhada | par: pública + privada      |
| Velocidade | rápida                          | lenta                       |
| Uso        | cifrar o volume de dados        | troca de chave, assinatura  |
| Exemplos   | AES, DES/3DES                   | RSA, ECC                    |

### Hash, assinatura, certificado, TDE, esteganografia

**Hash** (MD5, SHA-256): resumo de tamanho fixo, **unidirecional** — não existe "descriptografar" um hash. Garante **integridade**, não confidencialidade (MD5 e SHA-1 hoje são considerados fracos, com colisões conhecidas). **Assinatura digital**: o hash do documento é cifrado com a **chave privada** do emissor → garante autenticidade, integridade e não repúdio ao mesmo tempo. **Certificado digital / ICP-Brasil**: vincula uma identidade a uma chave pública, emitido por uma AC (Autoridade Certificadora) — é o que resolve o problema de "como sei

que esta chave pública é realmente da pessoa X". **TDE** (*Transparent Data Encryption*): cifra os dados **em repouso** (arquivos do banco/disco), de forma transparente à aplicação — protege contra o **roubo do arquivo físico**, não contra acesso lógico já autorizado ao banco. **Esteganografia** é diferente de criptografia: ela **oculta a existência** da mensagem (esconde dado dentro de uma imagem/áudio), enquanto a criptografia oculta o **conteúdo**, sem esconder que há uma mensagem.

#### HMAC: o cenário do *webhook*

**HMAC** (*Hash-based Message Authentication Code*) é um hash **combinado a uma chave secreta compartilhada**. Como só quem tem a chave produz o código correto, o HMAC entrega **integridade** (a mensagem não foi alterada) e **autenticidade** (veio de quem tem a chave). O que ele **não** entrega: **confidencialidade** (a mensagem viaja legível, só o código de autenticação vai junto) e **não repúdio** (a chave é **simétrica**, e as duas pontas conseguem gerar o mesmo código — então não dá para provar, perante um terceiro, qual das duas partes gerou a mensagem).

O cenário clássico: um sistema externo envia um *webhook* (uma chamada HTTP de notificação) e é preciso garantir que a requisição veio mesmo do parceiro combinado, e não foi adulterada no caminho. A resposta é assinar o corpo com **HMAC** usando um segredo combinado entre os dois lados, e o receptor recalcula o código e comparar. Hash puro não serve (qualquer um recalcularia sem precisar de segredo); cifrar o corpo resolveria confidencialidade, que não é o problema descrito.

#### Cuidado — confusão comum

HMAC **não** garante confidencialidade (não há cifra do conteúdo em si) nem não repúdio (para isso é preciso **assinatura digital** com chave **privada**, que só o emissor possui). Guarde o corte: chave **simétrica** → autenticidade + integridade; chave **privada** → acrescenta o não repúdio.

## 8.3 HTTPS, SSL e TLS

**HTTPS = HTTP sobre TLS/SSL**. Garante confidencialidade, integridade e autenticação do servidor (o cliente sabe que está falando com o servidor legítimo). **TLS substitui o SSL**: corrige vulnerabilidades conhecidas das versões antigas do SSL. SSL está obsoleto; TLS 1.2/1.3 é o padrão atual.

#### Cuidado — confusão comum

SSL **não** é mais seguro/moderno que TLS — é o contrário: TLS sucedeu e corrigiu o SSL. Os dois não são intercambiáveis apesar de frequentemente citados juntos.

## 8.4 Controle de acesso

### Quatro políticas, quatro bases de decisão diferentes

| Política             | Base da decisão                                                                                        |
|----------------------|--------------------------------------------------------------------------------------------------------|
| DAC (discricionário) | o <b>dono do recurso</b> concede acesso, a seu critério                                                |
| MAC (mandatário)     | <b>rótulos de segurança</b> comparados a autorizações; a regra é central e o usuário <b>não decide</b> |
| RBAC (por papéis)    | acesso concedido via <b>papéis/funções</b> organizacionais                                             |
| ABAC (por atributos) | decisão calculada a partir de <b>atributos</b> de usuário, recurso e contexto no momento do acesso     |

**Princípio do menor privilégio:** conceder só o acesso estritamente necessário para a função (um gerente de RH acessa só o que sua função exige, não tudo que tecnicamente seria possível liberar).

### OAuth2, OIDC e SSO

**OAuth2:** protocolo de **autorização delegada** baseado em tokens — resolve "este aplicativo pode agir em meu nome, com este escopo de permissão limitado", **sem** entregar a senha ao aplicativo. É diferente de autenticação. **OpenID Connect (OIDC):** construído **sobre** o OAuth2, é quem efetivamente faz **autenticação** — adiciona um ID Token com *claims* padronizados: **iat** (*issued at*, momento da emissão), **exp** (expiração), **sub** (*subject*, identificador único do usuário), **jti** (identificador único do próprio token). **SSO (Single Sign-On):** um único login dá acesso a vários sistemas, sem repetir autenticação em cada um.

### Cuidado — confusão comum

MAC significa rótulos e classificações impostos pelo **sistema** (o usuário não decide, diferente do DAC, em que o **proprietário** decide). OAuth2 é **autorização**, não autenticação — confundir os dois é o erro mais comum do tema.

## 8.5 OWASP Top 10

### Duas edições vigentes: 2025 (atual) e 2021 (a mais citada em prova)

A edição **vigente** é a **2025** (oitava da série). A **2021** continua relevante como referência amplamente citada em provas. Ao ler qualquer enunciado sobre o tema, **verificar qual ano ele cita** antes de contar posição é essencial, porque a ordem e até a existência de categorias mudou entre as duas edições.

| #  | Top 10:2025                                   | Top 10:2021                                     |
|----|-----------------------------------------------|-------------------------------------------------|
| 1  | Broken Access Control (absorveu o SSRF)       | Broken Access Control                           |
| 2  | Security Misconfiguration (subiu de 5º)       | Cryptographic Failures                          |
| 3  | Software Supply Chain Failures (nova)         | Injection (SQLi; <b>XSS absorvido aqui</b> )    |
| 4  | Cryptographic Failures (caiu de 2º)           | Insecure Design (nova em 2021)                  |
| 5  | Injection (caiu de 3º)                        | Security Misconfiguration                       |
| 6  | Insecure Design                               | Vulnerable and Outdated Components              |
| 7  | Authentication Failures                       | Identification and Authentication Failures      |
| 8  | Software <i>or</i> Data Integrity Failures    | Software <i>and</i> Data Integrity Failures     |
| 9  | Security Logging and <b>Alerting</b> Failures | Security Logging and <b>Monitoring</b> Failures |
| 10 | Mishandling of Exceptional Conditions (nova)  | Server-Side Request Forgery (SSRF)              |

O que mudou de 2021 para 2025: **duas categorias novas** (Software Supply Chain Failures e Mishandling of Exceptional Conditions) e **uma consolidação** (o SSRF, que era categoria própria em 2021, foi absorvido pelo Broken Access Control). Três categorias só **mudaram de nome**, mantendo a essência.

#### Cuidado — confusão comum

Desde 2021, **XSS entrou dentro de Injection** (era categoria própria até 2017) — não é mais listado separadamente. Itens como "proteção da cadeia de suprimentos/pipeline", "treinamento operacional" pertencem a outros frameworks (CI/CD, SSDF), não ao OWASP Top 10 do lado *web* tradicional — exceto que, a partir de 2025, cadeia de suprimentos **de software** virou categoria própria (A03), o que não retroage para tornar isso item de 2021.

## 8.6 Desenvolvimento seguro

**SDL** (*Security Development Lifecycle*): incorpora segurança em **todo** o ciclo de desenvolvimento, não só numa revisão final. **SAST** (estática): analisa o **código-fonte sem executá-lo** — é caixa-branca. **DAST** (dinâmica): testa a **aplicação em execução**, sem olhar o código-fonte — é caixa-preta. **IAST**: combina os dois, instrumentando a aplicação em tempo de execução para observar o comportamento interno enquanto ela roda. **DevSecOps**: automatiza essas verificações de segurança dentro do próprio pipeline de CI/CD.

**Cuidado — confusão comum**

SAST analisa código **parado**; DAST testa aplicação **rodando** — não inverter os dois.

## 8.7 X.800: a arquitetura de segurança OSI

### Vocabulário comum a normas e bancas

A recomendação X.800 (ITU-T) dá o vocabulário de segurança usado por diversas normas posteriores. Divide o assunto em três: **ataques**, **serviços** e **mecanismos**. O foco cobrado em prova é a divisão dos **mecanismos**.

Os cinco **serviços** de segurança: autenticação, controle de acesso, confidencialidade, integridade e **não repúdio** (a disponibilidade aparece como categoria adicional, fora dos cinco clássicos).

**Mecanismos**, divididos em dois grupos:

|                                                                          |                                                                                                                                                                                                        |
|--------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Específicos</b> (ligados a uma camada e a um serviço)                 | cifração, assinatura digital, controle de acesso, integridade de dados, troca de autenticação, <b>preenchimento de tráfego</b> ( <i>traffic padding</i> ), controle de roteamento e <b>notarização</b> |
| <b>Disseminados</b> ( <i>per-vasive</i> , não específicos de uma camada) | funcionalidade confiável, <b>rótulos de segurança</b> , detecção de eventos, <b>trilha de auditoria</b> de segurança e recuperação de segurança                                                        |

O critério de corte: um mecanismo **específico** implementa **um serviço** numa **camada** determinada; um mecanismo **disseminado** é de gestão, e vale para o sistema inteiro, sem se prender a uma camada só. **Preenchimento de tráfego** e **controle de roteamento** costumam surpreender por serem específicos — eles servem à confidencialidade do **fluxo** de comunicação (esconder padrões de tráfego), não à confidencialidade do **conteúdo**.

## 8.8 Gestão de riscos

### Ameaça + vulnerabilidade = risco, resultando em impacto

Um risco existe quando **uma ameaça encontra uma vulnerabilidade**, e disso resulta um **impacto** potencial. Retirar qualquer um dos três elementos e o risco desaparece.

|                        |                                                                                                                                                                                      |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Ameaça</b>          | o evento indesejado <b>em potencial</b> — incêndio, invasor, funcionário descuidado, <i>ransomware</i> . Vem de fora do seu controle direto: você <b>não elimina</b> a ameaça em si. |
| <b>Vulnerabilidade</b> | a fraqueza que permite a ameaça se concretizar — servidor sem <i>patch</i> , senha fraca, ausência de <i>backup</i> . É o <b>único dos três</b> sobre o qual você age diretamente.   |
| <b>Impacto</b>         | o dano, se a ameaça se concretizar. Você não impede o impacto em si, mas pode <b>reduzi-lo</b> (segmentar a rede, cifrar o dado que seria roubado) ou <b>transferi-lo</b> (seguro).  |

O **risco** é a combinação de **probabilidade** × **impacto** — é exatamente isso que uma matriz

de risco 5×5 desenha: probabilidade num eixo, impacto no outro, e a cor da célula indica a prioridade de tratamento. Consequência importante: um evento catastrófico mas muito improvável pode ficar **abaixo**, na prioridade, de um evento pequeno mas frequente.

### As quatro formas de tratar o risco

|                                  |                                                                                                                                                                                                                     |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mitigar</b> (reduzir)         | implanta um controle e <b>diminui</b> a probabilidade ou o impacto — é o caso mais comum: aplicar <i>patch</i> , instalar um WAF, treinar a equipe.                                                                 |
| <b>Transferir</b> (compartilhar) | passa a consequência <b>financeira</b> a um terceiro — seguro, terceirização com cláusula de responsabilidade. Transferir <b>não reduz</b> a probabilidade de o evento acontecer, só redistribui quem paga a conta. |
| <b>Evitar</b>                    | <b>deixa de fazer</b> a atividade que gera o risco — descontinuar um sistema legado, não coletar um dado sensível desnecessário.                                                                                    |
| <b>Aceitar</b> (reter)           | decide conviver com o risco, <b>formalmente</b> e no nível hierárquico adequado, porque o controle custaria mais que o dano potencial.                                                                              |

**Risco residual** é o que **sobra depois** do tratamento aplicado — nunca é zero, e a norma exige que seja **aceito formalmente** por quem tem alçada para isso; aceitar risco residual é decisão de gestão, não do time técnico isoladamente.

### Cuidado — confusão comum

Transferir o risco **não elimina** o risco — o seguro paga o prejuízo, mas o incidente acontece do mesmo jeito; quem reduz a **chance** de acontecer é mitigar. **Aceitar** não é o mesmo que **ignorar**: aceitar é um ato formal, documentado e aprovado; deixar sem tratamento por simples omissão não é uma das quatro opções formais.

## 8.9 Continuidade de negócio: RTO x RPO

### Dois objetivos, guardados pela unidade do que se perde

| Sigla      | Nome                            | O que limita                                                                  |
|------------|---------------------------------|-------------------------------------------------------------------------------|
| <b>RTO</b> | <i>Recovery Time Objective</i>  | <b>TEMPO</b> : quanto o serviço pode ficar indisponível até ser restabelecido |
| <b>RPO</b> | <i>Recovery Point Objective</i> | <b>DADO</b> : até que ponto no passado se aceita perder informação            |

É o **RPO** que determina a **frequência do backup**: aceitar perder no máximo 15 minutos de dado obriga a proteger os dados a cada 15 minutos, no máximo. O RTO, por sua vez, cobra da infraestrutura de recuperação (redundância, sítio alternativo, tempo de restauração).

Siglas vizinhas frequentemente oferecidas junto: **MTBF** (*Mean Time Between Failures*, tempo médio **entre** falhas sucessivas — métrica de confiabilidade, não objetivo de plano); **MTTR** (*Mean Time To Repair*, tempo médio de reparo); **SLA** (*Service Level Agreement*, o **acordo**

contratual de nível de serviço, não uma métrica em si).

#### Cuidado — confusão comum

Ancore numa letra: **T** de RTO é **Tempo**; **P** de RPO é **Ponto** no tempo (dado). Um cenário que dá dois números na ordem "pode ficar fora X, pode perder Y" e depois inverte as siglas na resposta é o erro clássico deste tema.

## 8.10 Detecção e resposta a incidentes

**IDS** (*Intrusion Detection System*): **alerta** sobre uma intrusão, de forma passiva — não bloqueia. **IPS** (*Intrusion Prevention System*): **bloqueia** ativamente, posicionado *inline* no caminho do tráfego. **SIEM** (*Security Information and Event Management*): correlaciona *logs* e eventos de várias fontes para detecção e resposta.

**Resposta a incidentes** (referência: NIST SP 800-61), em quatro fases ordenadas:

1. **Preparação**: antes de qualquer incidente — time, ferramentas, políticas prontas.
2. **Detecção e Análise**: identificar que um incidente está ocorrendo e entender sua natureza.
3. **Contenção, Erradicação e Recuperação**: isolar o problema, remover a causa raiz, restaurar serviços e dados.
4. **Atividade pós-incidente / Lições Aprendidas**: documentar o ocorrido e ajustar controles para prevenir recorrência — essa fase realimenta a Preparação, fechando o ciclo.

#### Cuidado — confusão comum

Detecção e Análise vêm **antes** da Contenção — não se contém o que ainda não foi identificado. Depois de conter e erradicar, vem **recuperação seguida de lições aprendidas** — essa ordem costuma ser embaralhada em prova.

## 8.11 ISO/IEC 27001 e 27002

### SGSI x guia de controles

A **ISO/IEC 27001:2022** especifica os **requisitos** de um Sistema de Gestão de Segurança da Informação (SGSI), incluindo seu Anexo A. A **ISO/IEC 27002:2022** é o **guia de controles** detalhado — **93 controles**, organizados em **4 temas**: organizacionais (37), pessoas (8), físicos (14), tecnológicos (34).

Exemplos de classificação temática: trabalho remoto → tema de **pessoas**; mídia de armazenamento → tema **físico**; criptografia → tema **tecnológico**; definição de políticas → tema **organizacional**.

### NIST Cybersecurity Framework e STRIDE

**NIST CSF 1.1**: cinco funções — *Identify, Protect, Detect, Respond, Recover*; a versão **2.0** acrescentou uma sexta função, **Govern**, no centro do modelo.

**STRIDE** (modelagem de ameaças): seis categorias — *Spoofing* (falsificar identidade), *Tampering* (adulterar dado), *Repudiation* (negar uma ação realizada), *Information disclosure* (vazamento de informação), *Denial of service* (indisponibilidade forçada), *Elevation of privilege*

(ganhar mais permissão do que deveria).

## Capítulo 9

# Programação

### Ficha do edital

Desenvolvimento em Java/JavaEE/JakartaEE/JPA/JavaScript; frameworks JUnit, Hibernate, JSF, Primefaces, Spring, Spring Cloud, Spring Boot; mobile (Android/iOS); low-code/no-code; análise estática (clean code, SonarQube); padrões XML, XSLT, UDDI, REST, JSON; DevOps; Git; codificação (transacional, analítico, mobile, API); reuso; blockchain. (Java em si tem capítulo próprio: Capítulo 10.)

## 9.1 Ecossistema Spring

### O nó que o Spring desfaz

Antes do Spring, uma classe de negócio construía suas próprias dependências com **new**: o serviço de pedidos decidia, dentro do próprio código, qual repositório concreto usar. Isso amarrava três coisas de uma vez — trocar a implementação exigia editar e recompilar quem a usava; testar exigia o banco de verdade, porque não havia onde encaixar um duplê; e a configuração ficava espalhada por todas as classes que precisavam dela. O ecossistema abaixo existe para desfazer exatamente esse nó.

| Framework    | Papel                                                                                                            |
|--------------|------------------------------------------------------------------------------------------------------------------|
| Spring       | contêiner de inversão de controle (IoC) e injeção de dependência                                                 |
| Spring Boot  | Spring com autoconfiguração e servidor embutido; elimina configuração manual e código repetitivo                 |
| Spring Cloud | ferramentas para sistemas distribuídos/microserviços (configuração centralizada, descoberta de serviço, gateway) |
| Hibernate    | ORM (mapeamento objeto-relacional), implementação de JPA                                                         |
| JUnit        | framework de testes de unidade                                                                                   |

### IoC e DI: o que o contêiner realmente faz

**Inversão de controle (IoC)** é isto: quem decide **qual** implementação usar e **quando** construí-la deixa de ser a própria classe e passa a ser o **contêiner**. A classe apenas **declara** do que precisa e recebe pronto de fora — o controle foi invertido, daí o nome.

**Injeção de dependência (DI)** é a forma concreta de fazer IoC: entregar a dependência por **construtor** (a forma preferida — a dependência se torna obrigatória e o objeto já nasce em estado válido), por *setter*, ou por campo anotado. DI e IoC não são conceitos rivais: DI é o

mecanismo, IoC é o princípio que ele implementa.

O ganho é duplo: **trocar a implementação** sem tocar em quem a usa, e **testar** injetando um dublê no lugar do recurso real (um banco de dados de teste em vez do de produção, por exemplo). É o *Dependency Inversion* — o D do SOLID (Capítulo 4) — executado automaticamente por um contêiner.

O objeto gerenciado pelo contêiner chama-se **bean**, e o escopo padrão do Spring é *singleton* — **uma instância por definição de bean, dentro daquele contêiner específico**. Isso **não** é o padrão Singleton do GoF (que faz a própria classe impedir uma segunda instância via construtor privado) — é o contêiner quem garante a instância única, a classe em si nem sabe disso. Mesmo nome, mecanismo e garantias diferentes.

#### Cuidado — confusão comum

Spring Boot **não exige** configuração manual de servidor (ele embute o Tomcat) nem um servidor de aplicação *standalone*; "Spring só serve para monólito" é falso. Hibernate é ORM/persistência, não framework de teste. JUnit testa, não faz mapeamento objeto-relacional. Spring Cloud endereça sistemas **distribuídos**; Spring Boot serve a uma aplicação **individual**.

## 9.2 Formatos de dados: XML, JSON, XSLT

|           | XML                                   | JSON                                       |
|-----------|---------------------------------------|--------------------------------------------|
| Estrutura | marcação com tags, verboso            | pares chave-valor, leve                    |
| Uso hoje  | legado, arquivo de configuração, SOAP | <b>APIs web</b> , transporte leve de dados |

**XSLT**: linguagem que **transforma XML** em outro formato (HTML, outro XML, texto puro) — **não** transforma JSON, porque opera sobre a estrutura de árvore específica do XML. **REST** tipicamente usa JSON como formato de troca; **SOAP** usa XML.

#### Cuidado — confusão comum

XSLT não transforma JSON — é exclusivo de XML. XML e JSON não são equivalentes/idênticos: JSON não tem comentários nativos nem um tipo **date** nativo (datas em JSON são convenção, geralmente string).

## 9.3 Mobile e low-code

**Nativo, multiplataforma, híbrido**

| Abordagem       | O que é                                                       | Ferramentas / linguagem                                                                   |
|-----------------|---------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Nativo          | app específico para cada sistema operacional                  | Android: <b>Kotlin/Java</b> ;<br>iOS: <b>Swift/Objective-C</b>                            |
| Multiplataforma | um código-fonte para vários sistemas operacionais             | <b>Flutter (Dart)</b> , <b>React Native (JavaScript)</b> ,<br>Xamarin (.NET), Ionic (web) |
| Híbrido         | conteúdo web dentro de um contêiner nativo ( <i>WebView</i> ) | Ionic, Cordova                                                                            |

**Flutter** usa **Dart** como linguagem; **React Native** usa **JavaScript**, mas renderiza componentes verdadeiramente nativos (diferente do *WebView* usado pelo híbrido, que renderiza uma página web dentro de um contêiner). Nativo entrega melhor desempenho e acesso completo ao hardware; multiplataforma reduz custo e tempo de desenvolvimento por manter um único código-fonte.

**Low-code/no-code**: desenvolvimento visual, com pouca ou nenhuma escrita manual de código; acelera a entrega e permite que pessoas sem formação em programação ("*citizen developers*") construam soluções — mas limita customização e pode gerar dependência (*lock-in*) da plataforma escolhida.

**Cuidado — confusão comum**

Pares que se confundem: Dart pertence ao Flutter, Kotlin ao Android nativo, Swift ao iOS nativo — não trocar entre si.

**9.4 Java EE / web server-side: JSF e Primefaces**

**JSF (*JavaServer Faces*)**: framework de UI baseado em componentes, executado no servidor, orientado a eventos — segue o padrão MVC. Tem um ciclo de vida de requisição com fases bem definidas (restaurar view, aplicar valores, validação, atualizar modelo, invocar aplicação, renderizar resposta).

**Primefaces**: biblioteca de componentes ricos construída sobre JSF (tabelas avançadas, gráficos, Ajax pronto) — não é um framework independente, é uma extensão de componentes do JSF.

**JSP/Servlet**: a camada web mais antiga do Java EE — o Servlet processa a requisição, o JSP gera a apresentação (view). JSF é a evolução baseada em componentes que veio depois.

**Cuidado — confusão comum**

JSF é *server-side* baseado em componentes — diferente de uma SPA *client-side* como React, que roda no navegador. Primefaces é biblioteca de componentes de JSF, não um framework à parte.

## 9.5 Versionamento com Git

### Git é distribuído

Cada clone do repositório tem o **histórico completo** — diferente do SVN, que é centralizado e lida mal com arquivos binários (cada alteração exige ida e volta ao servidor central).

### As três áreas, onde mora quase toda pegadinha de Git

diretório de trabalho  $\xrightarrow{\text{git add}}$  **stage** (index)  $\xrightarrow{\text{git commit}}$  repositório local  $\xrightarrow{\text{git push}}$  remoto

O **commit** grava **o que está no stage**, não o que está no diretório de trabalho. Editar um arquivo já versionado e fazer **commit sem git add** antes não inclui a alteração no commit — o commit sai "vazio" daquela mudança específica, silenciosamente, sem erro nenhum.

### Vocabulário de comandos

| Comando                      | O que faz                                                                              |
|------------------------------|----------------------------------------------------------------------------------------|
| <code>git clone</code>       | cria a cópia local inicial de um repositório remoto                                    |
| <code>git add</code>         | leva a alteração para o stage                                                          |
| <code>git commit</code>      | grava no repositório local o que está no stage                                         |
| <code>git fetch</code>       | traz referências e objetos do remoto <b>sem</b> tocar na cópia de trabalho             |
| <code>git pull</code>        | <b>fetch</b> + integração automática (merge ou rebase)                                 |
| <code>git push</code>        | envia commits locais ao remoto                                                         |
| <code>git merge</code>       | une duas linhas de desenvolvimento criando um <b>commit de junção</b>                  |
| <code>git rebase</code>      | <b>reaplica</b> os commits sobre outra base, produzindo histórico linear               |
| <code>git revert</code>      | cria um commit <b>novo</b> que anula outro (seguro em branch já pública/compartilhada) |
| <code>git reset -hard</code> | move a branch e <b>descarta</b> alterações locais                                      |
| <code>git cherry-pick</code> | copia <b>commits escolhidos</b> para a branch atual                                    |

**Regra de ouro do rebase:** ele reescreve commits (produz novos identificadores para eles). Não deve ser usado numa branch já compartilhada com outras pessoas — é a razão de ser chamado de "perigoso" nesse contexto: quem já baixou os commits antigos entra em conflito com os reescritos.

**Fluxos de trabalho comuns:** branch por funcionalidade + *pull request*; **Git Flow** (branches `main`, `develop`, `feature/*`, `release/*`, `hotfix/*`, cada uma com papel definido); **trunk-based** (integrações curtas e frequentes diretamente no tronco principal, casa melhor com integração contínua).

**Cuidado — confusão comum**

`fetch` **não** altera a cópia de trabalho (só atualiza referências locais do remoto); `pull` altera, porque integra automaticamente. Se o enunciado disser "**sem alterar** a cópia de trabalho", é `fetch`.

## 9.6 CI, entrega contínua e implantação contínua

### Três práticas, cada uma "terminando" num ponto diferente

| Prática                                           | Onde termina                                                                                     |
|---------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <b>Integração contínua</b> (CI)                   | integra e <b>testa</b> a cada <i>push</i>                                                        |
| <b>Entrega contínua</b> ( <i>delivery</i> )       | artefato <b>sempre pronto</b> para produção; a subida efetiva depende de <b>aprovação manual</b> |
| <b>Implantação contínua</b> ( <i>deployment</i> ) | o artefato aprovado vai a produção <b>automaticamente</b> , sem intervenção humana               |

Entrega contínua **pressupõe** integração contínua — não dá para ter um artefato "sempre pronto" sem antes integrar e testar continuamente. **DevOps**: cultura de aproximar desenvolvimento e operação, com automação, medição e responsabilidade compartilhada pelo que está em produção. **DevSecOps**: desloca a segurança para a **esquerda** (*shift left*) do pipeline — presente desde o início do desenvolvimento, não como uma auditoria só no fim.

**Cuidado — confusão comum**

Se o enunciado disser que a subida a produção **espera aprovação humana**, é entrega contínua (*delivery*), não implantação contínua (*deployment*) — essa distinção é o par mais trocado do tema.

## 9.7 Contêineres

**Docker** constrói (a partir de um `Dockerfile`) e executa a **imagem**, que empacota a aplicação junto com suas dependências. **Docker Compose** orquestra vários contêineres em **um único host** (uso típico: ambiente de desenvolvimento local). **Kubernetes** orquestra contêineres num **cluster** de várias máquinas: mantém o número desejado de réplicas, reinicia o que falha, distribui carga entre elas e realiza atualizações graduais (*rolling update*) — os detalhes de arquitetura estão no Capítulo 5.

## 9.8 Qualidade de código

**Clean Code:** princípios de nomes claros, funções curtas, baixo acoplamento — o código deve se explicar por si, sem exigir comentário extenso. **SonarQube:** ferramenta de análise **estática** (sem executar o código) que detecta *code smells*, bugs em potencial, vulnerabilidades de segurança, dívida técnica acumulada e mede cobertura de teste.

### Onde o resto do edital de Programação está tratado

- **SOLID e orientação a objetos** (herança, polimorfismo, sobrescrita): Capítulo 10.
- Os **23 padrões GoF** e o par alta coesão x baixo acoplamento: Capítulo 4.
- **REST na prática** (métodos HTTP, códigos de resposta, idempotência): Capítulo 5.
- **Reuso, testes e ciclo de vida:** Capítulo 3.

## 9.9 Blockchain

### O que cada bloco guarda — e o que ele não guarda

Uma cadeia de blocos encadeados por **hash do bloco anterior**, tornando a cadeia **imutável** na prática (alterar um bloco antigo mudaria seu hash, quebrando o encadeamento com todos os blocos seguintes). Cada bloco tipicamente contém: hash do bloco anterior, *timestamp*, dados/transações, *nonce* (usado no consenso por prova de trabalho), e a raiz de uma árvore de Merkle (resumo criptográfico eficiente de todas as transações do bloco).

**O saldo das carteiras não é armazenado diretamente em nenhum bloco — é derivado** do histórico de transações (modelo UTXO no Bitcoin: soma-se todo o dinheiro recebido por aquele endereço e subtrai-se o que já foi gasto). É descentralizado, com mecanismo de consenso (PoW — *Proof of Work*, ou PoS — *Proof of Stake*); Ethereum acrescenta **contratos inteligentes** (código executável armazenado e rodado na própria cadeia).

## 9.10 Estruturas de dados

### Cada estrutura é um contrato de custo, não uma definição solta

A forma mais útil de entender estrutura de dados não é decorar "o que é uma pilha— é reconhecer, diante de um **requisito** ("inserções e remoções frequentes no meio", "busca por chave em tempo constante", "listar em ordem e consultar por faixa"), qual estrutura atende com o menor custo. A escolha da estrutura fixa o custo de cada operação antes mesmo da primeira linha de código, e é uma escolha cara de desfazer depois, porque todo o código em volta se apoia nela.

| Estrutura              | O que é barato                                                       | O que é caro / o preço                                                    |
|------------------------|----------------------------------------------------------------------|---------------------------------------------------------------------------|
| Vetor ( <i>array</i> ) | acesso pela posição em $O(1)$                                        | inserir ou remover no meio é $O(n)$ : os demais elementos <b>deslocam</b> |
| Lista encadeada        | inserir/remover em $O(1)$ — <b>desde que você já esteja no ponto</b> | chegar ao $i$ -ésimo elemento é $O(n)$ : precisa percorrer desde o início |
| Duplamente encadeada   | remover um nó já conhecido em $O(1)$ , percorrer nos dois sentidos   | dois ponteiros por nó (memória extra)                                     |
| Pilha (LIFO)           | empilhar/desempilhar no topo em $O(1)$                               | só o topo é acessível                                                     |
| Fila (FIFO)            | enfileirar/desenfileirar em $O(1)$                                   | só as duas pontas são acessíveis                                          |
| Tabela hash            | busca/inserção/remoção em $O(1)$ <b>em média</b>                     | <b>não guarda ordem</b> ; pior caso $O(n)$                                |
| Árvore balanceada      | busca em $O(\log n)$ <b>mantendo a ordem</b>                         | mais lenta que o hash na busca pontual isolada                            |

Duas frases resolvem a maioria das questões do tema: **vetor troca inserção por acesso, lista encadeada troca acesso por inserção** — e o  $O(1)$  da lista pressupõe que você já tem o ponteiro em mãos; se for preciso **achar** a posição primeiro, soma-se o  $O(n)$  da busca por cima. E: **hash e árvore não competem pela mesma vaga** — se o enunciado pede **faixa** de valores ou **listagem ordenada**, a resposta é a árvore, mesmo que a tabela hash apareça entre as alternativas com seu  $O(1)$  sedutor (hash não preserva ordem, então não serve a essas consultas).

**Busca binária:**  $O(\log n)$ , mas **exige a coleção já ordenada** — sem essa condição, não se aplica. Em arquivo desordenado e sem índice, a única saída é a **busca sequencial**,  $O(n)$ .

### Por que uma tabela hash degrada: fator de carga e colisão

Uma tabela hash com  $m$  posições guardando  $n$  chaves tem **fator de carga**  $\alpha = n/m$ . Duas chaves diferentes podem cair na mesma posição (colisão), tratada de duas formas:

- **Encadeamento separado:** cada posição aponta para uma lista. As chaves moram **fora** do vetor, então  $\alpha$  **pode passar de 1** (mais chaves que posições, ficam em listas mais longas); o custo médio de busca cresce com o comprimento da lista naquela posição.
- **Endereçamento aberto:** a chave mora **dentro** do próprio vetor, numa posição livre encontrada por *sondagem*. Como cada posição comporta uma única chave,  $\alpha \leq 1$  **sempre** — é o que limita o fator de carga a 1 nesse esquema.

Caso concreto: endereçamento aberto com **sondagem linear** ("ocupado? tente a próxima posição") sob ocupação alta,  $\alpha \approx 0,8$ . Só uma posição em cinco está livre, e as ocupadas tendem a se juntar em **blocos contíguos**. Uma chave que caia em **qualquer** ponto de um bloco precisa caminhar até o fim dele — e, ao se acomodar ali, **aumenta o bloco**, tornando mais provável que a próxima chave também caia nele. Esse laço de realimentação é o **agrupamento primário** (*primary clustering*), e é ele que empurra a busca do  $O(1)$  médio anunciado para perto do  $O(n)$  na prática sob alta ocupação. Remédios: sondagem quadrática ou hash duplo (espalham a sequência de tentativas de forma menos previsível) e, sobretudo, **redimensionar a**

**tabela** quando  $\alpha$  ultrapassa um limiar — é o  $\alpha$ , não o número absoluto de chaves, que governa o desempenho.

A mesma lógica reaparece no índice hash de um SGBD (Capítulo 6): encontra a linha por igualdade em tempo constante, mas é inútil para **BETWEEN** ou **ORDER BY** — exatamente porque hash não preserva ordem. Estrutura de dados e índice de banco são a mesma discussão, vista em dois níveis diferentes.

### Cuidado — confusão comum

Não existe acesso  $O(1)$  ao  $i$ -ésimo elemento de uma **lista encadeada** (o  $O(1)$  é só da inserção, com o ponteiro já em mãos). Não existe inserção  $O(1)$  no meio de um **vetor** (há deslocamento de elementos). Uma tabela hash garante busca em tempo constante **em média**, nunca no pior caso absoluto — desconfie de afirmações que omitem essa ressalva.

## 9.11 Leitura ativa de código

### Protocolo para questões que mostram trecho de código

Quando a prova mostra um trecho de código pequeno e pergunta a **saída** ou o **efeito**, decorar conceito não resolve — o antídoto é **rodar o código mentalmente**, linha a linha, como um interpretador, anotando o estado das variáveis num rascunho.

1. **Não leia por cima:** releia a linha inteira antes de assumir o que ela faz — é comum um enunciado mudar um único parâmetro, operador ou índice em relação ao que "pareceria óbvio" numa leitura apressada.
2. **Anote o estado:** para cada variável relevante, escreva o valor depois de cada linha executada — não confie na memória depois da terceira linha de código.
3. **Desconfie de referência x cópia:** a armadilha mais comum em qualquer linguagem — duas variáveis apontando para o **mesmo** objeto/array, de modo que mudar uma "muda" a outra também, ou o contrário, quando a API faz uma cópia de fato.
4. **Confirme contra a assinatura real** da função/método, não contra o que pareceria fazer sentido.

### Quatro fatos de linguagem que decidem a resposta

- **Compilada x interpretada x bytecode:** uma linguagem **compilada** traduz para código de máquina **antes** de rodar — um erro de tipo aparece já na compilação. Uma **interpretada** executa comando a comando — o mesmo erro só aparece **na linha em que a execução chega**. **Java** é o meio-termo: compila para **bytecode** (`.class`), e a **JVM** executa esse bytecode, compilando os trechos "quentes" para código nativo em tempo de execução (JIT — *Just-In-Time*).
- **Passagem por valor x por referência:** em Java e em Python, o que se passa para uma função é sempre o **valor da referência** ao objeto. Consequência prática: **reatribuir** o parâmetro dentro da função **não** muda a variável de quem chamou; **mutar** o objeto apontado (alterar seu conteúdo interno) muda para os dois lados, porque ambos apontam para o mesmo objeto. Diante do código, a pergunta certa não é "é por valor ou por referência?" — é "esta linha reatribui ou muda internamente (*muta*) o objeto?"
- **Imutável x mutável:** em Python, `str`, `int` e `tuple` são imutáveis; `list`, `dict` e `set` são mutáveis. A consequência cobrada: só um objeto imutável (e por isso *hashável*) serve como **chave de dicionário** ou elemento de `set` — uma tupla pode, uma lista não. Em Java,

a mesma ideia aparece na `String` imutável, que é a razão de existir o `StringBuilder` (Capítulo 10).

- **Duck typing:** o que importa é o objeto **responder** ao método chamado, não a que classe ele formalmente pertence — a compatibilidade é verificada **no uso**, não na declaração de tipo. É por isso que Python dispensa a interface explícita que Java exige.

**Aviso de homônimo:** o *decorator* do Python (sintaxe `@`) é uma função que recebe outra função e devolve uma versão embrulhada dela — é metaprogramação da linguagem. **Não** é o padrão Decorator do GoF (Capítulo 4), que é composição de objetos em tempo de execução. Mesmo nome, assuntos completamente diferentes.

### NumPy: referência x cópia

| Código                             | Execução mental                                                               |
|------------------------------------|-------------------------------------------------------------------------------|
| <code>a = np.array([1,2,3])</code> | <code>a = [1,2,3]</code> (original)                                           |
| <code>b = a.view()</code>          | <b>b compartilha</b> o buffer de <code>a</code> ;<br><code>b.base is a</code> |
| <code>a[0] = 99</code>             | muda o buffer compartilhado                                                   |
| <code>print(b[0])</code>           | imprime <b>99</b> (reflete, pois é <i>view</i> )                              |

Se a segunda linha fosse `b = a.copy()`, `b[0]` continuaria **1** mesmo depois de alterar `a[0]`, porque `copy()` rompe o vínculo com o buffer original — `b.base` seria `None`.

**Dicionário e a função `max`:** `max(dic)` compara as **chaves** do dicionário; `max(dic, key=dic.get)` compara os **valores** associados a cada chave, mas devolve a **chave** vencedora — nunca o valor. O parâmetro `key=` de `max/min/sorted` diz **por onde comparar**, nunca **o que devolver**.

**Fatiamento de listas:** `lista[a:b]` inclui o índice `a` e **exclui** o índice `b` — `nums[1:4]` devolve **três** elementos (o tamanho do resultado é `b - a`). Índice negativo conta a partir do fim (`-1` é o último elemento).

## Capítulo 10

# Java

### Ficha do edital

Java (versão 6+), JavaEE (6+), JakartaEE, JPA (2+), frameworks JUnit, Hibernate, JSF, Primefaces, Spring/SpringCloud/SpringBoot. É a linguagem-base do perfil.

### 10.1 Exceções: *checked* x *unchecked*

|             | Checked (verificada)             | Unchecked (não verificada)                                                     |
|-------------|----------------------------------|--------------------------------------------------------------------------------|
| Superclasse | Exception (não RuntimeException) | RuntimeException                                                               |
| Compilador  | exige try-catch ou throws        | não exige nada                                                                 |
| Exemplos    | IOException, SQLException        | NullPointerException, ArrayIndexOutOfBoundsException, IllegalArgumentException |

A ideia por trás da divisão: exceções *checked* representam condições que um programa **razoável deveria antecipar e tratar** (um arquivo pode não existir, uma conexão de rede pode cair) — o compilador força a lidar com elas. Exceções *unchecked* tipicamente representam **erros de programação** (acessar índice inválido, referência nula) que, em teoria, um código correto nunca deveria disparar — por isso não são forçadas no nível do compilador.

### Cuidado — confusão comum

`NullPointerException` é **unchecked** (estende `RuntimeException`); `IOException` é **checked**. Uma exceção *unchecked* **não** exige `throws` obrigatório — é o contrário do que soa intuitivo pelo nome.

### 10.2 Polimorfismo: *overload* x *override*

|            | Overload (sobrecarga)               | Override (sobrescrita)    |
|------------|-------------------------------------|---------------------------|
| Onde       | mesma classe                        | subclasse                 |
| Assinatura | <b>muda</b> (parâmetros diferentes) | <b>igual</b> à do pai     |
| Resolução  | <i>compile-time</i> (estática)      | <i>runtime</i> (dinâmica) |
| Anotação   | —                                   | @Override                 |

A **assinatura** de um método é o nome mais a lista de parâmetros — o **tipo de retorno não entra nela**. Consequência direta: dois métodos no mesmo escopo que diferem **apenas** no tipo de retorno (`int calc(int x)` e `long calc(int x)`) **não compilam** — o compilador não os reconhece como sobrecarga válida, é declaração duplicada da mesma assinatura.

### Cuidado — confusão comum

Sobrecarga = mesmo nome, assinatura diferente, decidida em compilação. Sobrescrita = mesma assinatura, na subclasse, decidida em execução. Sobrescrever **não** viola Liskov por si só (ver Seção 10.3.1) — viola se quebrar o contrato esperado pelo cliente, não pelo simples fato de existir.

## 10.3 Orientação a objetos e SOLID

### Os quatro pilares: o mesmo propósito, alvos diferentes

Os quatro pilares clássicos (abstração, encapsulamento, herança, polimorfismo) têm o **mesmo** propósito: limitar o que o resto do sistema precisa saber sobre uma parte dele. O que muda entre eles é **o que** exatamente está sendo escondido:

| Pilar          | Esconde...                                      | Como aparece no código                                                                                            |
|----------------|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Abstração      | os detalhes irrelevantes para quem usa          | a classe expõe <i>o que</i> faz ( <b>sacar</b> ), não <i>como</i>                                                 |
| Encapsulamento | o <b>estado</b> interno                         | atributo <b>private</b> + operações públicas que <b>protegem a regra</b> de negócio                               |
| Herança        | a parte comum, escrita uma única vez            | <b>extends</b> : relação <b>é-um</b>                                                                              |
| Polimorfismo   | <b>qual</b> implementação vai atender a chamada | <code>conta.sacar()</code> executa o método da classe <b>real</b> do objeto, decidido em <b>tempo de execução</b> |

Duas confusões comuns sobre esses pilares. **Encapsulamento não é "gerar getter e setter para todo atributo"**: um `setSaldo` público expõe o estado tão livremente quanto um atributo público faria, e a regra de negócio ("não sacar além do saldo") fica sem dono. Encapsular é expor **comportamento** controlado, não simplesmente esconder o campo atrás de dois métodos que fazem a mesma coisa que acesso direto faria. O **polimorfismo que importa** é o de **sobrescrita**, resolvido em *runtime* pela classe real do objeto — a sobrecarga é resolvida em

**compilação**, pelos tipos declarados, e por isso alguns autores nem a contam como polimorfismo "de verdade".

Herança é o pilar com **contraindicação**: ela acopla a subclasse à implementação interna do pai. Quando a relação real é "**tem-um**" e não "**é-um**", a escolha correta é **composição** (Capítulo 4).

### SOLID — os cinco princípios

| Letra | Princípio                                                                          |
|-------|------------------------------------------------------------------------------------|
| S     | <i>Single Responsibility</i> — uma responsabilidade por classe                     |
| O     | <i>Open/Closed</i> — aberta a extensão, fechada a modificação                      |
| L     | <b>Liskov</b> — subtipo substitui o tipo base sem quebrar o programa               |
| I     | <i>Interface Segregation</i> — interfaces pequenas e específicas                   |
| D     | <i>Dependency Inversion</i> — depender de abstração, não de implementação concreta |

#### 10.3.1 Liskov em detalhe: a violação mais cobrada

##### Retângulo e Quadrado — a herança que compila e ainda assim quebra

| O código                                                                                                                                                                                                                                      | O que acontece                                                                                                                                                                                                                                                                        |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>class Retangulo {   setLargura; setAltura;   area() }  class Quadrado extends Retangulo — e cada <i>setter</i> muda <b>os dois</b> lados, para manter o quadrado quadrado  r.setLargura(5); r.setAltura(4); assert r.area() == 20;</pre> | <p>quem usa esta classe assume: mudar a largura não muda a altura</p> <p>a classe está coerente consigo mesma...</p> <p>... mas com um <b>Quadrado</b> no lugar do <b>Retangulo</b>, a área dá <b>16</b>. O código do cliente, que nunca soube da existência do quadrado, quebrou</p> |

Liskov não fala sobre a subclasse isoladamente: fala sobre **quem consome o supertipo**. O critério prático: a subclasse não pode **exigir mais** do que o pai exigia (fortalecer uma pré-condição), não pode **entregar menos** do que o pai entregava (enfraquecer uma pós-condição), e não pode quebrar uma **invariante** que o cliente já podia supor como verdadeira. O exemplo canônico do outro lado é o **Pinguim** que herda de **Ave** com um método **voar()**: a hierarquia parece natural biologicamente, mas o contrato de código não fecha.

##### Cuidado — confusão comum

O próprio compilador do Java já barra **três** violações de Liskov na sobrescrita: o método sobrescrito não pode **reduzir a visibilidade** em relação ao pai, não pode declarar uma exceção **checked mais ampla** que a do pai, e o tipo de retorno precisa ser o mesmo ou um **subtipo** dele (covariante). O que o compilador **não** consegue barrar é a quebra de contrato **semântico** (como

no exemplo do Retângulo/Quadrado) — e é exatamente ali que a prova cobra o entendimento real do princípio.

#### Cuidado — confusão comum

ISP (interface enxuta) e SRP (uma responsabilidade por classe) não são o mesmo princípio, apesar de soarem parecidos — ISP fala de **interface**, SRP fala de **classe**.

## 10.4 Interface x classe abstrata

|                       | Classe abstrata                               | Interface                                       |
|-----------------------|-----------------------------------------------|-------------------------------------------------|
| Construtor            | <b>tem</b>                                    | <b>não tem</b>                                  |
| Atributo de instância | tem (com estado)                              | só <code>public static final</code> (constante) |
| Herança múltipla      | <b>não</b> — só uma classe pode ser estendida | <b>sim</b> — várias podem ser implementadas     |
| Método concreto       | sim, desde sempre                             | sim, a partir do Java 8 (default e static)      |
| Palavra-chave         | <code>extends</code>                          | <code>implements</code>                         |

Regra de bolso: uma classe **estende uma** classe (abstrata ou não) e **implementa várias** interfaces. Use classe abstrata quando há **estado e comportamento comuns** genuínos a compartilhar entre subclasses; use interface quando o que importa é apenas o **contrato** a ser cumprido.

#### Cuidado — confusão comum

Desde o Java 8, interfaces têm métodos `default` com corpo implementado — "interface não pode ter implementação" está desatualizado e é falso hoje. O que a interface continua **sem ter** é **construtor** e **atributo de instância** — é por aí que a distinção real se sustenta. Java não tem herança múltipla de **classes** (só de **tipo**, via múltiplas interfaces implementadas).

## 10.5 Coleções (*Collections*)

#### Contrato x implementação — por que o framework é desenhado assim

`List`, `Set` e `Map` são **interfaces** — dizem que **garantia** você tem (ordem, unicidade, associação chave-valor) e nada sobre **como** ela é entregue internamente. `ArrayList` e `LinkedList` são duas **implementações** diferentes da mesma interface `List`, com os custos opostos das estruturas de dados vistas no Capítulo 9 (Seção 9.10). Por isso a boa prática é declarar pela interface (`List<String> lista = new ArrayList<>()`): trocar a implementação depois não exige mexer em quem a usa.

| Interface | Implementações                     | Característica               |
|-----------|------------------------------------|------------------------------|
| List      | ArrayList,<br>LinkedList           | ordenada, permite duplicatas |
| Set       | HashSet, TreeSet,<br>LinkedHashSet | sem duplicatas               |
| Map       | HashMap, TreeMap,<br>LinkedHashMap | associação chave-valor       |

**ArrayList** (array dinâmico, acesso  $O(1)$  por índice) x **LinkedList** (lista ligada, inserção/remoção  $O(1)$  no meio se já se tem o nó). **HashMap** (sem ordem garantida) x **LinkedHashMap** (preserva ordem de inserção) x **TreeMap** (mantém ordenação pela chave). **getOrDefault** retorna o valor **padrão** fornecido se a chave não existir — não lança exceção nem retorna um código de erro. **==** compara **referência** (mesmo objeto na memória); **.equals()** compara **conteúdo**.

### O contrato equals/hashCode: por que um objeto "desaparece" de um HashSet

Coleções com "Hash" no nome não procuram um elemento comparando com **todos** os demais — elas calculam o **hashCode()** do objeto para achar o **compartimento** correto, e só dentro dele usam **equals()** para confirmar a igualdade exata. Isso impõe um contrato obrigatório:

se **a.equals(b)** é **true**, então **a.hashCode() == b.hashCode()** **obrigatoriamente**

A recíproca **não** vale — dois objetos diferentes podem compartilhar o mesmo *hash* (é colisão, e é um evento normal e esperado).

A consequência prática cobrada em prova: sobrescrever **equals** e **esquecer** de sobrescrever **hashCode** junto produz um bug silencioso. Dois objetos considerados "iguais" pelo **equals** caem em compartimentos **diferentes** do hash (porque o **hashCode** herdado de **Object** usa identidade de memória), então o **HashSet** aceita a duplicata sem reclamar, e **contains/get** de um **HashMap** **não encontra** um objeto que está fisicamente lá dentro. Nada estoura — só dá a resposta errada silenciosamente, o pior desenho possível para um item de "qual a saída deste código". **TreeSet** e **TreeMap** **não** usam **hashCode** — eles ordenam os elementos, e por isso dependem de **compareTo()** (via **Comparable**) ou de um **Comparator** explícito. Uma classe sem nenhum dos dois dispara **ClassCastException** ao ser inserida num **TreeSet** — e ao mesmo tempo funciona perfeitamente bem num **HashSet**.

#### 10.5.1 String: imutabilidade e StringBuilder

**String** é **imutável**: toda operação que parece "alterar" (**concat**, **+**, **replace**, **toUpperCase**) na verdade devolve um **objeto novo**, deixando o original intacto. Por isso, concatenar dentro de um laço cria um objeto novo descartado a cada iteração — em concatenação intensiva, usa-se **StringBuilder** (mutável, não sincronizado, mais rápido) ou **StringBuffer** (mutável e sincronizado entre threads, mais lento).

Literais de string iguais compartilham o **string pool** (uma área especial de memória), então **"a" == "a"** costuma dar **true**; já **new String("a") == "a"** dá **false**, porque o **new** força a criação de um objeto **fora** do pool, mesmo com o mesmo conteúdo. Comparação de conteúdo deve sempre usar **.equals()**, nunca **==**.

**Cuidado — confusão comum**

"String é mutável porque posso reatribuir a variável" é falso: o que muda ao reatribuir é a **referência** guardada na variável, não o objeto **String** original, que permanece intacto na memória. **StringBuilder** (não sincronizado, mais rápido) e **StringBuffer** (sincronizado, mais lento) não são intercambiáveis.

## 10.6 JVM, JPA e Java EE

**JVM: por que existe, e as duas áreas de memória**

O compilador Java não gera código de máquina diretamente — gera **bytecode** (`.class`), uma instrução intermediária que nenhuma CPU física executa nativamente. Quem executa é a **JVM**, e existe uma implementação de JVM para cada sistema operacional — é exatamente daí que vem o lema *write once, run anywhere*: o `.class` é idêntico em Windows e Linux, o que muda é o intérprete que o executa.

A separação de memória explica boa parte das perguntas sobre o tema:

| Área         | O que guarda                                                                                                                                            |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Stack</b> | uma por <b>thread</b> : um quadro por chamada de método, com parâmetros, variáveis locais e <b>referências</b> . O quadro morre quando o método retorna |
| <b>Heap</b>  | os <b>objetos</b> em si, compartilhados por todas as threads. Sobrevivem ao método que os criou, enquanto alguma referência ainda os alcançar           |

A variável local mora na *stack*; o objeto que ela referencia mora no *heap*. É por isso que "passar o objeto" para um método é, na prática, passar uma cópia da referência (Capítulo 9) — e é por isso que a *stack* é literalmente uma pilha (a mesma estrutura LIFO da Seção 9.10, agora funcionando como mecanismo de execução): recursão sem fim estoura a pilha (**StackOverflowError**); criar objetos demais estoura o heap (**OutOfMemoryError**).

**Coleta de lixo (*garbage collection*)**: o coletor libera o que **não é mais alcançável** a partir das raízes (variáveis vivas nas pilhas de todas as threads, campos estáticos) — não simplesmente o que "não está mais sendo usado" no sentido intuitivo. Consequências cobradas: Java **não tem destrutor determinístico** (não se sabe **quando** exatamente um objeto será coletado) e `System.gc()` é apenas uma **sugestão** à JVM, não uma ordem imediata. Um recurso externo (arquivo, conexão de rede) precisa ser fechado explicitamente, com **try-with-resources** — o coletor de lixo cuida de memória, não de recursos externos (*handles*).

**JPA x Hibernate, EAGER x LAZY, problema N+1**

**JPA** é a **especificação** de persistência (a interface padronizada); **Hibernate** é uma **implementação** concreta dessa especificação (o motor de ORM que faz o trabalho de fato). **EAGER** (carregamento antecipado, junto com a entidade principal) x **LAZY** (carregamento sob demanda, só quando a relação é efetivamente acessada).

O **problema N+1**: buscar uma lista de  $N$  entidades e, para cada uma, disparar uma consulta **separada** para carregar sua relação associada — resultando em  $N + 1$  consultas ao banco em vez de uma só. Resolve-se combinando **LAZY** (não carrega a relação por padrão) com **JOIN FETCH** explícito na consulta específica que realmente precisa da relação naquele momento —

trazendo tudo numa única consulta otimizada, só quando necessário.

### Cuidado — confusão comum

Diante do problema N+1, "usar EAGER em tudo" ou "deixar o JPA decidir sozinho" não são a solução — a resposta correta é **LAZY** como padrão, combinado com **JOIN FETCH** pontual na consulta que precisa da relação.

### Java EE / Jakarta EE

**Java EE/Jakarta EE** reúne especificações como EJB, JPA, JMS e JSF. **Jakarta** é a continuação do Java EE sob governança da Eclipse Foundation (o projeto migrou da Oracle) — a mudança visível mais citada é o **namespace: jakarta.\*** substituiu **javax.\*** nos pacotes.

## 10.6.1 Anotações que aparecem em leitura de código

| Anotação                                   | Papel                                                                                                                                       |
|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Spring — estereótipos de componente</i> |                                                                                                                                             |
| @Component                                 | bean genérico gerenciado pelo contêiner                                                                                                     |
| @Service                                   | camada de regra de negócio                                                                                                                  |
| @Repository                                | camada de <b>acesso a dados</b> ; acrescenta <b>tradução automática de exceções</b> de persistência para a hierarquia de exceções do Spring |
| @Controller / @RestController              | camada web; o @RestController já embute @ResponseBody                                                                                       |
| @Autowired                                 | injeta a dependência automaticamente                                                                                                        |
| <i>JPA — mapeamento objeto-relacional</i>  |                                                                                                                                             |
| @Entity                                    | marca a classe como entidade persistente                                                                                                    |
| @Id                                        | identifica a chave primária (com @GeneratedValue para geração automática)                                                                   |
| @Table / @Column                           | renomeiam tabela e coluna (opcionais, senão usam o nome do campo/classe)                                                                    |
| @ManyToOne                                 | <b>muitos</b> registros desta entidade para <b>um</b> da outra (é o lado dono da relação, que guarda a chave estrangeira)                   |
| @OneToMany                                 | o inverso do anterior; costuma vir acompanhado de mappedBy                                                                                  |
| @OneToOne, @ManyToMany                     | relação um-para-um e muitos-para-muitos                                                                                                     |

O par mínimo para uma entidade JPA funcionar é **@Entity + @Id** — todo o resto das anotações é opcional.

**Cuidado — confusão comum**

A cardinalidade de `@ManyToOne` se lê do lado de quem a declara para o outro lado: **muitos** pedidos para **um** cliente — inverter essa leitura é o erro mais comum. Atenção também a nomes de anotação inventados que soam plausíveis mas não existem (`@Persistent`, `@PrimaryKey`) — o nome exato importa.

## 10.7 Recursos modernos (Java 17/21 LTS)

**Sealed, threads virtuais, streams**

**sealed/non-sealed/final**: uma classe **sealed** exige que suas subclasses sejam explicitamente declaradas (via `permits`, ou estando no mesmo arquivo), e cada subtipo deve ser marcado como **final**, **sealed** ou **non-sealed** (que reabre a hierarquia para herança livre a partir dali).

**Threads virtuais (*Project Loom*) x threads de plataforma**: milhares de threads **virtuais** compartilham um número muito menor de *carrier threads* de plataforma (não é uma relação um-para-um) — brilham especialmente em cenários dominados por espera de I/O.

**Stream: um pipeline preguiçoso**

Um *stream* **não é uma coleção** — é um **fluxo de processamento** sobre uma fonte de dados, com três partes:

fonte (`lista.stream()`) → operações **intermediárias** → operação **terminal**

Operações **intermediárias** (`filter`, `map`, `sorted`, `distinct`, `limit`) devolvem **outro stream** e são **preguiçosas** — nenhuma delas executa no momento em que é escrita no código. Operações **terminais** (`collect`, `forEach`, `reduce`, `count`, `anyMatch`) **disparam** a execução de todo o pipeline acumulado e o encerram.

Três consequências diretas dessa lógica preguiçosa: um pipeline **sem operação terminal não executa nada** (nem o `map` chega a rodar); o stream **não altera a fonte original** — `map` devolve valores novos, a lista original permanece intacta; e um stream é de **uso único** — tentar reaproveitar o mesmo objeto de stream depois de consumido dispara `IllegalStateException`. Trio funcional, resumido: **filter** escolhe **quais** elementos passam adiante (não transforma nenhum); **map** transforma **cada** elemento, um a um; **reduce** combina todos num **único** resultado final. Uma **lambda** é apenas a forma curta de escrever a implementação de uma interface de método único (`Predicate`, `Function`) no lugar de uma classe anônima — é açúcar de sintaxe, não um novo mecanismo de tipo.

### 10.7.1 var, record e switch como expressão

**var (Java 10)**: infere o tipo de uma variável **local** a partir do que a inicializa, em **tempo de compilação** — e o tipo inferido **fica fixo** a partir daí; Java continua estaticamente tipado, não virou dinamicamente tipado. Exige um inicializador (`var x`; sozinho não compila) e não se aplica a atributo de instância, parâmetro de método nem tipo de retorno.

**record (Java 16)**: classe de dados imutável, declarada por um cabeçalho compacto — `record Ponto(int x, int y) { }`. O compilador gera automaticamente: um construtor canônico (que pode ser sobrescrito na forma compacta para adicionar validação), métodos de acesso no formato `p.x()`, `p.y()` (**sem** prefixo `get`), `equals/hashCode` comparando componente a componente, e

`toString` no formato `Ponto[x=1, y=2]`. Os campos de um record são  **finais**; o record em si é implicitamente **final** e já estende `java.lang.Record`, portanto **não pode estender outra classe** — mas **pode implementar interfaces**. Não aceita campo de instância adicional além dos declarados no cabeçalho (só campos estáticos).

**switch como expressão (Java 14)**: devolve um valor diretamente e, na forma com seta (`->`), executa **só** o ramo correspondente — sem **break** e sem *fall-through* entre ramos (use `yield` dentro de um bloco com chaves quando precisar de lógica mais elaborada num ramo).

### Cuidado — confusão comum

"`var` torna a variável de tipo dinâmico" é falso: depois de inferido, atribuir um valor de tipo incompatível continua sendo erro de compilação. "O record gera *getters* no padrão `getX()`" é falso — o acessor chama-se apenas `x()`, sem prefixo. **sealed não é sinônimo de final**: o **final** proíbe qualquer herança; o **sealed** permite herança apenas pelos tipos explicitamente autorizados.

## 10.8 Leitura ativa de código em Java

Java é a linguagem onde erros gostam de se esconder **dentro** de uma estrutura que parece correta à primeira leitura — um **try-catch** que compila mas nunca captura a exceção certa, um escopo de variável que parece visível mas não é. O protocolo de leitura ativa (Capítulo 9, Seção 9.11) se aplica igualmente aqui: releia linha a linha, sem assumir.

### 10.8.1 Checked/unchecked disfarçadas em try-catch

- **Ordem dos catch**: um `catch (Exception e)` **antes** de um `catch (IOException e)` **não compila** — o `catch` mais genérico precisa vir **por último**, porque um `catch` mais específico depois de um genérico nunca seria alcançado.
- **Catch que não cobre o tipo lançado**: um método declara `throws SQLException`, mas o bloco que o chama só tem `catch (RuntimeException e)` — isso **não compila**, porque `SQLException` é **checked** e exige tratamento explícito ou propagação com `throws` no método chamador.
- **finally que engole a exceção**: um `return` dentro do bloco `finally` **sobrescreve** qualquer exceção lançada no `try/catch` anterior — a exceção original se perde silenciosamente, sem rastro.
- **Multi-catch**: `catch (IOException | SQLException e)` exige que as duas exceções listadas **não tenham relação de herança** entre si (uma não pode ser subtipo da outra) — senão não compila, por redundância.

### Cuidado — confusão comum

Um bloco com a ordem dos `catch` invertida (genérico antes do específico) **não compila e não funciona normalmente** — é erro de compilação, não um comportamento "que funciona mas é estranho". Uma exceção **checked** lançada dentro de um método sem `throws` e sem `try-catch` **não compila** — não é "ignorada silenciosamente em runtime". Erro de compilação e exceção em tempo de execução são fenômenos diferentes, e a prova testa se essa distinção está clara.

### 10.8.2 Escopo de variável

- **Shadowing** (sombreamento): uma variável local com o **mesmo nome** de um atributo da classe **esconde** o atributo dentro daquele método — `this.atributo` continua acessível explicitamente, mas `atributo` sozinho passa a se referir à variável local.
- **Variável efetivamente final em lambda/classe anônima**: uma variável local usada dentro de uma lambda ou classe anônima precisa ser **final** ou **efetivamente final** (nunca reatribuída depois de inicializada) — código que reatribui essa variável depois de criar a lambda **não compila**.
- **Escopo de bloco em for**: uma variável declarada **dentro** do cabeçalho do **for** não existe fora dele — usá-la depois do laço termina é erro de compilação, não um "valor residual" da última iteração.

#### Cuidado — confusão comum

Usar uma variável declarada dentro de um bloco (`if`, `for`, `try`) fora desse bloco é erro de compilação por variável fora de escopo — diferente de JavaScript, onde `var` pode vazar para o escopo da função inteira; em Java o escopo é sempre de **bloco**.

### 10.8.3 Armadilhas de threads virtuais: *pinning*

Uma thread virtual roda sobre uma *carrier thread* de plataforma. Se o código dentro de um bloco **synchronized** faz uma chamada **bloqueante** (I/O, banco de dados), a thread virtual pode ficar **presa** (*pinned*) à sua *carrier* em vez de liberá-la para outra thread virtual usar — isso anula justamente a vantagem de escala das threads virtuais, porque agora uma única operação bloqueante ocupa uma *carrier* inteira, impedindo outras threads virtuais de progredir nela. O cenário de risco típico: **synchronized** combinado com chamada bloqueante **dentro** do bloco sincronizado, sob alta concorrência.

#### Cuidado — confusão comum

Threads virtuais **não eliminam** toda preocupação com bloqueio ou concorrência — o cenário de **synchronized** + I/O bloqueante ainda pode prender a thread virtual à sua *carrier*. Elas reduzem o **custo de criar** threads (permitindo milhares delas), não eliminam o cuidado com seções críticas de código.

## Capítulo 11

# Frontend

### Ficha do edital

Tecnologias e práticas frontend web — HTML, CSS, UX, Ajax, frameworks (VueJS, Angular, React); padrões de frontend; SPA e PWA; UX (sistemas de gestão de conteúdo, arquitetura de informação, portais, workflow, acessibilidade, usabilidade).

## 11.1 HTML e CSS

### Por que semântica HTML é assunto, não decoração

Visualmente, `<div class="menu">` e `<nav>` podem renderizar de forma idêntica — a diferença é que só o segundo **informa o papel** daquele bloco de conteúdo. Quem lê esse papel são **programas**: o leitor de tela, que anuncia "navegação" e permite ao usuário saltar direto para ela; o buscador, que pondera o conteúdo de `<main>` de forma diferente do conteúdo de `<footer>`; e o próprio navegador, que já entrega comportamento de teclado pronto em elementos nativos (como `<button>` responder a Enter e Espaço automaticamente). É daí que vem a "regra número um do ARIA" vista adiante: se existe tag nativa com a semântica certa, ela vence qualquer atributo acrescentado depois. A separação clássica — **HTML estrutura, CSS apresenta, JS comporta** — é a mesma lógica de baixo acoplamento aplicada à construção de uma página.

### Box model

**content** → **padding** (espaço interno, dentro da borda) → **border** (a borda em si) → **margin** (espaço externo, fora da borda). O tamanho final **visível** de um elemento é **content + padding + border** — a **margin** conta apenas para o espaço **ao redor** do elemento, não para o tamanho do próprio elemento.

Com **box-sizing**: **border-box**, a propriedade **width** já **inclui** padding e border dentro do valor declarado. Com o padrão (**content-box**), **width** se refere só ao conteúdo, e padding/border se **somam por fora** do valor declarado — essa diferença de **box-sizing** é onde nasce boa parte das pegadinhas de cálculo de tamanho de elemento.

### Flexbox x Grid

**Flexbox** é **unidimensional** — distribui itens ao longo de **um** eixo por vez (linha **ou** coluna). **Grid** é **bidimensional** — define linhas e colunas simultaneamente. Um não substitui o outro: Grid é natural para a **estrutura geral** da página; Flexbox é natural para **alinhar o conteúdo** dentro de cada área já definida.

### Especificidade de seletores CSS

Quando duas regras declaram a mesma propriedade para o mesmo elemento, vence a de **maior especificidade**: id (peso 100) > classe / atributo / pseudoclasse (peso 10) > elemento (peso

1). A **ordem de declaração** no arquivo só desempata quando as especificidades são **iguais** — uma regra de especificidade menor declarada **depois** ainda perde para uma de especificidade maior declarada antes. **!important** ignora toda essa hierarquia e vence por cima de tudo, o que o torna desaconselhado como prática normal.

#### @import x <link>

Os dois trazem uma folha de estilo externa, mas **@import** é resolvido **dentro do CSS**, em série (bloqueia o carregamento seguinte até resolver), enquanto **<link>** no HTML permite **download em paralelo** de várias folhas — por isso **<link>** é a prática recomendada em produção. **@import não é** um mecanismo de responsividade, apesar de aceitar condição de mídia como parâmetro.

#### Cuidado — confusão comum

Padding é interno, margin é externo — não inverter. "A última regra declarada sempre vence" é falso: só vence entre especificidades **iguais**; um seletor de **id** ganha de um de classe mesmo declarado antes dele no arquivo.

## 11.2 SPA x PWA

|            | SPA ( <i>Single Page Application</i> )                      | PWA ( <i>Progressive Web App</i> )                               |
|------------|-------------------------------------------------------------|------------------------------------------------------------------|
| O que é    | app numa única página; troca conteúdo <b>sem recarregar</b> | site que se comporta como um app nativo                          |
| Chave      | roteamento no cliente, sem <i>full reload</i>               | <b>Service Worker</b> , funcionamento offline, <b>instalável</b> |
| Depende de | JavaScript no navegador                                     | recursos web progressivos                                        |

O **Service Worker** é um script que roda **em segundo plano**, separado da página em si, funcionando como um **proxy** entre a aplicação e a rede — é daí que vêm o cache e o funcionamento offline: ele intercepta as requisições e decide servir da rede ou de um cache local. Não tem acesso direto ao DOM e **exige HTTPS** (só **localhost** é exceção durante desenvolvimento), precisamente porque interceptar requisições numa conexão não cifrada seria um vetor de ataque grave. O **manifest** (**manifest.json**) é o outro pilar da PWA: define nome, ícones e modo de exibição que permitem a **instalação** no dispositivo.

#### Cuidado — confusão comum

Service Worker é característica da **PWA**, não requisito de SPA — atribuí-lo à SPA é um erro comum. "SPA se instala no sistema operacional como um app nativo" é também característica de **PWA**, não de SPA em si. Nenhuma das duas depende exclusivamente de um framework específico — ambas podem ser feitas com JavaScript puro.

### 11.2.1 Onde o HTML é montado: CSR, SSR, SSG e *hydration*

#### O problema que a SPA criou, e as três respostas

A SPA resolveu a navegação fluida, mas criou um problema novo: se o HTML é montado inteiramente pelo JavaScript no navegador, o servidor entrega uma página **vazia** de início, e o usuário espera o pacote de código baixar e executar antes de ver qualquer conteúdo. Daí três estratégias, distinguidas por **onde** e **quando** o HTML é efetivamente gerado:

| Estratégia                        | HTML é gerado...                           | O que se ganha e o que se paga                                                                                                      |
|-----------------------------------|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>CSR</b> ( <i>client-side</i> ) | no <b>navegador</b> , a cada tela          | servidor leve e navegação instantânea depois da carga inicial; <b>primeira</b> pintura lenta e indexação por buscadores mais frágil |
| <b>SSR</b> ( <i>server-side</i> ) | no <b>servidor</b> , a cada requisição     | conteúdo visível e indexável de imediato; custo de CPU do servidor a cada requisição                                                |
| <b>SSG</b> (estático)             | no momento do <b>build</b> , uma única vez | o mais rápido e barato de servir; o conteúdo só muda com uma nova publicação                                                        |

**Hydration** é o que costura SSR e SPA no mesmo produto: o servidor manda HTML já pintado visualmente, mas HTML puro não tem comportamento interativo — quando o JavaScript chega ao navegador, ele "hidrata" aquela marcação, reassociando componentes, estado interno e ouvintes de evento a ela. A consequência prática que costuma virar item de prova: existe uma **janela de tempo** em que a página **parece pronta visualmente e não responde a cliques**, porque foi pintada mas ainda não foi hidratada.

SSR/CSR decidem **quem monta o HTML**; Service Worker e cache offline decidem **o que acontece quando não há rede** — são eixos **independentes**: uma SPA renderizada no cliente pode ser uma PWA completa, e uma página com SSR pode não ser PWA nenhuma.

## 11.3 Frameworks

|           | React                   | Angular                     | Vue                   |
|-----------|-------------------------|-----------------------------|-----------------------|
| Tipo      | biblioteca de UI (Meta) | framework completo (Google) | framework progressivo |
| Linguagem | JS/JSX                  | TypeScript                  | JS                    |

React é deliberadamente uma **biblioteca focada em UI** (a composição de outras peças — roteamento, gerenciamento de estado global — fica a cargo do desenvolvedor); Angular é um **framework completo**, com roteamento, injeção de dependência e formulários já embutidos e opinativos sobre como o código deve ser organizado.

**Cuidado — confusão comum**

React não é escrito nativamente em TypeScript (embora aceite ser usado com ele) — o par React/framework-completo e Angular/biblioteca costuma ser invertido em prova.

## 11.4 Ajax

**Ajax** (*Asynchronous JavaScript and XML*, nome histórico) é o conjunto de técnicas para fazer requisições **assíncronas** ao servidor sem recarregar a página inteira — é a base do comportamento de uma SPA. Hoje implementado tipicamente via **fetch** ou **XMLHttpRequest** (XHR), com dados trafegando majoritariamente em JSON, apesar do "X" histórico do nome se referir a XML.

## 11.5 UX, usabilidade e acessibilidade

**Três recortes que se cruzam, mas não são sinônimos**

| Recorte        | Pergunta que responde                                                                    | Como se mede                                                                  |
|----------------|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Usabilidade    | o usuário <b>típico</b> consegue cumprir a tarefa com facilidade?                        | eficácia, eficiência e satisfação <b>num contexto de uso</b> declarado        |
| Acessibilidade | <b>pessoas com deficiência</b> conseguem usar o sistema?                                 | conformidade com critérios objetivos (WCAG A/AA/AAA)                          |
| UX             | como é a <b>experiência inteira</b> , inclusive antes e depois do uso propriamente dito? | percepção, expectativa, confiança — mais ampla e menos diretamente mensurável |

Duas relações resolvem a maior parte das questões sobre o tema. **UX contém** usabilidade como um de seus atributos — não são a mesma coisa, e **UI** é apenas a camada visual através da qual a experiência acontece, um subconjunto ainda menor. **Acessibilidade não é um caso particular de usabilidade**: um sistema pode ser cômodo para um usuário sem deficiência e **impossível** de usar para quem navega por leitor de tela; e um sistema formalmente acessível (que passa em todos os critérios técnicos) ainda pode ser péssimo de usar na prática. Os dois conceitos se cruzam, não se contêm um no outro.

A diferença de **natureza**: usabilidade é objetivo de **qualidade** de produto; acessibilidade é **requisito com base normativa** — WCAG (W3C) como critério técnico internacional, **eMAG** como o modelo derivado dele adotado pelo governo brasileiro, e a Lei Brasileira de Inclusão como obrigação legal em serviços públicos.

**Nielsen x ISO 9241-11: duas listas, não misture**

Os **cinco atributos de usabilidade de Nielsen**: **facilidade de aprendizado** (*learnability*), **eficiência**, **memorabilidade** (o usuário que volta depois de um tempo relembra como usar), **poucos erros** (e recuperáveis quando acontecem), e **satisfação**. A **ISO 9241-11** usa outro

trio, mais compacto: **eficácia**, **eficiência** e **satisfação**, sempre avaliados **num contexto de uso** explicitamente declarado (usuários, tarefas, equipamentos e ambiente específicos).

#### Cuidado — confusão comum

"Eficácia" pertence à lista da ISO 9241-11; "memorabilidade" e "poucos erros" pertencem à lista de Nielsen — misturar as duas listas é o erro mais comum ao responder sobre o tema.

#### Acessibilidade: WCAG, níveis de conformidade, ARIA

**WCAG** (W3C) organiza a acessibilidade em quatro princípios — **POUR**: **P**erceptível, **O**perável, **C**ompreensível (*Understandable*), **R**obusto. Cada critério de sucesso individual tem um **nível de conformidade**: **A** (mínimo), **AA** (o exigido na maioria das normas e contratos públicos), **AAA** (máximo, nem sempre alcançável no site inteiro sem comprometer outros objetivos de design). No Brasil, o **eMAG** adapta esses critérios para o governo.

**ARIA** (*Accessible Rich Internet Applications*): conjunto de atributos (`role`, `aria-label`, `aria-hidden`, `aria-live`) que acrescenta **semântica** a elementos **dinâmicos** ou construídos sem uma tag nativa correspondente, para que leitores de tela anunciem função, nome e estado corretamente. A **regra número um do ARIA é não usar ARIA**: se existe um elemento HTML nativo com a semântica certa (`<button>`, `<nav>`, `<label>`), o nativo deve ser usado — ARIA é complemento para o que o HTML puro não cobre, e ARIA mal aplicado pode piorar a acessibilidade em vez de melhorá-la (por exemplo, dizer a um leitor de tela que algo é um botão quando ele não se comporta como um).

#### Arquitetura de informação e CMS

**Arquitetura de informação**: como o conteúdo de um site é organizado, categorizado e navegado, de forma que o usuário encontre o que procura. **CMS** (sistema de gestão de conteúdo): plataforma que permite criar e gerenciar conteúdo sem escrever código a cada publicação (WordPress, portais corporativos com fluxo editorial/*workflow* de aprovação).

## 11.6 Mobile: multiplataforma x nativo

**Flutter** usa **Dart**; **React Native** usa JavaScript; **Ionic** usa tecnologias web; **Xamarin** usa C#. **Mobile nativo**: Android usa **Kotlin** (linguagem recomendada atualmente, com Java como opção mais antiga); iOS usa **Swift** (com Objective-C como opção mais antiga).

## 11.7 Leitura ativa de código: CSS e React

Boa parte das questões de frontend não pede definição — pede para **prever a renderização** de um trecho de HTML/CSS/JS. O protocolo de leitura ativa do Capítulo 9 se aplica aqui: resolva no papel, propriedade por propriedade, antes de comparar com alternativas prontas.

### 11.7.1 Pegadinhas visuais em CSS

#### A ordem certa para somar regras de CSS

1. **Box model primeiro:** já visto acima — confira `box-sizing` antes de calcular qualquer tamanho.
2. **Pseudo-elementos:** `::before` insere conteúdo **antes** do conteúdo real do elemento (mas ainda **dentro** dele, no DOM renderizado); `::after`, **depois**. Sem uma declaração `content: ...`, o pseudo-elemento **não aparece** de jeito nenhum, mesmo que outras propriedades estejam definidas nele.
3. **content: attr(x):** imprime o **valor** do atributo `x` do elemento HTML como texto — se o elemento não tiver esse atributo, o pseudo-elemento aparece **vazio**, sem gerar erro.
4. **Flex/Grid por último:** `flex-direction: row-reverse` inverte a **ordem visual** dos itens, sem alterar a ordem deles no DOM/HTML subjacente; `justify-content` age no **eixo principal**, `align-items` no **eixo transversal** — trocar os dois é o distrator mais comum de layout.

#### Exemplo

Um `<span data-nivel="urgente">` com CSS `span::before { content: attr(data-nivel) ": "; font-weight: bold; }` produz, **antes** do texto original do `span`, o literal **"urgente: "** em negrito — não o texto `"data-nivel"` (o nome do atributo, e não seu valor, que é o erro mais comum de leitura de `attr()`).

#### Cuidado — confusão comum

Questões de CSS costumam somar duas ou três regras na mesma pergunta (`box-sizing` + pseudo-elemento + `flex-direction`, por exemplo), e o erro de interpretação costuma estar na **combinação**, não em nenhuma regra isolada. Resolva cada camada separadamente antes de somar o resultado final.

### 11.7.2 Escopo e *closures* em JavaScript

#### O clássico laço com `var`

`var` tem escopo de **função** (não de bloco) e sofre *hoisting* (é içada para o topo da função em que está). Dentro de um laço, isso significa que existe **uma única** variável, compartilhada por todas as funções criadas dentro do laço — quando essas funções forem finalmente executadas (por exemplo, num *callback* assíncrono), todas leem o valor **final** que a variável tinha ao término do laço.

| Código                                                                            | Saída                                                                               |
|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <pre>for (var i = 0; i &lt; 3; i++)   fs.push(() =&gt;     console.log(i));</pre> | <b>3, 3, 3</b> — um único <code>i</code> compartilhado, que vale 3 ao final do laço |
| <pre>for (let i = 0; i &lt; 3; i++)   fs.push(() =&gt;     console.log(i));</pre> | <b>0, 1, 2</b> — <code>let</code> cria uma variável nova a cada iteração            |

`let` e `const` (ES6) têm escopo de **bloco**. `const` impede a **reatribuição** da própria variável, mas **não congela** o objeto referenciado — `const obj = {}; obj.x = 1;` é perfeitamente válido, porque o que está protegido é a ligação nome-valor, não o conteúdo interno do objeto. `==` faz **coerção de tipo** antes de comparar ("`1`" `==` `1` resulta `true`); `===` compara valor e tipo, sem coerção.

#### Cuidado — confusão comum

Um laço com `var` **não** imprime 0, 1, 2 — essa saída descreve o comportamento de `let`, não de `var`. Em Java, o escopo de bloco é sempre a regra (Capítulo 10) — não confundir o comportamento das duas linguagens.

### 11.7.3 Renderização no React

**key em listas:** React usa a `key` para identificar qual item de uma lista mudou, foi adicionado ou removido entre duas renderizações sucessivas. Usar o **índice do array** como `key` numa lista que pode reordenar, inserir ou remover itens **quebra** esse rastreamento (o React pode reaproveitar o componente errado para o item errado, causando bugs visuais sutis) — a prática recomendada é usar um **identificador estável** vindo dos próprios dados, nunca o índice posicional.

**useState é assíncrono e agrupado (*batched*):** chamar a função que atualiza o estado **não** atualiza a variável **na mesma linha** de código — o novo valor só se reflete na **próxima renderização** do componente. Um trecho que chama `setContador(contador + 1)` duas vezes seguidas **não** soma 2 ao contador, porque as duas chamadas leem o mesmo valor "antigo" de `contador` dentro do mesmo ciclo de evento — para acumular corretamente usa-se a forma funcional `setContador(c => c + 1)`, que recebe o valor mais atualizado em cada chamada.

**Reconciliação (Virtual DOM):** o React compara a árvore virtual nova com a anterior e aplica no DOM real **só as diferenças** — não recria a página inteira a cada renderização. Criar uma **função anônima nova a cada renderização** (por exemplo, `onClick={() => ...}` sem memoização) não quebra a reconciliação em si, mas pode gerar **re-renderizações desnecessárias** em componentes filhos que foram otimizados com `memo`.

#### Cuidado — confusão comum

Usar o índice do array como `key` **não** é sempre seguro — falha justamente quando a lista pode reordenar ou ter itens inseridos/removidos no meio. `setState/useState` **não** atualiza de forma síncrona e imediata — é assíncrono e agrupado dentro do mesmo *handler* de evento. Virtual DOM (estrutura em memória usada para calcular diferenças) não é o mesmo que o DOM real

do navegador.

## Capítulo 12

# Gestão e Governança de TI

### Ficha do edital

Gerenciamento de projetos (conceitos, áreas, projetos/programas/portfólio; abordagens tradicional, híbrida e ágil — Scrum, Lean, Kanban; Guia Scrum); processos e grupos de processos; gestão de riscos; ITIL v4; COBIT 2019; gestão e modelagem de processos com BPMN.

Este capítulo explica cada framework de gestão e governança do zero: o que ele resolve, por que tem a estrutura que tem, e como aplicar seus conceitos a um cenário. A versão resumida — focada no que vira pegadinha de prova — está no capítulo correspondente da **apostila**/; aqui o objetivo é entender o **porquê** de cada estrutura, não só memorizar o número certo.

## 12.1 Gerenciamento de projetos (PMBOK)

### 12.1.1 Projeto, programa e portfólio

#### Três escalas de coordenação de trabalho

As três palavras descrevem **escalas diferentes** de esforço organizado, uma contendo a outra:

- **Projeto**: esforço **temporário** (tem início e fim definidos) empreendido para criar um resultado **único** (produto, serviço ou resultado). Terminou de entregar o que foi pedido, o projeto acaba — não é um esforço contínuo.
- **Programa**: um **grupo de projetos relacionados**, gerenciados de forma coordenada, porque juntos entregam benefícios que nenhum deles entregaria isoladamente. A coordenação existe porque os projetos compartilham dependências (um alimenta o outro) ou objetivo comum.
- **Portfólio**: o **conjunto** de programas, projetos e outros trabalhos de uma organização, agrupados para viabilizar o **gerenciamento estratégico** — os itens de um portfólio não precisam ter relação direta entre si, o que os une é estarem todos alinhados aos objetivos estratégicos da organização.

#### Do projeto ao portfólio

Uma empresa quer "se tornar líder em atendimento digital". Isso vira um **portfólio** de iniciativas estratégicas. Dentro dele, um **programa** "Modernização do Atendimento" agrupa três **projetos** relacionados: (1) construir um novo app de atendimento, (2) migrar o banco de dados de clientes, (3) treinar a equipe de suporte. Os três são coordenados porque o app depende do banco migrado, e o treinamento depende do app pronto — não fazem sentido soltos. Já o portfólio da empresa também contém outros programas sem relação com este (ex.: "Redução de custos operacionais"), unidos só pelo fato de servirem à mesma estratégia.

### 12.1.2 PMBOK 6ª edição: processos, grupos e áreas

#### A matriz de 5 grupos × 10 áreas

O PMBOK 6 descreve o gerenciamento de projetos como uma **matriz de processos**: 49 processos, cada um cruzando um **grupo de processos** (quando, no ciclo de vida do projeto, ele acontece) com uma **área de conhecimento** (sobre o que ele trata).

**5 grupos de processos** (a dimensão *temporal* — não são fases sequenciais rígidas, um projeto pode voltar à Execução depois de passar pelo Monitoramento, por exemplo):

1. **Iniciação**: autoriza formalmente o projeto (ou uma fase dele) a começar.
2. **Planejamento**: define e refina os objetivos, e planeja a ação necessária para alcançá-los.
3. **Execução**: realiza o trabalho definido no plano.
4. **Monitoramento e Controle**: acompanha, revisa e regula o progresso e desempenho; identifica desvios e dispara ajustes.
5. **Encerramento**: finaliza formalmente o projeto, a fase ou o contrato.

**10 áreas de conhecimento** (a dimensão *temática* — o assunto de cada processo): integração, escopo, cronograma, custo, qualidade, recursos, comunicação, riscos, aquisições, partes interessadas.

#### Grupo de processos não é "fase do projeto"

É tentador ler os 5 grupos como 5 etapas sequenciais tipo cascata, mas não são: um projeto pode ter vários ciclos de Planejamento-Execução-Monitoramento dentro de uma única fase (é exatamente assim que um projeto ágil, iterativo, usa a mesma matriz do PMBOK 6). O grupo de processos é uma **categoria de natureza da atividade**, não uma marcação de calendário.

### 12.1.3 PMBOK 7ª edição: princípios e domínios de desempenho

#### Por que a 6ª virou 7ª

O PMBOK 6 é **prescritivo**: descreve *o que fazer*, processo por processo, com entrada → ferramenta → saída. Funciona bem quando o escopo do projeto é conhecido de antemão (obra, implantação de sistema com requisitos fechados) e mal quando o projeto **descobre** o escopo andando (a maioria dos projetos de software modernos, que usam abordagem ágil) — seguir os 49 processos como receita nesse cenário vira burocracia que não ajuda a entregar valor.

A 7ª edição inverteu o eixo: saiu do **processo** (uma sequência fixa de passos) e foi para o **princípio** (uma diretriz de julgamento, que o gerente aplica com discernimento, adaptando ao contexto). Com os princípios vieram os **domínios de desempenho** — áreas de resultado a *acompanhar* continuamente ao longo do projeto, não etapas a *executar* em ordem.

#### Os 12 princípios

- Ser um **administrador diligente/zeloso** (*steward*) — cuidado ético, responsável e íntegro com os recursos confiados ao gerente.
- Criar um **ambiente colaborativo** de equipe.
- Envolver-se efetivamente com as **partes interessadas**.
- **Focar no valor** — o resultado de negócio, não a entrega técnica em si.
- Reconhecer, avaliar e responder às interações do sistema (**pensamento sistêmico**) — o projeto não existe isolado, ele afeta e é afetado pelo ambiente ao redor.

- Demonstrar comportamentos de **liderança**.
- **Adaptar** (*tailoring*) a abordagem ao contexto do projeto — não existe um método único que sirva a todo projeto.
- Incorporar a **qualidade** nos processos e resultados.
- Navegar na **complexidade**.
- Otimizar as respostas aos **riscos**.
- Adotar **adaptabilidade e resiliência** — a capacidade de mudar de rumo e de se recuperar de contratemplos.
- Permitir a **mudança** para alcançar o estado futuro previsto.

### Associando cenário a princípio

"A analista percebeu que uma mudança na API de pagamentos de terceiros ia afetar o cronograma de três equipes diferentes, e ajustou o plano considerando esses efeitos cruzados.— isso é **pensamento sistêmico**: a analista reconheceu que o projeto interage com um sistema maior e ajustou a resposta considerando essas interações, não só o efeito isolado sobre sua própria equipe.

"Depois de receber feedback dos usuários no meio do desenvolvimento, o time repriorizou o backlog para entregar primeiro a funcionalidade que os usuários mais valorizavam.— isso é **foco no valor**: a decisão foi guiada pelo benefício de negócio percebido, não por seguir o plano original à risca.

### Como não confundir os 12 princípios entre si

Pergunte "que **palavra-chave** do enunciado aponta para qual princípio": mudança de rumo por causa de feedback do cliente → foco em valor; efeito colateral em outra parte do sistema/organização → pensamento sistêmico; decisão ética sobre uso de recursos → administrador zeloso; recuperação de um imprevisto sem travar o projeto → adaptabilidade e resiliência. A banca sempre oferece um princípio **plausível mas não o do cenário** — o critério de desempate é a palavra que o enunciado destaca.

### 8 domínios de desempenho

São as áreas de foco contínuo (não sequenciais, todas rodam o tempo todo em paralelo): partes interessadas; equipe; abordagem de desenvolvimento e ciclo de vida; planejamento; trabalho do projeto; entrega; medição; incerteza.

## 12.1.4 Tripla restrição e abordagens de execução

### O triângulo de ferro

**Escopo, tempo e custo** são as três restrições clássicas de todo projeto, e a lógica do "triângulo" é que elas são **interdependentes**: mexer em uma força ajuste nas outras duas (ou na qualidade, que sofre o impacto quando os três vértices são espremidos ao mesmo tempo). Reduzir prazo sem aumentar custo ou cortar escopo tende a comprometer a qualidade — não existe almoço grátis.

### Tradicional, ágil e híbrida

- **Tradicional/preditiva** (cascata): escopo definido e **congelado** no início, planejamento detalhado antecipado, execução segue o plano. Boa quando os requisitos são estáveis e bem conhecidos.
- **Ágil/adaptativa**: escopo **emerge** ao longo do projeto, por ciclos curtos com feedback constante. Boa quando os requisitos são incertos ou mudam com frequência.
- **Híbrida**: **combina** as duas — por exemplo, planejar o orçamento e o cronograma macro de forma preditiva (porque o cliente exige previsibilidade financeira), mas executar o desenvolvimento em sprints ágeis (porque o produto em si precisa de iteração).

### Híbrido não é "um pouco de ágil de vez em quando"

Híbrido é uma escolha deliberada de **combinar** elementos das duas abordagens conforme a parte do projeto que se beneficia de cada uma — não é "ágil malfeito" nem um meio-termo vago. Uma alternativa que descreve híbrido como "somente ágil, mas com menos rigor" ou "cascata com reuniões diárias" está errada: o que caracteriza híbrido é a combinação **intencional** de práticas preditivas e adaptativas em partes diferentes do mesmo projeto.

## 12.2 Scrum, Kanban e Lean

### 12.2.1 Scrum: os time-boxes

### Por que os eventos do Scrum são *time-boxed*

Um evento *time-boxed* tem duração **máxima** fixa — pode terminar antes, nunca depois. O motivo é limitar o desperdício de tempo em reunião: sem um teto, planejamento e retrospectiva tendem a se alongar indefinidamente. O *time-box* força o time a ser objetivo e cria um ritmo previsível (*cadence*) de trabalho.

| Evento               | Duração máx.<br>(Sprint de 1 mês) | Quem conduz / para quê                                                                                                                                  |
|----------------------|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Daily Scrum          | 15 min                            | os <b>Developers</b> , todo dia, no mesmo horário e lugar, para inspecionar o progresso rumo à <b>Meta da Sprint</b> e adaptar o plano das próximas 24h |
| Sprint Planning      | 8h                                | o time todo, no início da Sprint, para definir o que será feito e como                                                                                  |
| Sprint Review        | 4h                                | time + stakeholders, para inspecionar o incremento e adaptar o Product Backlog                                                                          |
| Sprint Retrospective | 3h                                | o time, sobre o <i>processo</i> de trabalho (não sobre o produto)                                                                                       |

Os 15 minutos da Daily são **fixos, independentemente** do tamanho da Sprint (mesmo numa Sprint de uma semana, continua sendo 15 min); os demais valores são proporcionais ao tamanho da Sprint e **encolhem** em Sprints menores que um mês.

### A Daily não é status report para o gerente

O propósito da Daily é o time **se replanejar entre si** rumo à Meta da Sprint — não é uma prestação de contas individual ("o que eu fiz ontem" reportado a um chefe). É por isso que quem **conduz** a Daily são os próprios Developers, não o Scrum Master nem um gerente de projeto.

## 12.2.2 Kanban: fluxo contínuo e limite de WIP

### Sem sprints, sem iterações fechadas

O Kanban gerencia o trabalho como um **fluxo contínuo**: os itens entram no quadro e vão avançando de coluna em coluna conforme ficam prontos — não há um ciclo fechado tipo Sprint que reinicia periodicamente. O mecanismo central é o **limite de trabalho em progresso** (WIP — Work In Progress): cada coluna do quadro tem um número máximo de itens que pode conter simultaneamente.

O WIP limitado força o time a **terminar** antes de **começar** mais coisa nova — se uma coluna está no limite, ninguém puxa item novo para ela até que algo saia. Isso combate o multitarefa (que na prática atrasa tudo, porque troca de contexto tem custo) e torna gargalos **visíveis**: uma coluna que trava no limite mostra exatamente onde o fluxo está entupido.

## Lead time, cycle time e throughput

| Métrica           | O que mede                                                                                                                                                                                                        |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Lead time</b>  | do momento da <b>solicitação</b> (entrada do item no sistema, sob a ótica de quem pediu) até a <b>entrega</b> . Inclui todo o tempo que o item passou esperando em fila antes de alguém começar a trabalhar nele. |
| <b>Cycle time</b> | do momento em que o trabalho <b>começa de fato</b> até a entrega. É sempre um <b>trecho</b> do lead time — nunca maior.                                                                                           |
| <b>Throughput</b> | <b>quantidade</b> de itens entregues por unidade de tempo (ex.: 8 itens/semana). Não é uma medida de tempo, é uma contagem.                                                                                       |

Como o cycle time é um pedaço do lead time, vale sempre **lead time**  $\geq$  **cycle time**. A diferença entre os dois é exatamente o tempo que o item ficou parado **em fila**, esperando alguém puxá-lo.

## Calculando lead time e cycle time

Um chamado é aberto pelo cliente na segunda-feira às 9h (solicitação). Um analista começa a trabalhar nele na quarta-feira às 9h (início do trabalho) e entrega a solução na quinta-feira às 17h. **Lead time** = da segunda 9h até quinta 17h = 3 dias e 8h. **Cycle time** = da quarta 9h até quinta 17h = 1 dia e 8h. A diferença de 2 dias é o tempo que o chamado ficou esperando na fila antes de ser puxado.

## Por que reduzir WIP reduz o lead time — a Lei de Little

A Lei de Little relaciona as três grandezas: **tempo médio no sistema** = **itens em progresso** / **taxa de conclusão**. Se a taxa de conclusão do time é relativamente estável, **diminuir o WIP** (menos itens disputando a atenção do time ao mesmo tempo) reduz diretamente o tempo médio que cada item passa no sistema — ou seja, reduz o lead time. É o argumento formal por trás da prática de "limitar o trabalho em progresso" do Kanban: não é só disciplina, é consequência matemática.

## Não inverta lead time e cycle time

O erro mais comum é dar ao **lead time** a definição do **cycle time** ("tempo do início do trabalho até a entrega"). A âncora para não errar: a palavra **solicitação/pedido do cliente** — se a contagem do tempo começa aí, é lead time; se começa quando alguém do time **pega** o item para trabalhar, é cycle time. E "itens entregues por semana" nunca é uma das duas — é **throughput**, porque é contagem, não duração.

### 12.2.3 Lean

#### Eliminar desperdício, maximizar valor

O Lean (originado no Sistema Toyota de Produção) enxerga qualquer atividade que não agrega valor percebido pelo cliente como **desperdício** (*muda*), e busca eliminá-lo sistematicamente — espera, retrabalho, superprodução, movimentação desnecessária, etc. O Kanban, como ferramenta visual de fluxo, é uma das técnicas usadas para implementar princípios Lean no desenvolvimento de software.

## 12.3 ITIL 4

### 12.3.1 Cocriação de valor: a mudança de eixo da v4

#### Por que "cocriação" e não "entrega"

A ITIL v3 descrevia o gerenciamento de serviços em termos de **processos**: sequências que o provedor de TI executa e "entrega" ao cliente, como um produto que sai de uma linha de montagem. A v4 muda o vocabulário para **práticas** e o modelo mental para **cocriação**, porque um serviço só gera valor quando é efetivamente **usado** — o provedor disponibiliza a capacidade, mas o valor concreto (a economia de tempo, a informação obtida, o problema resolvido) só existe no momento em que o consumidor interage com o serviço. Provedor e consumidor produzem o resultado **juntos**: um sistema de streaming não gera valor por existir no servidor, gera valor quando alguém assiste.

**Serviço**, na definição da ITIL, é o meio de permitir que o cliente alcance um resultado que ele quer **sem precisar gerenciar os custos e riscos específicos** envolvidos em produzi-lo — é por isso que se contrata um serviço de e-mail em vez de manter servidor próprio: o cliente quer o resultado (comunicação), não a responsabilidade de operar a infraestrutura.

**Prática** substitui **processo** como unidade central: um conjunto de **recursos organizacionais** (pessoas, processos, informação/tecnologia, parceiros) aplicados a um propósito específico — mais amplo que "processo", porque reconhece que cumprir um propósito envolve pessoas e ferramentas, não só uma sequência de passos.

#### Trocar "prática" por "processo" é o erro de vocabulário mais comum

Sempre que um enunciado sobre ITIL 4 usa a palavra "processo" para descrever um dos 34 elementos do framework, desconfie — a v4 abandonou esse termo propositalmente. "Processo" ainda existe como conceito genérico de gestão (inclusive dentro de uma prática pode haver processos), mas a **unidade organizacional** do modelo ITIL 4 é a prática.

### 12.3.2 Incidente, problema, requisição e mudança

#### Quatro coisas diferentes que chegam pelo mesmo balcão

Todas passam pela **central de serviços** (o ponto único de contato entre usuários e provedor — que **não é quem resolve** tudo, é quem recebe e direciona), mas são objetos de gestão completamente diferentes, com objetivos de tratamento diferentes:

| O que chegou                 | O que é                                                          | Objetivo do tratamento                                                                                                                   |
|------------------------------|------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Incidente</b>             | interrupção não planejada ou queda de qualidade do serviço       | <b>restaurar o serviço</b> o mais rápido possível — uma solução de contorno ( <i>workaround</i> ) já resolve, mesmo sem eliminar a causa |
| <b>Problema</b>              | a <b>causa</b> (conhecida ou suspeita) de um ou mais incidentes  | achar a <b>causa-raiz</b> e eliminá-la (ou documentar um erro conhecido) — não tem a mesma urgência de minutos do incidente              |
| <b>Requisição de serviço</b> | pedido <b>normal e previsto</b> (acesso, informação, senha nova) | atender com fluxo padronizado — <b>não houve falha</b> nenhuma, é operação normal                                                        |
| <b>Mudança</b>               | qualquer alteração que possa afetar serviços                     | maximizar mudanças <b>bem-sucedidas</b> , avaliando o risco antes de autorizar                                                           |

### Separando os quatro num cenário só

O sistema de folha de pagamento fica fora do ar (**incidente**: o time restaura reiniciando o serviço em 20 minutos, sem saber ainda por que caiu). Investigando depois, descobre-se que a causa foi um vazamento de memória em uma rotina específica (**problema**: abre-se investigação para achar a causa-raiz). Enquanto isso, outro funcionário pede acesso ao mesmo sistema para um novo colaborador (**requisição de serviço**: fluxo padrão de concessão de acesso, nada quebrou). Depois de identificada a causa-raiz, o time submete a correção do código para aprovação antes de subir para produção (**mudança**: avaliação de risco e autorização formal).

### O teste rápido para não confundir incidente com requisição

Pergunte: **algo estava funcionando e parou/piorou**, ou **o usuário está pedindo algo que nunca teve, de forma prevista**? Se houve degradação, é incidente. Se é um pedido normal dentro do catálogo de serviços (nova senha, novo acesso, nova informação), é requisição — *não houve falha*. E o incidente pode ser **fechado** assim que o serviço é restaurado, mesmo com o problema (a causa) ainda em aberto — os dois são rastreados separadamente.

## 12.3.3 Sistema de Valor de Serviço

### Como as peças da ITIL 4 se encaixam

O **Sistema de Valor de Serviço (SVS)** é o modelo que descreve como todos os componentes e atividades da organização trabalham juntos para **facilitar a criação de valor** através de serviços. Seus componentes:

- **Princípios orientadores** (7, ver abaixo): recomendações que guiam a organização em qualquer circunstância.
- **Governança**: como a organização é dirigida e controlada.

- **Cadeia de valor de serviço** (6 atividades, ver abaixo): o modelo operacional central.
- **Práticas** (34, ver abaixo): os conjuntos de recursos organizados por propósito.
- **Melhoria contínua**: o modelo recorrente de aprimoramento em todos os níveis.

## 7 princípios orientadores

Focar no **valor**; começar de onde se está (não descartar o que já existe e funciona); progredir **iterativamente** com feedback; colaborar e promover **visibilidade**; pensar e trabalhar de forma **holística**; manter **simples** e prático; otimizar e **automatizar**.

## 4 dimensões

Toda prática precisa ser olhada sob quatro ângulos, para não deixar lacuna: (1) **organizações e pessoas**; (2) **informação e tecnologia**; (3) **parceiros e fornecedores**; (4) **fluxos de valor e processos**. Um projeto de mudança que só olha a tecnologia e ignora as pessoas afetadas, por exemplo, está desconsiderando uma dimensão inteira.

## Cadeia de valor de serviço — 6 atividades

O modelo operacional que transforma demanda em valor, através de combinações flexíveis das seis atividades (não é uma sequência fixa — cada combinação de atividades forma um **fluxo de valor** diferente, adaptado à situação):

**Planejar, Melhorar, Engajar** (com clientes, usuários, fornecedores — manter relacionamento), **Desenhar e transicionar, Obter/construir, Entregar e suportar**.

## 34 práticas em 3 grupos

- **14 práticas gerais de gestão**: adaptadas do mundo geral de negócio para o gerenciamento de serviços (ex.: gestão de risco, gestão de projeto, gestão de portfólio, melhoria contínua, gestão financeira).
- **17 práticas de gerenciamento de serviço**: desenvolvidas especificamente no contexto de gerenciamento de serviços (ex.: gerenciamento de incidentes, gestão de mudança, gerenciamento de problemas, central de serviços, gerenciamento de nível de serviço).
- **3 práticas de gerenciamento técnico**: adaptadas do desenvolvimento e gestão de tecnologia (gestão de implantação, gestão de infraestrutura e plataforma, desenvolvimento e gerenciamento de software).

$14 + 17 + 3 = 34$ . As técnicas são só três — é o grupo que a memória tende a inflar por engano.

## Propósito de práticas parecidas

Práticas com nomes próximos têm propósitos distintos que a prova cobra literalmente:

- **Gerenciamento de infraestrutura e plataforma**: monitorar e prover as soluções tecnológicas da organização — é sobre manter a base técnica rodando.
- **Gerenciamento de liberação** (*release*): tornar serviços e funcionalidades disponíveis para uso — é sobre mover algo pronto para produção.
- **Central de serviços**: ser o ponto único de contato entre o provedor e os usuários — é sobre **comunicação**, não sobre resolver tecnicamente.

Confundir "quem monitora a infraestrutura" com "quem libera para produção" é o tipo de troca

que a banca planta entre práticas de nome parecido.

## 12.4 COBIT 2019

### 12.4.1 Governança × gestão

#### A distinção que organiza o framework inteiro

COBIT separa **governança** de **gestão** como duas responsabilidades **estruturalmente diferentes**, não dois sinônimos de "administrar":

|                  | Governança                                                                                 | Gestão                                                                     |
|------------------|--------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Pergunta central | " <b>o quê</b> e por quê?— que direção seguir, que risco aceitar, que prioridade vale mais | " <b>como</b> e quando?— executar o trabalho dentro da direção já definida |
| Verbos           | <b>avaliar, dirigir, monitorar</b> (EDM)                                                   | <b>planejar, construir, executar, monitorar</b> (APO, BAI, DSS, MEA)       |
| Quem responde    | o <b>conselho</b> /alta administração                                                      | a <b>diretoria</b> executiva de TI                                         |

Note que "**monitorar**" aparece nos dois lados — e é aí que mora a diferença mais sutil: a governança monitora **se a direção definida está sendo seguida** (o conselho pergunta "estamos indo aonde decidimos ir?"); a gestão monitora **o desempenho operacional do que está rodando** (a diretoria pergunta "o sistema está funcionando dentro do esperado?"). O mesmo verbo, dois objetos de observação diferentes.

#### Classificando uma atividade como governança ou gestão

"O conselho de administração define que a empresa vai priorizar segurança da informação sobre velocidade de lançamento de novos produtos, dado o histórico do setor." → **governança** (EDM): é uma decisão de direção e apetite a risco.

"A equipe de TI implementa um pipeline de testes automatizados de segurança antes de cada deploy, seguindo a diretriz aprovada." → **gestão** (BAI/DSS): é a execução concreta daquilo que a governança decidiu.

#### Por que a governança é "sob medida— fatores de desenho

O COBIT 2019 insiste que **não existe um conjunto único de objetivos de governança que sirva a toda organização**. Tamanho, setor, apetite a risco, estratégia e ambiente regulatório mudam qual objetivo deve receber mais prioridade e recursos. Os **fatores de desenho** (*design factors*) são justamente os critérios usados para **customizar** o sistema de governança à realidade de cada organização — "implantar o COBIT inteiro, exatamente como vem no guia" contraria essa lógica.

### 12.4.2 5 domínios, 40 objetivos

#### EDM é governança; os outros quatro são gestão

- **EDM** — *Evaluate, Direct and Monitor* (**governança**): avaliar opções estratégicas, dirigir a organização para a direção escolhida, monitorar se a direção está sendo seguida.
- **APO** — *Align, Plan and Organize* (gestão): alinhar a estratégia de TI com a organização, planejar e organizar os recursos.
- **BAI** — *Build, Acquire and Implement* (gestão): construir, adquirir e implementar as soluções.
- **DSS** — *Deliver, Service and Support* (gestão): entregar e dar suporte operacional aos serviços.
- **MEA** — *Monitor, Evaluate and Assess* (gestão): monitorar o desempenho operacional e a conformidade.

Ao todo, **40 objetivos de governança e gestão** distribuídos nesses 5 domínios.

#### Onde encaixar um objetivo específico

Se o enunciado fala em **especificar/priorizar requisitos, construir** ou **implementar** uma solução, pense **BAI** primeiro (é o domínio de "construir, adquirir, implementar") — é um erro comum jogar isso em APO (que é sobre alinhamento e planejamento estratégico, mais abstrato) ou em DSS (que já pressupõe a solução em operação). O critério: APO decide **o plano**, BAI **constrói a solução**, DSS **opera o que já foi construído**, MEA **mede o resultado**.

### 12.4.3 Os dois conjuntos de princípios

#### 6 princípios do sistema, 3 do framework — não se misturam

O COBIT 2019 tem **duas** listas de princípios, para **dois objetos diferentes**, e a diferença é literalmente "de quem é o princípio":

|                                                                                                      |                                                                                                                                                                                                                                                                                                     |
|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>6 princípios do sistema de governança</b> (descrevem a governança que a <i>organização</i> monta) | entregar <b>valor</b> às <b>partes interessadas</b> ; abordagem <b>holística</b> ; sistema de governança <b>dinâmico</b> ; governança <b>distinta</b> da gestão; <b>sob medida</b> para as necessidades da organização; sistema <b>de ponta a ponta</b> (cobre a empresa toda, não só a área de TI) |
| <b>3 princípios do framework de governança</b> (descrevem o guia que a <i>ISACA</i> publica)         | baseado em <b>modelo conceitual</b> ; <b>aberto e flexível</b> ; <b>alinhado</b> aos principais padrões e normas do mercado                                                                                                                                                                         |

Os seis descrevem **o que você constrói** dentro da sua organização; os três descrevem **o guia em si**, como produto publicado pela ISACA. Uma alternativa que oferece "aberto e flexível" como princípio do *sistema* de governança (em vez do *framework*), ou "de ponta a ponta" como princípio do *framework* (em vez do *sistema*), trocou os dois grupos entre si. E dizer que são "cinco" princípios é o número do **COBIT 5** (edição anterior), não do 2019.

### Componentes, não "enablers"

O COBIT 2019 usa o termo **componentes** para os elementos que constroem e sustentam um sistema de governança (processos, estruturas organizacionais, políticas, cultura, habilidades, serviços/infraestrutura etc.) — o COBIT 5, edição anterior, chamava o mesmo tipo de elemento de *enablers* (habilitadores). Se uma questão pergunta "para que serve o COBIT 2019", a resposta correta é algo como "definir os componentes que constroem e sustentam um sistema de governança— não "definir uma estratégia de TI" nem "criar um inventário de ativos"(isso é gestão de ativos, um objetivo específico dentro do framework, não o propósito do framework como um todo).

## 12.5 BPMN (modelagem de processos)

### Os elementos básicos de notação

BPMN (*Business Process Model and Notation*) é uma notação gráfica padronizada para representar processos de negócio de forma que tanto analistas de negócio quanto desenvolvedores consigam ler:

- **Eventos** (círculos): algo que **acontece** durante o processo. Distinguem-se pela espessura da borda: **início** (borda fina), **intermediário** (borda dupla), **fim** (borda grossa).
- **Atividades** (retângulos de cantos arredondados): trabalho executado — tarefas (atômicas) ou subprocessos (que se expandem em outro diagrama).
- **Gateways** (losangos): pontos de **decisão** ou **sincronização** do fluxo.
- **Raias** (*pools/lanes*): compartimentos que mostram **quem** executa cada parte do processo (papel, área, organização).
- **Fluxos**: de **sequência** (linha sólida — ordem das atividades dentro de uma mesma raia/processo) × de **mensagem** (linha tracejada — comunicação entre participantes/raias diferentes).

### Os três gateways — divisão e junção têm semânticas diferentes

O gateway é o único elemento do BPMN que se comporta de forma **diferente** dependendo de estar **abrindo** o fluxo (um caminho vira vários) ou **fechando** (vários caminhos voltam a virar um). É a semântica de **fechamento** que costuma ser mal compreendida:

| Gateway                | Na divisão (abertura)                                                                | Na junção (fechamento)                                                                                                                                                                                                     |
|------------------------|--------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Exclusivo (XOR)</b> | segue por <b>exatamente um</b> caminho — o da primeira condição verdadeira           | apenas <b>repassa</b> o que chegar de qualquer um dos ramos — não espera nada, porque só um ramo estava ativo                                                                                                              |
| <b>Paralelo (AND)</b>  | dispara <b>todos</b> os caminhos ao mesmo tempo, <b>sem</b> avaliar condição nenhuma | <b>sincroniza</b> : espera <b>todos</b> os caminhos chegarem antes de deixar o fluxo continuar                                                                                                                             |
| <b>Inclusivo (OR)</b>  | segue por <b>um ou mais</b> caminhos, todos os que tiverem condição verdadeira       | espera apenas os caminhos que <b>foram de fato ativados</b> na abertura correspondente — é o único gateway cuja junção precisa "saber" quantos ramos abriram, para não travar esperando um caminho que nunca foi disparado |

### Modelando três cenários com o gateway certo

"O pedido é avaliado e, então, **ou** é aprovado **ou** é recusado, nunca as duas coisas." → **exclusivo (XOR)**: os caminhos são mutuamente excludentes.

"Ao receber uma proposta, o time jurídico e o time financeiro analisam **simultaneamente**, e o processo só segue adiante quando **os dois** pareceres estiverem prontos." → **paralelo (AND)**: dispara os dois ramos juntos na abertura e sincroniza esperando ambos no fechamento.

"Dependendo do valor do pedido, **pode ser necessária** uma, duas ou as três verificações (crédito, estoque, fraude) — pelo menos uma sempre acontece." → **inclusivo (OR)**: mais de um caminho pode ser ativado, e o fechamento precisa saber exatamente quais foram para não esperar por um ramo que nunca abriu.

### Trocar o AND de fechamento por XOR é o erro de modelagem mais grave

Se dois ramos abrem em paralelo (AND) mas o modelador fecha com um gateway **exclusivo**, o processo segue adiante assim que **o primeiro** ramo chegar — **sem esperar o segundo**. Na prática, isso significa que o processo avança com metade do trabalho feito (ex.: aprova o pedido antes do parecer financeiro estar pronto). É o tipo de erro que passa despercebido no diagrama (visualmente parece só "um losango a menos") mas quebra a lógica de negócio inteira.

## 12.6 Gestão de riscos (ISO 31000)

### O ciclo de gestão de riscos

A norma ISO 31000 organiza a gestão de riscos em um processo cíclico e contínuo, aplicável a qualquer tipo de risco (não só de TI): **identificar** os riscos, **analisar** (entender causas, consequências, probabilidade), **avaliar** (comparar contra critérios, decidir prioridade), **tratar** (aceitar, mitigar, transferir ou evitar o risco) e **monitorar** continuamente (riscos e o contexto ao redor mudam com o tempo, então a análise não é um evento único).

### As quatro respostas possíveis a um risco identificado

Depois de avaliado, todo risco recebe uma destas respostas: **aceitar** (o custo de tratar é maior que o impacto do risco); **mitigar** (reduzir probabilidade ou impacto com uma ação concreta); **transferir** (repassar o impacto financeiro a terceiro, como um seguro ou contrato); **evitar** (eliminar a atividade que gera o risco). Não existe uma quinta opção de "ignorar sem decidir—mesmo não fazer nada é, na prática, uma decisão de aceitar.

## 12.7 Roteiro de revisão do capítulo

### Os números que amarram cada framework

**PMBOK**: 6ª edição = 5 grupos de processos × 10 áreas de conhecimento; 7ª edição = 12 princípios + 8 domínios de desempenho (não é baseada em processos). **ITIL 4**: 4 dimensões, 7 princípios orientadores, 6 atividades da cadeia de valor, 34 práticas (14 gerais + 17 de serviço + 3 técnicas). **COBIT 2019**: 5 domínios (1 de governança — EDM — e 4 de gestão), 40 objetivos, 6 princípios do sistema de governança + 3 do framework (o "5" pertence ao COBIT 5, edição anterior). Sempre que dois números de frameworks diferentes se parecem, volte a esta lista antes de responder.

## Capítulo 13

# Redes de Computadores

Este capítulo cobre os fundamentos de redes que aparecem na prova, com a mecânica por trás de cada conceito — não só o rótulo. A versão resumida, com o recorte do que realmente cai no Perfil 3, está na **apostila**/; aqui o objetivo é entender **por que** a pilha de protocolos funciona do jeito que funciona.

### 13.1 Modelos de referência: OSI e TCP/IP

#### 13.1.1 Encapsulamento: a ideia que sustenta o modelo em camadas

**Camada é divisão de trabalho, não gaveta de classificação**

O problema de fazer dois programas em máquinas diferentes conversarem é grande demais para resolver de uma vez — envolve sinal elétrico, entrega ao vizinho físico, roteamento entre redes, garantia de entrega, e o significado dos dados em si. O modelo em camadas **divide esse problema** em fatias menores, cada uma resolvida por uma camada diferente, que não precisa saber como as outras resolvem o próprio pedaço.

O mecanismo que viabiliza isso é o **encapsulamento**: ao descer a pilha (enviando), cada camada recebe os dados da camada de cima, faz seu trabalho e **acrescenta um cabeçalho próprio** antes de passar adiante. Ao subir a pilha (recebendo), cada camada **remove** o cabeçalho que corresponde a ela e entrega o resto para cima. É como uma carta dentro de um envelope, dentro de uma caixa, dentro de outra caixa — cada camada só abre a própria embalagem.

## OSI — 7 camadas

| # | Camada       | Problema que resolve                                                            | Exemplos                      |
|---|--------------|---------------------------------------------------------------------------------|-------------------------------|
| 7 | Aplicação    | serviço que o usuário/programa efetivamente quer                                | HTTP, DNS, SMTP               |
| 6 | Apresentação | formato dos dados — tradução, cifra, compressão                                 | codificação, TLS (discutível) |
| 5 | Sessão       | controle do diálogo — abrir, manter, retomar de um <i>checkpoint</i>            | —                             |
| 4 | Transporte   | entregar ao <b>processo</b> certo (porta), com ou sem garantia de ordem/entrega | TCP, UDP                      |
| 3 | Rede         | achar o caminho <b>entre redes diferentes</b>                                   | IP, roteador                  |
| 2 | Enlace       | entregar ao vizinho <b>dentro do mesmo segmento</b> físico (endereço MAC)       | switch, Ethernet              |
| 1 | Física       | converter bits em sinal no meio físico                                          | cabo, rádio                   |

Cada camada tem uma unidade de dados própria (PDU), e o nome dessa unidade denuncia a camada em qualquer questão: **segmento** (TCP) na Transporte, **pacote** na Rede, **quadro** na Enlace, **bit** na Física.

## TCP/IP — 4 camadas, a pilha realmente implementada

O modelo OSI é o **referencial de ensino** (didático, nunca foi implementado à risca como está descrito); a pilha que roda de verdade na Internet é a **TCP/IP**, mais enxuta: Acesso à Rede (funde OSI 1–2), Internet (equivale a OSI 3), Transporte (equivale a OSI 4), Aplicação (funde OSI 5–7). Quando um enunciado menciona "camada de sessão" ou "camada de apresentação", está necessariamente falando do modelo **OSI** — essas duas não têm correspondência separada no TCP/IP.

## Identificando a camada pela função descrita, não pelo nome

"Um protocolo insere marcadores no fluxo de dados para permitir que, se a conexão cair no meio de uma transferência grande, ela retome do ponto interrompido em vez de recomençar do zero." → isso é controle de **diálogo com checkpoint** → camada de **Sessão** (OSI 5) — não é Transporte (que garante entrega e ordem, mas não gerencia retomada de diálogo em alto nível) nem Apresentação (que trata de formato dos dados).

"Um roteador recebe um pacote e decide por qual próximo salto encaminhá-lo, com base no endereço de destino." → camada de **Rede** (OSI 3) — é sobre achar caminho entre redes.

"Um switch recebe um quadro e decide para qual porta física repassá-lo, com base na tabela de endereços MAC aprendidos." → camada de **Enlace** (OSI 2) — a decisão fica dentro do mesmo segmento físico.

**Roteiro para identificar a camada em qualquer cenário**

Pergunte, nesta ordem: o problema é **entre duas máquinas do mesmo segmento físico**? Enlace. É **entre redes diferentes** (precisa escolher rota)? Rede. Chegou **à máquina certa mas falta garantir processo/porta, ordem ou retransmissão**? Transporte. É sobre **retomar/gerenciar um diálogo** (checkpoint, sessão)? Sessão. É sobre **formato, cifra ou compressão** dos dados em si? Apresentação. É o **serviço final** que o usuário/programa pediu? Aplicação. A banca sempre descreve o comportamento, nunca pergunta "qual é o número da camada X" isoladamente — o número é consequência de identificar a função.

## 13.2 Equipamentos de rede por camada

**Hub, switch e roteador — a inteligência cresce com a camada**

- **Hub** (camada 1 — Física): recebe um sinal em uma porta e **repete para todas as outras**, sem entender endereço nenhum. É "burro" por definição — não sabe distinguir destinatários, então gera colisão e desperdiça banda.
- **Switch** (camada 2 — Enlace): aprende quais endereços **MAC** estão em quais portas (observando o tráfego que passa) e encaminha cada quadro **só para a porta certa** — elimina a repetição cega do hub.
- **Roteador** (camada 3 — Rede): usa o endereço **IP** de destino para decidir por qual interface encaminhar um pacote, inclusive **entre redes diferentes** — é o único dos três capaz de interligar duas redes distintas.

Existe switch de camada 3 (que também roteia), mas o par cobrado com mais frequência é a associação direta **switch = camada 2, roteador = camada 3**.

## 13.3 TCP × UDP

**Os dois protocolos de transporte resolvem trade-offs opostos**

Ambos entregam dados a um **processo** (via porta), mas fazem concessões diferentes entre confiabilidade e velocidade:

|                | TCP                                                                                      | UDP                                                                                                                                                |
|----------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Conexão        | orientado a conexão (estabelece com <i>handshake</i> de 3 vias antes de trocar dados)    | sem conexão — envia direto                                                                                                                         |
| Confiabilidade | garante entrega e ordem (retransmite o que se perde, reordena o que chega fora de ordem) | não garante nada disso — se um pacote se perde, não há retransmissão automática                                                                    |
| Custo          | mais overhead (cabeçalho maior, controle de estado da conexão)                           | mais leve e rápido                                                                                                                                 |
| Uso típico     | web, e-mail, transferência de arquivo — onde perder um byte é inaceitável                | streaming, DNS, VoIP — onde chegar rápido importa mais que chegar perfeito (um pacote perdido de voz não vale a pena retransmitir, já ficou velho) |

### O *three-way handshake* do TCP

Antes de trocar qualquer dado, as duas pontas trocam três mensagens para **sincronizar números de sequência** — o mecanismo que depois permite ao TCP detectar perda e reordenar:

1. **SYN**: o cliente propõe um número de sequência inicial.
2. **SYN-ACK**: o servidor confirma o número do cliente e propõe o próprio.
3. **ACK**: o cliente confirma o número do servidor.

Depois desse acordo, as duas pontas sabem contar exatamente quais bytes já chegaram e quais faltam — é isso que viabiliza a garantia de entrega e ordem. O handshake **não autentica** identidade, **não negocia** criptografia, e **não controla fluxo** de vazão — essas são outras funções (a autenticação/criptografia fica a cargo de camadas superiores como TLS; o controle de fluxo é outro mecanismo do TCP, separado do handshake). E só existe no TCP: o UDP não tem handshake porque não estabelece conexão.

## 13.4 Endereçamento e roteamento IP

### 13.4.1 IPv4, IPv6 e faixas privadas

#### Tamanho do endereço e por que existem faixas privadas

**IPv4**: endereço de **32 bits**, escrito em 4 octetos decimais (0–255). O espaço de endereços é finito (cerca de 4,3 bilhões), e não é suficiente para dar um endereço público único a cada dispositivo do mundo — daí a RFC 1918 reservar faixas **privadas**, que podem ser reutilizadas livremente dentro de redes internas diferentes (porque nunca saem para a Internet pública diretamente): 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16. O NAT (adiante) é o que permite esses endereços privados acessarem a Internet pública.

**IPv6:** endereço de **128 bits**, criado justamente para resolver a escassez do IPv4 — o espaço de endereços é astronomicamente maior, o que elimina a necessidade estrutural de NAT (ainda que ele possa existir por outros motivos). Prefixo de sub-rede típico: **/64**.

### Notação simplificada do IPv6

Um endereço IPv6 completo tem 8 grupos de 4 dígitos hexadecimais. Duas regras de abreviação, aplicadas **nesta ordem**:

1. Apague os **zeros à esquerda** de cada grupo individualmente ( $0DB8 \rightarrow DB8$ ;  $0001 \rightarrow 1$ ; um grupo totalmente zero,  $0000$ , vira só  $0$ ).
2. Substitua **uma única** sequência contígua de grupos inteiramente nulos por  $::$  — só pode ser usada **uma vez** no endereço inteiro, porque usá-la duas vezes tornaria impossível saber quantos grupos cada  $::$  está escondendo.

### Abreviando um endereço IPv6

Endereço completo:  $2001:0DB8:0000:0000:0000:0000:FE00:0001$ .

Passo 1 (zeros à esquerda de cada grupo):  $2001:DB8:0:0:0:0:FE00:1$ .

Passo 2 (a maior sequência de grupos zerados vira  $::$  — aqui, os quatro grupos  $0:0:0:0$  no meio):  $2001:DB8::FE00:1$ .

## 13.4.2 Sub-redes: calculando hosts utilizáveis

### Por que se subtrai 2 do total de endereços

Um endereço IPv4 tem 32 bits, divididos entre a parte de **rede** e a parte de **host**. O prefixo  $/n$  informa que os primeiros  $n$  bits identificam a rede, sobrando  $h = 32 - n$  bits para identificar hosts dentro dela. Com  $h$  bits, existem  $2^h$  combinações possíveis — mas duas delas são **reservadas** e não podem ser atribuídas a um host:

- o endereço com todos os bits de host em **0** identifica a **própria rede** (não um host);
- o endereço com todos os bits de host em **1** é o **broadcast** da sub-rede (não um host).

Por isso, **hosts utilizáveis**  $= 2^h - 2$ .

### Aplicando a fórmula

| Prefixo | Máscara         | Bits de host<br>( $h$ ) | Hosts utilizáveis<br>( $2^h - 2$ ) |
|---------|-----------------|-------------------------|------------------------------------|
| $/24$   | 255.255.255.0   | 8                       | $2^8 - 2 = 254$                    |
| $/25$   | 255.255.255.128 | 7                       | $2^7 - 2 = 126$                    |
| $/26$   | 255.255.255.192 | 6                       | $2^6 - 2 = 62$                     |
| $/27$   | 255.255.255.224 | 5                       | $2^5 - 2 = 30$                     |
| $/28$   | 255.255.255.240 | 4                       | $2^4 - 2 = 14$                     |

**Do requisito de hosts para a máscara certa**

Quando o enunciado dá uma necessidade ("a sub-rede precisa comportar até 30 hosts") e pede a máscara, procure a **menor** sub-rede (o maior prefixo /n) que ainda **comporta** o número pedido — usar uma sub-rede maior que o necessário desperdiça endereços. Trinta hosts cabem exatamente em /27 (30 utilizáveis); /28 (14) não cabe; /26 (62) cabe mas desperdiça 32 endereços à toa.

**13.4.3 Roteamento e distância administrativa****Rota estática × dinâmica, e como o roteador desempata**

**Rota estática:** configurada manualmente pelo administrador, fixa até alguém mudar. **Rota dinâmica:** aprendida automaticamente por um protocolo de roteamento que troca informação com outros roteadores (OSPF, RIP, BGP) — se adapta sozinha a mudanças na topologia.

Quando o roteador aprende **mais de uma rota** para o mesmo destino, por fontes diferentes, ele precisa de um critério de desempate: a **distância administrativa** (um número que representa a "confiabilidade" atribuída a cada fonte de rota, menor = mais confiável). Valores padrão (Cisco): rede diretamente conectada = 0, rota estática = 1, OSPF = 110, RIP = 120. **A menor distância administrativa vence**, sempre — é um critério puramente numérico, sem exceção por "protocolo mais moderno" ou situação especial.

**Não existe prioridade "inteligente" no roteamento**

Alternativas que descrevem comportamento adaptativo além da distância administrativa — "OSPF sempre tem prioridade sobre rota estática", "o roteador faz balanceamento de carga automático entre rotas concorrentes", "descarta pacotes quando há rotas conflitantes" — estão inventando comportamento. A regra real é mecânica e determinística: compara a distância administrativa das rotas disponíveis para o mesmo destino e usa a menor. Ponto.

**13.4.4 ACL Cisco e wildcard mask****Wildcard mask é o inverso lógico da máscara de sub-rede**

Uma *wildcard mask* usa a lógica oposta da máscara normal: bit **0** significa "esse bit **tem** que bater exatamente" e bit **1** significa "esse bit é **ignorado**" (pode ser qualquer valor). É obtida invertendo cada bit da máscara de sub-rede correspondente.

**Construindo a wildcard mask para "só endereços ímpares"**

Um endereço IPv4 é ímpar quando o **bit menos significativo do último octeto** vale 1. Para uma ACL casar *apenas* endereços ímpares, esse bit específico precisa estar **fixo** (wildcard = 0 nesse bit), e todos os demais bits do último octeto podem ser qualquer coisa (wildcard = 1). Isso dá, no último octeto, 11111110 em binário = 254 em decimal. Wildcard resultante: 0.0.0.254. O raciocínio é sempre **bit a bit**: monte o octeto em binário decidindo "fixo (0)" ou "livre (1)" posição por posição, e só depois converta para decimal — decorar valores prontos leva a erro quando o padrão muda.

### 13.4.5 MPLS, VLAN e NAT

#### Três mecanismos que resolvem problemas diferentes na rede

- **MPLS** (*Multiprotocol Label Switching*): em vez de cada roteador olhar o endereço IP de destino e recalculer a rota a cada salto, o MPLS anexa um **rótulo** (*label*) simples ao pacote na entrada da rede, e os roteadores internos só olham o rótulo para encaminhar — muito mais rápido que analisar IP a cada salto. Por operar "entre" a camada de enlace e a de rede (usa rótulos como a camada 2, mas serve para encaminhar como a camada 3), é apelidado de "**camada 2.5**". Permite criar caminhos dedicados (LSP) para engenharia de tráfego e qualidade de serviço (QoS).
- **VLAN** (*Virtual LAN*): segmenta **logicamente** a camada 2 — cria domínios de **broadcast** separados dentro do **mesmo** switch físico, sem precisar de switches físicos distintos para cada grupo de máquinas. Um pacote de broadcast de uma VLAN não vaza para outra, mesmo compartilhando o mesmo hardware.
- **NAT** (*Network Address Translation*): traduz endereços **privados** (dentro da rede interna) para um endereço **público** (ao sair para a Internet) e vice-versa na volta — resolve dois problemas ao mesmo tempo: economiza endereços IPv4 públicos (muitos hosts internos compartilham um só IP público) e oculta a topologia interna da rede de quem está de fora.

#### MPLS não é "roteamento IP mais rápido"; VLAN não é segmentação física

MPLS não substitui o roteamento IP — ele acelera o encaminhamento **depois** que a rota já foi decidida, usando rótulo em vez de recalculer IP a cada salto; não é a mesma coisa que "um protocolo de roteamento melhor". VLAN segmenta a camada **2** (domínio de broadcast) por configuração lógica — não é uma separação de cabos ou hardware, é a mesma infraestrutura física dividida por software.

## 13.5 Tipos de rede corporativa

#### Internet, intranet, extranet e portal

Os quatro termos descrevem **quem tem acesso** a uma rede, não sua tecnologia (todos rodam sobre a mesma pilha TCP/IP):

- **Internet**: rede **pública** e global, acessível a qualquer um.
- **Intranet**: rede **interna** de uma organização, acesso restrito aos seus membros.
- **Extranet**: extensão controlada da intranet para **parceiros ou clientes externos específicos** — não é pública como a Internet, mas também não é fechada só aos funcionários como a intranet; é um meio-termo de acesso seletivo.
- **Portal**: um **ponto único de acesso** organizado a serviços/informações (pode ser a interface tanto de uma intranet quanto de uma extranet).

## 13.6 Segurança de rede

### 13.6.1 Firewall: *stateless* × *stateful*

#### A diferença é lembrar (ou não) do que já aconteceu

|                      | Stateless (filtro de pacotes)                                                            | Stateful (com estado)                                                                                                                           |
|----------------------|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| O que examina        | cada pacote <b>isoladamente</b> — IP origem/destino, porta, protocolo                    | o pacote <b>no contexto da conexão/sessão</b> a que pertence                                                                                    |
| Como decide          | aplica a regra estática ao pacote, sem memória do que passou antes                       | mantém uma <b>tabela de estado</b> das conexões ativas e usa esse histórico na decisão                                                          |
| Consequência prática | para permitir a resposta de uma conexão, é preciso escrever uma regra explícita de volta | a resposta de uma conexão iniciada de dentro é aceita <b>automaticamente</b> , porque o firewall já sabe que ela pertence a uma sessão legítima |

Ambos operam nas camadas 3 e 4 (IP e porta). Quem inspeciona o **conteúdo** da camada de aplicação (camada 7) é outra categoria de equipamento — o **firewall de próxima geração** (NGFW) ou o **WAF** (*Web Application Firewall*).

#### Não misture "com estado" com "opera na aplicação"

Dizer que o firewall *stateful* "atua exclusivamente na camada de aplicação" é um absoluto errado — ele continua trabalhando nas camadas 3/4, só que **lembrando** do contexto da conexão. Subir até a camada 7 é uma capacidade **adicional** de outra categoria de equipamento (NGFW/WAF), não uma consequência automática de ser *stateful*.

### 13.6.2 SSH substituindo o Telnet

#### A mesma função, com e sem cifra

Telnet e SSH resolvem o mesmo problema — **administração remota de outro equipamento pela rede** — mas com uma diferença que os tornou incompatíveis em qualquer ambiente que leve segurança a sério:

- **Telnet** (porta 23): transmite **tudo em texto claro**, inclusive o usuário e a senha do administrador. Qualquer um capaz de capturar o tráfego no caminho (um *sniffer* na mesma rede, por exemplo) lê a sessão inteira, senha incluída.
- **SSH** (porta 22): faz o mesmo trabalho, mas **cifra toda a comunicação**, incluindo a etapa de autenticação — ninguém capturando o tráfego consegue ler usuário, senha ou comandos.

A razão de o SSH ter substituído o Telnet é estritamente **segurança (criptografia)**, não velocidade ou qualquer outra vantagem operacional.

### 13.6.3 TLS: por que a criptografia é híbrida

#### Assimétrica só para combinar a chave; simétrica para o volume

O TLS (o que torna HTTP em HTTPS) resolve dois problemas que pedem ferramentas diferentes, e por isso combina os dois tipos de criptografia:

- **Criptografia assimétrica — só no *handshake*:** o certificado digital do servidor comprova sua identidade (é ele mesmo quem diz ser), e o par de chaves pública/privada permite às duas pontas **combinarem uma chave simétrica de sessão sem que ela trafegue em claro** pela rede.
- **Criptografia simétrica — para todo o resto:** uma vez combinada a chave de sessão, **todo** o volume de dados trocado durante a conexão é cifrado com ela, porque criptografia simétrica é **ordens de magnitude mais rápida** que assimétrica.

O motivo do híbrido: a assimétrica sozinha seria lenta demais para cifrar todo o tráfego de uma sessão inteira; a simétrica sozinha não resolveria o problema de *como* as duas pontas combinam uma chave secreta por um canal que, até aquele momento, ainda não é seguro.

#### O que o handshake NÃO faz

A parte assimétrica do TLS **não** cifra o tráfego inteiro da sessão (só participa da negociação da chave) e **não** autentica o usuário final por login/senha — ela autentica o **servidor**, via certificado digital. Autenticação do usuário (se houver) é uma camada posterior, dentro da aplicação, não parte do handshake TLS.

#### SSL não é "mais moderno" que TLS

TLS é o **sucessor** do SSL, criado justamente para corrigir vulnerabilidades encontradas no protocolo antigo. Qualquer alternativa que descreva o SSL como mais seguro, mais recente ou mais recomendado que o TLS inverteu a linha do tempo.

## 13.7 Roteiro de revisão do capítulo

#### Sequência de raciocínio para qualquer questão de redes

1. O enunciado descreve uma **função**? Identifique a camada OSI pelo comportamento (Seção anterior), não tente decorar "camada X = tal coisa" sem entender o porquê.
2. Envolve **dois hosts trocando dados**? Pergunte se é TCP (precisa de confiabilidade/ordem) ou UDP (velocidade importa mais).
3. Envolve **endereçamento**? Verifique se é cálculo de sub-rede ( $2^h - 2$ ), faixa privada (RFC 1918) ou notação IPv6.
4. Envolve **decisão de caminho**? Verifique se é rota estática/dinâmica e aplique a distância administrativa (menor vence).
5. Envolve **segurança**? Separe o que é do protocolo de transporte/rede (TCP handshake, firewall stateful) do que é de criptografia de aplicação (TLS, SSH) — são mecanismos independentes que a prova gosta de combinar no mesmo cenário.

## Capítulo 14

# Órfãos — Administração de BD e Temas Coringa

Este é o capítulo mais heterogêneo do livro: reúne administração física de banco de dados (o lado operacional que o Capítulo 6 não cobre, focado que está em modelagem e SQL), arquitetura de computadores e sistema operacional, sistemas de informação corporativos, e aprendizado de máquina. São temas que aparecem em prova mas não têm um capítulo natural no edital do Perfil 3 — daí o nome. No fim do capítulo há um mapa indicando onde os demais temas "coringa" (blockchain, virtualização, 2PC etc.) estão tratados em profundidade nos outros capítulos deste livro.

## 14.1 Administração de Dados × Administração de Banco de Dados

### Dois papéis com o mesmo objeto e focos opostos

Os dois cuidam do mesmo recurso — o dado da organização — mas em níveis de abstração diferentes, e é essa diferença de **nível**, não de "quem é mais importante", que a prova cobra:

|            | Administração de Dados (AD)                                                                       | Administração de BD (DBA)                                                                        |
|------------|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Foco       | <b>negócio/lógico</b> — o dado como recurso corporativo, independente de onde ele está armazenado | <b>técnico/físico</b> — o SGBD específico funcionando                                            |
| Atividades | modelagem conceitual, políticas de dados, governança, dicionário de dados                         | instalação, ajuste de desempenho ( <i>tuning</i> ), backup, segurança de acesso, disponibilidade |
| Natureza   | estratégica, <b>independe</b> do produto de SGBD escolhido                                        | operacional, <b>ligada</b> a um produto específico (Oracle, PostgreSQL, SQL Server...)           |

Uma forma de fixar a distinção: a AD decidiria *que* dado a empresa precisa guardar sobre um cliente e *que* regras de qualidade esse dado deve seguir; o DBA decidiria *em que servidor*, com *que configuração de índice* e *que rotina de backup* esse dado efetivamente vai residir.

### Não inverta os dois papéis

Descrever a AD como "quem cuida do servidor de banco" ou o DBA como "quem define a política corporativa de qualidade de dados" troca os dois níveis entre si. A pergunta de desempate: a atividade descrita **depende de qual produto de SGBD está instalado**? Se sim, é DBA. Se a resposta seria a mesma independentemente do SGBD escolhido, é AD.

## 14.2 Recuperação e transações: o protocolo ARIES

### Write-Ahead Logging — registrar antes de gravar

A maioria dos SGBDs usa o protocolo **ARIES** para se recuperar de uma queda (falta de energia, crash do processo) sem perder transações confirmadas nem deixar o banco num estado inconsistente. O princípio-base é o **WAL** (*Write-Ahead Logging*): antes de gravar uma alteração **nos dados em si**, o SGBD grava um registro dessa alteração **no log**, em disco. Isso garante a **durabilidade** (o D do ACID, ver Capítulo 6) sem precisar gravar cada página de dado imediatamente a cada operação — o log, mais compacto e sequencial, é o que vai para disco com prioridade; o dado em si pode ser escrito depois, em lote, com mais eficiência.

### As três fases da recuperação, em ordem fixa

Depois de uma queda, o ARIES reconstrói o estado do banco em três fases, **sempre nesta ordem**:

1. **Analysis** (Análise): varre o log a partir do último *checkpoint* para reconstruir **que transações** estavam ativas e **que páginas** estavam sujas (modificadas, mas não gravadas) no momento da queda.
2. **Redo** (Refazer): reaplica **todas** as operações registradas no log a partir do ponto identificado — inclusive as de transações que **não chegaram a confirmar** (*commit*). O princípio é "*repeating history*": primeiro reconstrói o estado exato de antes da queda, sem julgar ainda o que deveria ou não ter acontecido.
3. **Undo** (Desfazer): só depois de reconstruído o estado exato do momento da queda, desfaz as operações das transações que **não tinham confirmado** — porque uma transação sem commit não deve deixar efeito permanente.

### Por que Redo vem antes de Undo

Se o Undo rodasse antes do Redo, o sistema tentaria desfazer operações de transações **sem primeiro saber com certeza** quais páginas realmente tinham sido alteradas em disco no momento da queda (algumas mudanças podem ter chegado ao disco, outras não) — o Redo existe exatamente para eliminar essa incerteza, recriando um estado **conhecido e completo** antes de começar a reverter o que não deveria ter ficado. Inverter a ordem quebraria a garantia de consistência que o protocolo promete.

### A troca mais comum: inverter Redo e Undo

Um enunciado que descreve a recuperação como "primeiro desfaz as transações não confirmadas, depois refaz as confirmadas" inverteu a lógica do ARIES. A ordem correta refaz **tudo primeiro** (inclusive o que depois será desfeito) e só desfaz **depois**, num segundo momento.

## 14.3 Armazenamento físico e desempenho

### Tablespace: a unidade lógica que organiza os arquivos físicos

**Tablespace** (termo Oracle; *filegroup* no SQL Server) é uma unidade lógica de armazenamento que agrupa objetos do banco (tabelas, índices) e os associa a um ou mais arquivos físicos em disco. O motivo de separar tablespaces distintas para dados, índices e área temporária é **isolar**

**o padrão de I/O:** dados e índices costumam ser lidos juntos mas escritos em ritmos diferentes, e a área temporária (usada por ordenações e junções grandes) tem um padrão de uso totalmente diferente — separar fisicamente evita que um tipo de operação compita com outro pelo mesmo disco.

### Otimizador de consultas e o papel das estatísticas

O **otimizador de consultas** (*query optimizer*) decide o **plano de execução** de um comando SQL — em que ordem fazer os JOINS, se usar um índice ou percorrer a tabela inteira (*full scan*), qual algoritmo de junção aplicar. Essa decisão se baseia em **estatísticas** sobre os dados (quantas linhas tem cada tabela, quão distribuídos estão os valores de cada coluna) — não em executar a consulta de verdade e comparar. Quando as estatísticas ficam **desatualizadas** (a tabela cresceu, a distribuição de valores mudou, mas o otimizador ainda raciocina com o retrato antigo), o plano escolhido deixa de ser o melhor, e a consulta que era rápida vira lenta sem que nenhum código tenha mudado. Por isso, **atualizar as estatísticas** é frequentemente o primeiro passo de diagnóstico quando uma consulta piora de repente — antes de sair reescrevendo SQL ou criando índice novo.

### Particionamento: dividir para escalar

**Particionamento** divide uma tabela grande em pedaços menores e mais administráveis:

- **Horizontal** (também chamado *sharding* quando os pedaços vão para servidores diferentes): divide por **linhas** — por exemplo, uma tabela de pedidos particionada por ano, cada partição com um subconjunto das linhas.
- **Vertical**: divide por **colunas** — separa colunas acessadas com frequência das raramente acessadas (ou muito grandes, como um campo de texto longo), para que consultas comuns leiam menos dado.

O ganho em ambos os casos é permitir que uma consulta toque só a parte relevante dos dados, em vez de varrer a tabela inteira.

### Índices: reveja no capítulo de banco de dados

A escolha entre índice B+ Tree, hash e bitmap já está detalhada na Seção de Índices do Capítulo 6 — as duas perguntas que decidem o tipo certo são *a consulta pede faixa ou igualdade?* e *quantos valores distintos tem a coluna?*. Não repita esse raciocínio do zero aqui; ele é o mesmo independentemente de o cenário vir rotulado como "administração de BD" ou "modelagem".

## 14.4 Backup e recuperação

### Completo, incremental e diferencial — a diferença é a referência

Os três tipos de backup diferem em **a partir de que ponto** eles medem "o que mudou":

| Tipo                      | O que copia                                                                                                                  |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <b>Completo</b><br>(full) | <b>tudo</b> , sem depender de nenhum backup anterior                                                                         |
| <b>Incremental</b>        | só o que mudou <b>desde o último backup, de qualquer tipo</b> (o completo mais recente <b>ou</b> o incremental mais recente) |
| <b>Diferencial</b>        | tudo que mudou <b>desde o último backup completo</b> — ignora incrementais/diferenciais intermediários                       |

A consequência prática é o que costuma ser cobrado: o **diferencial cresce** a cada dia que passa sem um novo completo (porque sempre conta a partir do mesmo ponto fixo), enquanto o **incremental** tende a ficar pequeno e constante (porque cada um só cobre o intervalo desde o anterior, que é sempre recente). Em compensação, restaurar a partir de incrementais exige reaplicar **toda a cadeia** de incrementais em ordem desde o último completo; restaurar a partir de um diferencial exige só o completo + o diferencial mais recente.

#### RPO e RTO — as duas perguntas de continuidade de negócio

**RPO** (*Recovery Point Objective*): quanto **dado** a organização aceita perder, medido em tempo — se o RPO é de 1 hora, o backup/replicação precisa garantir que, na pior hipótese, no máximo 1 hora de transações seja perdida. **RTO** (*Recovery Time Objective*): em quanto **tempo** o serviço precisa voltar a operar depois de um incidente. São eixos independentes: um sistema pode ter RPO baixo (não pode perder quase nada de dado) mas RTO alto (pode ficar fora do ar por horas), ou o contrário.

#### Backup quente × frio

**Quente** (*online*): feito com o banco **no ar**, atendendo requisições normalmente durante a cópia. **Frio** (*offline*): feito com o banco **parado** — mais simples de garantir consistência, mas exige uma janela de indisponibilidade.

## 14.5 Segurança e acesso no SGBD

#### Camadas de controle de acesso dentro do banco

**GRANT/REVOKE** (comandos DCL, ver Capítulo 6) concedem e revogam privilégios diretamente. **Papéis/roles** (RBAC — *Role-Based Access Control*) agrupam privilégios e são atribuídos a usuários, evitando conceder/revogar privilégio individual a cada pessoa. **Views** podem restringir o acesso a **colunas** específicas (o usuário enxerga só um subconjunto de campos de uma tabela). **Row-level security** restringe o acesso por **linha** (ex.: o gerente de uma filial só vê as linhas daquela filial). Auditoria registra quem acessou o quê, para rastreamento posterior.

O princípio que organiza tudo isso é o **menor privilégio**: cada usuário/aplicação recebe exatamente o acesso necessário para sua função, nada além — o gerente de RH acessa dados de folha de pagamento, não acessa esquema de outro sistema que não lhe diz respeito.

## 14.6 Arquitetura de computadores e sistema operacional

### Fora do escopo estrito do edital, mas cai em prova

O edital do Perfil 3 fala em arquitetura *de software* (Capítulo 5), não em arquitetura *de hardware*. Ainda assim, provas de concurso costumam reaproveitar questões de "informática básica"/arquitetura de computadores entre perfis diferentes, e esse conteúdo aparece. É rápido de cobrir e vale a pena saber.

### 14.6.1 Modelo de Von Neumann

#### Um só barramento para dados e instruções — e o gargalo que isso cria

A arquitetura de Von Neumann descreve um computador com **memória única** compartilhada entre **instruções** (o programa) e **dados** (as informações que o programa manipula), acessada por um **único barramento**. A CPU busca uma instrução, depois busca/grava dados, pelo mesmo canal — nunca simultaneamente. Esse compartilhamento é conhecido como o **gargalo de Von Neumann**: por mais rápida que a CPU fique, ela é limitada pela velocidade com que consegue trafegar instrução e dado pelo mesmo barramento, um de cada vez.

### 14.6.2 CPU: Unidade de Controle e ULA

#### Duas unidades, duas responsabilidades

- **Unidade de Controle (UC)**: **coordena** a execução — busca a instrução, decodifica o que ela significa, e ativa os sinais de controle necessários para executá-la (não realiza o cálculo em si, apenas orquestra).
- **Unidade Lógica e Aritmética (ULA)**: executa as operações **de fato** — soma, subtração, comparações lógicas (E, OU, NÃO).

A analogia útil: a UC é o maestro (decide o que tocar e quando), a ULA é quem toca o instrumento.

### 14.6.3 Ciclo de instrução e *pipeline*

#### Busca, decodifica, executa

Toda instrução de máquina passa por um ciclo básico: **busca** (*fetch* — traz a instrução da memória), **decodifica** (*decode* — interpreta o que ela pede) e **executa** (*execute* — realiza a operação, geralmente na ULA). Um processador simples faz esse ciclo inteiro para uma instrução antes de começar o próximo.

#### Pipeline: sobrepor os ciclos

Um processador com **pipeline** não espera uma instrução terminar todo o ciclo antes de começar a próxima: enquanto a instrução A está sendo **executada**, a instrução B já está sendo **decodificada** e a instrução C já está sendo **buscada** — como uma linha de montagem, em que cada estação trabalha numa instrução diferente ao mesmo tempo. O ganho é em **taxa de conclusão de instruções por segundo** (*throughput*), não em fazer uma instrução isolada terminar mais rápido.

### 14.6.4 Sistemas de numeração e complemento de dois

#### Conversão entre bases — o mesmo valor, notações diferentes

Um computador só entende binário (base 2) internamente, mas humanos leem mais facilmente octal (base 8) ou hexadecimal (base 16) — ambas escolhidas por serem **potências de 2** ( $8 = 2^3$ ,  $16 = 2^4$ ), o que torna a conversão **dígito a dígito** sem precisar de conta longa: agrupe os bits binários em grupos de 3 (octal) ou 4 (hexadecimal), da direita para a esquerda, e converta cada grupo isoladamente.

#### Convertendo binário para hexadecimal

Binário: 1011 1100. Agrupe de 4 em 4, da direita para a esquerda: 1011 e 1100.  $1011 = 8+0+2+1 = 11 = \text{B}$ .  $1100 = 8+4+0+0 = 12 = \text{C}$ . Resultado: 0xBC.

#### Complemento de dois: como o computador representa negativos

Para representar números negativos em binário sem precisar de um circuito separado para subtração, usa-se o **complemento de dois**: para negar um número, inverte-se todos os bits (complemento de um) e soma-se 1. A vantagem é que **a mesma unidade de soma** da ULA resolve subtração também — " $A - B$ " vira " $A + (\text{complemento de dois de } B)$ ", sem circuito dedicado.

#### Overflow × carry — dois tipos de "estourou"

- **Carry** (vai-um): ocorre quando uma soma **sem sinal** ultrapassa a capacidade de bits disponível — é o "vai um" que sobra da última posição.
- **Overflow**: ocorre quando o resultado de uma operação **com sinal** (complemento de dois) produz um valor que **não cabe** na faixa representável com aquele número de bits — por exemplo, somar dois números positivos grandes e o resultado "virar" negativo por estourar o bit de sinal.

Os dois podem ocorrer independentemente um do outro na mesma operação — não são sinônimos, e um enunciado que os trata como a mesma coisa está simplificando demais um conceito que a prova cobra separado.

### 14.6.5 ROM, firmware e periféricos

#### Memória que sobrevive sem energia

**ROM** (*Read-Only Memory*) é memória **não volátil** (mantém o conteúdo sem energia), tipicamente usada para armazenar **firmware** — o software de baixo nível gravado de fábrica que inicializa o hardware antes mesmo do sistema operacional carregar (ex.: a BIOS/UEFI de um computador). **Periférico** é qualquer dispositivo de entrada/saída conectado à CPU (teclado, monitor, disco), que se comunica com o resto do sistema através de controladores e barramentos de E/S.

### 14.6.6 Memória virtual: paginação, MMU, TLB e *thrashing*

#### O problema: programas maiores que a RAM física disponível

**Memória virtual** é a técnica que permite a um programa "enxergar" mais memória do que a RAM física realmente tem, usando o disco como extensão. O sistema operacional divide a memória em blocos de tamanho fixo chamados **páginas**, e mantém uma **tabela de páginas** que mapeia **endereço virtual** (o que o programa usa) para **endereço físico** (onde o dado realmente está — na RAM ou, se não estiver presente no momento, no disco).

#### MMU e TLB: o hardware que acelera a tradução

A **MMU** (*Memory Management Unit*) é o circuito de hardware que traduz endereço virtual em físico a cada acesso à memória, consultando a tabela de páginas. Como consultar a tabela de páginas na RAM a **cada** acesso seria lento, a MMU mantém um pequeno cache dedicado chamado **TLB** (*Translation Lookaside Buffer*) com as traduções mais recentes/frequentes — um acerto na TLB evita ter que consultar a tabela de páginas completa.

#### *Thrashing*: quando a paginação vira o próprio problema

Se a memória física disponível é pequena demais para as páginas que os processos ativos realmente precisam a cada instante, o sistema passa a gastar mais tempo **trocando páginas entre RAM e disco** (*swapping*) do que executando trabalho útil — esse colapso de desempenho é o **thrashing**. O sintoma característico é a CPU parecer **ociosa** mesmo com processos "rodando": na verdade eles estão travados esperando página chegar do disco, não executando instrução.

### 14.6.7 E/S bloqueante, troca de contexto e DMA

#### E/S bloqueante x troca de contexto

Quando um processo pede uma operação de **entrada/saída bloqueante** (ex.: ler um arquivo em disco), ele fica **suspenso** esperando a operação terminar — não faz sentido a CPU ficar parada nesse meio tempo, então o sistema operacional aproveita para dar a CPU a **outro** processo pronto para rodar. Essa troca — salvar o estado do processo suspenso e carregar o estado do próximo — é a **troca de contexto** (*context switch*), o mecanismo que viabiliza multitarefa numa CPU que só executa uma instrução por vez em cada núcleo.

#### DMA: tirar a CPU do meio do caminho

**DMA** (*Direct Memory Access*) é um controlador dedicado que transfere dados diretamente entre um periférico (ex.: disco) e a memória RAM, **sem** que a CPU precise mediar byte a byte a transferência — a CPU apenas configura a operação (origem, destino, tamanho) e é **interrompida** quando a transferência termina, ficando livre para fazer outra coisa enquanto o DMA trabalha. Sem DMA, toda transferência de E/S consumiria ciclos de CPU copiando dado por dado.

## 14.7 Sistemas de informação corporativos

### Cada sistema atende um nível gerencial diferente

Sistemas de informação corporativos se diferenciam pelo **nível de decisão** que atendem — do operacional ao estratégico:

- **SPT** (Sistema de Processamento de Transações, também chamado transacional): registra as operações do **dia a dia** operacional (uma venda, um pagamento) — nível mais baixo, alto volume, baixa complexidade de decisão por transação.
- **SIG** (Sistema de Informação Gerencial): consolida dados operacionais em **relatórios** para a gerência tática acompanhar desempenho.
- **SAD/SSD** (Sistema de Apoio/Suporte à Decisão): fornece **análise** para decisões **não estruturadas** ou semiestruturadas, tipicamente de nível mais estratégico — simulações, cenários "e se".
- **ERP** (*Enterprise Resource Planning*): integra processos de **múltiplas** áreas (financeiro, estoque, produção, RH) num único sistema com base de dados compartilhada, eliminando retrabalho de dados duplicados entre sistemas isolados.
- **CRM** (*Customer Relationship Management*): foca no relacionamento com o **cliente** — histórico de contato, vendas, suporte.
- **SCM** (*Supply Chain Management*): foca na **cadeia de suprimentos** — fornecedores, logística, estoque entre elos da cadeia.

A pirâmide organizacional clássica (operacional na base, tático no meio, estratégico no topo) ajuda a lembrar: SPT nasce na base, SIG no meio, SAD no topo; ERP/CRM/SCM são **transversais**, cortando vários níveis ao mesmo tempo, cada um por um recorte de processo diferente (recursos internos, cliente, cadeia de fornecimento).

## 14.8 Aprendizado de máquina

### Supervisionado, não supervisionado e semissupervisionado

A distinção central é se os dados de treino trazem **rótulo** (*label* — a resposta certa já conhecida):

- **Supervisionado**: treina com dados **rotulados** — cada exemplo já vem com a resposta esperada. Usado em **classificação** (rótulo categórico: "é spam?") e **regressão** (rótulo numérico: "qual será o preço?").
- **Não supervisionado**: treina com dados **sem rótulo** — o algoritmo busca **estrutura** escondida nos próprios dados, sem saber de antemão qual é a "resposta certa". Usado em **clusterização** (agrupar exemplos parecidos) e **associação** (achar padrões de coocorrência).
- **Semissupervisionado**: usa uma **pequena** quantidade de dados rotulados junto com uma **grande** quantidade de dados sem rótulo — útil quando rotular é caro (exige um humano), mas coletar dado bruto é barato.

### Não inverta "rotulado" entre os dois

Um enunciado que descreve o **não supervisionado** como "usa dados rotulados para prever categorias" trocou a definição com o **supervisionado**. O critério de desempate é sempre: **existe resposta certa conhecida de antemão no conjunto de treino, ou o algoritmo tem que descobrir a estrutura sozinho?**

### Overfitting e underfitting

**Overfitting:** o modelo **decora** particularidades do conjunto de treino (inclusive ruído) em vez de aprender o padrão geral — desempenho ótimo no treino, ruim em dados novos. **Underfitting:** o modelo é **simples demais** para captar sequer o padrão do treino — desempenho ruim em ambos, treino e dados novos. O tratamento detalhado da relação viés×variância por trás desses dois fenômenos está no Capítulo 18.

### Regularização L1 e L2: como conter o overfitting pela penalidade

Um modelo *overfit* costuma ter **coeficientes grandes** — é o que permite a curva ajustada oscilar bruscamente para passar exatamente por cada ponto do treino. Regularizar significa treinar minimizando **erro + uma penalidade sobre o tamanho dos coeficientes**, tornando caro manter coeficiente grande sem necessidade real:

|                   | Penaliza                                           | Efeito                                                                                                                    |
|-------------------|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>L1 (Lasso)</b> | soma dos <b>valores absolutos</b> dos coeficientes | empurra coeficientes <b>até zero</b> — remove atributos do modelo ( <b>seleção de atributos</b> , modelo <i>esparso</i> ) |
| <b>L2 (Ridge)</b> | soma dos <b>quadrados</b> dos coeficientes         | <b>encolhe</b> todos proporcionalmente, mas <b>não zera nenhum</b> — todos os atributos permanecem, com peso menor        |

A razão da diferença está no **formato** da penalidade: elevar ao quadrado (L2) pune muito um coeficiente grande e quase nada um já pequeno — não compensa terminar de zerá-lo. A L1 pune o mesmo valor por unidade em qualquer faixa, então zerar de vez sai "de graça" e compensa. Use L1 quando o objetivo é **descartar** atributos irrelevantes de um conjunto com muitas variáveis; use L2 quando o objetivo é apenas **estabilizar** um modelo com atributos correlacionados, mantendo todos.

### Métricas de avaliação

**MAE** (*Mean Absolute Error*): erro médio absoluto, para **regressão** — a distância média entre previsão e valor real, sem distinguir se errou para mais ou para menos. **F1-score**: média **harmônica** entre precisão (dos que o modelo disse positivo, quantos eram de fato) e recall/revocação (dos que eram de fato positivos, quantos o modelo capturou) — usada quando as classes são desbalanceadas e acurácia simples enganaria. **ROC/AUC**: mede a capacidade do modelo de **separar classes** variando o limiar de decisão — quanto mais perto de 1 a área sob a curva (AUC), melhor a separação.

### Drift: o modelo não piora sozinho, o mundo muda ao redor

Um modelo em produção perde acurácia ao longo do tempo mesmo sem nenhuma mudança de código, porque foi treinado numa "fotografia" da realidade que deixa de valer. Duas causas distintas:

|                      | O que mudou                                                                                     | Exemplo                                                                                                                          |
|----------------------|-------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>Data drift</b>    | a <b>distribuição da entrada</b> — os dados que chegam já não se parecem com os de treino       | o perfil de usuários do sistema mudou (faixa etária, região), mas a relação entre perfil e comportamento continua a mesma        |
| <b>Concept drift</b> | a <b>relação entre entrada e saída</b> — o próprio padrão que o modelo aprendeu deixou de valer | uma lei nova (ou mudança de mercado) faz o mesmo perfil de cliente se comportar de um jeito diferente do que se comportava antes |

A diferença tem consequência prática: data drift às vezes se resolve reamostrando/rebalanceando os dados de treino; concept drift **exige retreinar com rótulos novos**, porque a resposta certa em si mudou — não adianta só ajustar a proporção dos dados antigos. Detectar os dois é papel do monitoramento contínuo de modelos em produção (MLOps).

## 14.9 Mapa: onde estão os demais temas coringa

Vários temas que também caem sob a etiqueta "sem capítulo próprio" já recebem tratamento profundo em outros capítulos deste livro — não os repita aqui, use este mapa:

| Tema                                                       | Onde está             |
|------------------------------------------------------------|-----------------------|
| Blockchain (hash encadeado, raiz de Merkle, <i>nonce</i> ) | Capítulo 9            |
| Servidor web × servidor de aplicação                       | Capítulo 5            |
| 2PC ( <i>two-phase commit</i> ) × Raft (consenso)          | Capítulo 5, Seção 5.6 |
| Virtualização: hipervisor Tipo 1 × Tipo 2 × contêiner      | Capítulo 5            |
| Low-code / no-code                                         | Capítulo 9            |
| RPA ( <i>Robotic Process Automation</i> )                  | Capítulo 3            |
| Os "V"s do Big Data                                        | Capítulo 6            |
| Índices (B+ Tree, hash, bit-map)                           | Capítulo 6            |
| Internet × intranet × extranet × portal                    | Capítulo 13           |
| IA generativa, LLM, RAG, ética e regulação                 | Capítulo 18           |

## 14.10 Roteiro de revisão do capítulo

### Pares que se invertem com frequência

Memorize estes pares em bloco, porque a confusão entre os dois lados de cada par é o erro mais comum deste capítulo: AD (negócio)  $\times$  DBA (técnico); incremental (desde o último backup qualquer)  $\times$  diferencial (desde o último completo); RPO (quanto dado perder)  $\times$  RTO (quanto tempo até voltar); ordem Analysis  $\rightarrow$  Redo  $\rightarrow$  Undo do ARIES; carry (sem sinal)  $\times$  overflow (com sinal); supervisionado (rotulado)  $\times$  não supervisionado (sem rótulo); L1 (zera, seleciona)  $\times$  L2 (encolhe, mantém todos); data drift (mudou quem entra)  $\times$  concept drift (mudou a regra).

## Parte II

### Módulo I — Conhecimentos Gerais (peso 1, 40 questões)

## Capítulo 15

# Língua Portuguesa

### Ficha do edital

Compreensão e interpretação de textos; tipos e gêneros textuais; ortografia; coesão (referenciação, conectores, tempos/modos verbais); estrutura morfosintática (classes de palavras, coordenação, subordinação, pontuação, concordância verbal e nominal, regência, crase, colocação pronominal); reescrita e significação.

Este capítulo cobre cada tópico do edital do zero, com a definição, o raciocínio por trás da regra e exemplos resolvidos. A versão resumida — focada só no que costuma virar pegadinha de prova — está no capítulo correspondente da **apostila**/; aqui o objetivo é fixar o conceito de verdade, não só reconhecer o padrão de banca.

## 15.1 Compreensão e interpretação de textos

### 15.1.1 Três níveis de leitura

Toda questão de interpretação testa um destes três níveis, e a maior parte dos erros acontece por confundir um com o outro:

#### Informação explícita, inferência e opinião

- **Informação explícita:** está escrita literalmente no texto. A pergunta se responde apontando a frase. Erro típico: não achar a frase por estar em outro parágrafo, ou porque o examinador parafraseou (trocou as palavras mas manteve o sentido).
- **Inferência:** não está escrita, mas é a única conclusão possível a partir do que está escrito + conhecimento de mundo compartilhado. Uma inferência válida é **obrigatória**, não uma entre várias leituras possíveis — se dá pra imaginar um cenário em que a inferência é falsa, ela não é uma inferência válida, é um chute.
- **Opinião/juízo de valor:** pertence ao autor do texto (ou a um personagem citado), não ao leitor nem ao examinador. A alternativa erra quando atribui ao autor uma opinião que ele não expressou, ou quando o candidato concorda/discorda pessoalmente do texto em vez de reportar o que o texto diz.

#### Explícito vs. inferência vs. extrapolação

Texto: *"Depois de três tentativas frustradas de aprovar o projeto, a diretoria decidiu arquivá-lo."*

- Explícito: "a diretoria arquivou o projeto— está escrito.
- Inferência válida: "o projeto não foi aprovado nas tentativas anteriores— não está escrito com essas palavras, mas é a única leitura possível de "três tentativas frustradas".
- Extrapolação (não é inferência válida): "a diretoria considera o projeto inviável para sempre— o texto não diz isso; arquivar não é o mesmo que descartar definitivamente. É o tipo de leitura que parece razoável mas vai além do que o texto sustenta.

**Extrapolar não é inferir**

A diferença entre inferência válida e extrapolação é a diferença entre "a única leitura possível" e "uma leitura plausível". Toda vez que uma alternativa usa um verbo mais forte que o texto (o texto "sugere" e a alternativa diz que ele "prova"; o texto descreve uma dificuldade e a alternativa diz que é "impossível"), desconfie: é extrapolação.

**15.1.2 Tipos textuais x gêneros textuais**

Esse par é cobrado quase sempre em conjunto e a diferença entre os dois conceitos é a base de qualquer questão sobre "que tipo de texto é este":

**Tipo é a estrutura; gênero é a embalagem social**

**Tipo textual** é uma categoria **estrutural**, definida pela natureza linguística predominante da composição — são poucos e fixos:

- **Narração**: fatos organizados no tempo, com personagens, enredo, conflito (verbos de ação, tempo passado predominante).
- **Descrição**: apresenta características de algo/alguém, sem progressão temporal (predomínio de adjetivos, verbos de estado).
- **Dissertação/argumentação**: defende um ponto de vista com argumentos, geralmente sobre um tema abstrato.
- **Exposição**: explica, informa, expõe um assunto sem necessariamente defender uma tese (diferença sutil com a dissertação: expor é informar, dissertar é convencer).
- **Injunção**: instrui, ordena, orienta uma ação (verbos no imperativo ou infinitivo).

**Gênero textual** é a forma **social e concreta** que um texto assume no mundo real, definida pela função comunicativa e pelo suporte — são inúmeros e mudam com o tempo (um gênero pode nascer com uma nova tecnologia, como "post de rede social"): notícia, editorial, bula de remédio, receita culinária, carta, e-mail, resenha, ata de reunião, edital, regulamento, poema, conto, contrato.

**Um gênero mistura tipos**

Uma **bula de remédio** (gênero) é predominantemente **injuntiva** ("tome 1 comprimido a cada 8 horas") mas tem trechos **descritivos** ("comprimido branco, redondo") e **expositivos** ("o princípio ativo age bloqueando..."). Isso é a prova de que tipo e gênero são eixos diferentes: um gênero (categoria fixa, social) é composto de vários tipos (categorias estruturais) misturados.

**Regra prática**

Se a pergunta é "que **tipo**" de trecho é esse, procure a estrutura predominante (tem verbo de ação e linha do tempo? narração. Só adjetivos, sem ação? descrição). Se a pergunta é "que **gênero**" é o texto, procure o nome social que ele teria fora da prova (editorial, e-mail, bula). Uma reportagem de jornal (gênero) é majoritariamente narrativa com trechos expositivos (tipo) — a pergunta certa nunca mistura os dois níveis.

### 15.1.3 Coerência x coesão

#### Dois requisitos diferentes de um texto bem construído

**Coerência** é a lógica **do sentido**: as ideias do texto não se contradizem e formam um todo que faz sentido para quem lê — é uma propriedade semântica/pragmática, depende do conhecimento de mundo do leitor. **Coesão** é a amarração **gramatical** entre as partes do texto: os recursos linguísticos visíveis (pronomes, conectores, repetição controlada) que ligam frase a frase, parágrafo a parágrafo.

Um texto pode ser **coesos e incoerentes**: todas as frases conectadas gramaticalmente, mas o conteúdo é contraditório ou absurdo. E pode ser **coerente e pouco coeso**: o sentido geral se entende, mas faltam conectivos, e a leitura fica truncada. Os dois são independentes.

## 15.2 Ortografia

A ortografia da FGV cai pouco isoladamente (mais como parte de reescrita ou concordância) porque o vigente é o Acordo Ortográfico de 1990, já consolidado no ensino desde 2016. Os pontos que ainda geram dúvida real:

#### Casos que sobrevivem como pegadinha

- **Trema**: extinto para o português do Brasil (não se escreve mais "lingüiça", "seqüência" — ficou "linguiça", "sequência"). Só permanece em nomes próprios e suas derivações (Müller, mülleriano).
- **Hífen em palavras compostas com prefixo**: a regra prática (não memorize todos os prefixos, olhe a **terminação do prefixo x início da palavra seguinte**): mesma letra em ambos → hífen ("micro-ondas", "anti-inflamatório", com prefixo terminado em vogal + palavra iniciada pela mesma vogal, ou prefixo terminado em consoante + palavra iniciada pela mesma consoante, como "inter-regional"); prefixo terminado em vogal + palavra iniciada em **r** ou **s** → dobra a letra, sem hífen ("antirreligioso", "contrassenso"); demais casos, funde sem hífen ("autoescola", "extraoficial").
- **Acentuação de ditongos abertos "ei" e "oi" em paroxítonas**: não se acentuam mais ("ideia", "plateia", "heroico", "jiboia" — antes eram "idéia", "platéia", "heróico"). Continuam acentuados os ditongos abertos em **oxítonas** ("herói", "chapéu", "céu").
- **Acento diferencial**: caiu na reforma, com poucas exceções que sobrevivem (**pôde** passado x **pode** presente; **pôr** verbo x **por** preposição). "Para" (verbo parar, 3ª pessoa) e "pára" não se distinguem mais por acento.

#### Reforma ortográfica não é "fonte volátil qualquer"

O Acordo Ortográfico de 1990 está em vigor no Brasil desde 1º de janeiro de 2016 (fim do período de transição). Se uma questão datada de antes de 2016 usar "idéia" ou "vôo" com acento, não é erro do material — é o texto escrito sob a norma anterior. Não corrija exemplo histórico citado como histórico.

## 15.3 Coesão textual: referenciação, conectores e tempos verbais

### 15.3.1 Coesão referencial

#### Anáfora, catáfora e elipse

São os três mecanismos que evitam repetir a mesma palavra o tempo todo:

- **Anáfora:** um termo retoma algo já dito **antes**. É o caso mais comum. *"O relatório chegou atrasado. **Ele** continha erros."* ("ele" retoma "o relatório").
- **Catáfora:** um termo **anuncia** algo que só vem depois. *"Só quero uma coisa: **que você estude**."* (o "isso" implícito aponta para frente).
- **Elipse:** o termo é **omitido**, mas recuperável pelo contexto. *"Ele estudou de manhã; ela, à tarde."* (omitiu-se "estudou" na segunda oração — também chamado de zeugma quando a omissão é de um termo já citado).

O que retoma pode ser um **pronome** (ele, o qual, isso), um **sinônimo** ou **hiperônimo** (repetir "o documento" chamando de "o papel" ou de "o texto"), ou uma **elipse**.

### 15.3.2 Conectores e o valor semântico que carregam

A reescrita/interpretação cobra menos o nome da conjunção e mais **que relação lógica** ela expressa — porque a mesma relação pode ser expressa por conectores de classes gramaticais diferentes.

| Relação            | Conectores que a expressam                                                         |
|--------------------|------------------------------------------------------------------------------------|
| Adição             | e, nem, também, além disso, ainda                                                  |
| Oposição/contraste | mas, porém, contudo, todavia, entretanto, no entanto, embora, ainda que, apesar de |
| Causa              | porque, já que, uma vez que, visto que, por causa de, devido a                     |
| Consequência       | portanto, logo, por isso, de modo que, de forma que                                |
| Condição           | se, caso, contanto que, desde que, a menos que                                     |
| Finalidade         | para que, a fim de que, com o objetivo de                                          |
| Tempo              | quando, enquanto, assim que, logo que, antes que, depois que                       |
| Comparação         | como, tal qual, assim como, mais/menos... do que                                   |
| Conformidade       | conforme, segundo, de acordo com, consoante                                        |

#### O mesmo conector muda de valor com o contexto

"Como" pode ser causal ("Como chovia, não saiu" = "porque chovia"), comparativo ("Ele é como o pai") ou conformativo ("Como diz o ditado..."). "Que" pode ser conjunção integrante (introduz substantiva), pronome relativo (introduz adjetiva) ou até conjunção consecutiva ("tão cansado que dormiu"). A prova nunca pergunta só "que classe é esta palavra" — pergunta "que **relação/função** ela introduz **neste** contexto", e a resposta muda frase a frase.

### 15.3.3 Correlação/sequência de tempos verbais

#### O tempo da oração principal governa o da subordinada

Quando duas orações do período estão relacionadas no tempo, o verbo da subordinada tem que ser compatível com o tempo do verbo da principal — é a mesma lógica do "sequence of tenses" do inglês, só que menos rígida em português. Padrões mais cobrados:

- Principal no **presente** → subordinada pode ir para qualquer tempo compatível com o sentido: "*Acho que ele **chegou/chega/vai chegar**.*"
- Principal no **pretérito** → a subordinada tende a "puxar" para o pretérito também (discurso indireto clássico): "*Ele disse que **viria***" (não "que virá") — futuro do pretérito no lugar do futuro do presente, porque a fala foi relatada no passado.
- **Condicional**: "se" + **pretérito imperfeito do subjuntivo** na condição pede **futuro do pretérito** (não futuro do presente) na consequência: "*Se eu **tivesse** tempo, eu **estudaria***" (não "eu estudarei"). É um dos erros mais comuns de concordância verbal em prova de reescrita.

## 15.4 Classes de palavras

#### As dez classes, divididas em variáveis e invariáveis

**Variáveis** (flexionam em gênero, número, e às vezes grau/pessoa/modo/tempo): substantivo, artigo, adjetivo, numeral, pronome, verbo. **Invariáveis** (forma fixa): advérbio, preposição, conjunção, interjeição.

O ponto que a prova explora não é decorar a lista, é reconhecer que **a mesma palavra muda de classe conforme a função na frase** (derivação imprópria / hibridismo funcional):

- "Amanhã" → substantivo ("o amanhã é incerto") ou advérbio ("chego amanhã").
- "Que" → pronome relativo, conjunção integrante, conjunção consecutiva, ou interjeição de espanto ("Que confusão!").
- "Mal" → advérbio ("ele foi mal") ou substantivo ("um mal necessário") — não confundir com "mau" (adjetivo, antônimo de "bom").

#### Regra prática

Para classificar uma palavra ambígua, **substitua-a mantendo a função**: se dá para trocar por "isso"/"aquilo" e ainda ocupar posição de sujeito/objeto, é substantivo; se modifica um verbo/adjetivo/outro advérbio sem flexionar, é advérbio. Nunca classifique a palavra isolada — classifique a palavra **na frase dada**.

## 15.5 Sintaxe do período: coordenação e subordinação

### 15.5.1 Oração e período

#### Contando orações

**Oração** é todo enunciado organizado em torno de um núcleo verbal (verbo ou locução verbal). **Período simples** tem uma oração só; **período composto** tem duas ou mais. Truque de contagem: conte os verbos/locuções verbais do período — cada núcleo verbal é, em regra, uma oração (locução verbal conta como uma só).

#### Exemplo

"O candidato estudou muito porque a prova é difícil." → dois núcleos verbais ("estudou", "é")  
→ período composto de duas orações.

O período composto se organiza por dois mecanismos: **coordenação** (orações independentes entre si) e **subordinação** (uma oração é termo sintático da outra). Os dois podem coexistir no mesmo período (período misto).

### 15.5.2 Coordenação

#### Orações coordenadas

São **sintaticamente independentes**: nenhuma delas exerce função de termo (sujeito, objeto etc.) dentro da outra — cada uma "sobrevive sozinha" gramaticalmente. Podem vir **sem** conectivo (**assindéticas**, separadas por vírgula/ponto e vírgula: "*Cheguei, vi, venci*") ou **com** conjunção coordenativa (**sindéticas**), que se dividem em cinco tipos:

| Tipo        | Conjunções típicas                                   | Ideia               |
|-------------|------------------------------------------------------|---------------------|
| Aditiva     | e, nem, mas também                                   | soma                |
| Adversativa | mas, porém, contudo, todavia, entretanto, no entanto | contraste/oposição  |
| Alternativa | ou, ou...ou, ora...ora, seja...seja                  | escolha/alternância |
| Conclusiva  | logo, portanto, por isso, pois (depois do verbo)     | conclusão           |
| Explicativa | pois (antes do verbo), porque, que                   | justificativa       |

#### A conjunção "pois" muda de classe com a posição

"Estude, pois a prova é difícil." — "pois" logo depois da vírgula, **antes** do verbo → **explicativa** (equivale a "porque"). "A prova é difícil; estude, pois." / "Estude; a prova, pois, é difícil." — "pois" deslocado, intercalado **no meio ou no fim** da oração → **conclusiva** (equivale a "portanto"). Regra prática: pois colado após a vírgula = explicativa; pois "solto" no meio da frase = conclusivo.

### 15.5.3 Subordinação

#### Orações subordinadas

Uma oração **depende sintaticamente** da outra: ela ocupa, sozinha, a função que uma **palavra** ocuparia (sujeito, objeto, adjunto...). A pergunta-chave para classificar é sempre: **"que classe gramatical essa oração inteira está imitando?"** — substantivo, adjetivo ou advérbio. Daí os três grandes grupos: **substantivas, adjetivas, adverbiais**.

#### 15.5.3.1 Substantivas

##### A oração no lugar de um substantivo

Iniciadas por conjunção **integrante** ("que" ou "se" — não confundir com pronome relativo). Teste rápido: troque a oração inteira por **"isso"**; se a frase continuar de pé, é substantiva, e a posição de "isso" na frase revela a função:

| Subtipo            | Função                 | Exemplo                                                                |
|--------------------|------------------------|------------------------------------------------------------------------|
| Subjetiva          | sujeito                | <i>É necessário <b>que você estude</b>.</i> ("Isso é necessário")      |
| Objetiva direta    | objeto direto          | <i>Espero <b>que você passe</b>.</i> ("Espero isso")                   |
| Objetiva indireta  | objeto indireto        | <i>Preciso <b>de que você me ajude</b>.</i> ("Preciso disso")          |
| Completiva nominal | completa<br>nome       | <i>Tenho certeza <b>de que ele passou</b>.</i> ("Tenho certeza disso") |
| Predicativa        | predicativo<br>sujeito | <i>O problema é <b>que faltou tempo</b>.</i> ("O problema é isso")     |
| Apositiva          | aposto                 | <i>Só quero uma coisa: <b>que você estude</b>.</i>                     |

##### Objetiva indireta x completiva nominal

As duas têm preposição, e é fácil trocar uma pela outra. O critério: se a preposição é **exigida pelo verbo** (regência verbal), é objetiva indireta; se é exigida por um **nome abstrato** da oração principal (substantivo/adjetivo como "certeza", "medo", "necessidade", "capaz"), é completiva nominal. *"Preciso de que você ajude"* (verbo "precisar" rege "de") é objetiva indireta; *"Tenho certeza de que ele passou"* ("certeza de—" o nome "certeza" rege "de") é completiva nominal.

#### 15.5.3.2 Adjetivas

##### A oração no lugar de um adjetivo

Sempre iniciadas por **pronome relativo** (que, quem, o qual, cujo, onde, quando referidos a um antecedente), caracterizam um substantivo/pronome da oração principal (o **antecedente**). Dividem-se em:

- **Restritivas** (**sem** vírgula): delimitam o universo do antecedente — retirar a oração **muda o sentido** da frase. *"Os candidatos **que estudaram** passaram."* (só os que estudaram, não todos).

- **Explicativas (com vírgula):** acrescentam uma informação extra sobre **todo** o antecedente — retirar a oração **não muda** o sentido essencial. "*Os candidatos, que estudaram muito, passaram.*" (todos estudaram; é um comentário à parte).

### A vírgula muda o sentido da frase inteira

Este é o ponto mais cobrado do capítulo inteiro de sintaxe: a mesma sequência de palavras, com e sem vírgula, afirma coisas diferentes. "*Os alunos que estudaram passaram*" não garante que todos os alunos passaram (só os que estudaram); "*Os alunos, que estudaram, passaram*" afirma que **todos** os alunos estudaram e **todos** passaram. Ao comparar duas versões de uma frase numa questão de reescrita, a vírgula antes do "que" é o primeiro lugar a olhar.

### Cujo

"Cujo" é sempre pronome relativo (introduz adjetiva), concorda em gênero/número com a coisa **possuída** (não com o possuidor), e nunca vem seguido de artigo. "*O candidato cujo caderno sumiu*" (não "cujo o caderno").

### 15.5.3.3 Adverbiais

#### A oração no lugar de um advérbio

Fazem o papel de adjunto adverbial (circunstância) da oração principal. Nove tipos, decorados pela conjunção-gatilho:

| Tipo         | Conjunções típicas                     | Ideia        |
|--------------|----------------------------------------|--------------|
| Causal       | porque, já que, uma vez que, visto que | causa        |
| Consecutiva  | que (após "tão, tal, tanto")           | consequência |
| Condicional  | se, caso, contanto que, desde que      | condição     |
| Concessiva   | embora, ainda que, mesmo que           | concessão    |
| Comparativa  | como, (do) que                         | comparação   |
| Conformativa | conforme, segundo, como                | conformidade |
| Final        | para que, a fim de que                 | finalidade   |
| Temporal     | quando, enquanto, assim que, logo que  | tempo        |
| Proporcional | à medida que, à proporção que          | proporção    |

#### Causal x consecutiva

"*Estudou tanto que passou.*" **Não** é causal — é **consecutiva**: a conjunção correlata é "tanto...que", e a ideia é consequência (o resultado de estudar tanto foi passar), não a causa de algo.

### Concessiva não é "adversativa disfarçada"

As duas contrapõem ideias, mas por mecanismos sintáticos diferentes. **Adversativa** é **coordenação**: duas orações independentes contrapostas ("*Estudou, mas não passou*"). **Concessiva** é **subordinação**: uma oração se subordina admitindo um obstáculo que não impede a outra ("*Embora tenha estudado, não passou*"). Trocar "mas" por "embora" (ou vice-versa) muda a estrutura sintática do período inteiro, não só o conectivo.

#### 15.5.4 Orações reduzidas

##### Sem conectivo, sem verbo flexionado

Quando a subordinada não tem conectivo nem verbo flexionado (infinitivo, gerúndio ou particípio), ela é **reduzida**. A versão com conectivo e verbo flexionado é a **desenvolvida**.

| Forma nominal | Reduzida                        | Desenvolvida equivalente                                |
|---------------|---------------------------------|---------------------------------------------------------|
| Infinitivo    | <i>Ao chegar, avisou.</i>       | <i>Quando chegou, avisou.</i> (valor temporal)          |
| Gerúndio      | <i>Estudando, você passa.</i>   | <i>Se você estudar, você passa.</i> (valor condicional) |
| Particípio    | <i>Terminada a prova, saiu.</i> | <i>Quando a prova terminou, saiu.</i> (valor temporal)  |

##### Regra prática

Para classificar uma reduzida, **desenvolva-a mentalmente** (acrescente o conectivo que faz sentido) e classifique pelo **valor semântico** resultante (causal, temporal, condicional...) — a forma nominal (infinitivo/gerúndio/particípio) é só a "casca"; o sentido é que define o subtipo, porque não há conjunção explícita ali para entregar a resposta.

#### 15.5.5 Roteiro de classificação

##### Ordem de decisão para classificar um período composto

1. Conte os núcleos verbais → quantas orações tem o período.
2. Cada oração (a partir da segunda) é independente ou depende de outra como se fosse um termo dela?
  - Independente → **coordenada**: identifique a conjunção, classifique (aditiva/adversativa/alternativa/conclusiva/explicativa) ou marque assindética.
  - Depende (funciona como termo da outra) → **subordinada**: pergunte que classe gramatical ela imita.
    - Substantivo → substantiva → identifique a função sintática (troque por "isso") → subtipo.
    - Adjetivo (tem antecedente + pronome relativo) → adjetiva → tem vírgula? restritiva/explicativa.
    - Advérbio (circunstância) → adverbial → identifique a conjunção-gatilho →

subtipo.

3. Verbo no infinitivo/gerúndio/particípio sem conectivo? → **reduzida**, classifique pelo sentido (desenvolva mentalmente).

## 15.6 Pontuação

### A vírgula tem uma lista curta e fechada de usos válidos

Cada vírgula "correta" cabe em uma destas categorias — se não cabe em nenhuma, está errada:

- **Enumeração:** separa itens de uma série (*"Trouxe caneta, lápis, borracha e régua."* — repare que não há vírgula antes do último item ligado por "e", salvo se os itens tiverem sujeitos diferentes).
- **Aposto:** explica/identifica um termo, isolado por vírgulas (*"Machado de Assis, o maior escritor brasileiro, nasceu no Rio."*).
- **Vocativo:** chama alguém (*"Lucas, venha aqui."*).
- **Oração intercalada:** interrompe a frase principal (*"O réu, segundo o juiz, mentiu."*).
- **Adjunto adverbial deslocado:** quando o adjunto sai do lugar padrão (fim da frase) e vai para o início/meio (*"Ontem, o sistema caiu."* — vírgula opcional se curto, recomendada se longo).
- **Oração subordinada adverbial anteposta à principal** (*"Se chover, cancelamos o evento."* — a vírgula aqui é **obrigatória**; se a ordem inverter, *"Cancelamos o evento se chover"*, some).
- **Oração adjetiva explicativa** (ver seção anterior).
- **Elipse de verbo** (*"Ele estudou de manhã; ela, à tarde."*).

### Vírgula proibida

Nunca se separa por vírgula: sujeito do predicado (*"O candidato, passou"* está errado), verbo do complemento (*"Ele precisa, de ajuda"* está errado), salvo quando entre eles se intercala aposto ou oração intercalada (aí a vírgula existe por causa do intercalado, não "entre sujeito e verbo").

### Ponto e vírgula e dois-pontos

**Ponto e vírgula:** separa orações coordenadas quando pelo menos uma já tem vírgula interna (evita ambiguidade), ou itens de uma enumeração longa com sub-itens. **Dois-pontos:** anuncia uma enumeração, uma citação, uma explicação ou um aposto que resume o que veio antes (*"Só quero uma coisa: que você estude."*).

## 15.7 Concordância verbal

### 15.7.1 Regra geral e sujeito composto

#### O verbo concorda com o sujeito

Regra base: o verbo flexiona em número e pessoa para concordar com o núcleo do sujeito. Com **sujeito composto** anteposto ao verbo, o verbo vai para o **plural** ("*João e Maria **chegaram**.*"). Com sujeito composto **posposto** ao verbo, admite-se concordância no plural **ou** com o núcleo mais próximo ("***Chegaram** João e Maria*" ou, mais raro em prova, "*Chegou João e Maria*").

### 15.7.2 Verbos impessoais

#### Verbos sem sujeito ficam travados na 3ª pessoa do singular

Um pequeno grupo de verbos, em certos sentidos, **não tem sujeito** — e por isso nunca vão para o plural, não importa o que venha depois deles:

| Verbo                   | Quando é impessoal                  | Certo (e o erro comum)                                            |
|-------------------------|-------------------------------------|-------------------------------------------------------------------|
| <b>fazer</b>            | tempo decorrido                     | <i>Faz</i> dois anos — nunca "Fazem dois anos"                    |
| <b>haver</b>            | sentido de "existir" ou tempo       | <i>Havia</i> muitas falhas — nunca "Haviam muitas falhas"         |
| <b>haver</b> + auxiliar | o auxiliar também trava             | <i>Deve haver</i> relatórios — nunca "Devem haver"                |
| <b>existir</b>          | <b>não</b> é impessoal, tem sujeito | <i>Existem</i> muitos processos — nunca "Existe muitos processos" |

O contraste que a banca explora: "haver" (no sentido de existir) e "existir" significam quase a mesma coisa, mas um é impessoal (não varia) e o outro tem sujeito (varia normalmente).

### 15.7.3 Sujeito coletivo e sujeito oracional

**Sujeito coletivo** (multidão, maioria, grupo, equipe) concorda no **singular** quando não especificado ("*A **maioria** **chegou**.*"), mas admite plural quando o coletivo vem seguido de um complemento no plural ("*A **maioria dos candidatos** **chegou/chegaram***" — as duas são aceitas; a FGV costuma cobrar a forma no singular como a "mais formal/tradicional"). **Sujeito oracional**: quando o sujeito é uma oração subordinada substantiva subjetiva inteira, o verbo da principal fica sempre no **singular** ("***É** preciso que todos estudem.*" — não "são preciso").

## 15.8 Concordância nominal

#### Adjetivo concorda com o substantivo a que se refere

Regra geral simples; os casos que a prova cobra são os que **parecem** exceção mas seguem lógica própria:

- "Anexo", "incluso", "obrigado", "próprio", "mesmo", "quite": são adjetivos e

concordam normalmente (*"Seguem **anexas** as fotos."*, *"Ela mesma resolveu."*, *"Ele está **obrigado**, ela está **obrigada**."*) — o erro comum é tratá-los como advérbios invariáveis.

- **"Meio"**advérbio (= **"um pouco"**) é **invariável** (*"Ela está **meio** confusa."* — não *"meia confusa"*); **"meio"**numeral (= metade) concorda (*"Comeu **meia** maçã."*).
- **"É proibido/necessário/preciso" + substantivo**: fica **invariável** (masculino singular) quando o substantivo **não** tem determinante (artigo/pronome) antes (*"**É proibido** entrada."*); **concorda** quando o substantivo vem com determinante (*"**É proibida** a entrada."*).
- **"Bastante"**: como advérbio (modifica verbo/adjetivo) é invariável (*"Elas estudaram **bastante**."*); como adjetivo/pronome (modifica substantivo, = "muitos/muitas") concorda (*"**Bastantes** candidatos desistiram."*).

### O padrão por trás de "é proibido"/"é bastante"

Não são regras soltas: a lógica comum é **"a palavra concorda quando funciona como adjetivo do substantivo; fica fixa quando funciona como advérbio/predicativo genérico sem se prender a um substantivo determinado"**. Uma vez internalizado esse padrão, "meio", "bastante" e "é proibido" deixam de ser três regras decoradas e viram uma regra só aplicada três vezes.

## 15.9 Regência verbal e nominal

### Regência é sobre a preposição que o verbo/nome exige

**Regência verbal**: a preposição (ou ausência dela) que cada verbo exige para introduzir seu complemento — e o mesmo verbo pode mudar de sentido conforme a regência:

| Verbo                  | Regência e sentido                                              | Exemplo                                      |
|------------------------|-----------------------------------------------------------------|----------------------------------------------|
| assistir               | "a"= ver/presenciar (transitivo indireto)                       | Assisti <b>ao</b> filme.                     |
| assistir               | sem prep. = ajudar, dar assistência (transitivo direto)         | O médico assistiu o paciente.                |
| visar                  | "a"= ter como objetivo (formal)                                 | O projeto visa <b>a</b> reduzir custos.      |
| visar                  | sem prep. = mirar, rubricar                                     | Visou o documento; visou o alvo.             |
| obedecer / desobedecer | sempre "a"(transitivo indireto)                                 | Obedeça <b>às</b> normas.                    |
| esquecer               | sem prep. (transitivo direto)<br>OU pronominal "esquecer-se de" | Esqueci o nome. / Esqueci-me <b>do</b> nome. |
| lembrar                | sem prep. (transitivo direto)<br>OU pronominal "lembrar-se de"  | Lembrei o nome. / Lembrei-me <b>do</b> nome. |
| implicar               | sem prep. = acarretar (transitivo direto, sentido de "causar")  | O atraso implicou <b>a</b> perda do prazo.   |
| preferir               | "a"(nunca "do que") na norma culta                              | Prefiro café <b>a</b> chá.                   |

**Regência nominal:** substantivos e adjetivos abstratos também exigem preposição fixa ("obediente **a**", "apto **para**", "cheio **de**", "certeza **de**") — é a mesma preposição que aparece na oração completiva nominal (ver Seção 15.5.3.1 deste capítulo).

### Regência com duplo regime é área de controvérsia gramatical

Alguns pares de regência têm **mais de uma forma aceita** pela norma culta (ex.: "namorar" pode ou não levar preposição conforme a gramática consultada; "chegar/ir a/para" varia por região e por tradição/permanência pretendida). Nesses casos não existe "a" resposta absoluta — a banca, ao cobrar, costuma usar o caso **mais consolidado** (assistir, obedecer, visar, preferir da tabela acima), não os limítrofes. Se um caso limítrofe aparecer, desconfie de alternativas que tratam a variação como erro categórico.

## 15.10 Crase

### Crise é a fusão da preposição "a" com o "a" artigo/pronome

Crise só existe quando **duas condições se encontram ao mesmo tempo**: (1) um termo antes que **exige a preposição "a"** (regência verbal ou nominal) e (2) um termo depois que **admite artigo feminino "a(s)"**. Faltando qualquer uma das duas, não há crise.

- **Obrigatória:** locuções adverbiais femininas de instrumento/modo/causa ("à mão", "às pressas", "à força"); locuções conjuntivas/prepositivas femininas ("à medida que", "à

proporção que"); antes de "moda"/"maneira" subentendida ("sapato à Luís XV" = "à moda de"); horas determinadas quando o contexto pede "a" ("Cheguei às 8h").

- **Proibida:** antes de verbo ("*Começou a estudar*", nunca "*à estudar*"); antes de palavra masculina ("*Andar a cavalo*"); antes de pronome que não admite artigo ("*Entreguei a ela*", nunca "*à ela*" — pronomes pessoais não levam artigo, salvo exceções específicas); entre palavras repetidas ("*cara a cara*", "*dia a dia*"); antes de "casa" (sem especificador) e "terra" (sentido de chão firme, oposto a bordo) no sentido genérico.
- **Facultativa:** antes de pronome possessivo feminino ("*Entreguei a/à sua filha*"); antes de nome próprio de mulher ("*Entreguei a/à Maria*"); depois da preposição "até" ("*Foi até a/à porta*").

### Teste da substituição por palavra masculina

Troque o termo depois do "a" por um equivalente **masculino**. Se o masculino pede "ao", o feminino leva crase; se o masculino pede só "o" (sem preposição) ou "a" (sem artigo), não há crase. "*Foi à escola*" → masculino: "*Foi ao colégio*" → crase confirmada. "*Foi a pé*" → masculino: "*Foi a cavalo*" (não "*ao cavalo*") → sem crase.

### Crase em caso limítrofe também tem controvérsia gramatical

Casos como crase antes de nomes de lugares que admitem artigo de forma variável ("Vou a Bahia" x "Vou à Bahia", dependendo se o falante entende "Bahia" como admitindo artigo) ou antes de pronomes possessivos são **genuinamente controversos** entre gramáticos — não é uma lacuna do material, é a língua tendo duplo regime ali. A banca, quando quer resposta inequívoca, usa os casos obrigatórios/proibidos da lista acima, não os facultativos.

## 15.11 Colocação pronominal

### Próclise, mesóclise e ênclise

Trata da posição do pronome oblíquo átono (me, te, se, o, a, lhe, nos, vos, lhes) em relação ao verbo:

- **Próclise** (pronome **antes** do verbo): puxada por uma **palavra atrativa** antes do verbo — palavra de sentido negativo ("*Não me diga*"), pronome relativo ("*o homem que me ajudou*"), conjunção subordinativa ("*quando se formou*"), advérbio sem vírgula depois ("*Aqui se vive bem*"), pronome indefinido ou interrogativo ("*Ninguém me avisou*", "*Quem te contou?*").
- **Mesóclise** (pronome **no meio** do verbo): só ocorre com verbo no **futuro do presente** ou **futuro do pretérito**, e apenas quando não há palavra atrativa antes ("*Dir-lhe-ei a verdade*"; se houvesse atrativa antes, viraria próclise: "*Não lhe direi*").
- **Ênclise** (pronome **depois** do verbo): a posição "neutra"/padrão quando não há atrativa antes e o verbo não está no futuro — é a posição de **início de frase** ("*Diga-me a verdade*") e a recomendada na norma culta escrita quando não há motivo para próclise.

### Início de frase nunca leva próclise na norma culta escrita

"*Me diga a verdade*" é falado corrente no Brasil, mas na norma culta escrita (a que a prova cobra) frase iniciada por verbo exige ênclise: "**Diga-me a verdade.**" É um dos pontos onde o

português informal brasileiro mais diverge da norma cobrada em concurso.

## 15.12 Reescrita e significação

### O que muda e o que não pode mudar numa reescrita

Questão de reescrita pede para reescrever um trecho **mudando a forma sem mudar o sentido** (ou, menos comum, mudando intencionalmente o sentido, e aí a pergunta avisa). O que se testa:

- **Sinonímia:** troca de palavra por um sinônimo que preserva o sentido **no contexto** (nem todo sinônimo de dicionário serve em toda frase — "grande" pode ser "amplo" em um contexto e "importante" em outro).
- **Denotação x conotação:** sentido **literal/dicionário** (denotação) x sentido **figurado/contextual** (conotação). Reescrever trocando um sentido conotativo por uma paráfrase literal do mesmo sentido (não pelo sentido literal da palavra) é o padrão mais cobrado.
- **Ambiguidade:** mais de uma leitura sintática/semântica possível para a mesma frase — reescrita "corrige" ambiguidade reordenando termos ou trocando um pronome vago por um termo específico ("João disse a Pedro que **ele** passou" — "ele" é ambíguo; a reescrita sem ambiguidade precisa nomear quem passou).
- **Voz ativa x passiva (apassivação):** mesma informação, estrutura sintática diferente ("O réu cometeu o crime" ↔ "O crime foi cometido pelo réu") — o sujeito da ativa vira agente da passiva, o objeto direto da ativa vira sujeito da passiva.
- **Discurso direto x indireto:** o direto reproduz literalmente a fala (aspas/travessão, verbo de elocução: "disse:", "perguntou:"); o indireto relata a fala dentro de uma oração subordinada, com os ajustes de pessoa e tempo verbal da Seção 15.3.3 ("Ele disse: 'Eu vou'" → "Ele disse que **iria**").

### Reescrita preservando sentido

Original: "*Embora tenha estudado bastante, ele não passou.*" Reescritas equivalentes: "*Ele estudou bastante, mas não passou.*" (concessiva → adversativa, mudou de subordinação para coordenação, mesmo sentido); "*Apesar de ter estudado bastante, ele não passou.*" (troca de conjunção concessiva por locução prepositiva + infinitivo, reduzida).

### Regra prática

Para validar uma reescrita, pergunte: **quem fez o quê a quem**, mudou? Se as relações de sujeito/objeto/causa/consequência continuam as mesmas e só a **forma** (ordem das palavras, voz, classe gramatical do conectivo) mudou, a reescrita é válida. Se alguma relação lógica se inverteu ou sumiu, não é reescrita equivalente — é outra frase.

## Capítulo 16

# Língua Inglesa

Este capítulo trata a prova de Inglês pelo que ela realmente é: um teste de **leitura instrumental** sobre texto técnico/jornalístico curto, não de conversação nem de gramática decorada isoladamente. Cada seção cobre um mecanismo da língua que a interpretação de texto explora — referência pronominal, conectivos, modais, vocabulário em contexto — com o raciocínio para resolver, não só a lista de palavras. A calibração de estilo (que tipo de texto, que tipo de pegadinha) está na apostila/.

### 16.1 Como ler para prova: *skimming* e *scanning*

#### Duas técnicas de leitura, para dois tipos de pergunta

- **Skimming:** leitura **rápida e superficial** do texto inteiro, para captar a ideia geral, o gênero e o propósito — sem parar em palavra desconhecida. Use no primeiro contato com o texto, antes de olhar as perguntas, e para responder questões de "ideia principal" ou "propósito do texto".
- **Scanning:** busca **dirigida** por uma informação específica (um nome, uma data, uma palavra) — os olhos varrem o texto procurando o ponto exato, ignorando o resto. Use depois de ler a pergunta, quando ela pede um detalhe pontual (referência de pronome, sentido de uma palavra específica, se uma afirmativa bate com o texto).

#### A ordem que economiza tempo de prova

Não tente entender **cada palavra** do texto antes de ir às perguntas — é a armadilha mais comum de quem trata a prova como teste de vocabulário. Faça um *skimming* rápido (ideia geral + gênero), *depois* leia a primeira pergunta, e só então volte ao texto com *scanning* para achar o trecho exato que ela cobra. A resposta de questão de detalhe está sempre em uma frase ou duas, localizáveis — não exige reler o texto inteiro de novo.

### 16.2 Referência pronominal

#### O pronome aponta para trás — em regra, para o mais próximo relevante

Pronomes (*it, this, that, they, her, his, these, those*) existem para **evitar repetir** um substantivo já mencionado — exatamente como a anáfora do português (Capítulo 15). Resolver a referência é achar, **voltando ao texto**, a que substantivo/ideia o pronome se refere. A armadilha: a prova costuma colocar **dois** substantivos candidatos na mesma vizinhança do pronome, e um deles é gramaticalmente plausível mas semanticamente errado.

### Resolvendo um pronome ambíguo à primeira vista

*"The professor asked the student to review the report before her presentation."*

"Her" pode, à primeira vista, parecer se referir a "the student" (mais próximo na frase) — mas o sentido completo depende de **quem vai apresentar**: se o contexto no texto deixa claro que é a aluna quem vai apresentar, "her" = the student; se é a professora quem vai apresentar depois de receber a revisão, "her" = the professor. **Proximidade sintática não decide sozinha** — é preciso o sentido da frase completa, às vezes até do parágrafo anterior, para confirmar.

### O distrator do pronome usa o candidato mais óbvio, não o correto

Quando a prova pergunta a que um pronome se refere, uma das alternativas erradas quase sempre é o substantivo **fisicamente mais próximo** do pronome no texto — porque parece a resposta "óbvia" para quem não voltou a conferir. A técnica correta é sempre **retornar à frase** (às vezes à anterior) e testar a substituição: troque o pronome pelo candidato e veja se a frase mantém o sentido lógico do texto inteiro, não só da oração isolada.

## 16.3 Conectivos e o valor lógico que carregam

### Cada conectivo sinaliza uma relação lógica específica

Assim como em português (Capítulo 15, Seção 15.3.3), a interpretação em inglês cobra menos "que classe gramatical é essa palavra" e mais "**que relação lógica** ela introduz entre as duas partes do texto":

| Relação             | Conectivos                                        | Sinal                                                            |
|---------------------|---------------------------------------------------|------------------------------------------------------------------|
| Consequência/efeito | thus, therefore, hence, so, as a result           | o que vem depois é <b>resultado</b> do que veio antes            |
| Contraste/concessão | however, whereas, but, although, despite, yet     | o que vem depois <b>contradiz ou relativiza</b> o que veio antes |
| Adição              | moreover, furthermore, in addition, besides, also | reforça no <b>mesmo sentido</b> do que já foi dito               |
| Causa               | because, since, as, due to                        | o que vem depois é o <b>motivo</b> do que veio antes             |
| Condição            | if, unless, provided that                         | <i>unless</i> é <b>condição negativa</b> (= "if not")            |
| Tempo               | meanwhile, while, when, after, before             | situa <b>quando</b> algo acontece em relação a outra ação        |
| Exemplificação      | for example, for instance, such as                | introduz um <b>caso concreto</b> do que foi dito de forma geral  |

### Trocando o conectivo certo pela relação certa

"The company's servers were outdated. **Thus**, the migration to cloud infrastructure became urgent."

"Thus" marca que a migração urgente é **consequência** de os servidores estarem desatualizados — não é um exemplo, nem uma condição, nem um contraste. Se o texto trocasse "thus" por "however", o sentido mudaria para uma **oposição** — e a frase deixaria de fazer sentido lógico com o contexto dado.

### A banca confunde causa/consequência com contraste

O erro mais comum em questão de conectivo é a alternativa trocar a relação **real** do texto (efeito, adição, causa) por **contraste**, ou vice-versa — porque são categorias que soam parecidas quando lidas rápido demais. Antes de responder, releia a frase completa (a que vem *antes* e a que vem *depois* do conectivo) e pergunte: a segunda ideia **decorre** da primeira, **se opõe** a ela, ou só a **reforça**?

## 16.4 Verbos modais

### O que cada modal sinaliza sobre a atitude do autor

Modal não é só gramática de conjugação — cada um carrega um **grau de certeza ou obrigação** que muda a leitura do texto:

| Modal           | Sentido                                                                       | Força         |
|-----------------|-------------------------------------------------------------------------------|---------------|
| must            | obrigação/necessidade forte, ou dedução quase certa                           | alta          |
| should/ought to | sugestão, conselho, expectativa                                               | média         |
| have to         | obrigação (geralmente externa, imposta por regra/circunstância)               | alta          |
| can/could       | possibilidade, permissão, capacidade ( <i>could</i> é mais formal/hipotético) | variável      |
| may/might       | possibilidade — <i>might</i> indica <b>menor</b> certeza que <i>may</i>       | baixa a média |

O foco de prova não é decorar a tabela de regras de uso do modal em isolado, é reconhecer **que atitude o autor do texto está expressando** quando o usa — um "should" no meio de um parágrafo é uma **recomendação**, não um fato relatado como certo.

### O modal muda a força da afirmação

"Developers should review AI-generated code before merging it." — o autor está **recomendando** uma prática, não afirmando que é uma regra universal seguida por todos.

"Developers must review AI-generated code before merging it, according to the company's new policy." — aqui é **obrigação** formal, imposta por uma política — uma leitura mais forte que a anterior, mesmo a frase sendo quase idêntica.

## 16.5 Vocabulário em contexto

### A mesma palavra muda de classe — e de sentido — conforme a função

Assim como em português (Capítulo 15, Seção de Classes de Palavras), várias palavras do inglês técnico funcionam ora como **substantivo**, ora como **verbo**, e o sentido muda conforme a posição na frase — não existe glossário fixo que resolva sozinho, é preciso olhar a função sintática:

- **browse**: verbo ("to browse the web"= navegar) ou, mais raro, substantivo informal.
- **rate**: verbo ("to rate a product"= avaliar) ou substantivo ("a high failure rate"= uma taxa).
- **access**: verbo ("to access a system"= acessar) ou substantivo ("access to the system"= o acesso).
- **post**: verbo ("to post an update"= publicar) ou substantivo ("read the post"= a publicação).

### Como decidir a classe de uma palavra ambígua

Olhe a **posição** na frase: logo depois de *to*, de um modal (*can*, *should*), ou concordando como núcleo de uma oração → provavelmente **verbo**. Precedida de artigo (*a*, *the*) ou de adjetivo, ou seguida de preposição que a modifica → provavelmente **substantivo**. O mesmo teste de substituição usado em português funciona aqui: tente trocar por um verbo/substantivo inequívoco e veja se a frase continua de pé com a mesma estrutura.

### Falsos cognatos — parece português, não é

Palavras parecidas com o português enganam quando o sentido diverge: "*actually*" não é "atualmente" (é "na verdade"); "*eventually*" não é "eventualmente" (é "finalmente, eventualmente no sentido de resultado final"); "*pretend*" não é "pretender" (é "fingir"); "*parents*" não é "parentes" (é "pais"). Em texto técnico de TI, cuidado especial com "*library*" (biblioteca de código, não "livraria") e "*realize*" (perceber, dar-se conta — não necessariamente "realizar/executar").

## 16.6 Compreensão global: ideia principal, propósito e gênero

### Três perguntas diferentes sobre o texto como um todo

- **Ideia principal** (*main idea*): o resumo do que o texto **diz**, em uma frase — não um detalhe isolado de um parágrafo, mas o fio condutor de todos eles.
- **Propósito** (*purpose*): **por que** o autor escreveu — para informar, persuadir, alertar, vender, instruir. Um mesmo assunto pode ser tratado com propósitos diferentes (um texto sobre IA pode informar sobre uma pesquisa ou alertar sobre um risco).
- **Gênero** (*genre*): **que tipo** de texto é — anúncio, resenha, *abstract* acadêmico, notícia, post de blog de segurança. Reconhecer o gênero ajuda a prever o propósito (um *abstract*

acadêmico tem propósito de relatar achado de pesquisa; um anúncio tem propósito de vender).

### Onde procurar a ideia principal

Em textos curtos de prova, a ideia principal costuma estar **no primeiro parágrafo** (a tese/tópico) e ser **retomada** no último. Se a pergunta pede "o principal ponto do texto", desconfie de alternativas que citam um **detalhe** de um único parágrafo do meio — detalhe real não é a mesma coisa que ideia central, mesmo estando literalmente no texto.

## 16.7 Questões de julgamento (afirmativas verdadeiro/falso)

### Cada afirmativa é um teste independente contra o texto

O formato "julgue as afirmativas I, II, III e marque a alternativa que indica as corretas" exige comparar **cada** afirmativa, **isoladamente**, contra o que o texto **efetivamente** diz — o mesmo princípio da distinção entre informação explícita e extrapolação do capítulo de Português (Capítulo 15). Uma afirmativa é falsa se: (1) contradiz o texto diretamente; (2) inverte um detalhe (troca um número, um nome, uma condição); ou (3) acrescenta uma informação que o texto **não** sustenta, mesmo que pareça plausível no mundo real.

### Os três jeitos de uma afirmativa estar errada

Texto-base: *"The report, based on a survey of 500 developers, found that AI coding assistants reduced debugging time by 20% on average."*

- Contradição direta: *"The report found that AI assistants increased debugging time."* — inverte o resultado.
- Detalhe trocado: *"The survey included 5,000 developers."* — trocou o número (500 por 5.000).
- Extrapolação: *"The report proves that all developers should adopt AI assistants immediately."* — o texto relata um achado estatístico médio, não uma recomendação universal e imediata; "proves" e "should... immediately" vão além do que o texto sustenta.

### Resolva uma afirmativa de cada vez

Não tente julgar as três afirmativas simultaneamente nem "pelo padrão da resposta". Releia o texto **uma vez para cada afirmativa**, marque verdadeiro/falso individualmente, e só então combine com as alternativas de resposta (que geralmente cobrem combinações como "I e III" ou "apenas II"). Julgar em bloco é o que leva a arrastar um erro de uma afirmativa para a interpretação das outras.

## 16.8 Roteiro de revisão do capítulo

### Sequência para qualquer questão de interpretação em inglês

1. *Skimming* no texto inteiro: gênero, ideia geral, propósito.
2. Leia a pergunta antes de caçar a resposta no texto.
3. Pergunta de **pronome**? Volte à frase (ou à anterior) e teste a substituição — não confie no candidato mais próximo.
4. Pergunta de **conectivo**? Releia a oração antes e depois dele; identifique a relação lógica real (causa, contraste, adição, condição).
5. Pergunta de **vocabulário**? Olhe a função sintática da palavra na frase (verbo ou substantivo) antes de decidir o sentido.
6. Pergunta de **compreensão global**? Desconfie de alternativas que citam só um detalhe isolado, ou que acrescentam informação não sustentada pelo texto (extrapolação).
7. Afirmativas I/II/III? Julgue uma de cada vez, contra o texto, antes de olhar as combinações de resposta.

## Capítulo 17

# Raciocínio Lógico-Matemático

Este capítulo cobre as duas frentes do RLM — matemática aplicada e lógica formal — com o raciocínio por trás de cada fórmula, não só a fórmula pronta. Decorar "a fórmula de juro composto é  $M = C(1 + i)^n$ " sem entender *por que* ela tem essa cara falha assim que o enunciado troca o contexto; entender o mecanismo generaliza para qualquer variação que a prova inventar.

## 17.1 Porcentagem e taxas sucessivas

### Por que aumentos sucessivos se multiplicam, não se somam

Aplicar um aumento de 30% sobre um valor  $V$  resulta em  $V \times 1,30$  — o **fator multiplicativo** 1,30 já representa "o valor original mais 30% dele". Se depois aplicarmos outro aumento de 10%, esse segundo aumento incide sobre o valor **já aumentado**, não sobre o original:  $V \times 1,30 \times 1,10$ . Por isso os fatores se **multiplicam**, e o aumento total **não** é a soma ingênua ( $30\% + 10\% = 40\%$ ) — o fator composto é  $1,30 \times 1,10 = 1,43$ , ou seja, aumento real de **43%**. A diferença entre 40% e 43% é exatamente o "juro sobre o aumento": os 10% do segundo aumento incidiram também sobre os 30% do primeiro.

### Taxa média de dois aumentos sucessivos

Um produto sofre dois reajustes de 20% cada, em meses seguidos. Qual o aumento acumulado? Fator composto:  $1,20 \times 1,20 = 1,44$  — aumento acumulado de **44%** (não 40%). Se a pergunta fosse "qual a taxa **mensal única** que, aplicada duas vezes, produz o mesmo efeito", a resposta é a raiz quadrada do fator composto:  $\sqrt{1,44} = 1,2$ , ou seja, a própria taxa de 20% (porque os dois aumentos já eram iguais). Quando os dois aumentos são diferentes, a taxa média não é a média aritmética dos dois percentuais — é a raiz do produto dos fatores.

### Desconto funciona pelo mesmo princípio, com fator $< 1$

Um desconto de  $x\%$  corresponde ao fator  $(1 - x/100)$ . Desconto sucessivo de 10% e depois mais 10% **não** é 20% de desconto total: o fator é  $0,90 \times 0,90 = 0,81$ , ou seja, desconto real de **19%** — porque o segundo desconto incide sobre um valor já reduzido, que é menor que o original.

## 17.2 Razão, proporção e regra de três

### Direta x inversa — o sinal da relação entre as grandezas

Duas grandezas são **diretamente proporcionais** quando **crescem juntas** (mais trabalhadores, mais produção, no mesmo ritmo) e **inversamente proporcionais** quando uma **cresce enquanto a outra diminui** (mais trabalhadores, menos tempo para terminar a mesma obra). Antes de montar a regra de três, pergunte: se eu aumentar uma grandeza, a outra também aumenta (direta) ou diminui (inversa)? Errar essa pergunta inicial inverte a fração inteira e

produz um resultado absurdo (geralmente maior que o total, ou menor que zero).

### Divisão proporcional

Repartir uma quantia **proporcionalmente** a valores dados (ex.: prejuízo repartido conforme o capital investido por cada sócio) significa dividir o total **na mesma razão** dos valores de referência — não em partes iguais. Se os capitais estão na razão 3:2, o total se reparte em 3 partes para um e 2 partes para o outro, do total de 5 partes.

## 17.3 Análise combinatória

### Arranjo x combinação — a pergunta que decide qual usar

As duas contam **quantas seleções** de  $p$  elementos são possíveis a partir de um conjunto de  $n$ , mas diferem em uma pergunta central: **a ordem importa?**

- **Arranjo**  $A(n, p) = \frac{n!}{(n-p)!}$ : a ordem **importa** — "primeiro e segundo colocados" é diferente de "segundo e primeiro".
- **Combinação**  $C(n, p) = \frac{n!}{p!(n-p)!}$ : a ordem **não importa** — escolher um grupo de 3 pessoas para uma comissão não distingue quem foi "escolhido primeiro".

A combinação é o arranjo **dividido** por  $p!$  (o número de formas de reordenar os  $p$  elementos escolhidos) — porque cada grupo de  $p$  elementos, no arranjo, é contado uma vez para **cada ordem** possível dele, e a combinação quer contá-lo **uma vez só**.

### Identificando arranjo x combinação num enunciado

"De um grupo de 10 pessoas, quantas comissões de 3 pessoas podem ser formadas?— ordem não importa (a comissão A, B, C é a mesma comissão não importa a ordem de escolha) → **combinação**:  $C(10, 3) = \frac{10!}{3! \cdot 7!} = 120$ .

"De um grupo de 10 pessoas, de quantas formas posso eleger presidente, vice e secretário?— ordem importa (ser presidente é diferente de ser secretário, mesmo sendo a mesma pessoa escolhida) → **arranjo**:  $A(10, 3) = \frac{10!}{7!} = 720$ .

### Grafo completo e o atalho $C(n, 2)$

Quando o problema pede "quantas conexões/apertos de mão/estradas entre  $n$  pontos, ligando todos os pares", é uma combinação de  $n$  elementos tomados 2 a 2 (cada par se conecta uma única vez, sem ordem) —  $C(n, 2) = \frac{n(n-1)}{2}$ . É a fórmula do número de arestas de um grafo completo de  $n$  vértices, e aparece disfarçada em problemas de "cada pessoa cumprimenta todas as outras uma vez".

## 17.4 Probabilidade

### Casos favoráveis sobre casos possíveis — o total, não o outro grupo

$P(\text{evento}) = \frac{\text{casos favoráveis}}{\text{casos possíveis}}$ , onde **casos possíveis** é sempre o **total** de resultados igualmente prováveis — não o tamanho de outro subgrupo.

### O erro de usar o grupo errado no denominador

Uma urna tem 5 bolas azuis e 3 verdes (total 8).  $P(\text{azul}) = \frac{5}{8}$  — casos favoráveis (5 azuis) sobre o **total** (8 bolas). O erro comum é escrever  $\frac{5}{3}$  (favoráveis sobre o **outro** grupo, o que nem seria uma probabilidade válida, por passar de 1) ou invertê-la para  $\frac{3}{5}$  (trocando favorável por desfavorável).

## 17.5 Médias, mediana, moda e desvio padrão

### Três medidas de posição — cada uma resume o conjunto de um jeito

- **Média:** soma de todos os valores dividida pela quantidade. Sensível a valores extremos (um único valor muito alto/baixo puxa a média).
- **Mediana:** o valor **central** do conjunto **depois de ordenado** (com quantidade par de valores, a média dos dois centrais). **Não** é sensível a valores extremos — por isso é preferida quando o conjunto tem *outliers*.
- **Moda:** o valor que **mais se repete**. Pode não existir (todos os valores distintos) ou haver mais de uma (bimodal, multimodal).

### Esquecer de ordenar antes de tirar a mediana

A mediana só funciona no conjunto **ordenado**. Pegar "o valor do meio" na ordem em que os dados foram apresentados no enunciado — sem antes reordená-los do menor para o maior — é o passo que a banca conta que o candidato pule, e produz um valor errado que geralmente ainda parece plausível.

### Média ponderada

Média ponderada =  $\frac{\sum(\text{valor} \times \text{peso})}{\sum \text{pesos}}$  — cada valor contribui proporcionalmente ao seu **peso**, não igualmente. Consequência que a prova explora: colocar a **menor** nota no **maior** peso derruba a média final mais do que colocar a mesma nota baixa num peso pequeno — então uma afirmativa do tipo "a média **necessariamente** fica acima de X" pode ser falsa dependendo de **qual** nota recebeu o maior peso, não só de quais notas existem.

### Desvio padrão mede dispersão, não posição

O desvio padrão descreve o quanto os valores se **espalham** ao redor da média — não onde a média está. Duas séries podem ter a **mesma média** e desvios padrão bem diferentes: a de **menor** desvio é mais **regular/concentrada** em torno da média; a de **maior** desvio tem valores mais espalhados, para cima e para baixo.

**Desvio padrão não é amplitude, e "menor desvio" não implica "menor média"**

Dois erros recorrentes: tratar o desvio padrão como se fosse a **amplitude** (a diferença entre o maior e o menor valor — são conceitos diferentes, embora relacionados) e concluir que "desvio padrão menor" significa "média menor" (a média pode ser **idêntica**; o que muda é só o quanto os valores variam ao redor dela).

## 17.6 Juros simples e compostos

**Sobre o quê o juro incide, período após período**

**Juros simples:** o juro de cada período incide **sempre sobre o capital inicial**  $C$  —  $M = C(1 + i \cdot n)$ , crescimento **linear**. **Juros compostos:** o juro de cada período incide **sobre o montante do período anterior** (juro sobre juro) —  $M = C(1 + i)^n$ , crescimento **exponencial**. A diferença entre os dois regimes só aparece a partir do **segundo** período — no primeiro período, simples e composto rendem exatamente o mesmo.

**Onde exatamente nasce a diferença**

R\$ 10.000 a 10% ao ano, por 2 anos.

**Simple:**  $M = 10.000 \times (1 + 0,10 \times 2) = 10.000 \times 1,20 = \text{R\$ } 12.000$  — o juro do 2º ano (1.000) incidiu sobre os mesmos R\$ 10.000 do 1º ano.

**Composto:**  $M = 10.000 \times 1,10^2 = 10.000 \times 1,21 = \text{R\$ } 12.100$  — o juro do 2º ano incidiu sobre R\$ 11.000 (o montante já com o juro do 1º ano), dando R\$ 1.100.

A diferença de R\$ 100 é exatamente "o juro sobre o juro do primeiro ano" (10% de 1.000) — a assinatura do regime composto.

## 17.7 Progressões aritmética e geométrica

**PA: soma um valor fixo; PG: multiplica por um fator fixo**

**Progressão Aritmética (PA):** cada termo é o anterior **mais** uma razão fixa  $r$ . Termo geral:  $a_n = a_1 + (n - 1)r$ . Soma dos  $n$  primeiros termos:  $S_n = \frac{(a_1 + a_n)n}{2}$  (a média entre primeiro e último termo, vezes a quantidade de termos). **Progressão Geométrica (PG):** cada termo é o anterior **multiplicado** por uma razão fixa  $q$ . Termo geral:  $a_n = a_1 \cdot q^{n-1}$ .

**O expoente/multiplicador é  $n - 1$ , não  $n$** 

O termo geral conta quantas vezes a razão foi aplicada **a partir do primeiro termo** — o próprio  $a_1$  já é o "termo zero de aplicações da razão", então o  $n$ -ésimo termo aplicou a razão  $n - 1$  vezes, não  $n$ . Exemplo: com  $a_1 = 7$  e  $r = 4$ , o 10º termo é  $a_{10} = 7 + 9 \times 4 = 43$  — usar  $n = 10$  no lugar de  $n - 1 = 9$  daria erradamente  $7 + 10 \times 4 = 47$ .

## 17.8 Matrizes

### Multiplicação de matrizes: a regra das dimensões

O produto  $A \cdot B$  só existe quando o número de **colunas de  $A$**  é igual ao número de **linhas de  $B$**  — é essa igualdade que permite "casar" cada linha de  $A$  com cada coluna de  $B$  elemento a elemento na soma de produtos. A matriz resultante herda as **linhas de  $A$**  e as **colunas de  $B$** :  $(m \times n) \cdot (n \times p) = (m \times p)$ .

### O produto de matrizes não é comutativo

$A \cdot B$  geralmente é **diferente** de  $B \cdot A$  — e às vezes um dos dois produtos nem existe (se as dimensões não combinarem na ordem trocada). Nunca assuma que inverter a ordem dá o mesmo resultado, ao contrário da multiplicação de números.

## 17.9 Conjuntos: inclusão-exclusão

### Por que se subtrai a interseção

$|A \cup B| = |A| + |B| - |A \cap B|$ . Somar  $|A|$  com  $|B|$  **conta duas vezes** quem está nos dois conjuntos ao mesmo tempo — subtrair a interseção corrige essa dupla contagem, deixando cada elemento contado exatamente uma vez.

### Aplicando com "quem está fora dos dois"

De 60 candidatos, 35 sabem Java, 28 sabem Python, e 12 sabem os dois. Quantos sabem **pelo menos um** dos dois?  $|Java \cup Python| = 35 + 28 - 12 = 51$ . Quantos **não sabem nenhum** dos dois?  $60 - 51 = 9$  — o total menos a união, nunca o total menos a soma bruta (que esqueceria de somar de volta a interseção contada a menos).

## 17.10 Geometria plana essencial

### O que a prova cobra: área, perímetro e Pitágoras

**Perímetro:** soma de todos os lados da figura. **Área:** varia por figura (retângulo = base  $\times$  altura; triângulo = base  $\times$  altura / 2). **Teorema de Pitágoras**, para triângulo retângulo:  $a^2 = b^2 + c^2$ , onde  $a$  é a **hipotenusa** (lado oposto ao ângulo reto, sempre o maior) e  $b, c$  são os catetos. O erro mais comum é aplicar a fórmula com um cateto no lugar da hipotenusa.

## 17.11 Lógica sentencial: proposições e tabelas-verdade

### Proposição: uma sentença que é verdadeira ou falsa, nunca as duas

Uma **proposição** é uma sentença declarativa com valor lógico definido — verdadeira (V) ou falsa (F), nunca ambígua, nunca as duas ao mesmo tempo. "Está chovendo" é proposição; "Feche a porta!" (imperativo) e "Que dia é hoje?" (pergunta) não são — não têm valor V/F.

## Os cinco conectivos e quando cada um dá falso

Memorizar **quando cada conectivo é falso** (em vez de quando é verdadeiro) costuma ser mais rápido, porque a maioria tem só **um** padrão de falsidade:

| Conectivo                         | Símbolo               | É falso quando                                                                                |
|-----------------------------------|-----------------------|-----------------------------------------------------------------------------------------------|
| Conjunção ("e")                   | $p \wedge q$          | <b>ao menos um</b> dos dois é falso — precisa dos <b>dois</b> verdadeiros para ser verdadeira |
| Disjunção ("ou")                  | $p \vee q$            | <b>ambos</b> são falsos — basta <b>um</b> verdadeiro para ser verdadeira                      |
| Condicional ("se...então")        | $p \rightarrow q$     | <b>só</b> no caso V→F — o antecedente verdadeiro "prometeu"o consequente e ele não veio       |
| Bicondicional ("se e somente se") | $p \leftrightarrow q$ | os dois têm valores <b>diferentes</b>                                                         |
| Disjunção exclusiva ("ou...ou")   | $p \oplus q$          | os dois têm valores <b>iguais</b>                                                             |

O condicional só é falso em um único caso —  $V \rightarrow F$ 

É a linha da tabela-verdade que mais gera confusão: "se  $p$  então  $q$ " é **verdadeira** sempre que  $p$  é falso, **não importa** o valor de  $q$  (uma promessa condicionada a algo que nunca aconteceu não pode ser quebrada). "Se chover, levo guarda-chuva" continua **verdadeira** mesmo num dia sem chuva, **mesmo que** eu não leve guarda-chuva — porque a condição ( $p$ =chover) nunca se realizou.

## 17.12 Equivalências lógicas

## Contrapositiva — a única reformulação que preserva o valor lógico

$p \rightarrow q \equiv \neg q \rightarrow \neg p$  (**contrapositiva**): logicamente **idêntica** à condicional original — ambas verdadeiras ou ambas falsas nas mesmas circunstâncias. "Se é fã, então assiste ao show"  $\equiv$  "se **não** assiste, então **não** é fã".

A **recíproca** ( $q \rightarrow p$ ) e a **inversa** ( $\neg p \rightarrow \neg q$ ) **não** são equivalentes à condicional original — podem ter valor lógico diferente. "Se é fã, então assiste" não garante "se assiste, então é fã" (a recíproca): alguém pode assistir sem ser fã.

## Condicional como disjunção, e sua negação

$p \rightarrow q \equiv \neg p \vee q$  — outra forma de escrever a mesma condicional. Daí decorre a **negação da condicional**:  $\neg(p \rightarrow q) \equiv p \wedge \neg q$  — negar "se  $p$  então  $q$ " não é "se  $p$  então não  $q$ " (erro comum), é afirmar que  $p$  **aconteceu e**  $q$  **não** (a única situação que torna o condicional original falso).

**Leis de De Morgan**

$\neg(p \wedge q) \equiv \neg p \vee \neg q$ ;  $\neg(p \vee q) \equiv \neg p \wedge \neg q$ .

O mecanismo: negar uma conjunção ("os dois acontecem") é dizer que **pelo menos um não acontece** (vira "ou"); negar uma disjunção ("pelo menos um acontece") é dizer que **nenhum** acontece, ou seja, **os dois** deixam de acontecer (vira "e"). O conectivo sempre **troca de tipo** ao atravessar a negação.

**Aplicando De Morgan**

"Não é verdade que o sistema está online e seguro"  $\equiv$  "o sistema não está online **ou** não está seguro" (não necessariamente os dois ao mesmo tempo — basta um dos dois falhar para a afirmação original ser falsa).

**17.13 Argumentação: validade e falácias****Um argumento é válido pela forma, não pelo conteúdo ser "verdadeiro no mundo real"**

Um argumento é **válido** quando, **se** todas as premissas forem verdadeiras, a conclusão **necessariamente** também é — é uma propriedade da **estrutura** lógica, independente de as premissas serem factualmente verdadeiras ou não. A prova de RLM testa a forma, não o conteúdo.

**As formas válidas e as duas falácias mais cobradas**

- **Modus ponens** (válido):  $p \rightarrow q, p \vdash q$ . Se "se  $p$  então  $q$ " e " $p$  é verdade", conclui-se  $q$ .
- **Modus tollens** (válido):  $p \rightarrow q, \neg q \vdash \neg p$ . Se "se  $p$  então  $q$ " e " $q$  é falso", conclui-se que  $p$  também é falso — é exatamente a lógica da contrapositiva aplicada.
- **Silogismo hipotético** (válido):  $p \rightarrow q, q \rightarrow r \vdash p \rightarrow r$  — encadeia duas condicionais.
- **Afirmar o consequente** (falácia, inválido):  $p \rightarrow q, q \vdash p$ . Saber que  $q$  é verdade **não** garante que  $p$  causou isso — pode haver outro caminho para  $q$  ser verdadeiro.
- **Negar o antecedente** (falácia, inválido):  $p \rightarrow q, \neg p \vdash \neg q$ .  $p$  ser falso não impede  $q$  de ser verdadeiro por outro motivo.

**Por que "afirmar o consequente" falha**

"Se chove, a rua fica molhada. A rua está molhada. Logo, choveu." — **inválido**: a rua pode estar molhada porque alguém lavou a calçada, sem ter chovido. A condicional original só garante o caminho  $p \rightarrow q$ ; não proíbe  $q$  de acontecer por outras causas.

**As duas falácias parecem modus ponens/tollens de relance**

"Afirmar o consequente" parece com modus ponens porque também parte de uma premissa verdadeira reconhecida no condicional — a diferença é **qual** lado da condicional foi confirmado: modus ponens confirma o **antecedente** ( $p$ ); a falácia confirma o **consequente** ( $q$ ). "Negar o antecedente" parece com modus tollens pelo mesmo motivo invertido: modus tollens nega o **consequente** ( $\neg q$ ); a falácia nega o **antecedente** ( $\neg p$ ). Sempre confira **que lado** da condicional a segunda premissa afirma ou nega antes de validar o argumento.

## 17.14 Lógica de primeira ordem: quantificadores

### Todo, algum, nenhum — e como negar cada um

| Afirmação    | Significado                                       | Negação correta                                                                      |
|--------------|---------------------------------------------------|--------------------------------------------------------------------------------------|
| Todo A é B   | <b>cada</b> elemento de A também é B, sem exceção | <b>Algum</b> A <b>não</b> é B — basta <b>um</b> contraexemplo para derrubar o "todo" |
| Algum A é B  | <b>existe pelo menos um</b> elemento de A que é B | <b>Nenhum</b> A é B — nem um único caso                                              |
| Nenhum A é B | <b>não existe</b> elemento de A que seja B        | <b>Algum</b> A é B                                                                   |

O mecanismo por trás: negar "todo" não vira "nenhum" — vira "existe pelo menos uma exceção" (algum não). Para derrubar "todo A é B" basta **um único** A que não seja B; não é preciso que **nenhum** A seja B (isso seria uma afirmação bem mais forte que a simples negação exige).

### O erro mais comum: negar "todo" como "nenhum"

"Todo desenvolvedor testa o próprio código" — a negação **não** é "nenhum desenvolvedor testa o próprio código" (isso seria uma afirmação adicional muito mais forte, não a simples negação lógica). A negação correta é "**algum** desenvolvedor **não** testa o próprio código" — basta um único caso contrário para a afirmação original cair.

### Diagramas lógicos: testar o que é necessário, não o que é possível

Diagramas de Euler/Venn resolvem problemas de quantificador desenhando os conjuntos e testando o que **necessariamente** decorre das premissas — não o que **poderia** ser verdade em algum cenário imaginável. Se existe uma forma de desenhar os círculos que satisfaz as premissas mas contradiz a suposta conclusão, a conclusão **não** é válida — mesmo que exista **outra** forma de desenhar em que ela seria verdadeira. Validade exige que a conclusão valha em **todo** desenho possível compatível com as premissas, não só em algum.

## 17.15 Roteiro de revisão do capítulo

### Fórmulas e checagens para ter na ponta da caneta

**Matemática:** taxa acumulada = produto dos fatores (nunca soma);  $C(n, 2) = n(n-1)/2$ ; média ponderada =  $\Sigma(\text{valor} \times \text{peso}) / \Sigma \text{pesos}$ ; PA  $a_n = a_1 + (n-1)r$ ; juros compostos  $M = C(1+i)^n$  contra simples  $M = C(1+i \cdot n)$ ;  $|A \cup B| = |A| + |B| - |A \cap B|$ ; produto de matrizes  $(m \times n) \cdot (n \times p) = (m \times p)$ ; ordene a série antes de tirar a mediana.

**Lógica:** condicional só é falsa em  $V \rightarrow F$ ; contrapositiva equivale, recíproca **não**; De Morgan sempre troca "e" por "ou" (e vice-versa) ao negar; modus ponens/tollens são válidos, afirmar o conseqüente/negar o antecedente são falácias; negar "todo" dá "algum não" (nunca "nenhum").

# Atualidades e Inteligência Artificial

Este capítulo cobre as duas partes do bloco de Atualidades: o viés socioambiental clássico de prova de conhecimentos gerais, e o bloco novo de Inteligência Artificial (que entrou no edital da Dataprev só em 2026). Como é o capítulo mais volátil do livro — números, status de projeto de lei e métricas de mercado mudam — o objetivo aqui é fixar o **mecanismo** de cada conceito (por que ESG mede o que mede, por que viés e variância se comportam de forma oposta, por que um LLM alucina) para que o entendimento sobreviva mesmo quando o número específico for atualizado antes da prova.

## 18.1 Atualidades: o viés socioambiental

### ESG — avaliando o que o balanço financeiro não mostra

**ESG** (*Environmental, Social and Governance* — **A**mbiental, **S**ocial e **G**overnança) é o conjunto de critérios usado para avaliar o desempenho **não financeiro** de uma organização — o que o balanço contábil tradicional não captura, mas que afeta o risco e a reputação de longo prazo do negócio:

- **Ambiental (E)**: emissões de carbono, consumo de energia e água, gestão de resíduos, biodiversidade, exposição a mudanças climáticas.
- **Social (S)**: condições de trabalho, diversidade e inclusão, direitos humanos, relação com comunidades afetadas, práticas na cadeia de fornecedores.
- **Governança (G)**: composição e independência do conselho de administração, ética corporativa, transparência, combate à corrupção, prestação de contas aos acionistas e à sociedade.

O motivo de existir como critério **formal** de investimento e relatório corporativo: investidores e reguladores perceberam que riscos não financeiros (um desastre ambiental, um escândalo de corrupção) também destroem valor financeiro — então passaram a exigir que esse risco seja **medido e divulgado**, não só descoberto depois que já aconteceu.

### O "G" não é "Gestão", "Global" nem "Growth"

"Governança" é especificamente sobre **como a organização é dirigida e controlada** (estrutura de poder, prestação de contas) — não é sinônimo genérico de "boa gestão" nem está relacionado a expansão ("Growth") ou alcance internacional ("Global"). Da mesma forma, o "S" é **Social**, não "Sustentável" — sustentabilidade é o conceito guarda-chuva que o ESG inteiro serve, não uma das três letras especificamente.

### Como filtrar item de atualidades socioambientais

Esse tipo de questão costuma vir ancorada num **texto-base**, no formato de julgar afirmativas (I, II, III). Dois filtros rápidos, antes de reler o texto com calma: (1) item com termo **absoluto** ("invariavelmente", "sempre", "nunca", "exclusivamente") é quase sempre **falso** — a realidade socioambiental raramente é tão categórica; (2) item com dado **numérico** precisa ser **conferido**

**contra o texto-base**, nunca aceito de memória — é comum a alternativa inflar ou distorcer um número que o texto deu de forma mais modesta.

## 18.2 Fundamentos de Inteligência Artificial

### 18.2.1 IA, Machine Learning e Deep Learning: uma relação de contêiner

#### Três círculos concêntricos, não três coisas separadas

- **Inteligência Artificial (IA)**: o campo mais amplo — qualquer sistema capaz de executar tarefas que, se feitas por um humano, exigiriam inteligência (raciocinar, perceber, decidir).
- **Machine Learning (ML)**: um **subcampo** da IA em que o sistema **aprende padrões a partir de dados** e melhora seu desempenho com a experiência, em vez de seguir regras explicitamente programadas por um humano para cada situação.
- **Deep Learning**: um **subcampo do ML** que usa **redes neurais com muitas camadas** (daí "profundo", *deep*) para aprender representações cada vez mais abstratas dos dados, camada após camada.

A relação é de **contenção**:  $IA \supset ML \supset \text{Deep Learning}$  — todo Deep Learning é ML, todo ML é IA, mas a recíproca não vale (existe IA que não usa aprendizado de máquina, como sistemas baseados puramente em regras lógicas escritas à mão).

### 18.2.2 Tipos de aprendizado

#### O que diferencia os três — o que o sistema recebe como sinal de aprendizado

| Tipo               | Dados de treino                             | Objetivo                                   | Exemplos                                     |
|--------------------|---------------------------------------------|--------------------------------------------|----------------------------------------------|
| Supervisionado     | <b>rotulados</b> (resposta certa conhecida) | prever o rótulo de casos novos             | classificação, regressão                     |
| Não supervisionado | <b>sem rótulo</b>                           | achar estrutura escondida nos dados        | <i>clustering</i> (K-Means), associação, PCA |
| Por reforço        | recompensa/punição a cada ação              | aprender uma <b>política</b> de ação ótima | agentes, jogos, robótica                     |

O critério que separa os três é **que tipo de sinal** orienta o aprendizado: no supervisionado, o sinal é "aqui está a resposta certa para este exemplo"; no não supervisionado, não há resposta certa dada, só os dados brutos; no aprendizado por reforço, o sinal é indireto — "essa sequência de ações rendeu recompensa X", sem dizer qual ação específica foi a certa.

#### "Aprende com os dados e melhora ao longo do tempo" não é lógica booleana

Um enunciado que descreve um sistema "que aprende com a experiência e melhora seu desempenho ao longo do tempo" está descrevendo **redes neurais/aprendizado de máquina** — não

lógica booleana clássica nem programação linear determinística, que executam sempre a mesma regra fixa e não mudam de comportamento sozinhas com o uso.

## 18.3 Viés e variância: o trade-off central do aprendizado supervisionado

### Dois jeitos opostos de um modelo errar

Todo modelo supervisionado comete um dos dois tipos de erro sistemático (ou uma combinação dos dois), e eles têm causas **opostas**:

|                  | Alto viés ( <i>bias</i> )                                                  | Alta variância ( <i>variance</i> )                                                    |
|------------------|----------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Causa            | modelo <b>simples demais</b> para captar o padrão real dos dados           | modelo <b>complexo demais</b> , ajusta-se a ruído específico do treino                |
| Sintoma          | erra <b>tanto no treino quanto no teste</b> — nem decorou o treino direito | acerta muito bem o treino, erra feio em dados novos — "decorou" em vez de generalizar |
| Nome do fenômeno | <b>underfitting</b>                                                        | <b>overfitting</b>                                                                    |
| Como combater    | modelo mais expressivo, mais atributos relevantes                          | regularização (L1/L2), validação cruzada, mais dados de treino                        |

O motivo de ser um **trade-off** (e não duas falhas independentes que dá para zerar ao mesmo tempo): reduzir a complexidade do modelo para combater a variância tende a aumentar o viés (o modelo fica simples demais de novo); aumentar a complexidade para combater o viés tende a aumentar a variância (o modelo passa a se ajustar a detalhes específicos do treino). O objetivo prático não é zerar os dois, é achar o ponto de **equilíbrio** que minimiza o erro total em dados novos.

### Não inverta viés com variância

"Viés alto" **não** é o modelo que "decorou" o treino — isso é **variância alta** (overfitting). O jeito de fixar sem trocar: viés alto = modelo **"burro"** (simples demais até para o próprio treino); variância alta = modelo que **"decorou"** (bom no treino, ruim fora dele). E regularização combate especificamente a **variância/ overfitting**, não o viés — regularizar um modelo que já sofre de viés alto (underfitting) só pioraria a situação, tornando-o ainda mais restrito.

## 18.4 Métricas de avaliação

### Classificação: quatro métricas a partir da matriz de confusão

- **Acurácia** = acertos totais / total de casos. Parece a métrica mais natural, mas é enganosa em bases desbalanceadas.
- **Precisão**: dos casos que o modelo **apontou como positivos**, quantos realmente eram — importa quando o custo de um **falso positivo** é alto (ex.: barrar um e-mail legítimo como se fosse spam).
- **Recall** (revocação/sensibilidade): dos casos que **realmente eram positivos**, quantos o modelo conseguiu capturar — importa quando o custo de um **falso negativo** é alto (ex.: deixar passar uma fraude).
- **F1-score**: a **média harmônica** entre precisão e recall — harmônica, e não aritmética, justamente porque a média harmônica **pune mais** o desequilíbrio entre as duas (um modelo com precisão 100% e recall 1% teria média aritmética de 50,5%, uma nota enganosamente alta; a harmônica cairia para perto de 2%, refletindo melhor que o modelo é inútil na prática).

### Por que acurácia engana em base desbalanceada

Um conjunto de e-mails tem 99% legítimos e 1% spam. Um modelo "burro" que **sempre** responde "legítimo", sem olhar o conteúdo, acerta **99% de acurácia** — um número que parece excelente — mas detecta **zero** spam (recall igual a zero para a classe que realmente importa detectar). É exatamente o cenário que a prova costuma montar (spam, fraude, doença rara) para testar se o candidato confia demais na acurácia isolada.

### Regressão: MAE, RMSE e $R^2$

Para prever um **número** (não uma classe), as métricas de classificação (acurácia, precisão, F1) não se aplicam — usam-se **MAE** (erro médio absoluto, na mesma unidade do valor previsto), **RMSE** (raiz do erro quadrático médio — penaliza mais erros grandes, por elevar ao quadrado antes de tirar a média) e  **$R^2$**  (quanto da variação dos dados o modelo consegue explicar, de 0 a 1). Usar uma métrica de classificação num problema de regressão (ou vice-versa) é um distrator comum — confirme sempre se a saída do modelo é uma **categoria** ou um **número** antes de escolher a métrica.

## 18.5 Modelos generativos e LLMs

### Modelo generativo: aprende a distribuição para criar exemplos novos

Um **modelo generativo** aprende a **distribuição estatística** dos dados de treino e usa esse aprendizado para **gerar exemplos novos**, plausíveis dentro daquela distribuição — texto, imagem, áudio — em vez de apenas classificar ou prever um rótulo sobre um exemplo já existente (o que seria um modelo **discriminativo**).

### LLM: linguagem em escala, via Transformer e atenção

Um **LLM** (*Large Language Model*) é um modelo de linguagem treinado sobre um volume massivo de texto, tipicamente com a arquitetura **Transformer** e seu **mecanismo de atenção** — que permite ao modelo, ao processar cada palavra, "prestar atenção" dinamicamente em **quais**

**outras palavras do contexto** são mais relevantes para aquele ponto, em vez de processar a sequência de forma estritamente linear e igualmente ponderada. É esse mecanismo que permite ao modelo capturar dependências de longo alcance no texto (uma referência no início do parágrafo que só faz sentido lida junto com uma palavra no final).

### RAG: buscar antes de gerar

**RAG** (*Retrieval-Augmented Generation*) combina o LLM com uma etapa de **busca** (*retrieval*) numa base de conhecimento externa **antes** de gerar a resposta — em vez de responder só com o que aprendeu durante o treinamento (que tem uma data de corte e pode estar desatualizado ou não cobrir um domínio específico), o modelo recebe trechos relevantes recuperados da base e os usa como contexto adicional para fundamentar a resposta. É a técnica típica para tornar respostas mais **atuais** e **rastreáveis** a uma fonte.

### Alucinação: resposta fluente, mas não fundamentada

**Alucinação** é quando o modelo gera uma resposta **plausível e bem escrita**, mas **factualmente incorreta** ou inventada — o modelo não "sabe que não sabe": ele continua produzindo texto estatisticamente coerente mesmo quando não tem base real para a afirmação. É a razão pela qual saída de LLM em contexto que afeta decisão real (um atendimento ao cidadão, uma análise jurídica) exige **validação**, não confiança cega.

## 18.6 Ética, governança e privacidade em IA

### Viés algorítmico: o modelo herda o viés dos dados, não o inventa

**Viés algorítmico** ocorre quando os **dados de treino** refletem desigualdades ou padrões históricos discriminatórios, e o modelo **aprende e reproduz** esse padrão em suas decisões — o algoritmo não é "preconceituoso" por conta própria, ele generaliza exatamente o que os dados de treino mostraram, inclusive quando esses dados carregam discriminação histórica.

### Não usar o atributo sensível diretamente não elimina o viés

Um modelo que não usa "raça" ou "gênero" como atributo de entrada **não** está automaticamente livre de viés — outras variáveis (CEP, escola frequentada, histórico de consumo) podem funcionar como **proxy**, recriando indiretamente a mesma discriminação através de uma correlação estatística. A alternativa que descreve isso como "o modelo é neutro porque não usa o atributo sensível" é o distrator clássico de "neutralidade tecnológica" — neutralidade de dado não é garantida só por omitir uma coluna.

### Explicabilidade x acurácia: são eixos independentes

**Explicabilidade** é a capacidade de o modelo (ou de quem o opera) **justificar** uma decisão de forma compreensível a humanos. **Acurácia** é o quanto o modelo acerta. Os dois não andam necessariamente juntos: um modelo pode ser **muito acurado e pouco explicável** (as redes neurais profundas são o exemplo clássico de "caixa-preta" — acertam muito, mas é difícil apontar exatamente *por quê* decidiram assim). Essa tensão é o que justifica exigir explicabilidade como requisito **separado** de desempenho em contextos de decisão sensível (crédito, benefício social, Justiça).

**LGPD, art. 20 — direito à revisão de decisão automatizada**

A LGPD garante ao titular de dados o direito de **solicitar revisão** de decisões tomadas **unicamente** com base em tratamento automatizado que afetem seus interesses — incluindo decisões que definam perfil pessoal, profissional, de consumo ou de crédito. É a base legal que ancora cenários de prova envolvendo score de crédito, triagem de currículo automatizada ou concessão de benefício público por algoritmo: sistema que decide **sozinho**, sem qualquer possibilidade de revisão humana, contraria essa garantia.

**18.7 Regulação da IA: em vigor x ainda em tramitação****A distinção de estado que a prova mais explora**

Duas normas de referência sobre IA existem hoje em estágios **completamente diferentes**, e confundir os dois estágios é o erro mais fácil de cometer por quem só acompanha manchete de notícia:

- **União Europeia — Regulamento (UE) 2024/1689 ("AI Act")**: já está em **vigor** (desde 01/08/2024), com aplicação **escalonada** no tempo (as proibições entraram primeiro; as obrigações de sistemas de alto risco entram depois). Por ser um **regulamento** europeu, aplica-se diretamente nos Estados-membros, sem precisar de uma lei nacional para "ativá-lo". Estrutura-se por **nível de risco**: risco **inaceitável** (práticas proibidas, como pontuação social do cidadão pelo Estado), **alto risco** (obrigações pesadas de conformidade e supervisão humana), **risco limitado** (dever de transparência — avisar que se está interagindo com uma IA) e **risco mínimo** (sem obrigação especial).
- **Brasil — PL 2338/2023**: ainda é projeto de lei — não é lei em vigor. Segue o mesmo desenho baseado em **nível de risco** do modelo europeu, prevê **direitos** das pessoas afetadas (informação prévia, explicação da decisão, contestação e revisão humana), cria um sistema nacional de governança de IA sob coordenação da ANPD, e prevê multa por infração.

O mecanismo comum aos dois textos: quanto maior o **risco** que um sistema de IA representa para direitos das pessoas, maior a **obrigação** de conformidade imposta a quem o opera — é uma regulação **proporcional ao risco**, não uma proibição genérica de IA.

**As trocas de estado que passam despercebidas**

- Dizer que o **Brasil já tem** uma lei específica de IA em vigor é falso — o que está em vigor no Brasil e toca IA é a **LGPD** (por tabela, via o art. 20 sobre decisão automatizada), não uma lei de IA dedicada.
- Dizer que o **AI Act ainda é uma proposta** em discussão é falso — é regulamento **vigente** desde agosto de 2024.
- Atribuir ao PL 2338 uma abordagem "baseada em princípios, sem classificação de risco" é falso — ele é, como o europeu, **baseado em nível de risco**.

Como este é o assunto mais volátil do livro, **confira o estágio atual do PL 2338** próximo à data da prova — se ele for sancionado como lei antes disso, deixa de ser "projeto" e a resposta correta muda de categoria.

## 18.8 Reconhecendo o padrão em cenários de IA aplicada

### Quatro padrões de cenário que a prova reaproveita

O edital pede "fundamentos e aplicações"— ou seja, a mesma pergunta técnica pode vir embrulhada num cenário concreto (um órgão público, um banco, uma prefeitura). Reconhecer o padrão por trás do cenário resolve a questão:

- **Triagem/priorização de atendimento** (fila de posto de saúde, concessão de benefício, análise de currículo): tipicamente **classificação supervisionada** (existe um rótulo conhecido: aprovado/negado, prioritário/não) — e o cenário costuma testar se há **direito a explicação/revisão humana** da decisão.
- **Detecção de fraude ou anomalia** (transação suspeita, glosa irregular): tipicamente **não supervisionado** ou semissupervisionado — porque casos rotulados de fraude são raros e o padrão fraudulento **muda com o tempo** (concept drift, ver Capítulo 14).
- **Chatbot/atendimento ao cidadão via LLM**: testa se o candidato reconhece o risco de **alucinação** e a necessidade de **validação humana** antes que a resposta do modelo produza efeito sobre um direito do cidadão.
- **Score de crédito/seguro/benefício por algoritmo**: cenário clássico de **viés algorítmico** — se o histórico de treino reflete desigualdade passada, o modelo tende a reproduzi-la, mesmo sem usar diretamente o atributo sensível (via proxy).

## 18.9 Interseção IA × ESG: energia e sustentabilidade

### Por que IA e sustentabilidade aparecem juntas em prova

O crescimento do uso de IA em escala tem um custo energético real, e é exatamente o tipo de interseção que uma prova de "Atualidades e IA" gosta de explorar — une os dois temas mais recorrentes do bloco:

- O consumo de um data center se divide, a grosso modo, entre **processamento** (a maior fatia) e **resfriamento** (refrigerar o hardware para não superaquecer é, depois do próprio processamento, o maior custo energético de operar um data center em escala).
- **PUE** (*Power Usage Effectiveness*): métrica-padrão de eficiência energética de data center — razão entre a energia total consumida e a energia usada especificamente pela TI. Quanto mais próximo de 1,0, mais eficiente (menos energia "desperdiçada" em overhead como refrigeração).
- Como parte da energia elétrica que alimenta data centers ainda vem de fontes fósseis, a expansão acelerada de IA **pressiona** as metas de descarbonização — é a mesma preocupação de mudança climática que já aparece na Parte 1 deste capítulo, agora aplicada especificamente à infraestrutura digital.

### Não caia no absoluto em nenhuma das duas direções

Dizer que a infraestrutura de IA usa "majoritariamente energia limpa" é otimismo não sustentado (a menos que o texto-base afirme isso explicitamente); dizer que IA é "incompatível com sustentabilidade" é o absoluto oposto, igualmente errado. O tratamento realista do tema é o **dilema**: benefício tecnológico e custo ambiental coexistindo, não um veredito fechado num sentido ou no outro.

## 18.10 Roteiro de revisão do capítulo

### Os pares que este capítulo mais inverte

Supervisionado (rotulado) × não supervisionado (sem rótulo); viés alto/underfitting (modelo simples demais, erra em tudo) × variância alta/overfitting (modelo decorou o treino); precisão (foco no falso positivo) × recall (foco no falso negativo); F1 é média **harmônica**, não aritmética; AI Act (**em vigor**) × PL 2338 (**ainda projeto**); explicabilidade × acurácia (eixos independentes); "não usa o atributo sensível" não é o mesmo que "livre de viés" (cuidado com proxy).

## Capítulo 19

# Legislação — Segurança da Informação e Proteção de Dados

Este capítulo explica o **porquê** por trás de cada regra das quatro leis do edital — não é decoreba de número de artigo, é entender *que problema* cada dispositivo resolve e *como as peças se encaixam* entre si (quem decide, quem executa, quem fiscaliza, o que acontece se algo dá errado). Os números e prazos específicos já foram conferidos contra a fonte primária no processo de auditoria do repositório; aqui o foco é fixar a lógica que sustenta cada um deles, para que o conhecimento resista mesmo quando um detalhe pontual for atualizado.

## 19.1 LGPD (Lei 13.709/2018)

### 19.1.1 Dado pessoal e dado pessoal sensível

#### Duas categorias, dois regimes de proteção

**Dado pessoal** (art. 5º, I) é qualquer informação relacionada a pessoa natural **identificada ou identificável** — nome, endereço, telefone, e-mail, placa de veículo, CPF. **Dado pessoal sensível** (art. 5º, II) é um **subconjunto fechado e menor**: origem racial ou étnica, convicção religiosa, opinião política, filiação sindical ou a organização de caráter religioso/filosófico/político, dado sobre **saúde** ou **vida sexual**, dado **genético** ou **biométrico**.

A razão de existir uma segunda categoria: esses dados específicos, se expostos ou usados indevidamente, geram um risco de **discriminação** muito maior do que um dado comum como um endereço — por isso a lei impõe um regime de tratamento **mais restrito** (art. 11) para eles, com menos hipóteses legais disponíveis para justificar o tratamento. Sensível não significa "proibido de tratar" — significa "tratado sob critério mais apertado".

### 19.1.2 Agentes de tratamento: o papel vem do poder de decisão

#### Controlador, operador e encarregado — o critério é a decisão, não o acesso físico

A LGPD distribui obrigações por **quem decide** sobre o tratamento de dados, não por quem fisicamente manipula o dado:

- **Controlador**: a quem competem as decisões referentes ao tratamento — define a **finalidade** (para quê) e os **meios** (como).
- **Operador**: realiza o tratamento **em nome do controlador** — executa segundo instruções, não escolhe a finalidade.
- **Encarregado (DPO)**: pessoa indicada como **canal de comunicação** entre controlador, titulares e ANPD — não decide, não executa, apenas intermedeia.

O art. 5º, IX define **agentes de tratamento** como **apenas o controlador e o operador** — o encarregado **não é agente de tratamento**. É uma consequência direta do papel de cada

um: quem **responde** perante a ANPD é quem decide ou executa; o encarregado é o telefone de contato, não o responsável pela decisão.

### O papel muda conforme o tratamento, não é um rótulo fixo

Um ministério contrata uma estatal de TI para processar sua folha de pagamento. O ministério decidiu a finalidade (processar a folha) e os meios: é o **controlador**. A estatal executa segundo essas instruções, mesmo sendo dela o banco de dados, os servidores e a equipe: é a **operadora**. Agora a mesma estatal resolve, por conta própria, usar essa base para treinar um modelo interno de detecção de fraude. Nesse tratamento **novo**, foi ela quem decidiu a finalidade — e, só para esse tratamento, ela **vira controladora**. O papel é definido **por tratamento**, não é um crachá permanente da empresa.

### 19.1.3 Bases legais: duas listas diferentes, arts. 7º e 11

#### Todo tratamento precisa de base legal escolhida antes de tratar

Consentimento é **apenas uma** das bases legais disponíveis — e a **mais frágil** de todas, porque o titular pode revogá-la a qualquer momento. A lei fixa duas listas separadas de bases legais, uma para cada categoria de dado:

- **Dado comum (art. 7º)**: dez hipóteses — consentimento; cumprimento de obrigação legal ou regulatória; execução de políticas públicas; realização de estudos por órgão de pesquisa; execução de contrato; exercício regular de direitos; proteção da vida; tutela da saúde; **legítimo interesse**; e **proteção do crédito**.
- **Dado sensível (art. 11)**: lista **própria e menor** — consentimento específico e destacado, ou, sem consentimento, sete hipóteses em que o tratamento seja **indispensável** (obrigação legal, políticas públicas, estudos por órgão de pesquisa, exercício regular de direitos, proteção da vida, tutela da saúde, e prevenção à fraude/segurança nos processos de identificação e autenticação).

O que **não** está na lista do art. 11 é justamente o ponto mais testado: **legítimo interesse** e **proteção do crédito não** autorizam o tratamento de dado sensível — são bases exclusivas do dado comum, porque a lei reservou hipóteses mais restritas para o dado de maior risco.

#### Trocar o artigo troca a lista inteira de opções disponíveis

Guardar o CPF de um usuário para executar um contrato entra pelo art. 7º, V (execução de contrato). Guardar a **digital** do mesmo usuário para liberar uma catraca é dado **biométrico** — portanto sensível — e não adianta invocar legítimo interesse (não está na lista do art. 11); a base correta é a hipótese de prevenção à fraude/segurança nos processos de identificação. Trocar de um dado comum para um sensível troca inteiramente o conjunto de justificativas legais disponíveis.

### 19.1.4 Princípios (art. 6º): finalidade, adequação e necessidade

#### Três princípios que soam sinônimos e não são

São dez princípios ao todo; três costumam ser confundidos entre si porque tratam de aspectos vizinhos do mesmo tratamento:

| Princípio          | O que exige                                                                                                   |
|--------------------|---------------------------------------------------------------------------------------------------------------|
| <b>Finalidade</b>  | propósitos <b>legítimos, específicos e explícitos, informados ao titular</b> , sem uso posterior incompatível |
| <b>Adequação</b>   | <b>compatibilidade</b> do tratamento com as finalidades informadas, <b>conforme o contexto</b>                |
| <b>Necessidade</b> | tratamento <b>limitado ao mínimo</b> indispensável para atingir a finalidade                                  |

A palavra-chave de cada um resolve a confusão: **finalidade** = "para quê, e o titular foi avisado"; **adequação** = "o que está sendo feito **combina** com o que foi avisado"; **necessidade** = "só o **mínimo**, nada além". Um tratamento pode ter finalidade bem informada, mas violar a adequação se o uso real não combinar com o que foi avisado (mesmo dado, contexto diferente do prometido); ou pode ser adequado ao contexto, mas violar a necessidade se coletar mais dado do que o estritamente exigido para aquele fim.

### 19.1.5 Decisão automatizada e explicabilidade (art. 20)

#### O direito à revisão e o dever de explicar

Quando um sistema decide **sozinho** sobre algo que afeta um titular (score de crédito, triagem de currículo, concessão de benefício), o art. 20 garante ao titular o direito de **solicitar a revisão** de decisões tomadas **unicamente** com base em tratamento automatizado que afetem seus interesses — inclusive as que definem perfil pessoal, profissional, de consumo ou de crédito. O **controlador** deve fornecer, quando solicitado, informações **claras e adequadas** sobre os critérios e procedimentos usados na decisão, **respeitados os segredos comercial e industrial**. O mecanismo por trás da ressalva de segredo industrial: a lei equilibra dois interesses — o direito do titular de entender por que foi decidido contra ele, e o interesse legítimo da empresa de não expor o funcionamento interno completo de seu sistema (o código-fonte, os pesos exatos de um modelo). O que se garante ao titular são os **fatores considerados** e o **procedimento seguido**, em linguagem compreensível — não o mecanismo técnico interno linha a linha.

#### O que "explicabilidade" concretamente garante

Um sistema recusa automaticamente um pedido de benefício. O titular pode pedir revisão e pode saber **quais critérios** pesaram na decisão — não o código-fonte nem os pesos do modelo (protegidos por segredo industrial), mas os fatores considerados e o procedimento seguido, de forma que um humano leigo consiga entender. Essa é exatamente a definição de **explicabilidade** de um sistema de IA: a propriedade de expressar os fatores importantes de uma decisão de maneira compreensível a humanos — não é o mesmo que "o modelo é simples" nem "o humano tende a confiar na decisão" (isso seria viés de automação, um fenômeno diferente).

### 19.1.6 Segurança e boas práticas (Capítulo VII): a lei que vira requisito de sistema

#### Os quatro deveres operacionais

Enquanto os capítulos anteriores da lei dizem *o que* pode ser tratado e *por quem*, o Capítulo VII diz *como* — é a parte que vira requisito técnico de projeto, não política de gaveta:

- **Segurança desde a concepção (art. 46):** os agentes devem adotar medidas técnicas e administrativas de proteção, observadas **desde a fase de concepção** do produto/serviço até sua execução — é o *privacy by design* com força de lei: segurança é requisito de projeto desde o início, não uma etapa final adicionada depois de pronto.
- **Sigilo que sobrevive ao fim (art. 47):** a obrigação de segurança alcança todos que intervêm em qualquer fase do tratamento, **mesmo depois** de ele terminar.
- **Comunicação de incidente (art. 48):** o **controlador** deve comunicar **à ANPD e ao titular** incidente que possa acarretar **risco ou dano relevante**, em prazo razoável — regulamentado hoje em **3 dias úteis** (Resolução CD/ANPD nº 15/2024), contados do conhecimento do incidente. O controlador deve ainda registrar **todos** os incidentes (inclusive os não comunicados) por, no mínimo, 5 anos.
- **Governança em privacidade (art. 50):** controladores e operadores podem formular boas práticas e programa de governança, proporcional à estrutura, escala e sensibilidade dos dados tratados.

#### Um vazamento e os três erros comuns de leitura

Um vazamento expõe e-mails e CPFs de mil titulares. Primeiro erro: achar que basta avisar a ANPD — a lei exige avisar **também** o titular. Segundo erro: achar que só se comunica vazamento de dado **sensível** — o critério é **risco ou dano relevante**, não a categoria específica do dado. Terceiro erro, o mais sutil: confundir o prazo da **lei** (que fala em "prazo razoável", sem número) com o prazo do **regulamento** da ANPD (que fixa 3 dias úteis) — dizer que "a LGPD estabelece 72 horas" confunde a lei brasileira com o regulamento europeu (GDPR) e atribui à lei um número que só está no regulamento infralegal.

### 19.1.7 Sanções administrativas (art. 52): um regime, não uma tabela de multa

#### Quem aplica, sob qual rito, com quais critérios

O art. 52 não é uma tabela de preços — é um **regime** completo: define quem aplica (a **ANPD**, sobre os **agentes de tratamento**), sob que rito (**procedimento administrativo** com ampla defesa, aplicação **gradativa, isolada ou cumulativa** — nunca sanção direta sem processo), com que critérios de dosagem (onze parâmetros: gravidade, boa-fé, vantagem auferida, condição econômica, reincidência, grau do dano, cooperação, mecanismos de mitigação, governança, medidas corretivas, proporcionalidade — **nacionalidade do infrator não é um deles**).

**A gradação:** as três sanções mais severas (suspensão parcial do banco de dados, suspensão da atividade, proibição parcial ou total) só podem ser aplicadas **depois** de já ter sido imposta, no mesmo caso, ao menos uma das sanções intermediárias (multa simples, multa diária, publicização, bloqueio ou eliminação) — a **advertência sozinha não satisfaz** esse requisito de gradação. A lógica: a lei constrói uma escada de severidade, e não permite pular direto para o topo sem passar por um degrau intermediário primeiro.

**Multa simples:** até **2%** do faturamento no Brasil no **último exercício**, excluídos tributos, limitada a **R\$ 50 milhões por infração**. **Órgão público não leva multa** — aplicam-se apenas as sanções não pecuniárias (advertência, publicização, bloqueio, eliminação, suspensão,

proibição). O **destino** da multa é o Fundo de Defesa de Direitos Difusos, **não** o titular lesado diretamente — a reparação individual do titular corre por via **civil** separada, não pela sanção administrativa.

### Duas confusões estruturais frequentes

Achar que o dinheiro da multa vai para quem sofreu o dano é misturar duas vias diferentes: a sanção administrativa financia um fundo **coletivo**; a reparação do titular específico vem de uma ação **civil** separada. E achar que "já ter sido advertido antes" habilita a ANPD a aplicar direto a suspensão de atividade ignora que a advertência **não conta** como uma das sanções que abrem caminho para as três mais severas — é preciso uma sanção do grupo intermediário (multa, publicização, bloqueio, eliminação) primeiro.

## 19.1.8 ANPD e CNPD: dois órgãos com funções opostas

### Quem decide e quem só aconselha

**ANPD** é a autoridade: **fiscaliza, regulamenta e aplica sanção**. **CNPD** é **consultivo**: propõe diretrizes, elabora estudos, **sugere** ações à ANPD, dissemina conhecimento — nenhum verbo do CNPD é verbo de decisão. Estruturalmente, o **CNPD integra a estrutura da ANPD** (não o contrário) — é um conselho **dentro** da autoridade, não uma instância superior a ela.

Não confundir com o **Conselho Diretor**: esse é o órgão máximo de direção da própria ANPD (cinco diretores, mandato de quatro anos) — diferente do CNPD, que é o colegiado consultivo externo com representantes de governo, sociedade civil, academia e setor empresarial.

A ANPD passou por uma trajetória institucional: nasceu como **órgão** da administração federal vinculado à Presidência da República (natureza declaradamente transitória), tornou-se **autarquia de natureza especial**, e depois passou a **Agência** Nacional de Proteção de Dados, vinculada ao Ministério da Justiça e Segurança Pública, com autonomia funcional, técnica, decisória, administrativa e financeira. Alternativas que a descrevam presa a um estágio anterior dessa trajetória (órgão da administração direta, vinculada à Presidência) descrevem um desenho já superado.

## 19.2 Marco Civil da Internet (Lei 12.965/2014)

### 19.2.1 Guarda de registros

#### Dois prazos, dois tipos de provedor

**Registros de conexão**: guarda obrigatória por **1 ano** (dever do provedor de **conexão** — quem dá acesso à Internet). **Registros de acesso a aplicações**: guarda por **6 meses** (dever do provedor de **aplicação** — quem oferece o serviço/site específico). São obrigações **diferentes**, atribuídas a **papéis** diferentes na cadeia — um mesmo incidente pode envolver os dois provedores, cada um com seu prazo próprio de guarda.

### 19.2.2 Neutralidade de rede

#### Tratar todo pacote de forma isonômica — e as duas exceções

O art. 9º impõe ao responsável pela transmissão/comutação/roteamento o dever de tratar de forma **isonômica** qualquer pacote de dados, sem distinção por conteúdo, origem, destino, serviço, terminal ou aplicação. Na prática: o provedor não pode degradar, priorizar ou bloquear tráfego **por causa do que ele carrega ou de quem ele vem**. As únicas exceções (§1º) são requisitos técnicos **indispensáveis** à prestação adequada dos serviços, e priorização de serviços de **emergência** — e mesmo essas exceções dependem de regulamentação, não são livre critério do provedor.

A razão de existir essa regra: sem neutralidade, o provedor de acesso poderia favorecer certos serviços (os seus próprios, ou os de quem paga mais) em detrimento de outros, distorcendo a concorrência na Internet a partir do controle da infraestrutura de transporte — que é, por natureza, um ponto de passagem obrigatório.

#### Discriminação "por eficiência" ainda é discriminação

Reduzir a velocidade de um **tipo específico** de tráfego (ex.: vídeo) no horário de pico, para "aliviar a rede", soa como uma medida técnica razoável — mas é discriminação **por aplicação**, que não está entre as exceções do §1º (essas cobrem requisito técnico **geral** de operação da rede, não a escolha de penalizar um tipo de conteúdo específico). O teste é sempre: a medida distingue pacotes **pelo conteúdo/origem/aplicação**, ou é uma exigência técnica que se aplicaria igualmente a qualquer tráfego?

### 19.2.3 Sanções do art. 12

#### Quatro sanções, aplicáveis isolada ou cumulativamente

Advertência (com prazo para correção); **multa de até 10%** do faturamento do grupo econômico **no Brasil**, no **último exercício**, excluídos tributos — **sem** teto nominal fixo (ao contrário da LGPD); suspensão temporária das atividades; proibição do exercício das atividades. Para empresa estrangeira, a filial/sucursal situada no país responde **solidariamente** pela multa — não há brecha para empresa estrangeira operar sem responsabilidade só por não ter sede formal no Brasil.

#### Não troque o percentual entre Marco Civil e LGPD

São regimes de multa **estruturalmente diferentes**: Marco Civil — até **10%** do faturamento, **sem** teto nominal em reais; LGPD — até **2%**, com teto de **R\$ 50 milhões por infração**. Guardar só um dos dois números e aplicá-lo à lei errada é o erro mais previsível dessa seção — decore o par completo (percentual + presença ou ausência de teto), não o número isolado.

### 19.2.4 Art. 19: responsabilidade dos provedores por conteúdo de terceiros

#### O problema que o art. 19 resolve

Quando um usuário publica conteúdo ofensivo/ilegal numa plataforma (post, comentário, vídeo), até que ponto a **plataforma** (não o autor do conteúdo) responde civilmente por isso? É o problema que o art. 19 do Marco Civil endereça — e a resposta **mudou** depois de uma decisão

do STF de 26/06/2025 (Temas 987 e 533, repercussão geral, 8 votos a 3), que declarou a redação original do artigo parcial e progressivamente inconstitucional.

|                                                                                                 | Antes (redação original)                                                       | Agora (pós-STF)                                                                                  |
|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Regra geral                                                                                     | provedor só responde se descumprir <b>ordem judicial específica</b> de remoção | <b>notice-and-takedown</b> : notificação <b>extrajudicial</b> já pode responsabilizar o provedor |
| Crimes contra a honra (calúnia, injúria, difamação)                                             | exigia ordem judicial                                                          | <b>continua exigindo</b> ordem judicial — única categoria que preserva a regra antiga            |
| Conteúdo de risco grave (crime contra o Estado, terrorismo, incitação ao racismo, risco à vida) | tratamento igual ao geral                                                      | exige atuação <b>proativa</b> do provedor, sem sequer esperar notificação                        |

#### Por que a mudança faz sentido como escala de gravidade

A nova estrutura não é uma regra única aplicada a tudo — é uma **escala**, proporcional à gravidade do conteúdo e à urgência de agir: para a maioria dos casos, uma notificação já move o dever de agir (equilíbrio entre proteger a vítima e não impor censura prévia generalizada); para os crimes contra a honra, mantém-se a exigência de ordem judicial (porque a fronteira entre crítica legítima e ofensa é mais sujeita a interpretação, exigindo um juiz para decidir); para risco grave à vida ou à ordem democrática, o provedor não pode esperar nem notificação — o dever de cuidado exige ação proativa, dada a gravidade do dano potencial de esperar.

#### Nem "nada mudou" nem "tudo mudou" — é uma reforma proporcional

Dois erros simétricos de leitura: (1) aplicar a redação **antiga** como se ainda fosse a regra geral (válida só até 25/06/2025); (2) achar que o STF **extinguiu totalmente** a exigência de ordem judicial — falso, ela permanece para os crimes contra a honra. E o novo regime de notificação não é "censura prévia" — é reação a uma notificação sobre conteúdo **já publicado**, não filtragem antes da publicação. A mudança veio de uma **decisão do STF** em controle de constitucionalidade, válida até que o Congresso legisle especificamente sobre o tema — não veio de uma lei nova aprovada pelo Legislativo.

## 19.3 LAI — Lei de Acesso à Informação (Lei 12.527/2011)

#### Publicidade é a regra; sigilo é a exceção que precisa de prazo

O princípio estruturante da LAI é a inversão da lógica antiga: informação pública é, por padrão, **acessível** — sigilo é a **exceção**, e toda exceção precisa de justificativa e de **prazo máximo**, nunca sigilo indefinido:

| Classificação | Prazo máximo de restrição                   |
|---------------|---------------------------------------------|
| Reservada     | 5 anos                                      |
| Secreta       | 15 anos                                     |
| Ultrasecreta  | 25 anos (prorrogável <b>uma única vez</b> ) |

Os prazos contam da **data de produção** da informação. Informação sobre **violação de direitos humanos** por agente público **não pode** ser objeto de restrição — é uma exceção à exceção, porque o interesse público em apurar violação de direitos humanos supera o interesse do sigilo.

#### A classificação não é definitiva nem discricionária

Três mecanismos garantem que o sigilo não vire permanente por inércia:

- **Findo o prazo**, a informação se torna **automaticamente** pública — não é preciso procedimento próprio nem decisão específica para liberar; o decurso do tempo já produz o efeito por si só.
- A lei permite tanto **desclassificar** quanto **reduzir o prazo** originalmente fixado, a qualquer momento — a reavaliação cabe à autoridade classificadora ou a autoridade hierarquicamente superior, de ofício ou mediante provocação.
- A decisão de classificar é **vinculada** a balizas legais (grau, prazo máximo, competência de quem pode classificar em cada grau) — não é escolha livre e discricionária do agente público.

#### Transparência ativa x passiva, e os decretos regulamentadores

**Transparência ativa:** o órgão publica a informação **de ofício** (ex.: portal de transparência), sem que ninguém precise pedir. **Transparência passiva:** o cidadão solicita informação especificamente, via e-SIC. **Decreto 7.724/2012:** regulamenta os procedimentos de acesso no Executivo federal (e-SIC, prazo de resposta de **20 dias**, prorrogável por mais **10**, recursos, competências de classificação). **Decreto 7.845/2012:** regula procedimentos de **credenciamento de segurança** e o tratamento de informação já classificada (custódia, acesso, controle).

## 19.4 Delitos informáticos: invasão de dispositivo (art. 154-A do CP)

#### O que caracteriza o crime, e o que não é elementar do tipo

A Lei 12.737/2012 tipificou a **invasão de dispositivo informático** — invadir dispositivo alheio, conectado ou não à rede, mediante violação indevida de mecanismo de segurança, para obter, adulterar ou destruir dados/informações, ou instalar vulnerabilidade. O ponto que mais gera confusão: a conexão à rede **não é elementar do tipo** — o crime se configura mesmo num dispositivo **offline**, desde que haja violação indevida de mecanismo de segurança. A pena aumenta conforme o sujeito passivo (autoridades específicas) e o prejuízo causado.

**Não exigir conexão à rede é o ponto central do tipo**

Uma descrição do crime que exija "o dispositivo estar conectado à Internet no momento da invasão" está errada — o bem jurídico protegido é a **segurança do dispositivo em si** (a violação do mecanismo de proteção), não a segurança da comunicação em rede. Um pendrive ou um notebook desconectado, invadido fisicamente com violação de mecanismo de segurança, já configura o tipo.

## 19.5 Roteiro de revisão do capítulo

**Os pares e números que este capítulo mais confunde**

**LGPD:** controlador (decide) × operador (executa) × encarregado (não é agente); dez bases no art. 7º, sete indispensáveis no art. 11 (legítimo interesse e proteção do crédito **não** servem para dado sensível); multa até **2%**, teto **R\$ 50 milhões**; ANPD (decide e sanciona) × CNPD (só aconselha, integra a estrutura da ANPD). **Marco Civil:** guarda de conexão **1 ano** × aplicações **6 meses**; multa até **10%**, sem teto nominal; art. 19 pós-STF — notificação extrajudicial basta, **exceto** crimes contra a honra (ordem judicial) e conteúdo de risco grave (dever proativo). **LAI:** reservada **5**, secreta **15**, ultrassecreta **25** anos (só esta prorroga, uma vez); sigilo é exceção, desclassificação automática ao fim do prazo. **Art. 154-A:** invasão independe de conexão à rede.